AutoMage-2-MVP
架 构 设 计 文 档

摘要

本文档面向 AutoMage-2 MVP 阶段，重点定义系统中 Staff、Manager、Executive 三级 Agent 的 Schema 契约结构、数据流转规则与节点决策机制，为后端接口开发、数据库设计、Agent 客制化和产品验收提供统一依据。

AutoMage-2 的核心目标是将传统企业中依赖人工传递、人工汇总、人工判断的组织运行流程，转化为可记录、可校验、可追溯、可聚合的结构化数据闭环。MVP 阶段采用三级节点架构：一线员工节点负责提交每日工作数据；中层管理节点负责读取本部门员工数据并生成部门汇总、风险识别和权限内决策；老板/高管节点负责读取组织级汇总数据，生成关键决策项、方案建议和任务拆解。系统通过标准 JSON Schema 约束各级 Agent 的输入输出，确保 Agent 生成的数据可以被后端稳定解析、存储和调用。

本文档主要完成四类设计：第一，定义 Staff、Manager、Executive 三类核心 Schema 的字段结构、必填规则、校验逻辑和数据生命周期；第二，明确三级节点之间的数据读取、聚合、决策上推和任务下发机制；第三，说明 Schema 与 API、数据库表、权限模型、数字签名和审计日志之间的映射关系；第四，给出 MVP 阶段可验证的最小闭环流程，即「员工提交日报 → 中层汇总分析 → 老板接收决策项 → 老板确认方案 → 系统生成任务 → 任务下发到一线员工」。

MVP 阶段不追求完整替代企业现有 OA、ERP 或项目管理系统，也不强依赖屏幕记忆、自动录屏或行业专用数据采集能力。当前阶段优先通过 IM 表单、Agent 辅助整理和后端 Schema 校验，跑通通用型组织数据闭环。后续版本可在此基础上扩展自动化数据采集、行业化 Schema 模板、复杂权限矩阵、绩效评估、组织诊断和更完整的 Dream 决策引擎。

本文档最终交付目标是锁定 AutoMage-2 MVP 的核心契约边界，降低后端、Agent、产品和前端之间的理解偏差，确保各模块可以围绕同一套 Schema、同一套权限规则和同一条业务闭环并行开发、联调和验收。

目录

0 文档基础信息 **27**
0.1 文档名称 . . . 27
0.2 文档版本 . . . 27
0.3 Owner 与协作方 . . . 28
0.4 文档适用范围 . . . 28
0.5 本文档解决的问题 . . . 29
0.6 本文档不解决的问题 . . . 29
0.7 术语表 . . . 30
0.7.1 Agent . . . 30
0.7.2 Staff Agent . . . 30
0.7.3 Manager Agent . . . 30
0.7.4 Executive Agent . . . 31
0.7.5 Schema . . . 31
0.7.6 Schema Contract . . . 31
0.7.7 Node . . . 32
0.7.8 Decision . . . 32
0.7.9 Dream . . . 32
0.7.10 Task . . . 33
0.7.11 Work Record . . . 33
0.7.12 Digital Signature . . . 33
0.7.13 IM Trigger . . . 33

1 MVP 总体目标 **35**
1.1 AutoMage-2 的核心目标 . . . 35
1.2 MVP 阶段的最小可验证闭环 . . . 35
1.3 MVP 不追求的能力边界 . . . 36
1.4 本阶段的核心验收目标 . . . 36
1.5 MVP 成功标准 . . . 37
1.6 与 AutoMage-1 的区别 . . . 38
1.7 与飞书/传统 OA/项目管理工具的区别 . . . 38

2 MVP 系统总架构 **40**
2.1 三层 Agent 架构总览 . . . 40
2.1.1 Staff Agent：岗位执行节点 . . . 41
2.1.2 Manager Agent：部门汇总与边界决策节点 . . . 41
2.1.3 Executive Agent：老板决策与战略拆解节点 . . . 42
2.2 数据流转总览 . . . 43

2.2.1 一线员工 → Staff Schema . . . 43
2.2.2 Staff Schema → 数据库 . . . 44
2.2.3 Manager Agent → 部门汇总 Schema . . . 45
2.2.4 Executive Agent → Dream 决策 . . . 45
2.2.5 老板确认 → 任务下发 . . . 46
2.2.6 Staff Agent → 获取每日任务 . . . 47
2.3 MVP 核心链路图 . . . 48
2.4 系统模块边界 . . . 49
2.5 角色与系统交互关系 . . . 50
2.6 IM、Agent、后端、数据库之间的职责划分 . . . 51
2.7 MVP 阶段推荐技术路径 . . . 52
2.7.1 先 IM 表单 . . . 53
2.7.2 后屏幕记忆/自动采集 . . . 53
2.7.3 先 Mock . . . 54
2.7.4 后真实 API . . . 54
2.7.5 先通用 Schema . . . 55
2.7.6 后行业模板 Schema . . . 55

3 Schema 契约设计总原则 57
3.1 Schema 的定义 . . . 57
3.2 为什么 AutoMage-2 必须使用 Schema 契约 . . . 57
3.3 Schema 在系统中的位置 . . . 58
3.4 Schema 与普通日报的区别 . . . 59
3.5 Schema 与数据库表的关系 . . . 59
3.6 Schema 与 Agent Prompt 的关系 . . . 60
3.7 Schema 与前端表单的关系 . . . 61
3.8 Schema 与权限控制的关系 . . . 61
3.9 Schema 与数字签名的关系 . . . 62
3.10 Schema 设计原则 . . . 63
3.10.1 字段稳定 . . . 63
3.10.2 类型明确 . . . 63
3.10.3 必填项明确 . . . 64
3.10.4 可校验 . . . 64
3.10.5 可追溯 . . . 65
3.10.6 可聚合 . . . 65
3.10.7 可版本化 . . . 65
3.10.8 可审计 . . . 66
3.10.9 可被 Agent 理解 . . . 66
3.10.10 人类最终确认 . . . 67

3.10.11 MVP 优先通用，后续支持行业化 . . . 67
3.10.12 后端校验优先于 Agent 自觉 . . . 67
3.10.13 数据库是事实源 . . . 68
3.10.14 先闭环，后优化 . . . 68

4 Schema 分层体系设计 70
4.1 Schema 分层总览 . . . 70
4.2 Staff Schema：一线执行数据 . . . 71
4.3 Manager Schema：部门汇总数据 . . . 71
4.4 Executive Schema：老板决策数据 . . . 72
4.5 Task Schema：任务下发数据 . . . 73
4.6 Incident Schema：异常上报数据 . . . 75
4.7 Decision Schema：决策记录数据 . . . 76
4.8 Summary Schema：周期性汇总数据 . . . 78
4.9 不同 Schema 之间的转换关系 . . . 79
4.10 Schema 生命周期 . . . 80

5 Staff 每日日报 Schema 设计 83
5.1 Schema 基本信息 . . . 83
5.2 Staff Schema 字段总览 . . . 84
5.3 字段定义明细 . . . 84
5.3.1 `schema_id`：Schema 标识 . . . 84
5.3.2 `schema_version`：Schema 版本 . . . 84
5.3.3 `timestamp`：提交时间 . . . 85
5.3.4 `org_id`：组织 ID . . . 85
5.3.5 `department_id`：部门 ID . . . 85
5.3.6 `user_id`：员工 ID . . . 85
5.3.7 `node_id`：Agent 节点 ID . . . 86
5.3.8 `record_date`：日报日期 . . . 86
5.3.9 `work_progress`：今日完成事项 . . . 86
5.3.10 `issues_faced`：遇到的问题 . . . 87
5.3.11 `solution_attempt`：已尝试的解决方案 . . . 88
5.3.12 `need_support`：是否需要上级支持 . . . 88
5.3.13 `support_detail`：需要支持的具体内容 . . . 88
5.3.14 `next_day_plan`：明日计划 . . . 89
5.3.15 `resource_usage`：资源消耗 . . . 89
5.3.16 `artifacts`：产出物 . . . 90
5.3.17 `related_task_ids`：关联任务 ID . . . 90
5.3.18 `risk_level`：风险等级 . . . 90

5.3.19 employee_comment：员工补充说明 90
5.3.20 signature：数字签名信息 90
5.3.21 meta：扩展字段 91
5.4 Staff Schema JSON 示例 91
5.5 Staff Schema 校验规则 93
5.5.1 基础字段校验 93
5.5.2 工作内容校验 93
5.5.3 支持需求校验 94
5.5.4 风险等级校验 94
5.5.5 签名校验 94
5.5.6 权限校验 94
5.5.7 重复提交校验 94
5.6 Staff Schema 必填字段 94
5.7 Staff Schema 可选字段 95
5.8 Staff Schema 错误示例 95
5.8.1 错误示例一：把 boolean 写成字符串 95
5.8.2 错误示例二：工作进展不是数组 95
5.8.3 错误示例三：需要支持但未填写支持内容 95
5.8.4 错误示例四：缺少签名信息 95
5.8.5 错误示例五：把关键数据塞进补充说明 95
5.9 Staff Agent 生成 Schema 的 Prompt 约束 95
5.9.1 角色约束 95
5.9.2 信息收集约束 95
5.9.3 输出格式约束 96
5.9.4 不编造约束 96
5.9.5 风险提示约束 96
5.9.6 员工确认约束 96
5.10 Staff Agent 提交失败后的重试逻辑 96
5.10.1 422 Schema 校验失败 96
5.10.2 401 鉴权失败 96
5.10.3 403 权限不足 96
5.10.4 409 重复提交 96
5.10.5 5xx 服务错误 96
5.10.6 网络超时 97
5.11 Staff Schema 与数据库字段映射 97
5.11.1 映射到 work_records 97
5.11.2 映射到 work_record_items 97
5.11.3 映射到 artifacts 98
5.11.4 映射到 incidents 98

5.11.5 映射到 tasks . . . 98
5.11.6 映射到 audit_logs . . . 98
5.11.7 当前数据库映射建议 . . . 98

6 Manager 汇总日报 Schema 设计 99
6.1 Schema 基本信息 . . . 99
6.1.1 Schema ID：schema_v1_manager . . . 100
6.1.2 使用节点：Manager Agent . . . 100
6.1.3 触发方式：每日定时读取部门 Staff Schema . . . 100
6.1.4 数据来源：本部门下属员工日报 . . . 100
6.1.5 写入对象：部门汇总表 / summaries . . . 101
6.2 Manager Schema 字段总览 . . . 101
6.3 字段定义明细 . . . 102
6.3.1 timestamp：汇总生成时间 . . . 102
6.3.2 manager_user_id：中层负责人 ID . . . 103
6.3.3 manager_node_id：中层 Agent 节点 ID . . . 103
6.3.4 dept_id：部门 ID . . . 103
6.3.5 staff_report_count：读取的一线日报数量 . . . 103
6.3.6 missing_report_count：缺失日报数量 . . . 104
6.3.7 overall_health：部门整体健康度 . . . 104
6.3.8 aggregated_summary：部门综合进展摘要 . . . 105
6.3.9 top_3_risks：三大风险 . . . 105
6.3.10 workforce_efficiency：部门效能评分 . . . 106
6.3.11 pending_approvals：待老板决策事项数量 . . . 106
6.3.12 highlight_staff：表现突出员工 . . . 107
6.3.13 blocked_items：阻塞事项 . . . 107
6.3.14 manager_decisions：权限内已处理决策 . . . 108
6.3.15 need_executive_decision：需上推老板的事项 . . . 109
6.3.16 next_day_adjustment：次日部门调整建议 . . . 109
6.3.17 source_record_ids：引用的一线日报 ID . . . 110
6.3.18 signature：Manager 确认签名 . . . 110
6.4 Manager Schema JSON 示例 . . . 111
6.5 Manager Schema 聚合逻辑 . . . 113
6.5.1 读取 Staff Schema . . . 114
6.5.2 过滤无效数据 . . . 114
6.5.3 按事项归类 . . . 115
6.5.4 提炼部门进展 . . . 115
6.5.5 判断风险和权限边界 . . . 115
6.5.6 输出 Manager Schema . . . 116

6.6 Manager Schema 生成 Prompt 约束 . . . 116
6.6.1 不复制员工日报 . . . 116
6.6.2 不编造未出现的信息 . . . 116
6.6.3 明确区分事实和判断 . . . 117
6.6.4 高风险必须说明来源 . . . 117
6.6.5 上推事项必须整理成决策题 . . . 117
6.6.6 保留中层处理价值 . . . 118
6.7 Manager Schema 风险识别规则 . . . 118
6.7.1 风险来源 . . . 118
6.7.2 风险等级判断 . . . 118
6.7.3 风险进入 `top_3_risks` 的条件 . . . 119
6.7.4 缺失日报风险 . . . 119
6.7.5 风险与异常的转换 . . . 119
6.8 Manager Schema 与 Staff Schema 的关系 . . . 119
6.8.1 字段对应关系 . . . 120
6.8.2 一对多与多对一关系 . . . 120
6.8.3 Staff 数据不完整时的处理 . . . 120
6.8.4 Staff 原始数据不得被覆盖 . . . 121
6.9 Manager Schema 与数据库字段映射 . . . 121
6.9.1 映射到 `summaries` . . . 121
6.9.2 映射到 `summary_source_links` . . . 122
6.9.3 映射到 `tasks` . . . 122
6.9.4 映射到 `incidents` . . . 122
6.9.5 映射到决策记录 . . . 123
6.9.6 映射到 `audit_logs` . . . 123
6.10 Manager Agent 读取权限规则 . . . 123
6.10.1 基础读取范围 . . . 123
6.10.2 读取 Staff Schema 的条件 . . . 124
6.10.3 跨部门事项处理 . . . 124
6.10.4 数据下钻权限 . . . 125
6.10.5 权限失败处理 . . . 125
6.10.6 Manager 汇总的写入权限 . . . 125
6.10.7 审计要求 . . . 126
6.11 本章小结 . . . 126

7 Executive 决策 Schema 设计 127
7.1 Schema 基本信息 . . . 127
7.1.1 Schema ID：`schema_v1_executive` . . . 128
7.1.2 使用节点：Executive Agent . . . 128

7.1.3 触发方式：每日早间 / 定时 Dream . . . 128
7.1.4 数据来源：Manager Schema、历史决策、外部变量 . . . 128
7.1.5 输出对象：老板决策卡片、任务拆解 . . . 129
7.2 Executive Schema 字段总览 . . . 129
7.3 字段定义明细 . . . 130
7.3.1 timestamp：生成时间 . . . 130
7.3.2 executive_user_id：老板/高管用户 ID . . . 131
7.3.3 executive_node_id：老板 Agent 节点 ID . . . 131
7.3.4 org_id：组织 ID . . . 131
7.3.5 business_summary：公司级业务摘要 . . . 131
7.3.6 key_risks：关键风险 . . . 132
7.3.7 decision_required：是否需要老板决策 . . . 132
7.3.8 decision_items：待决策事项列表 . . . 133
7.3.9 option_a：方案 A . . . 134
7.3.10 option_b：方案 B . . . 134
7.3.11 recommended_option：Agent 推荐方案 . . . 135
7.3.12 reasoning_summary：推荐理由摘要 . . . 135
7.3.13 expected_impact：预期影响 . . . 135
7.3.14 generated_tasks：可下发任务 . . . 136
7.3.15 source_summary_ids：引用的部门汇总 ID . . . 136
7.3.16 human_confirm_status：人工确认状态 . . . 137
7.3.17 confirmed_by：确认人 . . . 137
7.3.18 confirmed_at：确认时间 . . . 137
7.4 Executive Schema JSON 示例 . . . 138
7.5 Executive Schema 决策卡片格式 . . . 140
7.6 Executive Schema 与 Dream 机制关系 . . . 141
7.7 Executive Schema 与任务下发关系 . . . 142
7.8 Executive Schema 与数据库字段映射 . . . 143
7.8.1 映射到组织级 summaries . . . 143
7.8.2 映射到 Decision 记录 . . . 144
7.8.3 映射到 tasks . . . 144
7.8.4 映射到 audit_logs . . . 144
7.8.5 映射到 summary_source_links . . . 145
7.9 老板确认后的状态变化 . . . 145
7.10 未确认时的处理规则 . . . 146
7.10.1 等待确认 . . . 146
7.10.2 超时未确认 . . . 147
7.10.3 老板要求补充信息 . . . 147
7.10.4 老板驳回 . . . 147

7.10.5 决策卡片修改 . . . 148
7.10.6 未确认数据的可见范围 . . . 148
7.10.7 未确认事项的复盘 . . . 149
7.11 本章小结 . . . 149

8 三级节点决策机制设计 150
8.1 决策机制总览 . . . 150
8.2 决策权限分层原则 . . . 150
8.3 Staff Agent 的决策边界 . . . 151
8.3.1 可自主记录 . . . 152
8.3.2 可自主提醒 . . . 152
8.3.3 不可替员工做重大承诺 . . . 153
8.3.4 不可越级读取他人数据 . . . 153
8.4 Manager Agent 的决策边界 . . . 154
8.4.1 可读取本部门数据 . . . 154
8.4.2 可生成部门汇总 . . . 155
8.4.3 可处理权限内异常 . . . 155
8.4.4 可下发部门级任务 . . . 156
8.4.5 超权限事项上推老板 . . . 156
8.5 Executive Agent 的决策边界 . . . 157
8.5.1 可读取组织级汇总 . . . 157
8.5.2 可生成决策方案 . . . 158
8.5.3 可推荐方案 . . . 159
8.5.4 不可绕过老板自动执行重大决策 . . . 159
8.6 决策等级定义 . . . 159
8.6.1 L0：Agent 可自动处理 . . . 160
8.6.2 L1：员工确认后处理 . . . 160
8.6.3 L2：Manager 确认后处理 . . . 160
8.6.4 L3：Executive 确认后处理 . . . 161
8.6.5 L4：需要人工会议/线下审批 . . . 161
8.7 决策上推规则 . . . 162
8.7.1 Staff 向 Manager 上推 . . . 162
8.7.2 Manager 向 Executive 上推 . . . 163
8.7.3 上推时机 . . . 163
8.7.4 上推失败处理 . . . 164
8.8 决策下发规则 . . . 164
8.8.1 Executive 向 Manager 下发 . . . 164
8.8.2 Manager 向 Staff 下发 . . . 165
8.8.3 下发前校验 . . . 165

8.8.4 下发后的状态追踪 . . . 166
8.9 决策冲突处理 . . . 166
8.9.1 常见冲突类型 . . . 166
8.9.2 冲突处理优先级 . . . 166
8.9.3 冲突发现方式 . . . 167
8.9.4 冲突处理流程 . . . 167
8.10 决策撤回机制 . . . 167
8.10.1 可撤回范围 . . . 168
8.10.2 撤回权限 . . . 168
8.10.3 撤回流程 . . . 168
8.10.4 决策修改与重新提交 . . . 169
8.11 决策审计机制 . . . 169
8.11.1 审计对象 . . . 169
8.11.2 审计事件 . . . 169
8.11.3 审计记录要求 . . . 170
8.11.4 审计与原始数据关系 . . . 170
8.12 决策日志字段定义 . . . 171
8.12.1 决策日志主字段 . . . 171
8.12.2 决策状态字段 . . . 171
8.12.3 候选方案字段 . . . 172
8.12.4 签名字段 . . . 172
8.12.5 决策日志 JSON 示例 . . . 173
8.13 本章小结 . . . 174

9 Dream 机制设计 175
9.1 Dream 的定义 . . . 175
9.2 Dream 在 AutoMage-2 中的作用 . . . 175
9.3 Dream 的每日运行时间 . . . 176
9.4 Dream 输入数据 . . . 177
9.4.1 Staff Schema . . . 177
9.4.2 Manager Schema . . . 178
9.4.3 历史 Summary . . . 178
9.4.4 历史 Decision . . . 179
9.4.5 未完成任务 . . . 180
9.4.6 外部业务目标 . . . 180
9.5 Dream 输出数据 . . . 181
9.5.1 老板决策项 . . . 181
9.5.2 风险预警 . . . 182
9.5.3 任务拆解 . . . 183

9.5.4 次日重点 . . . 184
9.5.5 部门调整建议 . . . 185
9.6 Dream 运行流程 . . . 186
9.6.1 触发运行 . . . 186
9.6.2 读取数据 . . . 186
9.6.3 过滤无效数据 . . . 187
9.6.4 归并同类事项 . . . 187
9.6.5 识别风险 . . . 187
9.6.6 生成决策项 . . . 188
9.6.7 生成任务草案 . . . 188
9.6.8 写入结果 . . . 188
9.6.9 推送结果 . . . 188
9.7 Dream Prompt 结构 . . . 189
9.7.1 角色定义 . . . 189
9.7.2 输入说明 . . . 189
9.7.3 输出目标 . . . 189
9.7.4 约束规则 . . . 190
9.7.5 输出格式 . . . 190
9.7.6 自检要求 . . . 191
9.8 Dream 结果校验 . . . 191
9.8.1 结构校验 . . . 191
9.8.2 来源校验 . . . 191
9.8.3 权限校验 . . . 192
9.8.4 业务校验 . . . 192
9.8.5 人工确认校验 . . . 192
9.9 Dream 与 Executive Schema 的关系 . . . 193
9.10 Dream 与任务生成的关系 . . . 194
9.10.1 任务草案生成 . . . 194
9.10.2 正式任务生成 . . . 194
9.10.3 任务分发 . . . 195
9.10.4 任务结果回流 . . . 195
9.11 Dream 失败时的降级策略 . . . 195
9.11.1 常见失败类型 . . . 196
9.11.2 降级策略一：生成简版摘要 . . . 196
9.11.3 降级策略二：只转发 Manager 上推事项 . . . 196
9.11.4 降级策略三：延迟重试 . . . 197
9.11.5 降级策略四：人工接管 . . . 197
9.11.6 失败记录 . . . 197
9.12 Dream 结果的人工确认机制 . . . 198

9.12.1 不同输出的确认要求 . . . 198
9.12.2 老板确认 . . . 198
9.12.3 Manager 确认 . . . 199
9.12.4 修改确认 . . . 199
9.12.5 驳回与补充信息 . . . 199
9.12.6 人工确认后的审计 . . . 200
9.13 本章小结 . . . 200

10 任务生成与反向下发机制 201
10.1 任务生成来源 . . . 201
10.2 Staff 日报生成任务 . . . 202
10.3 Manager 汇总生成任务 . . . 203
10.4 Executive 决策生成任务 . . . 205
10.5 老板确认后任务下发 . . . 206
10.6 任务分配规则 . . . 208
10.6.1 优先分配给明确负责人 . . . 208
10.6.2 只有角色时先分配给 Manager . . . 208
10.6.3 跨部门任务由多个 Manager 承接 . . . 209
10.6.4 老板任务不直接压给一线 . . . 209
10.6.5 一人多任务时需要优先级排序 . . . 209
10.6.6 分配失败处理 . . . 210
10.7 任务优先级规则 . . . 210
10.8 任务截止时间规则 . . . 211
10.9 任务状态流转 . . . 212
10.9.1 待处理 . . . 212
10.9.2 进行中 . . . 213
10.9.3 阻塞 . . . 213
10.9.4 已完成 . . . 214
10.9.5 已关闭 . . . 214
10.9.6 已取消 . . . 215
10.10 任务 Schema 字段定义 . . . 215
10.10.1 Task Schema 字段总览 . . . 216
10.10.2 Task Schema JSON 示例 . . . 216
10.10.3 字段说明 . . . 217
10.11 任务与 Staff Agent 的交互方式 . . . 217
10.11.1 每日任务推送 . . . 218
10.11.2 任务开始 . . . 218
10.11.3 任务阻塞 . . . 218
10.11.4 任务完成 . . . 219

10.11.5 任务进入日报 . . . 219
10.11.6 员工拒绝或无法接收任务 . . . 219
10.12 任务与数据库字段映射 . . . 219
10.12.1 映射到 tasks . . . 220
10.12.2 映射到 task_assignments . . . 220
10.12.3 映射到 task_updates . . . 221
10.12.4 映射到 artifacts . . . 221
10.12.5 映射到 audit_logs . . . 222
10.12.6 当前数据库映射建议 . . . 222
10.13 本章小结 . . . 223

11 异常与风险识别机制 224
11.1 异常定义 . . . 224
11.2 风险定义 . . . 224
11.3 Staff 层异常 . . . 225
11.4 Manager 层异常 . . . 226
11.5 Executive 层异常 . . . 227
11.6 异常等级 . . . 228
11.6.1 low . . . 228
11.6.2 medium . . . 229
11.6.3 high . . . 229
11.6.4 critical . . . 230
11.7 异常上报规则 . . . 230
11.7.1 Staff 向 Manager 上报 . . . 230
11.7.2 Manager 向 Executive 上报 . . . 231
11.7.3 自动上报条件 . . . 232
11.7.4 上报去重 . . . 232
11.8 异常处理流程 . . . 233
11.8.1 发现异常 . . . 233
11.8.2 确认异常 . . . 233
11.8.3 分配处理人 . . . 234
11.8.4 生成处理任务 . . . 234
11.8.5 跟进异常状态 . . . 235
11.8.6 上推处理 . . . 235
11.9 异常关闭规则 . . . 235
11.9.1 异常关闭条件 . . . 236
11.9.2 不允许关闭的情况 . . . 236
11.9.3 关闭权限 . . . 236
11.9.4 重新打开异常 . . . 237

11.10 异常 Schema 字段定义 . . . 237
11.10.1 Incident Schema 字段总览 . . . 238
11.10.2 异常类型枚举 . . . 238
11.10.3 异常状态枚举 . . . 239
11.10.4 Incident Schema JSON 示例 . . . 239
11.11 异常与任务的关系 . . . 240
11.11.1 一个异常生成一个任务 . . . 241
11.11.2 一个异常生成多个任务 . . . 241
11.11.3 多个异常合并为一个任务 . . . 241
11.11.4 任务阻塞反向生成异常 . . . 242
11.11.5 异常关闭依赖任务结果 . . . 242
11.12 异常与审计日志的关系 . . . 242
11.12.1 需要审计的异常事件 . . . 243
11.12.2 审计记录内容 . . . 243
11.12.3 审计与责任划分 . . . 244
11.12.4 审计与复盘 . . . 244
11.13 本章小结 . . . 245

12 数字签名与数据可信机制 246
12.1 为什么需要数字签名 . . . 246
12.2 MVP 阶段的数字签名定义 . . . 246
12.3 数字签名适用对象 . . . 247
12.3.1 Staff 日报 . . . 248
12.3.2 Manager 汇总 . . . 248
12.3.3 Executive 决策 . . . 249
12.3.4 任务确认 . . . 250
12.3.5 异常处理 . . . 250
12.4 签名字段设计 . . . 251
12.5 签名生成流程 . . . 253
12.5.1 生成待确认内容 . . . 253
12.5.2 展示给用户 . . . 254
12.5.3 用户执行确认动作 . . . 254
12.5.4 后端校验身份和权限 . . . 255
12.5.5 后端规范化 payload . . . 255
12.5.6 生成 payload_hash . . . 256
12.5.7 写入签名字段 . . . 256
12.5.8 写入审计日志 . . . 256
12.6 签名校验流程 . . . 257
12.6.1 检查签名字段 . . . 257

12.6.2 检查签名人权限 . . . 258
12.6.3 重新计算 payload_hash . . . 258
12.6.4 校验结果处理 . . . 258
12.6.5 何时触发校验 . . . 258
12.7 签名失败处理 . . . 259
12.7.1 未签名 . . . 259
12.7.2 签名人无权限 . . . 259
12.7.3 内容被修改 . . . 260
12.7.4 签名超时 . . . 260
12.7.5 签名被拒绝 . . . 260
12.7.6 签名系统异常 . . . 261
12.8 签名与责任划分 . . . 261
12.8.1 Agent 的责任 . . . 261
12.8.2 员工的责任 . . . 262
12.8.3 Manager 的责任 . . . 262
12.8.4 Executive / 老板的责任 . . . 262
12.8.5 后端系统的责任 . . . 263
12.8.6 责任边界示例 . . . 263
12.9 签名与审计日志 . . . 264
12.9.1 签名字段记录当前状态 . . . 264
12.9.2 审计日志记录过程 . . . 264
12.9.3 需要写入审计的签名事件 . . . 264
12.9.4 审计日志字段建议 . . . 265
12.9.5 签名后修改的审计要求 . . . 266
12.10 MVP 阶段可接受的简化实现 . . . 266
12.10.1 简化实现目标 . . . 266
12.10.2 推荐简化方案 . . . 267
12.10.3 最小落地字段 . . . 267
12.10.4 MVP 阶段签名规则建议 . . . 268
12.10.5 不同阶段的演进路线 . . . 268
12.10.6 MVP 不应省略的底线 . . . 268
12.11 本章小结 . . . 269

13 前端 / IM / Agent 校验逻辑 270
13.1 校验逻辑总览 . . . 270
13.2 前端表单校验 . . . 270
13.3 Agent 生成前校验 . . . 271
13.4 Agent 生成后校验 . . . 272
13.5 后端 Schema 校验 . . . 273

13.6 字段类型校验 274
13.6.1 string 字段 275
13.6.2 boolean 字段 275
13.6.3 array 字段 275
13.6.4 object 字段 276
13.6.5 datetime 字段 276
13.7 必填字段校验 277
13.7.1 Staff Schema 必填字段 277
13.7.2 Manager Schema 必填字段 277
13.7.3 Executive Schema 必填字段 278
13.8 枚举值校验 278
13.8.1 风险等级 278
13.8.2 部门健康度 279
13.8.3 任务优先级 279
13.8.4 任务状态 279
13.8.5 决策状态 279
13.8.6 签名状态 280
13.9 权限校验 280
13.9.1 Staff Agent 权限 280
13.9.2 Manager Agent 权限 281
13.9.3 Executive Agent 权限 281
13.10 签名校验 282
13.11 重试机制 282
13.11.1 可以重试的场景 283
13.11.2 不建议自动重试的场景 283
13.12 422 错误处理 283
13.13 401/403 权限错误处理 285
13.13.1 401 处理 285
13.13.2 403 处理 285
13.14 超时与幂等处理 286
13.14.1 为什么需要幂等 286
13.14.2 推荐幂等键设计 286
13.14.3 超时处理 287
13.14.4 幂等返回 287
13.14.5 重复提交处理 287
13.14.6 超时后的用户提示 288
13.15 本章小结 288

14 API 契约设计 **289**
14.1 API 设计原则 . . . 289
14.1.1 Schema 优先 . . . 289
14.1.2 后端最终校验 . . . 289
14.1.3 数据库是事实源 . . . 290
14.1.4 幂等优先 . . . 290
14.1.5 错误可被 Agent 理解 . . . 290
14.1.6 权限由后端强制执行 . . . 291
14.1.7 接口版本化 . . . 291
14.2 统一响应格式 . . . 291
14.2.1 成功响应格式 . . . 291
14.2.2 失败响应格式 . . . 292
14.2.3 分页响应格式 . . . 292
14.2.4 幂等重放响应格式 . . . 293
14.3 统一错误码 . . . 293
14.4 Agent 初始化接口 . . . 294
14.4.1 `POST /api/v1/agent/init` . . . 294
14.4.2 请求参数 . . . 294
14.4.3 响应参数 . . . 295
14.4.4 错误码 . . . 296
14.5 Staff 日报提交接口 . . . 296
14.5.1 `POST /api/v1/report/staff` . . . 296
14.5.2 请求参数 . . . 296
14.5.3 响应参数 . . . 298
14.5.4 Schema 校验规则 . . . 298
14.6 Manager 汇总提交接口 . . . 299
14.6.1 `POST /api/v1/report/manager` . . . 299
14.6.2 请求参数 . . . 299
14.6.3 响应参数 . . . 301
14.7 Executive 决策提交接口 . . . 302
14.7.1 `POST /api/v1/decision/commit` . . . 302
14.7.2 请求参数 . . . 302
14.7.3 响应参数 . . . 303
14.8 任务获取接口 . . . 304
14.8.1 `GET /api/v1/tasks` . . . 304
14.8.2 请求参数 . . . 304
14.8.3 响应参数 . . . 304
14.9 汇总读取接口 . . . 305
14.9.1 `GET /api/v1/summaries` . . . 305

14.9.2 请求参数 . . . 306
14.9.3 响应参数 . . . 306
14.9.4 来源追溯接口 . . . 307
14.10 异常上报接口 . . . 307
14.10.1 `POST /api/v1/incidents` . . . 307
14.10.2 请求参数 . . . 308
14.10.3 响应参数 . . . 308
14.10.4 异常状态更新接口 . . . 309
14.11 审计日志接口 . . . 309
14.11.1 `GET /api/v1/audit-logs` . . . 309
14.11.2 请求参数 . . . 309
14.11.3 响应参数 . . . 310
14.11.4 审计写入规则 . . . 310
14.12 API 与 Schema 的映射关系 . . . 311
14.13 API 与数据库的映射关系 . . . 312
14.13.1 Staff 日报提交映射 . . . 312
14.13.2 Manager 汇总提交映射 . . . 313
14.13.3 Executive 决策确认映射 . . . 313
14.13.4 任务读取映射 . . . 313
14.13.5 汇总读取映射 . . . 314
14.13.6 异常上报映射 . . . 314
14.13.7 审计日志读取映射 . . . 315
14.14 本章小结 . . . 315

15 数据库映射设计 316
15.1 数据库映射总览 . . . 316
15.2 Staff Schema 与 ‘work_records‘ 的映射 . . . 317
15.3 Staff Schema 与 ‘work_record_items‘ 的映射 . . . 318
15.4 Manager Schema 与 ‘summaries‘ 的映射 . . . 319
15.5 Executive Schema 与决策表的映射 . . . 321
15.5.1 MVP 临时映射方式 . . . 321
15.5.2 建议正式映射方式 . . . 321
15.6 Task Schema 与 ‘tasks‘ 的映射 . . . 322
15.7 Task Assignment 与 ‘task_assignments‘ 的映射 . . . 324
15.8 Incident Schema 与 ‘incidents‘ 的映射 . . . 324
15.9 Artifact 与 ‘artifacts‘ 的映射 . . . 326
15.10 Audit 与 ‘audit_logs‘ 的映射 . . . 327
15.11 当前 DDL 缺口说明 . . . 328
15.11.1 Agent 节点表 . . . 328

15.11.2 Decision 记录表 . . . 329
15.11.3 Digital Signature 字段 . . . 329
15.11.4 Dream Run 表 . . . 329
15.11.5 Schema Contract 表 . . . 330
15.12 建议新增表结构清单 . . . 331
15.12.1 `agent_nodes` . . . 331
15.12.2 `decisions` . . . 332
15.12.3 `decision_source_links` . . . 333
15.12.4 `digital_signatures` . . . 333
15.12.5 `dream_runs` . . . 334
15.12.6 `schema_contracts` . . . 334
15.12.7 可选补充表：`incident_source_links` . . . 335
15.12.8 可选补充表：`incident_task_links` . . . 335
15.12.9 新增表优先级建议 . . . 336
15.13 本章小结 . . . 336

16 权限与数据隔离设计 338
16.1 权限设计原则 . . . 338
16.2 组织级隔离 . . . 339
16.3 部门级隔离 . . . 340
16.4 用户级隔离 . . . 341
16.5 Agent 节点级隔离 . . . 342
16.6 Staff 权限 . . . 344
16.7 Manager 权限 . . . 345
16.8 Executive 权限 . . . 346
16.9 跨级读取规则 . . . 347
16.9.1 Staff → Manager . . . 347
16.9.2 Manager → Staff . . . 348
16.9.3 Executive → Manager . . . 348
16.9.4 Executive → Staff . . . 348
16.9.5 下级读取上级数据 . . . 349
16.10 跨部门读取规则 . . . 349
16.10.1 跨部门协作 . . . 349
16.10.2 跨部门汇总 . . . 350
16.10.3 跨部门明细读取 . . . 350
16.10.4 跨部门任务权限 . . . 350
16.10.5 跨部门授权记录 . . . 351
16.11 权限失败处理 . . . 351
16.11.1 401 未认证 . . . 351

16.11.2 403 无权限 . . . 352
16.11.3 404 不存在或不可见 . . . 352
16.11.4 权限失败的 Agent 处理 . . . 353
16.11.5 权限失败与用户体验 . . . 353
16.12 权限审计 . . . 353
16.12.1 需要审计的权限事件 . . . 353
16.12.2 权限审计字段 . . . 354
16.12.3 审计粒度 . . . 355
16.12.4 权限变更审计 . . . 355
16.12.5 权限审计与异常联动 . . . 356
16.12.6 权限审计的查询入口 . . . 356
16.13 本章小结 . . . 357

17 MVP 核心 Demo 流程 358
17.1 Demo 目标 . . . 358
17.2 Demo 角色 . . . 358
17.3 Demo 前置数据 . . . 359
17.3.1 组织与用户数据 . . . 359
17.3.2 Agent 节点数据 . . . 359
17.3.3 Schema 与模板数据 . . . 360
17.3.4 Demo 业务场景数据 . . . 360
17.4 Demo 流程总览 . . . 361
17.5 Step 1：Staff Agent 初始化 . . . 362
17.5.1 目标 . . . 362
17.5.2 调用接口 . . . 362
17.5.3 成功表现 . . . 363
17.6 Step 2：员工提交每日工作 Schema . . . 363
17.6.1 目标 . . . 363
17.6.2 员工输入示例 . . . 363
17.6.3 Staff Agent 整理结果 . . . 364
17.6.4 员工确认 . . . 365
17.7 Step 3：Staff Schema 写入数据库 . . . 365
17.7.1 目标 . . . 365
17.7.2 调用接口 . . . 366
17.7.3 后端处理 . . . 367
17.7.4 响应示例 . . . 367
17.7.5 成功表现 . . . 368
17.8 Step 4：Manager Agent 定时读取部门数据 . . . 368
17.8.1 目标 . . . 368

17.8.2 触发方式 . . . 368
17.8.3 Manager Agent 读取条件 . . . 368
17.8.4 读取结果示例 . . . 369
17.8.5 成功表现 . . . 369
17.9 Step 5：Manager 生成部门汇总 Schema . . . 369
17.9.1 目标 . . . 369
17.9.2 Manager Schema 示例 . . . 369
17.9.3 提交接口 . . . 372
17.9.4 响应示例 . . . 372
17.9.5 成功表现 . . . 372
17.10 Step 6：Executive Agent 读取部门汇总 . . . 373
17.10.1 目标 . . . 373
17.10.2 读取接口 . . . 373
17.10.3 返回数据 . . . 373
17.10.4 成功表现 . . . 373
17.11 Step 7：Dream 生成老板决策项 . . . 374
17.11.1 目标 . . . 374
17.11.2 Executive Schema 示例 . . . 374
17.11.3 成功表现 . . . 376
17.12 Step 8：IM 推送老板决策卡片 . . . 376
17.12.1 目标 . . . 376
17.12.2 决策卡片内容 . . . 376
17.12.3 卡片按钮 . . . 377
17.12.4 成功表现 . . . 377
17.13 Step 9：老板选择方案 . . . 377
17.13.1 目标 . . . 377
17.13.2 老板操作 . . . 377
17.13.3 调用接口 . . . 377
17.13.4 后端处理 . . . 378
17.13.5 响应示例 . . . 378
17.14 Step 10：系统自动生成任务 . . . 379
17.14.1 目标 . . . 379
17.14.2 任务生成规则 . . . 379
17.14.3 写入数据库 . . . 380
17.14.4 幂等要求 . . . 380
17.15 Step 11：任务下发到 Staff Agent . . . 380
17.15.1 目标 . . . 380
17.15.2 下发逻辑 . . . 380
17.15.3 Staff Agent 收到任务通知 . . . 381

17.15.4 成功表现 . . . 381
17.16 Step 12：员工查询今日任务 . . . 381
17.16.1 目标 . . . 381
17.16.2 调用接口 . . . 381
17.16.3 响应示例 . . . 381
17.16.4 Staff Agent 展示 . . . 382
17.16.5 可选演示：员工开始处理 . . . 382
17.17 Demo 成功判定标准 . . . 383
17.17.1 主链路成功标准 . . . 383
17.17.2 数据追溯成功标准 . . . 383
17.17.3 权限成功标准 . . . 384
17.17.4 签名成功标准 . . . 384
17.17.5 幂等成功标准 . . . 384
17.17.6 演示话术建议 . . . 384
17.18 本章小结 . . . 385

18 MVP 验收标准 386
18.1 Schema 验收标准 . . . 386
18.2 Agent 验收标准 . . . 387
18.2.1 Staff Agent 验收标准 . . . 387
18.2.2 Manager Agent 验收标准 . . . 388
18.2.3 Executive Agent 验收标准 . . . 388
18.2.4 Dream 验收标准 . . . 389
18.3 API 验收标准 . . . 389
18.3.1 Agent 初始化接口 . . . 390
18.3.2 Staff 日报提交接口 . . . 390
18.3.3 Manager 汇总提交接口 . . . 390
18.3.4 决策确认接口 . . . 391
18.4 数据库验收标准 . . . 391
18.4.1 主链路落表验收 . . . 392
18.4.2 数据追溯验收 . . . 392
18.4.3 数据库缺口验收 . . . 392
18.5 权限验收标准 . . . 393
18.5.1 组织级权限 . . . 393
18.5.2 用户级权限 . . . 393
18.5.3 部门级权限 . . . 393
18.5.4 Executive 权限 . . . 394
18.5.5 Agent 节点权限 . . . 394
18.6 决策机制验收标准 . . . 394

18.6.1 决策分层验收 . . . 394
18.6.2 上推机制验收 . . . 395
18.6.3 老板确认验收 . . . 395
18.7 Dream 机制验收标准 . . . 395
18.7.1 Dream 输入验收 . . . 395
18.7.2 Dream 输出验收 . . . 396
18.7.3 Dream 运行验收 . . . 396
18.8 任务闭环验收标准 . . . 396
18.8.1 任务生成验收 . . . 397
18.8.2 任务分配验收 . . . 397
18.8.3 任务状态验收 . . . 397
18.8.4 Staff Agent 任务交互验收 . . . 397
18.9 异常处理验收标准 . . . 398
18.9.1 异常识别验收 . . . 398
18.9.2 异常字段验收 . . . 398
18.9.3 异常流转验收 . . . 399
18.9.4 异常与任务关系验收 . . . 399
18.10 演示验收标准 . . . 400
18.10.1 Demo 主链路验收 . . . 400
18.10.2 Demo 数据追溯验收 . . . 400
18.10.3 Demo 签名与审计验收 . . . 401
18.10.4 Demo 失败判定 . . . 401
18.10.5 Demo 最终通过标准 . . . 401
18.11 本章小结 . . . 402

19 风险与待确认问题 403
19.1 Schema 是否过于通用 . . . 403
19.2 员工填报真实性问题 . . . 404
19.2.1 强制结构化，不强制复杂化 . . . 404
19.2.2 员工确认后再入库 . . . 404
19.2.3 任务状态与日报交叉校验 . . . 405
19.2.4 Manager 汇总中标记数据完整度 . . . 405
19.2.5 不把系统做成强监控工具 . . . 405
19.3 屏幕记忆方案是否进入 MVP . . . 405
19.4 Agent 是否允许自动决策 . . . 406
19.5 Manager 权限边界是否清晰 . . . 407
19.6 老板侧是否必须做独立 App . . . 408
19.7 Dream 机制 MVP 阶段做到什么程度 . . . 409
19.8 后端是否新增 Decision 表 . . . 410

19.9 数字签名是否本阶段落地 . . . 412
19.10 IM 平台优先接飞书还是企业微信 . . . 413
19.11 Schema 版本如何管理 . . . 414
19.11.1 Schema ID 表示类型 . . . 414
19.11.2 schema_version 表示版本 . . . 415
19.11.3 后端维护支持版本 . . . 415
19.11.4 数据库保存版本信息 . . . 415
19.11.5 Schema Contract 后续落表 . . . 415
19.12 产品侧需要补充的问题清单 . . . 416
19.12.1 MVP 场景边界 . . . 416
19.12.2 员工侧体验 . . . 416
19.12.3 Manager 侧体验 . . . 416
19.12.4 老板侧体验 . . . 417
19.12.5 Dream 机制边界 . . . 417
19.12.6 数据可信与签名 . . . 417
19.12.7 权限与隐私 . . . 418
19.12.8 数据库与后端范围 . . . 418
19.12.9 IM 平台与交互 . . . 418
19.12.10 验收与交付 . . . 419
19.13 风险优先级与建议处理顺序 . . . 419
19.14 本章小结 . . . 420

20 后续版本规划 421
20.1 MVP v0.1：Mock 闭环 . . . 421
20.2 MVP v0.2：真实数据库联调 . . . 421
20.3 MVP v0.3：真实 IM 接入 . . . 421
20.4 MVP v0.4：老板决策卡片 . . . 421
20.5 MVP v1.0：完整三级数据闭环 . . . 421
20.6 后续行业化 Schema 模板 . . . 421
20.7 后续屏幕记忆自动采集 . . . 421
20.8 后续绩效分析与组织诊断 . . . 421

A Schema JSON 完整示例 422
A.1 Staff Schema 示例 . . . 422
A.2 Manager Schema 示例 . . . 422
A.3 Executive Schema 示例 . . . 422
A.4 Task Schema 示例 . . . 422
A.5 Incident Schema 示例 . . . 422
A.6 Decision Schema 示例 . . . 422

B 字段字典 **422**
B.1 通用字段 422
B.2 用户字段 422
B.3 部门字段 422
B.4 Staff 字段 422
B.5 Manager 字段 422
B.6 Executive 字段 422
B.7 任务字段 422
B.8 决策字段 422
B.9 异常字段 422
B.10 签名字段 422

C 错误码表 **422**
C.1 通用错误码 422
C.2 Schema 校验错误 422
C.3 权限错误 422
C.4 签名错误 423
C.5 Agent 执行错误 423
C.6 Dream 执行错误 423
C.7 数据库写入错误 423
C.8 IM 推送错误 423

D 待产品确认问题清单 **423**
D.1 老板侧交互问题 423
D.2 员工侧填报问题 423
D.3 Manager 权限问题 423
D.4 决策等级问题 423
D.5 数字签名问题 423
D.6 行业场景问题 423
D.7 MVP 演示问题 423

0 文档基础信息

0.1 文档名称

《AutoMage-2-MVP 架构设计文档（杨卓）》

本文档是 AutoMage-2 项目 MVP 阶段的核心架构设计文档，重点围绕三级 Agent 架构中的数据契约、Schema 结构、节点决策机制、任务流转机制与接口协作边界展开。

本文档不作为商业宣传材料使用，也不作为最终产品 PRD 使用，而是作为后端开发、Agent 客制化、产品设计、数据库设计和系统联调的统一技术依据。

0.2 文档版本

字段	内容
文档版本	v0.1
文档状态	MVP 初稿
创建日期	2026-05
当前阶段	架构设计与契约定义阶段
适用项目	AutoMage-2 企业级 AI 中枢 MVP
主要维护人	杨卓
更新机制	每次 Schema、接口、决策机制或数据库映射发生变更时同步更新

版本说明：

v0.1 版本主要用于锁定 MVP 阶段的核心系统边界，明确 Staff、Manager、Executive 三级 Agent 的 Schema 契约定义、数据流转规则和决策机制，为后续后端接口、数据库结构、Agent 模板和前端交互设计提供统一依据。

0.3 Owner 与协作方

模块	Owner	主要职责	协作方
Schema 契约设计	杨卓	定义 Staff、Manager、Executive 三级 Schema 结构、字段规范、校验逻辑和版本规则	后端、Agent 客制化、产品
三级节点决策机制	杨卓	定义 Staff、Manager、Executive 各级节点的决策边界、上推规则、下发规则和异常处理规则	徐少洋、后端、产品
数据库与接口映射	熊锦文	根据 Schema 和业务流程设计数据库表结构、API 接口、鉴权和校验机制	杨卓、胡文涛、徐少洋
Agent 客制化模板	胡文涛	开发 Staff Agent、Manager Agent、Executive Agent 的模板、Prompt、Skill 和配置机制	杨卓、后端
Dream 决策机制	徐少洋	设计日报汇总、决策生成、任务拆解、记忆整理和定时运行机制	杨卓、后端、Agent 客制化
老板侧交互	前端负责人	设计老板侧决策卡片、看板或 IM 交互入口	产品、杨卓、后端
产品边界确认	产品负责人	明确 MVP 阶段业务边界、用户路径、验收场景和交互细节	全体开发成员

本文档中杨卓主要负责的内容包括：Schema 契约定义、各级节点数据结构、节点决策机制、Schema 与数据库/API 的映射关系、MVP 核心数据闭环设计。

0.4 文档适用范围

本文档适用于 AutoMage-2 MVP 阶段的系统设计、开发、联调和验收。

具体适用范围包括：

1. Staff Agent、Manager Agent、Executive Agent 的数据结构定义。
2. Staff 每日日报 Schema、Manager 汇总 Schema、Executive 决策 Schema 的字段设计。
3. 三级 Agent 之间的数据读取、汇总、决策上推和任务下发规则。
4. Agent 生成数据时的 Schema 校验规则。
5. 后端 API 接收、校验、存储 Schema 数据时的结构依据。
6. 数据库表结构与 Schema 字段之间的映射关系。
7. IM 表单、Agent Prompt、后端接口之间的协作边界。
8. MVP 阶段的最小可验证业务闭环。
9. 后续产品、前端、后端、Agent 客制化团队之间的对齐依据。

本文档优先服务于 MVP 阶段，不覆盖完整商业化版本的全部功能。后续版本可在本文档基础上扩展行业化模板、复杂审批流、自动化数据采集、绩效分析、组织诊断和多租户高级权限能力。

0.5 本文档解决的问题

本文档主要解决 AutoMage-2 MVP 阶段开发中的「契约不统一、边界不清晰、数据不可联调」的问题。

具体解决以下问题：

1. 明确三级 Agent 架构中 Staff、Manager、Executive 各自的职责边界。
2. 明确一线员工每日工作数据应该以什么 Schema 结构提交。
3. 明确中层节点如何读取、聚合和分析本部门 Staff Schema。
4. 明确老板/高管节点如何基于 Manager 汇总生成决策项和任务拆解。
5. 明确哪些数据由 Agent 生成，哪些数据必须由人确认。
6. 明确 Schema 字段的类型、必填规则、校验规则和异常处理方式。
7. 明确 Schema 与数据库表、API 接口、Agent Prompt、IM 表单之间的对应关系。
8. 明确决策上推、任务下发、风险识别和异常处理的基础机制。
9. 明确 MVP 阶段的最小可运行闭环，避免各模块各自开发后无法联调。
10. 为后端、Agent 客制化、产品、前端提供统一的开发参照。

本文档的核心目标不是描述宏观愿景，而是把 AutoMage-2 的 MVP 落到可执行、可校验、可联调的系统契约上。

0.6 本文档不解决的问题

本文档不解决以下问题：

1. 不定义完整商业化版本的全部产品功能。
2. 不设计最终 UI 视觉稿、交互细节和页面样式。
3. 不覆盖所有行业场景下的定制化业务 Schema。
4. 不实现完整的 OA、ERP、CRM、项目管理系统替代能力。
5. 不强依赖屏幕记忆、自动录屏、鼠标行为采集等自动化数据采集能力。
6. 不在 MVP 阶段解决所有员工填报真实性问题，只定义数字签名和责任划分的基础机制。
7. 不在 MVP 阶段实现复杂绩效考核、薪酬计算和组织优化模型。
8. 不在本文档中详细定义数据库 DDL 的全部字段和索引。
9. 不在本文档中详细定义所有后端接口的最终 Swagger 规范。
10. 不在本文档中决定最终使用飞书、企业微信、钉钉或独立 App 作为唯一交互入口。
11. 不定义大模型选型、推理框架、部署成本和算力优化方案。
12. 不作为对外融资、销售或商务宣传材料使用。

本文档只负责定义 MVP 阶段系统能够跑通所需的核心架构、数据契约和节点协作机制。

0.7 术语表

0.7.1 Agent

Agent 指具备一定自主执行能力的 AI 节点。它可以根据预设角色、Prompt、工具权限和上下文数据，完成信息收集、数据整理、任务生成、风险识别、决策建议和结果写入等操作。

在 AutoMage-2 中，Agent 不是单纯聊天助手，而是组织结构中的数字节点。每个 Agent 都有明确的角色、权限、输入、输出和可调用工具。

0.7.2 Staff Agent

Staff Agent 指一线员工对应的岗位级 Agent。

它的主要职责是辅助员工整理每日工作内容，生成符合 Schema 契约的结构化日报数据，并将数据提交给后端系统。Staff Agent 也可以读取上级下发给该员工的任务，并通过 IM 或其他交互方式提醒员工执行。

Staff Agent 的核心能力包括：

1. 引导员工填写每日工作数据。
2. 将自然语言内容整理为标准 Staff Schema。
3. 提交工作记录。
4. 查询个人任务。
5. 上报阻塞问题和异常。
6. 在权限范围内提醒员工补充信息。

Staff Agent 不具备跨员工、跨部门读取数据的权限，也不应直接做重大业务决策。

0.7.3 Manager Agent

Manager Agent 指部门负责人或中层管理者对应的管理级 Agent。

它的主要职责是读取本部门 Staff Agent 提交的工作数据，进行部门级汇总、风险识别、异常归类、业绩标记和权限内决策，并生成 Manager Schema 写入系统。

Manager Agent 的核心能力包括：

1. 读取本部门员工日报数据。
2. 聚合部门每日工作进展。
3. 识别部门风险和阻塞事项。
4. 标记表现突出员工和异常员工。
5. 处理权限范围内的常规管理决策。
6. 将超出权限的事项上推给 Executive Agent。
7. 根据老板确认的决策拆解部门任务。

Manager Agent 是连接一线执行层和老板决策层的中间节点，其价值不再是单纯传话，而是负责处理异常、判断风险和完成部门级决策。

0.7.4 Executive Agent

Executive Agent 指老板、高管或公司领导层对应的决策级 Agent。

它的主要职责是读取各部门 Manager Schema，形成组织级业务摘要、风险预警、决策选项和任务拆解方案，并通过 IM 或前端界面推送给老板确认。

Executive Agent 的核心能力包括：

1. 读取组织级汇总数据。
2. 识别跨部门风险和关键经营问题。
3. 生成老板需要确认的决策项。
4. 提供 A/B 方案或多方案建议。
5. 根据老板确认结果生成任务拆解。
6. 将任务反向下发给 Manager Agent 或 Staff Agent。
7. 维护组织级 Dream 决策闭环。

Executive Agent 可以提出建议，但在 MVP 阶段不能绕过老板自动执行重大决策。

0.7.5 Schema

Schema 指系统中用于约束数据结构的标准格式。

在 AutoMage-2 中，Schema 用来规定 Agent 输出的数据必须包含哪些字段、字段类型是什么、哪些字段必填、哪些字段可选、字段之间是否存在逻辑关系，以及后端如何校验和解析这些数据。

Schema 的作用是将员工工作、部门汇总、老板决策、任务下发和异常处理从非结构化文本转化为可存储、可检索、可聚合、可审计的结构化数据。

0.7.6 Schema Contract

Schema Contract 指前端、Agent、后端、数据库之间共同遵守的数据契约。

普通 Schema 只描述数据结构，而 Schema Contract 进一步定义该结构在系统中的生成方式、校验方式、写入位置、读取权限、版本规则和错误处理机制。

在 AutoMage-2 中，Schema Contract 是各模块并行开发的基础。只要 Schema Contract 被锁定，Agent 可以按契约生成数据，后端可以按契约校验数据，数据库可以按契约存储数据，前端或 IM 可以按契约展示和收集数据。

0.7.7 Node

Node 指 AutoMage-2 系统中的组织节点或 Agent 节点。

一个 Node 可以对应一个员工、一个部门负责人、一个高管、一个部门、一个岗位或一个虚拟 Agent。每个 Node 都有自己的身份、层级、权限、上级节点、下级节点和可访问数据范围。

MVP 阶段主要包含三类 Node：

1. Staff Node：一线员工节点。
2. Manager Node：部门管理节点。
3. Executive Node：公司决策节点。

Node 是组织结构数字化的基础单位。

0.7.8 Decision

Decision 指系统中需要被记录、确认或执行的决策事项。

在 AutoMage-2 中，Decision 可以由 Staff Agent、Manager Agent 或 Executive Agent 生成，但不同级别的 Decision 拥有不同的处理规则。

低风险、低权限事项可以由 Agent 自动处理；中等风险事项需要 Manager 确认；重大事项必须由老板或指定负责人确认。所有关键 Decision 都应记录来源、上下文、候选方案、推荐理由、确认人、确认时间和后续任务。

Decision 是 AutoMage-2 从「数据汇总系统」走向「AI 决策中枢」的关键对象。

0.7.9 Dream

Dream 指 AutoMage-2 中用于每日自动汇总、反思、推理和生成次日决策建议的智能分析机制。

Dream 的输入通常包括 Staff Schema、Manager Schema、历史任务、历史异常、历史决策和组织目标。Dream 的输出通常包括老板决策项、风险预警、部门调整建议、任务拆解和次日重点。

在 MVP 阶段，Dream 不要求实现完整复杂的长期记忆系统，但至少需要跑通以下流程：

1. 定时读取各级汇总数据。
2. 生成组织级摘要。
3. 识别关键风险。
4. 生成老板待确认决策项。
5. 在老板确认后生成任务下发。

0.7.10 Task

Task 指系统中可被分配、执行、跟踪和关闭的任务对象。

Task 可以来源于员工日报、部门汇总、异常处理或老板确认后的决策拆解。每个 Task 应至少包含任务标题、任务描述、负责人、优先级、截止时间、状态、来源对象和创建时间。

在 AutoMage-2 中，Task 是决策落地的执行载体。没有 Task，老板确认的决策无法形成可追踪的执行闭环。

0.7.11 Work Record

Work Record 指一线员工在某一时间周期内提交的工作记录。

MVP 阶段的 Work Record 主要表现为 Staff 每日日报。它记录员工当天完成的事项、遇到的问题、尝试的解决方案、是否需要支持、明日计划、资源消耗和相关产出物。

Work Record 是 Manager Agent 生成部门汇总的主要数据来源，也是 Executive Agent 了解一线真实情况的基础数据。

0.7.12 Digital Signature

Digital Signature 指用于确认数据提交责任和数据完整性的签名机制。

在 MVP 阶段，Digital Signature 可以先采用简化实现，例如基于用户身份、提交时间、Schema 内容哈希和确认动作生成签名记录。其主要目的不是立即实现复杂密码学签名，而是明确「谁提交、何时提交、提交了什么、是否确认过」。

Digital Signature 主要用于以下场景：

1. 员工确认自己的日报内容。
2. Manager 确认部门汇总内容。
3. 老板确认关键决策。
4. 系统记录任务下发和异常处理责任。
5. 审计时追溯数据来源和责任主体。

0.7.13 IM Trigger

IM Trigger 指通过即时通讯工具触发 Agent 行为的机制。

MVP 阶段，员工、Manager 和老板可以优先通过 IM 完成系统交互，而不强制开发完整独立 App。IM Trigger 可以用于日报提醒、表单填写、补充信息、任务通知、决策确认、异常预警和催办提醒。

典型 IM Trigger 包括：

1. 每日下班前提醒员工填写日报。
2. 员工提交日报后触发 Schema 校验和写入。
3. Manager 定时收到部门汇总结果。

4. 老板早上收到待确认决策卡片。
5. 老板点击确认后触发任务生成。
6. 一线员工收到次日任务提醒。

IM Trigger 是 MVP 阶段降低产品开发成本、快速跑通三级数据闭环的重要交互方式。

1 MVP 总体目标

1.1 AutoMage-2 的核心目标

AutoMage-2 的核心目标是构建一套面向企业组织运行的三级 AI Agent 数据闭环系统，将传统企业中依赖人工汇报、人工传话、人工汇总和经验判断的管理流程，转化为由结构化数据驱动、由 Agent 辅助执行、由人类保留关键决策权的新型组织运行机制。

在 AutoMage-2 中，系统不再把 AI 仅作为单点工具使用，而是将 AI Agent 嵌入组织结构本身。每一线员工对应 Staff Agent，每个部门或中层管理者对应 Manager Agent，公司老板或高管对应 Executive Agent。不同层级的 Agent 按照明确的 Schema 契约生成、读取、汇总和传递数据，从而形成「员工执行数据 → 部门汇总分析 → 老板决策建议 → 任务反向下发 → 执行结果继续沉淀」的闭环。

MVP 阶段的核心目标不是一次性实现完整的企业 AI 操作系统，而是先验证以下关键假设：

1. 一线员工的日常工作可以被结构化为标准 Staff Schema。
2. 中层节点可以基于本部门 Staff Schema 自动生成 Manager 汇总。
3. 老板节点可以基于 Manager 汇总生成可确认的决策项。
4. 老板确认后的决策可以自动拆解为任务，并下发到对应执行节点。
5. 整个过程可以通过数据库、API、Agent 和 IM 交互形成可运行闭环。

因此，AutoMage-2 MVP 的核心目标可以概括为：**用最小系统跑通三级 Agent 的组织数据闭环，证明企业内部工作流可以被 Schema 化、节点化、决策化和任务化。**

1.2 MVP 阶段的最小可验证闭环

MVP 阶段必须优先跑通一个最小可验证闭环，而不是堆叠过多功能。该闭环应覆盖 Staff、Manager、Executive 三个层级，并能够完整证明系统的数据流、决策流和任务流可以贯通。

最小可验证闭环如下：

1. Staff Agent 在每日固定时间通过 IM 触发员工填写日报。
2. 员工提交今日工作进展、问题、解决尝试、明日计划和是否需要支持等信息。
3. Staff Agent 将员工输入整理为标准 Staff Schema。
4. 后端对 Staff Schema 进行字段、类型、权限和签名校验。
5. 校验通过后，数据写入数据库，形成 Work Record。
6. Manager Agent 定时读取本部门下属员工的 Work Record。
7. Manager Agent 聚合生成部门级 Manager Schema，包括部门进展、风险、阻塞、突出员工和待决策事项。
8. Executive Agent 读取各部门 Manager Schema，生成老板侧业务摘要和决策选项。

9. 老板通过 IM 或前端确认决策方案。
10. 系统根据老板确认结果生成任务。
11. 任务下发到对应 Manager Agent 或 Staff Agent。
12. 员工在次日通过 Staff Agent 获取自己的任务并继续执行。

这个闭环是 MVP 阶段的核心验证对象。只要该闭环能够稳定跑通，就可以证明 AutoMage-2 的三级节点架构、Schema 契约机制、数据汇总机制和决策下发机制具备基础可行性。

1.3 MVP 不追求的能力边界

MVP 阶段不追求完整替代企业现有 OA、ERP、CRM、飞书、企业微信、钉钉或项目管理系统，也不追求一次性完成所有自动化能力。当前阶段的重点是验证架构闭环，而不是追求功能完备。

MVP 阶段暂不追求以下能力：

1. 不追求完整的行业业务系统集成。
2. 不追求自动读取所有电脑屏幕、鼠标行为和软件操作记录。
3. 不追求完全自动判断员工真实工作量。
4. 不追求完整绩效考核、薪酬计算和组织优化模型。
5. 不追求复杂审批流、预算流、报销流和法务流的全量替代。
6. 不追求老板侧独立 App 的完整产品体验。
7. 不追求多模型、多工具、多插件生态的完整开放能力。
8. 不追求复杂的长期记忆、跨周期战略推演和高级 Dream 决策能力。
9. 不追求完全无人干预的 AI 自动经营公司。
10. 不追求在 MVP 阶段解决所有数据真实性和员工抵触问题。

MVP 阶段的实现策略是：**先用通用日报 Schema、IM 交互、数据库存储、Agent 汇总和人工确认决策跑通闭环，再逐步扩展自动采集、行业模板、复杂权限和高级决策能力。**

1.4 本阶段的核心验收目标

MVP 阶段的核心验收目标是验证 AutoMage-2 的三级 Agent 架构是否能够稳定完成一次完整的数据闭环和决策闭环。

本阶段验收目标包括：

1. Staff Agent 能够引导员工提交每日工作数据。
2. Staff Agent 能够生成符合 `schema_v1_staff` 的结构化数据。
3. 后端能够对 Staff Schema 进行校验、鉴权和写入。
4. 数据库能够保存员工每日工作记录，并支持按组织、部门、用户、日期查询。

5. Manager Agent 能够读取本部门员工数据。
6. Manager Agent 能够生成符合 `schema_v1_manager` 的部门汇总数据。
7. Manager Schema 能够体现部门整体健康度、综合进展、风险、阻塞和待决策事项。
8. Executive Agent 能够读取部门汇总数据。
9. Executive Agent 能够生成老板可理解、可选择、可确认的决策项。
10. 老板确认决策后，系统能够生成对应任务。
11. 任务能够被分配到指定员工或部门。
12. Staff Agent 能够读取并展示员工待执行任务。
13. 整个流程中关键数据均有来源记录、状态记录和基础审计记录。
14. 异常输入、Schema 校验失败、权限不足和接口失败能够返回明确错误信息。
15. MVP Demo 能够完整演示「日报提交 → 部门汇总 → 老板决策 → 任务下发 → 员工接收」的流程。

本阶段验收不以界面美观度、复杂功能数量或商业化完整度为主要标准，而以**闭环是否跑通、数据是否可解析、节点是否能协作、决策是否能落地**为主要标准。

1.5 MVP 成功标准

MVP 成功的判断标准不是系统功能有多少，而是是否证明 AutoMage-2 的核心架构假设成立。

MVP 成功标准如下：

1. **数据结构可用**
   Staff、Manager、Executive 三层 Schema 字段清晰、类型明确、可被 Agent 生成、可被后端校验、可被数据库存储。
2. **数据链路可通**
   一线员工提交的数据能够经过 Staff Agent 写入数据库，并被 Manager Agent 正确读取和汇总。
3. **汇总逻辑可跑**
   Manager Agent 能够基于多条 Staff 数据生成部门级摘要、风险识别、阻塞事项和待上推决策。
4. **决策项可生成**
   Executive Agent 能够基于 Manager 汇总生成老板可确认的决策项，而不是只输出普通文本总结。
5. **人工确认可介入**
   关键决策必须经过老板或指定负责人确认，系统不能在 MVP 阶段绕过人类执行重大决策。

6. **任务可以反向下发**
   老板确认后的决策能够被系统转化为具体任务，并分配到对应部门或员工。
7. **执行节点可接收任务**
   Staff Agent 能够读取员工自己的任务，并通过 IM 或其他方式展示给员工。
8. **权限边界清晰**
   Staff Agent 只能访问个人相关数据，Manager Agent 只能访问本部门数据，Executive Agent 才能访问组织级汇总数据。
9. **接口与数据库可联调**
   Agent、后端 API、数据库之间能够基于统一契约完成联调，不依赖口头约定。
10. **Demo 可稳定复现**
    系统能够在演示环境中稳定复现完整流程，且每一步都有可观察的数据输入、输出和状态变化。

    如果以上标准基本满足，即可认为 AutoMage-2 MVP 阶段达成初步成功。

1.6 与 AutoMage-1 的区别

AutoMage-1 的核心思路是「AI 中枢总控台」，即由一个中心化 AI 系统全面感知公司信息、分配任务、监控进度、做出决策，并将少数关键事项推送给管理层确认。该方案的优势是想象空间大、自动化程度高，但同时存在较高的信任风险、技术风险和组织变革阻力。

AutoMage-2 相比 AutoMage-1，核心变化在于从「单一 AI 中枢」转向「多级 Agent 节点架构」。

主要区别如下：

对比项	AutoMage-1	AutoMage-2 MVP
架构形态	单一 AI 中枢	Staff / Manager / Executive 三级 Agent
数据处理方式	中枢集中处理全量数据	分层处理、逐级汇总
决策方式	AI 做大量自动决策	Agent 提供建议，人类确认关键决策
组织接受度	冲击较大，容易引发抵触	保留原有层级，但重塑中层职责
技术压力	单点模型和系统压力较高	多节点分担，数据库作为契约中转层
数据透明度	中枢黑盒风险较高	每级 Schema 可追溯、可审计
中层角色	可能被直接替代	从传话者转为异常处理者和决策节点
MVP 落地难度	较高	更适合分阶段验证

AutoMage-2 并不是放弃 AI 中枢的目标，而是采用更稳妥的落地路径：先让每一层组织节点都具备数字分身，再通过标准 Schema 让数据逐级汇总，最终形成更可信、更可控、更容易被企业接受的 AI 组织操作系统。

1.7 与飞书/传统 OA/项目管理工具的区别

AutoMage-2 MVP 与飞书、传统 OA、项目管理工具存在本质区别。

飞书、企业微信、钉钉等工具主要解决的是沟通、协作、审批和信息承载问题。传统 OA 主要解决流程流转和表单审批问题。项目管理工具主要解决任务分配、进度追踪和团队协作问题。

AutoMage-2 的目标不是再做一个协作工具，而是构建一套由 Agent 驱动的组织运行机制。

主要区别如下：

对比项	飞书/企业微信/钉钉	传统 OA	项目管理工具	**AutoMage-2 MVP**
核心定位	沟通协作平台	流程审批系统	任务管理系统	AI 组织运行系统
数据来源	人主动发消息、开会、填文档	人提交审批表单	人创建和更新任务	Agent 引导生成 Schema
数据结构	大量非结构化消息和文档	固定审批表单	任务字段为主	Staff / Manager / Executive 多级 Schema
AI 角色	辅助总结、搜索、问答	较弱	辅助生成任务或总结	作为组织节点参与数据汇总和决策建议
管理方式	人看消息、人做判断	人按流程审批	人维护任务状态	Agent 分层读取、聚合、上推和下发
老板视角	需要自己查群、看文档、问中层	主要看审批结果	主要看项目进度	直接接收关键摘要、风险和决策选项
中层角色	仍以传话、协调、汇报为主	仍以流程节点为主	仍以任务推进为主	转为异常处理、风险判断和部门决策节点
闭环能力	沟通闭环为主	审批闭环为主	任务闭环为主	数据 → 汇总 → 决策 → 任务 → 再数据的组织闭环

AutoMage-2 可以接入飞书、企业微信或钉钉作为 IM 入口，但它本身不是 IM 工具。IM 在 MVP 阶段只是触发器和交互入口，真正的核心在于后端数据库中的结构化工作记录、分层汇总、决策日志和任务闭环。

因此，AutoMage-2 与传统工具的最大区别是：**传统工具让人更方便地沟通和管理，AutoMage-2 则尝试让组织运行过程本身变成可被 Agent 理解、处理和优化的数据系统。**

2 MVP 系统总架构

AutoMage-2 MVP 采用「三级 Agent + Schema 契约 + 数据库中转 + 人工确认」的系统架构。整体设计目标不是一次性替代企业现有所有管理系统，而是先跑通一条最小但完整的组织数据闭环：一线员工提交工作数据，部门节点汇总判断，老板节点接收决策项，决策确认后反向生成任务并下发到执行层。

MVP 阶段的系统重点不在于界面完整度，而在于验证以下几个关键问题：

1. 一线员工的日常工作能否被稳定转化为结构化 Schema。
2. 中层节点能否基于下属数据形成部门级汇总和风险判断。
3. 老板节点能否收到真正需要决策的事项，而不是大量原始信息。
4. 老板确认后的结果能否被拆解为任务并下发到具体执行人。
5. 全链路数据能否被记录、校验、追踪和复盘。

因此，本章主要定义 AutoMage-2 MVP 的总体架构、数据流转、模块边界、角色交互关系和推荐技术路径。

2.1 三层 Agent 架构总览

AutoMage-2 MVP 的核心架构是三级 Agent：

```
Executive Agent
老板决策与战略拆解节点
        ↑
          读取部门汇总、生成决策项、下发任务

Manager Agent
部门汇总与边界决策节点
        ↑
          读取员工日报、生成部门态势、处理异常

Staff Agent
岗位执行节点
```

这三级 Agent 分别对应企业组织中的三个关键层级：

层级	Agent 类型	对应组织角色	主要职责
L1	Staff Agent	一线员工	记录工作、提交日报、接收任务、反馈执行结果
L2	Manager Agent	部门负责人 / 中层管理者	汇总部门数据、识别风险、处理边界事项、上推决策
L3	Executive Agent	老板 / 高管	获取组织态势、确认关键决策、拆解战略任务

这套架构与传统「一个 AI 中枢统一处理所有信息」的方式不同。AutoMage-2 不要求单个中心模型直接读取和理解所有企业数据，而是让不同层级的 Agent 在各自权限范围内处理对应层级的数据。下级节点负责产出结构化数据，上级节点负责读取、汇总和判断。

这样做有三个好处：

第一，降低系统复杂度。基层数据先在基层节点被整理，部门数据先在部门节点被汇总，老板侧只接收经过筛选的关键事项。

第二，降低组织阻力。中层管理者不是被系统直接替代，而是从「传话者」转为「异常处理者」和「部门决策节点」。

第三，降低信任风险。关键决策不由 AI 自动拍板，而是由 Agent 生成选项和建议，再交给人类确认。

2.1.1 Staff Agent：岗位执行节点

Staff Agent 是一线员工对应的岗位级 Agent。它的定位是员工的「工作记录助手」和「任务接收助手」，负责把员工每天的工作过程转化为标准 Staff Schema。

Staff Agent 的主要职责包括：

1. 通过 IM 或其他入口提醒员工填写每日工作记录。
2. 引导员工补充今日完成事项、遇到的问题、解决尝试、明日计划和产出物。
3. 将员工输入的自然语言内容整理成标准 Staff Schema。
4. 在提交前向员工展示结构化结果，并要求员工确认。
5. 调用后端接口提交日报数据。
6. 查询上级下发给该员工的任务。
7. 当员工遇到阻塞时，生成异常或支持请求。
8. 将任务执行结果继续沉淀为新的工作记录。

Staff Agent 的权限边界必须严格控制：

可做	不可做
提交本人日报	读取他人日报
查询本人任务	查询部门整体数据
上报本人问题	代表其他员工提交数据
接收上级任务	自动做出部门级或老板级决策
辅助整理员工输入	编造员工未确认的工作内容

Staff Agent 的核心价值不是「帮员工写好看日报」，而是把员工真实工作过程转化为系统可读取、可汇总、可追踪的数据。

2.1.2 Manager Agent：部门汇总与边界决策节点

Manager Agent 是部门负责人或中层管理者对应的管理级 Agent。它的定位是部门数据的汇总节点和边界决策节点。

Manager Agent 的主要职责包括：

1. 定时读取本部门员工提交的 Staff Schema。
2. 汇总部门每日工作进展。
3. 识别部门风险、阻塞事项和资源缺口。

4. 标记表现突出员工、关键产出和异常事项。
5. 判断哪些问题可以在部门内解决。
6. 将超出权限的事项整理为待上推决策。
7. 接收老板确认后的任务，并拆解到具体员工或小组。
8. 生成部门级 Manager Schema，供 Executive Agent 读取。

Manager Agent 的权限边界如下：

可做	不可做
读取本部门 Staff Schema	读取其他部门员工明细
生成本部门汇总	直接修改员工原始日报
处理部门权限内异常	绕过老板处理重大事项
下发部门级任务	伪造员工确认
上推超权限决策	越权查看组织级敏感数据

Manager Agent 的核心价值在于把中层从「信息中转站」转为「异常处理者」。它不只是把员工日报复制给老板，而是要把零散的一线数据整理成部门态势：哪些事正常推进，哪些事有风险，哪些人需要支持，哪些事项必须上推。

2.1.3 Executive Agent：老板决策与战略拆解节点

Executive Agent 是老板或公司高管对应的决策级 Agent。它的定位是老板的信息过滤器、决策整理器和任务拆解器。

Executive Agent 的主要职责包括：

1. 定时读取各部门 Manager Schema。
2. 生成组织级业务摘要。
3. 识别跨部门风险、资源冲突和关键阻塞。
4. 将复杂问题整理成老板可判断的决策项。
5. 为每个决策项生成候选方案、推荐方案和影响分析。
6. 通过 IM 或老板侧前端推送决策卡片。
7. 接收老板确认结果。
8. 根据确认结果生成任务，并下发给 Manager Agent 或 Staff Agent。
9. 记录决策来源、确认人、确认时间和后续任务。

Executive Agent 的权限边界如下：

可做	不可做
读取组织级汇总数据	随意读取未授权隐私数据
生成老板决策项	绕过老板执行重大决策
推荐 A/B 方案	自动批准重大资源调整
生成任务拆解草案	删除原始业务记录
跟踪决策执行结果	篡改历史决策依据

Executive Agent 的重点不是生成长篇报告，而是把老板真正需要处理的问题变成选择题。

老板不需要从大量日报里自己找问题，而是直接看到「需要我决定什么、有哪些方案、推荐哪个、确认后会产生哪些任务」。

2.2 数据流转总览

AutoMage-2 MVP 的数据流转分为两条主线：

第一条是自下而上的数据汇总链路，用于让老板看到真实的一线情况。

第二条是自上而下的任务下发链路，用于让老板确认后的决策回到具体执行人。

完整链路如下：

```
一线员工
  ↓
Staff Agent
  ↓
Staff Schema
  ↓
数据库
  ↓
Manager Agent
  ↓
部门汇总 Schema
  ↓
Executive Agent
  ↓
Dream 决策
  ↓
老板确认
  ↓
任务下发
  ↓
Staff Agent 获取每日任务
  ↓
员工执行并形成新的工作记录
```

这个流程中，数据库是核心中转层。Agent 之间不依赖长连接互相通信，也不把 IM 聊天记录作为事实源。所有关键业务数据必须经过后端校验后写入数据库，再由上级节点按权限读取。

2.2.1 一线员工 → Staff Schema

数据流的第一步，是一线员工向 Staff Agent 提供每日工作信息。

MVP 阶段推荐通过 IM 触发日报填写。员工不需要打开复杂后台，只需要在 IM 中回答几个固定问题：

1. 今天完成了哪些事项？
2. 哪些事项还在进行中？

3. 遇到了什么问题？
4. 已经尝试过什么解决方式？
5. 是否需要上级支持？
6. 明天计划做什么？
7. 是否有文档、代码、链接或其他产出物？

Staff Agent 根据员工输入整理出 Staff Schema。整理过程中，Staff Agent 需要做三件事：

第一，结构化。把自然语言拆成标准字段，例如 `work_progress`、`issues_faced`、`next_day_plan`。

第二，补全。发现必填字段缺失时，继续追问员工，而不是自行编造。

第三，确认。提交前必须把整理结果展示给员工，由员工确认后再写入系统。

该阶段的输出是：

```
schema_v1_staff
```

这条 Staff Schema 是后续部门汇总的原始输入。它必须尽量稳定、清晰、可校验。

2.2.2 Staff Schema → 数据库

Staff Schema 生成后，不直接传给 Manager Agent，而是先通过后端 API 写入数据库。

这个环节的职责主要在后端。后端需要完成以下操作：

1. 校验用户身份。
2. 校验 Agent 身份。
3. 校验 Schema ID 和版本。
4. 校验必填字段。
5. 校验字段类型和枚举值。
6. 校验员工是否属于对应组织和部门。
7. 校验签名或确认状态。
8. 判断是否重复提交。
9. 写入工作记录主表和字段明细表。
10. 写入审计日志。

建议映射方式如下：

Staff Schema 内容	数据库对象
日报主信息	`work_records`
具体字段明细	`work_record_items`
文档、代码、链接等产出物	`artifacts` / `artifact_links`
需要支持或高风险问题	`incidents`
可执行后续事项	`tasks`
提交、确认、修改行为	`audit_logs`

数据库写入成功后，Staff Schema 才算正式进入系统。后续 Manager Agent 读取的是数据库中的正式记录，而不是员工与 Staff Agent 的聊天上下文。

2.2.3 Manager Agent → 部门汇总 Schema

Manager Agent 定时读取本部门员工当天或指定周期内的 Staff Schema，并生成部门汇总 Schema。

该环节的输入包括：

1. 本部门员工 Staff Schema。
2. 本部门未完成任务。
3. 本部门异常记录。
4. 历史部门汇总。
5. 老板此前下发的任务或决策。

Manager Agent 不应该简单拼接员工日报，而要完成整理和判断：

处理动作	说明
汇总进展	提炼部门当天主要推进事项
识别风险	从员工问题、阻塞、延期中提取关键风险
识别支持需求	判断哪些员工或事项需要部门介入
识别突出产出	标记有价值的工作成果
判断权限边界	区分部门内可处理事项和需要上推事项
形成次日建议	给出部门明日重点调整方向

该阶段输出的是：

```
schema_v1_manager
```

Manager Schema 应至少包含部门进展、健康度、风险、阻塞事项、突出员工、待处理事项、待上推决策和来源记录。

Manager Schema 的作用是给 Executive Agent 提供更高层级的输入，而不是让老板直接读取所有员工日报。

2.2.4 Executive Agent → Dream 决策

Executive Agent 读取各部门 Manager Schema 后，进入 Dream 决策流程。

这里的 Dream 不一定在 MVP 阶段实现复杂的长期记忆系统，但至少要完成一次组织级的定时分析：读取部门汇总，提取关键问题，生成老板需要确认的决策项。

Dream 决策的输入包括：

1. 各部门 Manager Schema。
2. 部门风险和异常。
3. 未完成关键任务。
4. 历史决策记录。

5. 老板近期目标或策略。
6. 必要的外部输入。

Dream 决策的输出包括：

1. 今日组织级摘要。
2. 当前最重要的风险。
3. 需要老板确认的事项。
4. 每个事项的候选方案。
5. Agent 推荐方案。
6. 推荐理由。
7. 预期影响。
8. 确认后可生成的任务草案。

Executive Agent 在这个阶段要特别注意信息密度。老板侧不应收到大量无筛选内容，而应收到少量高价值决策项。

示例输出形态：

```
今日需要确认 1 项决策：

事项：是否新增独立 decision_logs 表承载老板决策记录。

方案 A：新增独立决策表，审计清晰，后续扩展方便。
方案 B：暂时复用 audit_logs + tasks，开发更快，但后续追踪不够清晰。

推荐：方案 A。
原因：该系统后续核心价值在于决策可追溯，独立决策表更符合长期架构。
```

2.2.5 老板确认 → 任务下发

老板看到 Executive Agent 推送的决策项后，需要做确认动作。MVP 阶段可以先通过 IM 按钮、文字回复或简单前端卡片完成确认。

确认结果通常有几种：

确认动作	系统处理
选择方案 A	记录决策，生成对应任务
选择方案 B	记录决策，生成另一组任务
要求补充信息	决策状态变为待补充
驳回	决策关闭，不生成任务
改写方案	记录人工修改内容，再生成任务

老板确认后，系统需要把决策转化为可执行任务。任务不应只停留在「老板已确认」状态，而应进入任务系统，并明确：

1. 任务标题。
2. 任务说明。

3. 负责人。
4. 所属部门。
5. 优先级。
6. 截止时间。
7. 来源决策。
8. 确认人。
9. 当前状态。

任务可以下发给 Manager Agent，也可以直接下发给 Staff Agent。具体取决于任务粒度：

任务类型	下发对象
部门级调整	Manager Agent
具体执行事项	Staff Agent
跨部门事项	多个 Manager Agent
需要继续拆解的事项	Manager Agent 先接收，再拆给 Staff

这个阶段是决策闭环的关键。没有任务下发，Executive Agent 生成的决策项只是一份报告；有了任务下发，老板决策才进入执行链路。

2.2.6 Staff Agent → 获取每日任务

任务生成后，Staff Agent 需要在合适时间读取员工自己的待执行任务，并推送给员工。

典型触发时间包括：

1. 每天早上上班前。
2. 老板或 Manager 下发任务后。
3. 任务截止前。
4. 任务状态发生变化时。
5. 员工主动查询时。

Staff Agent 获取任务时，只能读取当前员工相关任务，不能读取其他员工或整个部门任务。

任务展示内容应简洁明确：

```
今日待执行任务：

1. 完成 decision_logs 表结构设计初稿
   优先级：高
   截止时间：今日 18:00
   来源：老板确认的“决策记录表设计方案 A”

2. 补充 Manager Schema 与 Summary 表映射说明
   优先级：中
   截止时间：明日 12:00
```

员工执行后，Staff Agent 应在当天日报中引导员工反馈这些任务的完成情况。这样任务执行结果会重新进入 Staff Schema，形成新的数据输入。

2.3 MVP 核心链路图

MVP 阶段只验证一条主链路：日报提交、部门汇总、老板决策、任务下发、员工执行反馈。

```
1. 员工通过 IM 填写日报

          ↓

2. Staff Agent 整理输入
   生成 Staff Schema

          ↓

3. 后端校验 Schema
   鉴权 / 签名 / 写库

          ↓

4. 数据库保存工作记录
   Work Record

          ↓

5. Manager Agent 读取
   本部门员工数据

          ↓

6. 生成部门汇总 Schema
   进展 / 风险 / 待决策

          ↓

7. Executive Agent 读取
   各部门汇总数据

          ↓

8. Dream 生成决策项
   A/B 方案 / 推荐理由

          ↓

9. 老板确认方案

          ↓

10. 系统生成任务
    分配到部门或员工

          ↓
```

```
11. Staff Agent 获取任务
    员工执行并反馈
```

该链路的验收重点如下：

1. Staff Schema 能否稳定生成。
2. 后端能否完成校验和写库。
3. Manager Agent 能否读取本部门数据。
4. Manager Agent 能否生成有效部门汇总。
5. Executive Agent 能否生成老板可确认的决策项。
6. 老板确认动作能否被记录。
7. 任务能否被生成并分配。
8. Staff Agent 能否读取员工任务。
9. 任务结果能否在下一次日报中继续沉淀。

MVP 阶段所有功能开发都应围绕这条链路展开。与该链路无关的复杂功能，可以暂缓。

2.4 系统模块边界

AutoMage-2 MVP 主要由六个模块组成：

1. IM 交互层。
2. Agent 层。
3. API 层。
4. 数据库层。
5. Dream 决策层。
6. 管理展示层。

各模块边界如下：

模块	主要职责	不负责的内容
IM 交互层	触发填报、展示消息、接收确认、推送任务	不保存最终事实数据，不做复杂权限判断
Agent 层	理解输入、生成 Schema、读取数据、汇总分析、生成建议	不直接改数据库，不越权读写，不伪造人工确认
API 层	鉴权、Schema 校验、权限判断、写库、状态流转、错误返回	不负责自然语言理解，不替代 Agent 推理
数据库层	保存组织、用户、日报、任务、异常、汇总、审计等事实数据	不生成业务判断，不做 Agent 推理
Dream 决策层	定时分析部门汇总，生成老板侧摘要和决策项	不绕过老板自动执行重大决策
管理展示层	展示老板决策、部门汇总、任务和风险	MVP 阶段非强依赖，可先由 IM 承载

需要强调的是，Agent 不是事实源，IM 也不是事实源。员工在 IM 中说过的话，只有经过 Agent 整理、员工确认、后端校验并写入数据库后，才成为正式业务数据。

系统边界可以概括为：

```
IM：负责交互
Agent：负责整理和生成
API：负责校验和控制
数据库：负责保存事实
Dream：负责分析和建议
管理展示层：负责呈现结果
```

2.5 角色与系统交互关系

MVP 阶段主要涉及四类用户角色和两类系统角色。

角色	入口	主要动作	主要看到的数据
一线员工	IM / Staff Agent	填写日报、确认提交、接收任务、反馈问题	自己的日报、自己的任务
部门负责人	IM / Manager Agent	查看部门汇总、处理异常、上推事项、拆解任务	本部门汇总、本部门风险、本部门任务
老板 / 高管	IM / Executive Agent / 看板	查看组织摘要、确认决策、查看执行结果	部门汇总、关键风险、决策项、任务结果
管理员	后台 / 配置系统	配置组织、用户、部门、权限、Agent 模板	组织结构、用户权限、系统运行状态
Agent 系统	Agent Runtime	生成 Schema、汇总数据、调用 API、推送消息	权限范围内的结构化数据
后端系统	API / DB	鉴权、校验、写库、审计、状态流转	全部授权业务数据

典型交互关系如下：

一线员工交互

```
Staff Agent：请填写今日工作记录。
员工：今天完成了 Staff Schema，遇到 Decision 表未确认的问题，明天继续补 Manager Schema。
Staff Agent：我整理成以下日报，请确认。
员工：确认提交。
Staff Agent：已提交，进入部门汇总。
```

一线员工侧强调低成本填报，尽量不要求员工学习复杂系统。

部门负责人交互

```
Manager Agent：今日部门 8 人提交日报，2 个事项存在阻塞，1 个事项建议上推。
部门负责人：查看上推事项。
Manager Agent：事项为“决策记录表结构未确认”，影响老板侧决策链路。
部门负责人：上推老板确认。
```

部门负责人侧强调风险和异常，不需要逐条阅读所有员工日报。

老板 / 高管交互

```
Executive Agent: 今日有 1 个关键决策需要确认。
事项: 是否新增独立 decision_logs 表。
方案 A: 新增独立表，审计清晰。
方案 B: 复用 audit_logs，开发更快。
推荐: 方案 A。
老板: 选择方案 A。
Executive Agent: 已确认，将生成任务并下发给后端负责人。
```

老板侧强调信息过滤和决策效率，不展示过多一线细节，除非老板主动下钻查看。

2.6 IM、Agent、后端、数据库之间的职责划分

MVP 阶段必须明确 IM、Agent、后端和数据库的职责，否则后续联调会出现数据重复、状态不一致和权限失控。

总体职责划分如下：

```
IM 负责触发和展示;
Agent 负责理解和生成;
后端负责校验和控制;
数据库负责保存事实。
```

详细划分如下：

组件	负责内容	不负责内容
IM	消息提醒、表单入口、按钮确认、任务推送	不保存最终业务状态，不承担复杂权限，不直接改库
Agent	理解用户输入、整理 Schema、生成汇总、提出建议、调用 API	不越权读取数据，不伪造确认，不绕过 API 直连数据库
后端 API	鉴权、权限校验、Schema 校验、幂等控制、状态流转、写库、错误码	不负责自然语言理解，不生成复杂业务判断
数据库	保存事实数据、支持查询、保留审计、记录任务和决策状态	不负责 Agent 推理，不直接决定业务动作

以员工提交日报为例：

```
员工在 IM 中输入日报
  ↓
IM 将内容传给 Staff Agent
  ↓
Staff Agent 整理成 Staff Schema
  ↓
员工确认
  ↓
Agent 调用后端 API
  ↓
后端校验身份、权限、Schema、签名
  ↓
```

```
数据库写入 Work Record
  ↓
审计日志记录提交行为
```

以老板确认决策为例：

```
Executive Agent 推送决策卡片
  ↓
老板在 IM 中点击确认
  ↓
IM 将确认动作传给后端
  ↓
后端记录决策确认
  ↓
系统生成任务
  ↓
任务写入数据库
  ↓
Staff / Manager Agent 读取任务并推送执行人
```

这个职责划分需要在实现中保持一致。不能出现以下情况：

1. Agent 说已经提交，但后端没有记录。
2. IM 里有确认动作，但数据库没有决策状态。
3. Manager Agent 读取聊天记录做汇总，而不是读取正式数据库记录。
4. Staff Agent 能看到其他员工日报。
5. 老板确认后只生成一段文字，没有生成正式任务。

2.7 MVP 阶段推荐技术路径

MVP 阶段的技术路径应以「尽快跑通闭环」为第一原则。复杂能力可以后置，先把关键链路打通。

推荐路线如下：

```
先 IM 表单
  ↓
后屏幕记忆 / 自动采集

先 Mock
  ↓
后真实 API

先通用 Schema
  ↓
后行业模板 Schema
```

这条路径的目标是降低早期开发不确定性。不要一开始就把屏幕自动采集、复杂行业模板、老板 App、长期记忆和完整权限系统全部放进 MVP 主链路。

2.7.1 先 IM 表单

MVP 阶段建议优先使用 IM 表单或 IM 问答完成数据采集。

原因如下：

1. 开发成本低。
2. 员工学习成本低。
3. 字段可控。
4. 便于快速验证 Schema 是否合理。
5. 不依赖复杂客户端权限。
6. 更适合两周内跑通 MVP 闭环。

员工侧可以先通过固定问题填报：

```
1. 今天完成了什么？
2. 遇到了什么问题？
3. 是否需要支持？
4. 明天准备做什么？
5. 有什么产出物？
```

Staff Agent 再将这些回答整理为 Staff Schema。

IM 表单不是最终产品形态，但它适合作为 MVP 的第一交互入口。

2.7.2 后屏幕记忆/自动采集

屏幕记忆、自动录屏、行为采集、办公软件插件等能力可以作为后续增强方向，但不建议作为 MVP 前置条件。

原因如下：

1. 自动采集涉及客户端权限和隐私问题。
2. 不同企业环境差异较大。
3. 开源项目集成成本不确定。
4. 自动采集结果仍然需要 Schema 化和人工确认。
5. 如果一开始依赖自动采集，MVP 交付风险会明显增加。

推荐阶段划分：

阶段	数据采集方式
MVP v0.1	IM 表单手动填报
MVP v0.2	Agent 追问和结构化整理
MVP v0.3	支持上传文档、代码、链接作为产出物
后续版本	接入屏幕记忆、办公插件、业务系统数据

后续即使接入自动采集，也不应绕过员工确认。自动采集只能作为草稿来源，最终进入系统的数据仍应经过员工确认和后端校验。

2.7.3 先 Mock

在真实后端接口完成前，Agent 侧可以先使用 Mock API 跑通流程。

Mock 阶段的目标不是模拟所有细节，而是先验证 Agent 之间的主流程是否成立。

Mock 阶段至少要跑通：

1. Staff Agent 提交日报。
2. Mock 后端保存 Staff Schema。
3. Manager Agent 读取日报并生成汇总。
4. Executive Agent 读取汇总并生成决策项。
5. 老板确认方案。
6. 系统生成任务。
7. Staff Agent 查询任务。

Mock 阶段需要保持接口形式尽量接近真实接口，避免后续切换真实 API 时大面积重写。

建议 Mock 数据结构提前采用正式字段名，例如：

```
/api/v1/report/staff
/api/v1/report/manager
/api/v1/decision/commit
/api/v1/tasks
```

2.7.4 后真实 API

Mock 流程跑通后，再逐步切换真实后端 API。

真实 API 阶段重点补齐以下能力：

1. 用户鉴权。
2. Agent 鉴权。
3. Schema 校验。
4. 组织和部门权限校验。
5. 数据库写入。
6. 幂等控制。
7. 错误码返回。
8. 审计日志。
9. 任务状态流转。
10. 签名记录。

真实 API 不应只是「能写入数据」，还要保证数据边界清楚。尤其是 Staff、Manager、Executive 三类节点的读取和写入权限必须在后端控制。

Agent 侧不得绕过 API 直接连接数据库。数据库读写必须统一经过后端接口，便于权限控制和审计追踪。

2.7.5 先通用 Schema

MVP 阶段建议先采用通用 Schema，不绑定某个具体行业。

通用 Staff Schema 重点覆盖：

1. 今日完成事项。
2. 遇到的问题。
3. 已尝试方案。
4. 是否需要支持。
5. 明日计划。
6. 资源消耗。
7. 工作产出物。

通用 Manager Schema 重点覆盖：

1. 部门综合进展。
2. 部门健康度。
3. 风险和阻塞。
4. 突出员工和产出。
5. 待处理事项。
6. 待上推决策。

通用 Executive Schema 重点覆盖：

1. 组织级摘要。
2. 关键风险。
3. 待老板确认事项。
4. 候选方案。
5. 推荐方案。
6. 预期影响。
7. 任务拆解草案。

先做通用 Schema 的好处是可以快速验证组织数据闭环，而不用在早期陷入复杂行业细节。

2.7.6 后行业模板 Schema

通用 Schema 跑通后，再逐步扩展行业模板 Schema。

行业模板不是重写整套系统，而是在通用 Schema 基础上增加行业字段和业务规则。

示例：

行业 / 场景	可扩展字段
销售	客户名称、跟进阶段、成交概率、合同金额、下一步动作
研发	需求 ID、代码提交、Bug 编号、测试状态、发布计划
客服	客户问题、处理结果、满意度、升级原因、响应时长
制造	工序、产量、良率、设备状态、异常原因
市场	活动名称、投放渠道、线索数量、ROI、转化率
项目管理	里程碑、依赖项、交付物、风险、延期原因

行业模板 Schema 的推进顺序建议如下：

```
通用 Schema 跑通
  ↓
选择一个试点行业
  ↓
补充行业字段
  ↓
保留通用字段兼容
  ↓
形成行业模板
  ↓
复制到更多行业
```

这样既能保证 MVP 阶段快速落地，也为后续商业化和行业化部署留下扩展空间。

3 Schema 契约设计总原则

3.1 Schema 的定义

在 AutoMage-2 中，Schema 指系统中用于描述和约束数据结构的标准格式。它规定了 Agent 生成的数据必须包含哪些字段、字段类型是什么、哪些字段必填、哪些字段可选、字段之间是否存在逻辑关系，以及后端应如何校验、解析、存储和使用这些数据。

Schema 的本质不是普通表单，也不是简单的 JSON 示例，而是 AutoMage-2 三级 Agent 之间进行数据传递的统一语言。Staff Agent、Manager Agent、Executive Agent 都必须按照各自层级的 Schema 生成结构化数据，避免不同 Agent 输出格式不一致，导致后端无法解析、数据库无法存储、上级节点无法汇总。

MVP 阶段所有核心业务数据都应优先通过 Schema 表达，包括员工日报、部门汇总、老板决策项、任务下发、异常上报和周期性总结。任何进入后端存储、跨节点读取、决策生成或审计追踪的数据，都必须有明确的 Schema 定义。

3.2 为什么 AutoMage-2 必须使用 Schema 契约

AutoMage-2 的核心不是让 Agent 输出一段自然语言总结，而是让 Agent 参与企业组织运行。组织运行要求数据稳定、可校验、可追溯、可聚合、可审计。自然语言文本无法直接满足这些要求，因此必须引入 Schema 契约。

使用 Schema 契约主要解决以下问题：

1. **解决 Agent 输出不稳定的问题**
   大模型天然容易输出格式不一致的内容。如果没有 Schema 约束，同一个日报可能今天输出成段落，明天输出成列表，后端无法稳定解析。

2. **解决前后端协作不一致的问题**
   前端、IM 表单、Agent、后端和数据库如果没有统一字段约定，联调阶段会出现大量字段名、字段类型、必填项不一致的问题。

3. **解决多级节点汇总困难的问题**
   Manager Agent 需要读取多个 Staff Schema，Executive Agent 需要读取多个 Manager Schema。只有下级数据结构稳定，上级节点才能自动聚合。

4. **解决决策依据不可追溯的问题**
   老板看到的决策建议必须能够追溯到具体部门汇总、员工日报和任务记录。Schema 可以记录来源字段和引用关系。

5. **解决权限控制模糊的问题**
   不同层级 Agent 能读取哪些字段、写入哪些字段、修改哪些字段，必须基于 Schema 和节点权限共同约束。

6. **解决数据长期沉淀的问题**

AutoMage-2 的目标是沉淀组织运行数据。只有结构化数据才能被后续用于搜索、统计、分析、绩效评估、组织诊断和 Dream 决策。

7. **解决产品从通用型走向行业化的问题**
   MVP 阶段可以先使用通用 Schema，后续可以扩展制造业、销售、研发、客服等行业模板。Schema 是行业化扩展的基础。

因此，Schema 契约是 AutoMage-2 的系统底座。没有 Schema，系统只能停留在「AI 总结工具」；有了 Schema，系统才能进入「AI 组织操作系统」的阶段。

3.3 Schema 在系统中的位置

Schema 位于 Agent、IM、后端 API、数据库和 Dream 决策机制之间，是各模块协同的中心契约。

在 MVP 阶段，Schema 的位置如下：

1. **IM 侧**
   IM 负责触发员工填报、推送提醒、展示决策卡片和任务通知。IM 收集到的内容需要被 Agent 整理成 Schema。

2. **Agent 侧**
   Agent 负责理解用户输入、补全字段、整理结构、生成符合 Schema 的 JSON 数据。Agent 不能随意输出不受约束的结构。

3. **前端侧**
   如果存在老板侧看板或员工侧表单，前端表单字段应直接来源于 Schema 定义，避免前端另起一套字段体系。

4. **后端 API 侧**
   后端负责接收 Schema 数据，并进行字段校验、类型校验、权限校验、签名校验和幂等处理。

5. **数据库侧**
   数据库负责将 Schema 中的核心字段实体化存储，将弹性字段存入 JSONB 或扩展字段，并建立可查询、可聚合的索引。

6. **Dream 机制侧**
   Dream 机制读取 Staff Schema、Manager Schema、历史任务和历史决策，生成组织级总结、风险判断和老板决策项。

7. **审计侧**
   审计日志需要记录 Schema 的提交人、提交时间、来源节点、校验结果、签名状态和后续影响对象。

可以理解为：**IM 是入口，Agent 是生成器，Schema 是契约，API 是网关，数据库是事实源，Dream 是推理层，任务是执行结果。**

3.4 Schema 与普通日报的区别

AutoMage-2 中的 Staff Schema 不是传统意义上的日报。传统日报通常是一段自然语言描述，主要用于人工阅读；Schema 则是一种可以被机器稳定读取、处理和聚合的数据契约。

二者区别如下：

对比项	普通日报	AutoMage-2 Schema
数据形态	自然语言文本	标准 JSON 结构
主要读者	人	Agent、后端、数据库、人
字段约束	弱约束或无约束	强字段、强类型、强校验
可聚合性	差，需要人工整理	强，可自动汇总
可追溯性	弱，难定位来源	强，可关联用户、部门、任务和时间
可审计性	弱	强，可记录签名、来源和变更
决策支持	依赖人工判断	可作为 Dream 和上级 Agent 的输入
后续利用	难以复用	可用于统计、任务生成、风险识别和组织分析

普通日报解决的是「让上级知道我今天做了什么」。AutoMage-2 Schema 解决的是「让系统能够理解、存储、汇总、判断和驱动下一步行动」。

因此，MVP 阶段即使从员工填报日报开始，也不能把日报当成普通文本处理，而必须把它设计成标准化 Work Record Schema。

3.5 Schema 与数据库表的关系

Schema 是数据进入系统前的契约，数据库表是数据进入系统后的事实存储。二者不是完全等同关系。

一个 Schema 可以映射到一张主表，也可以拆分映射到多张表。例如 Staff Schema 中的基础信息可以写入 `work_records`，具体字段明细可以写入 `work_record_items`，相关文件可以写入 `artifacts`，需要支持的事项可以进一步生成 `incidents` 或 `tasks`。

MVP 阶段建议采用以下映射原则：

Schema 类型	主要数据库对象	说明
Staff Schema	`work_records`、`work_record_items`	存储员工每日工作记录和字段明细
Manager Schema	`summaries`、`summary_source_links`	存储部门级汇总及其来源记录
Executive Schema	决策记录表 / `audit_logs` / `tasks`	存储老板决策项、确认结果和生成任务
Task Schema	`tasks`、`task_assignments`、`task_updates`	存储任务主体、负责人和任务动态
Incident Schema	`incidents`、`incident_updates`	存储异常、风险和处理过程
Artifact Schema	`artifacts`、`artifact_links`	存储工作产出物、附件和外部链接

Schema 与数据库的关系应遵循以下原则：

1. 高频查询字段必须实体化为数据库列。

2. 弹性扩展字段可以进入 `meta`、`payload` 或 `value_json`。
3. 关键业务对象必须有独立 ID，方便追踪和引用。
4. 所有跨节点读取的数据必须带 `org_id`、必要时带 `department_id`。
5. 所有可审计对象必须记录创建人、创建时间、来源对象和状态。
6. 不允许长期依赖 JSONB 中的字段完成高频过滤、排序和权限判断。

Schema 是「输入结构」，数据库是「事实沉淀」。系统设计时不能只看 Schema，也不能只看数据库表，必须保证二者可映射、可追溯、可演进。

3.6 Schema 与 Agent Prompt 的关系

Agent Prompt 负责告诉 Agent 应该如何理解角色、如何处理输入、如何生成输出；Schema 负责约束 Agent 最终输出的数据结构。

Prompt 与 Schema 的关系如下：

1. Prompt 定义 Agent 的行为边界。
2. Schema 定义 Agent 的输出格式。
3. Prompt 不能代替 Schema 校验。
4. Schema 不能代替 Prompt 中的业务判断。
5. Agent 生成数据时必须同时遵守 Prompt 和 Schema。

例如 Staff Agent 的 Prompt 可以要求：

- 你是员工的岗位执行官。
- 你需要引导员工填写今日工作。
- 你不能编造员工未提供的信息。
- 你必须将员工输入整理为 `schema_v1_staff`。
- 如果必填字段缺失，你必须追问员工。
- 如果员工提交内容不完整，你不能直接写入数据库。

但真正判断数据是否合格的，不应该只依赖 Prompt，而应该由 Schema 校验和后端校验共同完成。

MVP 阶段必须避免以下错误：

1. 只在 Prompt 中写「请输出 JSON」，但没有定义字段结构。
2. 只让 Agent 自己判断字段是否完整，后端不校验。
3. Agent 输出格式不稳定，但后端强行兼容。
4. Prompt 与后端字段命名不一致。
5. Prompt 更新后没有同步更新 Schema 版本。

正确做法是：**Prompt 引导 Agent 生成 Schema，Schema 约束 Agent 输出，后端负责最终校验。**

3.7 Schema 与前端表单的关系

MVP 阶段可能优先通过 IM 表单完成员工填报和老板确认，也可能后续增加 Web、小程序或 App 页面。不论交互形态如何，前端表单字段都应尽量与 Schema 保持一致。

前端表单与 Schema 的关系如下：

1. 前端表单字段应来源于 Schema 字段定义。
2. 前端表单的必填项应与 Schema 必填项一致。
3. 前端表单的枚举选项应与 Schema 枚举值一致。
4. 前端表单的字段类型应与 Schema 类型一致。
5. 前端校验只能作为第一层校验，后端仍需进行最终校验。
6. 表单展示文案可以面向用户优化，但字段 key 不应随意变化。

例如，前端可以把 `work_progress` 展示为「今日完成事项」，把 `issues_faced` 展示为「遇到的问题」，把 `need_support` 展示为「是否需要上级支持」。但提交给后端时，字段 key 必须保持 Schema 定义中的标准命名。

这样做的好处是：

1. 前端、Agent、后端使用同一套字段。
2. 后续可以根据 Schema 自动生成表单。
3. 表单改动可以被版本管理。
4. 不同企业或行业可以替换 Schema 模板，而不必重写整套前端逻辑。
5. Agent 和前端可以共享同一套校验规则。

因此，Schema 不只是 Agent 输出格式，也是前端表单设计的源头。

3.8 Schema 与权限控制的关系

Schema 不仅定义数据结构，也会影响权限控制。不同层级的 Agent 可以读取、生成、修改的 Schema 范围必须不同。

MVP 阶段的基本权限原则如下：

节点类型	可写入 Schema	可读取 Schema	禁止行为
Staff Agent	自己的 Staff Schema、个人任务更新、个人异常上报	自己的任务、自己的历史记录	读取他人日报、读取部门汇总、越级查看老板决策
Manager Agent	本部门 Manager Schema、部门任务、部门异常处理	本部门 Staff Schema、本部门任务和异常	读取其他部门明细、直接执行老板级决策
Executive Agent	Executive Schema、组织级决策、任务拆解	组织级 Manager Schema、关键汇总、风险和任务	绕过人工确认自动执行重大决策
系统后台	审计日志、Schema 校验结果、任务状态	按权限读取所有相关数据	无审计修改核心数据

Schema 权限控制需要包含以下维度：

1. **组织维度**：所有业务数据必须绑定 `org_id`。
2. **部门维度**：部门级数据必须绑定 `department_id`。

3. **用户维度**：个人数据必须绑定 `user_id`。
4. **节点维度**：Agent 操作必须绑定 `node_id` 或等价身份。
5. **角色维度**：不同角色拥有不同读取和写入范围。
6. **动作维度**：读取、创建、修改、确认、删除、下发任务应分别授权。
7. **字段维度**：某些字段可以展示，某些字段只能用于系统内部处理。

权限不能只写在 Prompt 中，必须由后端强制执行。Agent 即使生成了越权请求，后端也必须拒绝。

3.9 Schema 与数字签名的关系

数字签名用于解决 Schema 数据的责任归属和完整性问题。

在 AutoMage-2 中，Agent 可以辅助整理数据，但关键数据需要有人或指定节点确认。尤其是 Staff 日报、Manager 汇总、Executive 决策和任务确认，都需要记录提交主体、确认主体和提交内容的哈希信息。

MVP 阶段可以采用简化签名机制，不要求立即实现复杂的密码学签名，但至少应具备以下能力：

1. 记录是谁提交了 Schema。
2. 记录提交时间。
3. 记录提交时的 Schema 内容摘要。
4. 记录提交来源 Agent。
5. 记录是否经过人类确认。
6. 记录确认人和确认时间。
7. 记录后续是否被修改。
8. 记录校验是否通过。

建议签名相关字段包括：

字段	含义
`signature_required`	是否需要签名
`signature_status`	签名状态
`signed_by`	签名人用户 ID
`signed_at`	签名时间
`payload_hash`	Schema 内容哈希
`signature_type`	签名类型
`signature_source`	签名来源，如 IM、Web、Agent
`verify_status`	后端验签状态

数字签名与 Schema 的关系可以理解为：

- Schema 规定「数据长什么样」。
- 数字签名证明「这份数据是谁确认的」。

- 审计日志记录「这份数据后来产生了什么影响」。

没有签名，Schema 只能说明数据格式正确；有了签名，Schema 才能进入组织责任体系。

3.10 Schema 设计原则

MVP 阶段所有 Schema 设计必须遵守以下原则。

3.10.1 字段稳定

Schema 字段一旦进入前后端联调，不应频繁变更。字段名称、字段类型、必填规则和枚举值必须经过确认后再使用。

如果确实需要变更，应通过 Schema 版本管理解决，而不是直接修改旧字段含义。

错误示例：

```
{
  "today_work": "完成客户沟通"
}
```

下一版又改为：

```
{
  "work_progress": "完成客户沟通"
}
```

如果没有版本管理，会导致后端解析混乱。

正确做法是保持字段稳定，并通过版本号区分：

```
{
  "schema_id": "schema_v1_staff",
  "work_progress": "完成客户沟通"
}
```

3.10.2 类型明确

每个字段必须有明确类型，不能让同一字段有时是字符串，有时是数组，有时是对象。

例如：

```
{
  "need_support": true
}
```

不能写成：

```
{
  "need_support": "需要"
}
```

字段类型不稳定会直接影响后端校验、数据库存储和上级 Agent 聚合。

3.10.3 必填项明确

每个 Schema 必须区分必填字段和可选字段。

MVP 阶段 Staff Schema 至少应包含：

1. 提交时间。
2. 用户 ID。
3. 部门 ID。
4. 今日完成事项。
5. 遇到的问题。
6. 是否需要支持。
7. 明日计划。

如果必填字段缺失，Agent 应优先追问用户，不能直接提交后端。后端收到缺失必填字段的数据时，应返回明确错误。

3.10.4 可校验

Schema 必须能被程序校验，不能只依赖人工阅读。

校验至少包括：

1. 字段是否存在。
2. 字段类型是否正确。
3. 枚举值是否合法。
4. 字符串长度是否超限。
5. 数组数量是否超限。
6. 对象结构是否符合要求。
7. 时间格式是否正确。
8. 当前用户是否有权限提交该数据。
9. 签名状态是否符合要求。

Agent 生成 Schema 后，应先进行本地校验；后端收到 Schema 后，应进行最终校验。

3.10.5 可追溯

任何上级汇总、老板决策和任务下发，都必须能够追溯到原始来源。

例如 Manager Schema 应记录其引用了哪些 Staff Work Record。Executive Schema 应记录其引用了哪些 Manager Summary。任务应记录来源于哪个决策或异常。

建议所有汇总类 Schema 都包含来源字段：

```
{
  "source_record_ids": [1001, 1002, 1003],
  "source_summary_ids": [2001, 2002]
}
```

可追溯是后续审计、复盘和责任划分的基础。

3.10.6 可聚合

Schema 必须便于上级节点聚合。

Staff Schema 中的工作进展、问题、风险、资源使用和支持需求，应该能被 Manager Agent 汇总成部门进展、Top 风险、阻塞事项和待决策事项。

Manager Schema 中的部门健康度、风险、待审批事项和效率评分，应该能被 Executive Agent 汇总成组织级业务摘要和老板决策项。

因此，Schema 不能只写大段文本，必须包含可聚合字段。例如：

```
{
  "risk_level": "medium",
  "need_support": true,
  "resource_usage": {
    "hours": 6,
    "tools": ["CRM", "Feishu"]
  }
}
```

这样的字段才能被系统用于统计、筛选和排序。

3.10.7 可版本化

Schema 必须支持版本管理。

每条 Schema 数据建议包含：

```
{
  "schema_id": "schema_v1_staff",
  "schema_version": "1.0.0"
}
```

版本管理用于解决以下问题：

1. 旧数据仍然可以解析。
2. 新字段可以逐步加入。
3. 不同企业可以使用不同模板。
4. 行业化 Schema 可以从通用 Schema 扩展。
5. 后端可以根据版本选择不同校验逻辑。

禁止在没有版本号的情况下直接改变字段含义。

3.10.8 可审计

所有关键 Schema 操作都应该写入审计日志，包括创建、修改、确认、驳回、上推、下发任务和异常关闭。

审计信息至少包括：

1. 操作人。
2. 操作 Agent。
3. 操作时间。
4. 操作对象。
5. 操作类型。
6. 操作前后状态。
7. 请求来源。
8. 校验结果。

可审计不是后期功能，而是组织级 AI 系统的基础要求。没有审计，系统无法被企业信任。

3.10.9 可被 Agent 理解

Schema 不仅要方便后端处理，也要方便 Agent 理解。

字段名称应尽量语义清晰，避免过度缩写。例如应使用 `work_progress`，而不是 `wp`；应使用 `need_support`，而不是 `ns`。

字段说明中应明确告诉 Agent：

1. 字段含义。
2. 填写方式。
3. 是否允许为空。
4. 示例值。
5. 错误示例。
6. 与其他字段的关系。

如果 Schema 对 Agent 不友好，Agent 生成数据时会频繁出错，后端校验失败率会升高，用户体验会变差。

3.10.10 人类最终确认

MVP 阶段必须坚持关键数据和关键决策的人类确认原则。

Agent 可以生成日报草稿、部门汇总、风险判断、老板决策建议和任务拆解，但以下行为必须保留人类确认：

1. 员工日报最终提交。
2. Manager 部门汇总确认。
3. 超出权限的异常处理。
4. 老板级关键决策。
5. 重大任务下发。
6. 影响组织资源配置的操作。

这一原则用于降低黑盒 AI 决策带来的信任风险，也有助于企业客户接受系统。

3.10.11 MVP 优先通用，后续支持行业化

MVP 阶段 Schema 应优先采用通用结构，覆盖大多数电脑办公场景。不要一开始就绑定制造业、销售、客服、研发等特定行业流程。

通用 Staff Schema 应优先回答以下问题：

1. 今天做了什么？
2. 遇到了什么问题？
3. 如何尝试解决？
4. 是否需要支持？
5. 明天计划做什么？
6. 消耗了哪些资源？
7. 产生了哪些产出物？

后续行业化版本可以在通用 Schema 基础上扩展行业字段。例如销售场景可以增加客户名称、跟进阶段、成交概率；研发场景可以增加代码提交、Bug 编号、需求 ID；制造业场景可以增加工序、产量、良率、设备状态。

3.10.12 后端校验优先于 Agent 自觉

Agent 生成的数据不能直接被视为可信数据。即使 Agent 已经按照 Prompt 输出了 JSON，后端也必须进行严格校验。

后端至少需要校验：

1. Schema ID 是否存在。
2. Schema 版本是否支持。
3. 字段是否完整。
4. 类型是否正确。
5. 当前用户是否有权限。
6. 当前 Agent 是否有权限。
7. 签名是否满足要求。
8. 是否存在重复提交。
9. 来源对象是否真实存在。
10. 数据是否符合业务状态流转。

系统不能依赖「Agent 应该不会错」来保证稳定性。Agent 可以辅助生成，但后端必须负责兜底。

3.10.13 数据库是事实源

MVP 阶段可以使用 IM、Agent 记忆、缓存或临时文件作为交互辅助，但最终事实数据必须写入数据库。

以下数据必须以数据库为准：

1. 员工工作记录。
2. 部门汇总。
3. 老板决策。
4. 任务状态。
5. 异常状态。
6. 审计日志。
7. 签名记录。
8. Schema 校验结果。

Agent 本地记忆、聊天上下文和 IM 消息只能作为辅助，不应作为最终事实源。

3.10.14 先闭环，后优化

MVP 阶段 Schema 设计不追求一次性完美，而应优先服务于最小闭环。

当前阶段最重要的是让以下链路跑通：

```
Staff Schema
→ Work Record
→ Manager Schema
```

```
→ Department Summary
→ Executive Decision
→ Human Confirmation
→ Task
→ Staff Execution
```

只要该链路能够跑通，后续可以继续优化字段、补充行业模板、增强自动采集、加入绩效分析和复杂 Dream 机制。

因此，Schema 设计应避免过度复杂化。字段要够用、稳定、可校验、可扩展，但不应在 MVP 阶段引入过多暂时无法实现的复杂结构。

4 Schema 分层体系设计

4.1 Schema 分层总览

AutoMage-2 MVP 阶段的 Schema 采用分层设计，不把所有数据都塞进一个统一大表单，而是按照组织运行过程拆成不同层级、不同用途的数据结构。

这样设计的原因很简单：一线员工每天产生的是「执行数据」，中层需要的是「部门态势」，老板需要的是「决策依据」。如果三类角色都使用同一种数据结构，字段会变得非常臃肿，Agent 也很难稳定生成和解析。

MVP 阶段建议将 Schema 分为七类：

Schema 类型	所属层级	主要用途	主要生成方	主要读取方
Staff Schema	一线员工层	记录员工每日工作、问题、计划和资源消耗	Staff Agent	Manager Agent、员工本人
Manager Schema	部门管理层	汇总部门进展、风险、阻塞和待决策事项	Manager Agent	Executive Agent、部门负责人
Executive Schema	公司决策层	生成老板侧摘要、决策项和方案建议	Executive Agent	老板/高管
Task Schema	执行闭环层	承载决策拆解后的具体任务	Manager Agent / Executive Agent / 系统	Staff Agent、Manager Agent
Incident Schema	异常处理层	记录风险、阻塞、异常和处理过程	Staff Agent / Manager Agent	Manager Agent、Executive Agent
Decision Schema	决策记录层	记录决策项、候选方案、确认状态和执行结果	Manager Agent / Executive Agent	老板、审计、任务系统
Summary Schema	汇总沉淀层	记录周期性汇总、日报、周报、部门总结和组织级总结	Manager Agent / Executive Agent / Dream	上级节点、看板、审计

这七类 Schema 共同构成 MVP 阶段的组织数据骨架：

```
Staff Schema
  ↓
Manager Schema
  ↓
Executive Schema
  ↓
Decision Schema
  ↓
Task Schema
  ↓
Staff 执行
  ↓
新的 Staff Schema
```

Incident Schema 和 Summary Schema 贯穿整个链路，用于处理异常和沉淀周期性结果。

MVP 阶段不要求一次性把每类 Schema 都做得非常复杂，但必须先明确它们之间的边界。只有边界清楚，后端、Agent、数据库和产品交互才能稳定协作。

4.2 Staff Schema：一线执行数据

Staff Schema 是 AutoMage-2 中最底层、也是最重要的数据来源。它记录一线员工在一个工作周期内完成了什么、遇到了什么问题、是否需要支持、明天准备做什么，以及产生了哪些可追溯的工作产物。

MVP 阶段 Staff Schema 的核心目标不是做绩效考核，而是把原本分散在聊天、文档、会议和个人脑子里的工作过程，转化为系统可以读取的结构化记录。

Staff Schema 主要回答以下问题：

1. 这个员工今天做了什么？
2. 做这些事情对应什么业务目标或任务？
3. 哪些事情已经完成，哪些还在推进？
4. 遇到了什么问题？
5. 员工自己尝试过什么解决办法？
6. 是否需要上级、其他部门或老板介入？
7. 明天计划继续做什么？
8. 是否产生了文档、代码、表格、链接、图片等产出物？
9. 是否存在风险、延期或资源不足？

Staff Schema 的数据来源主要包括：

数据来源	MVP 阶段处理方式
员工主动填写	通过 IM 表单或 Agent 问答收集
Agent 辅助整理	将员工自然语言整理为标准字段
工作产出物	由员工上传链接、文档、代码或附件
自动采集数据	MVP 阶段可暂缓，后续接入屏幕记忆或办公软件插件

Staff Schema 生成后，后端应将其写入工作记录相关数据表。对于固定字段，例如用户、部门、日期、标题、状态等，应进入显式列。对于工作进展、问题、计划、资源消耗等弹性内容，可以进入字段明细或 JSON 结构。

Staff Schema 是后续 Manager 汇总、风险识别、任务生成和绩效分析的基础。如果 Staff Schema 不稳定，上层所有汇总都会失真。因此 MVP 阶段要优先保证 Staff Schema 的字段稳定性和可校验性。

4.3 Manager Schema：部门汇总数据

Manager Schema 是部门管理层的数据结构，由 Manager Agent 基于本部门下属员工的 Staff Schema 聚合生成。

它不是简单地把员工日报拼接起来，而是要从多个一线记录中提炼出部门级信息。Manager Schema 应该让部门负责人和老板快速知道：部门今天整体推进得怎么样，哪里有风险，谁需要支持，哪些事情需要上推决策。

Manager Schema 主要回答以下问题：

1. 今天部门整体进展如何？
2. 哪些事项按计划推进？
3. 哪些事项出现阻塞？
4. 哪些风险需要重点关注？
5. 哪些员工表现突出？
6. 哪些员工或事项需要跟进？
7. 有多少事项需要部门负责人处理？
8. 有哪些事项超出部门权限，需要上推老板？
9. 明天部门工作重点应如何调整？

Manager Schema 的输入主要包括：

输入数据	说明
本部门 Staff Schema	部门下属员工当日工作记录
未完成任务	本部门仍在执行中的任务
异常记录	员工上报或系统识别的风险与阻塞
历史部门汇总	用于判断趋势变化
老板历史决策	用于判断当前工作是否符合上级目标

Manager Schema 的输出主要包括：

输出内容	说明
部门综合摘要	对部门当天整体进展的简要总结
部门健康度	用 green / yellow / red 或评分表示
Top 风险	提炼最重要的几个风险
阻塞事项	需要协调或上推的问题
突出员工	有明显贡献或关键产出的员工
待审批事项	需要部门负责人或老板确认的事项
次日调整建议	对部门明天工作的建议
来源记录	本次汇总引用的 Staff 记录 ID

Manager Schema 的关键在于「提炼」和「分流」。能在部门内解决的事项，应由 Manager Agent 生成部门任务或提醒负责人处理；超出权限的事项，应进入 Executive 层的决策输入。

MVP 阶段不要求 Manager Agent 做复杂绩效判断，但至少要完成三件事：

1. 把部门 Staff 数据汇总成一段可读摘要。
2. 提取风险、阻塞和需要支持的事项。
3. 区分「部门内可处理」和「需要老板决策」的事项。

4.4 Executive Schema：老板决策数据

Executive Schema 是老板/高管层使用的数据结构，由 Executive Agent 基于多个部门的 Manager Schema 聚合生成。

它面向的对象不是普通员工，也不是部门负责人，而是老板。因此 Executive Schema 不能只是长篇总结，而应该直接服务于决策。老板看到的内容应该是：现在公司发生了什么、哪些风险最重要、需要我决定什么、有哪些方案、每个方案的影响是什么。

Executive Schema 主要回答以下问题：

1. 公司今天整体运行状态如何？
2. 哪些部门表现正常，哪些部门出现风险？
3. 哪些事项需要老板介入？
4. 每个待决策事项的背景是什么？
5. Agent 推荐哪些可选方案？
6. 各方案的影响、风险和资源消耗是什么？
7. 老板确认后应生成哪些任务？
8. 决策结果应下发给哪些部门或员工？

Executive Schema 的输入包括：

输入数据	说明
Manager Schema	各部门每日汇总
部门风险与异常	需要跨部门或老板处理的问题
历史决策记录	用于判断决策连续性
未完成关键任务	用于判断执行风险
组织目标	如 ROI、交付周期、销售目标等
外部变量	MVP 阶段可选，例如市场信息、客户反馈等

Executive Schema 的输出包括：

输出内容	说明
组织级业务摘要	面向老板的整体情况说明
关键风险	当前最需要老板关注的问题
决策项列表	需要老板确认的事项
候选方案	每个决策项对应的 A/B 或多方案
推荐方案	Agent 给出的建议选项
推荐理由	简短说明为什么推荐该方案
预期影响	对成本、进度、资源、风险的影响
任务拆解草案	老板确认后可生成的任务
来源汇总	引用的 Manager Summary ID

Executive Schema 的核心不是「替老板做决策」，而是「把老板需要判断的事项整理成选择题」。在 MVP 阶段，重大决策必须由老板确认，Executive Agent 只负责生成结构化建议和可执行的任务草案。

4.5 Task Schema：任务下发数据

Task Schema 是 AutoMage-2 中连接「决策」和「执行」的关键结构。

如果系统只做日报和总结，最多只能算一个信息汇总工具。只有当老板或 Manager 的决策可以被拆解成任务，并下发到具体执行人，AutoMage-2 才真正形成组织运行闭环。

Task Schema 主要回答以下问题：

1. 这是什么任务？
2. 为什么生成这个任务？
3. 任务来源于哪个日报、汇总、异常或决策？
4. 谁负责执行？
5. 谁负责确认？
6. 优先级是什么？
7. 截止时间是什么？
8. 当前状态是什么？
9. 如果阻塞，应向谁上报？
10. 任务完成后如何记录结果？

Task Schema 的来源主要有四类：

来源	示例
Staff Schema	员工日报中提出需要支持，生成跟进任务
Manager Schema	部门汇总中发现阻塞，生成协调任务
Executive Schema	老板确认决策后，生成执行任务
Incident Schema	异常处理过程中拆解出的补救任务

Task Schema 的最小字段建议包括：

字段	说明
`task_title`	任务标题
`task_description`	任务说明
`source_type`	来源类型，如 staff_report、manager_summary、decision、incident
`source_id`	来源对象 ID
`assignee_user_id`	执行人
`department_id`	所属部门
`priority`	优先级
`due_at`	截止时间
`status`	任务状态
`created_by_node_id`	创建任务的节点
`confirm_required`	是否需要人工确认
`result_summary`	完成后的结果摘要

Task Schema 的状态流转建议如下：

```
待领取 / 待执行
  ↓
进行中
  ↓
已完成
  ↓
```

已确认关闭

如果任务出现问题，则进入：

进行中
↓
阻塞
↓
异常处理 / 上推决策

MVP 阶段的 Task Schema 不需要覆盖复杂项目管理功能，但必须能完成最基础的任务生成、分配、查询和状态更新。

4.6 Incident Schema：异常上报数据

Incident Schema 用于记录风险、阻塞、异常和需要上级介入的问题。

在组织运行中，真正需要管理者处理的往往不是正常流程，而是异常情况。AutoMage-2 需要让异常从一线被及时记录、分类、上推和追踪，而不是埋在聊天记录或口头汇报里。

Incident Schema 主要回答以下问题：

1. 发生了什么异常？
2. 异常来自哪个员工、任务或部门？
3. 异常严重程度如何？
4. 当前是否影响交付、客户、成本或安全？
5. 员工是否已经尝试解决？
6. 需要谁介入？
7. 是否已经生成处理任务？
8. 当前处理状态是什么？
9. 最终如何关闭？

Incident Schema 的来源包括：

来源	示例
Staff Schema	员工日报中标记 `need_support = true`
Task Schema	任务执行中被标记为阻塞
Manager Schema	部门汇总中发现风险
Executive Schema	老板决策中要求重点跟进
系统规则	长时间未提交日报、任务逾期、重复失败等

Incident Schema 的最小字段建议包括：

字段	说明
incident_title	异常标题
description	异常说明
source_type	来源类型
source_id	来源对象 ID
reporter_user_id	上报人
department_id	所属部门
severity	严重程度
status	当前状态
related_task_id	关联任务
need_escalation	是否需要上推
suggested_action	Agent 建议处理方式
resolved_at	解决时间

Incident Schema 的严重程度建议先采用简单分级：

等级	含义
low	轻微问题，不影响整体进度
medium	需要部门负责人关注
high	影响关键任务，需要上推
critical	影响交付、客户或经营，需要老板决策

MVP 阶段 Incident Schema 不需要覆盖复杂风控体系，但至少要能把「问题」从普通日报中提取出来，进入可追踪对象。

4.7 Decision Schema：决策记录数据

Decision Schema 用于记录系统中所有需要被确认、选择或执行的决策事项。

AutoMage-2 的关键不是让 Agent 只做总结，而是让 Agent 把复杂情况整理成可决策项。Decision Schema 就是这个过程的结构化载体。

Decision Schema 主要回答以下问题：

1. 需要决策的事项是什么？
2. 决策事项来自哪里？
3. 为什么需要决策？
4. 有哪些可选方案？
5. Agent 推荐哪个方案？
6. 推荐理由是什么？
7. 谁有权限确认？
8. 最终选择了哪个方案？
9. 确认后生成了哪些任务？
10. 决策产生了什么结果？

Decision Schema 的来源包括：

来源	示例
Manager Schema	部门发现事项超出权限，需要老板判断
Executive Schema	Dream 生成老板待决策项
Incident Schema	异常严重，需要管理层确认处理方案
Task Schema	任务执行出现冲突，需要重新分配资源

Decision Schema 的最小字段建议包括：

字段	说明
`decision_title`	决策标题
`decision_context`	背景说明
`source_type`	来源类型
`source_id`	来源对象 ID
`decision_level`	决策等级
`options`	候选方案列表
`recommended_option`	Agent 推荐方案
`recommend_reason`	推荐理由
`expected_impact`	预期影响
`confirm_required`	是否需要人工确认
`confirmed_by`	确认人
`confirmed_at`	确认时间
`confirmed_option`	最终确认方案
`generated_task_ids`	决策后生成的任务 ID
`status`	决策状态

Decision Schema 的状态建议包括：

待生成
↓
待确认
↓
已确认
↓
已生成任务
↓
执行中
↓
已完成 / 已关闭

如果老板拒绝或要求修改，则进入：

待确认
↓
已驳回 / 需补充信息

MVP 阶段要特别注意：Decision Schema 不等于 Agent 的自由发挥。每个决策项都必须有来源、有选项、有推荐理由、有确认状态，不能只是一段泛泛的建议。

4.8 Summary Schema：周期性汇总数据

Summary Schema 用于沉淀一段时间内的汇总结果，例如员工日总结、部门日报、组织日报、周报、月报和 Dream 结果摘要。

在 AutoMage-2 中，不是每次查看数据都应该重新扫描所有明细记录。对于部门汇总、老板日报和周期性复盘，应通过 Summary Schema 固化结果，既方便读取，也方便后续追溯。

Summary Schema 主要回答以下问题：

1. 汇总对象是谁？
2. 汇总周期是什么？
3. 汇总范围是个人、部门还是组织？
4. 本周期内完成了什么？
5. 主要风险是什么？
6. 有哪些未完成事项？
7. 有哪些待决策事项？
8. 引用了哪些原始记录？
9. 由哪个 Agent 生成？
10. 是否经过人工确认？

Summary Schema 的类型可以包括：

类型	说明
staff_daily_summary	员工日总结
department_daily_summary	部门日总结
organization_daily_summary	组织日总结
department_weekly_summary	部门周总结
dream_summary	Dream 运行后的组织级摘要
decision_summary	决策执行结果摘要

Summary Schema 的最小字段建议包括：

字段	说明
`summary_type`	汇总类型
`scope_type`	汇总范围，如 user、department、organization
`scope_id`	对应用户、部门或组织 ID
`summary_date`	汇总日期
`title`	汇总标题
`content`	汇总正文
`key_points`	关键事项
`risks`	风险列表
`pending_items`	待处理事项
`source_count`	引用记录数量
`source_record_ids`	来源工作记录
`source_summary_ids`	来源汇总
`generated_by_node_id`	生成节点
`status`	状态

Summary Schema 的价值在于减少重复计算，并为老板看板、日报推送、组织复盘和后

续 Dream 分析提供稳定输入。

MVP 阶段可以先实现日级 Summary，后续再扩展周报、月报和长期趋势分析。

4.9 不同 Schema 之间的转换关系

AutoMage-2 的 Schema 不是孤立存在的。系统真正的价值来自不同 Schema 之间的逐级转换。

MVP 阶段建议按照以下转换关系设计：

```
Staff Schema
  → Work Record
  → Manager Schema
  → Department Summary
  → Executive Schema
  → Decision Schema
  → Task Schema
  → Staff 执行
  → 新的 Staff Schema
```

同时，异常链路如下：

```
Staff Schema / Task Schema
  → Incident Schema
  → Manager 处理
  → Decision Schema
  → Task Schema
  → Incident 关闭
```

汇总链路如下：

```
多个 Staff Schema
  → Manager Schema
  → Summary Schema

多个 Manager Schema
  → Executive Schema
  → Summary Schema / Decision Schema
```

各类转换关系说明如下：

来源 Schema	目标 Schema	触发条件	说明
Staff Schema	Manager Schema	每日定时汇总	Manager 读取本部门员工日报
Staff Schema	Incident Schema	`need_support = true` 或风险较高	员工问题进入异常处理
Staff Schema	Task Schema	员工提出明确后续事项	生成个人或部门任务
Manager Schema	Executive Schema	每日老板侧汇总	Executive 读取各部门数据
Manager Schema	Decision Schema	出现超权限事项	生成待老板确认决策
Executive Schema	Decision Schema	Dream 生成决策项	老板侧选择题
Decision Schema	Task Schema	决策确认后	自动拆解执行任务
Task Schema	Incident Schema	任务阻塞或逾期	进入异常处理
Task Schema	Staff Schema	员工执行后反馈	形成新的工作记录
Summary Schema	Executive Schema	周期复盘或 Dream 运行	支持更高层判断

设计时需要注意三点：

第一，每次转换都要记录来源对象，不能只保留转换后的结果。第二，转换后的 Schema 不能覆盖原始 Schema，原始数据应保持可追溯。第三，Agent 可以负责生成转换结果，但后端需要负责校验、写入和状态更新。

4.10 Schema 生命周期

每条 Schema 数据从生成到归档，应具备完整生命周期。生命周期设计的目的，是让系统知道一条数据当前处于什么状态、是否可信、是否已经被上级读取、是否已经产生后续任务或决策。

MVP 阶段建议将 Schema 生命周期分为十个阶段：

生成
↓
校验
↓
补全
↓
确认
↓
签名
↓
写入
↓
读取
↓
聚合
↓
决策 / 任务生成
↓
归档

各阶段说明如下：

阶段	说明	责任方
生成	Agent 根据用户输入或下级数据生成 Schema 草稿	Agent
校验	检查字段、类型、枚举、权限和格式	Agent 本地校验 + 后端校验
补全	必填信息缺失时，向用户或下级节点追问	Agent / IM
确认	关键数据由对应人员确认	员工 / Manager / 老板
签名	记录确认人、确认时间和内容哈希	后端
写入	将通过校验的数据写入数据库	后端
读取	上级节点按权限读取数据	Manager Agent / Executive Agent
聚合	将多条下级数据汇总为上级 Schema	Manager Agent / Executive Agent
决策 / 任务生成	根据汇总结果生成决策项或任务	Executive Agent / Manager Agent
归档	周期结束后保留为历史记录，用于审计和复盘	后端 / 数据库

不同 Schema 的生命周期略有差异。

Staff Schema 的典型生命周期：

```
员工填写
  → Staff Agent 整理
  → 员工确认
  → 后端校验
  → 写入 Work Record
  → Manager 读取
  → 参与部门汇总
```

Manager Schema 的典型生命周期：

```
Manager Agent 读取 Staff 数据
  → 聚合部门汇总
  → Manager 确认或系统标记
  → 写入 Summary
  → Executive 读取
  → 参与老板决策
```

Executive Schema 的典型生命周期：

```
Executive Agent 读取 Manager 汇总
  → 生成老板摘要和决策项
  → 推送老板确认
  → 写入 Decision
  → 生成 Task
  → 下发执行
```

Task Schema 的典型生命周期：

由决策或异常生成
→ 分配执行人
→ 员工接收
→ 执行中更新
→ 完成
→ 确认关闭

Incident Schema 的典型生命周期：

发现异常
→ 上报
→ 分配处理人
→ 处理跟进
→ 必要时上推决策
→ 关闭

Schema 生命周期中有几个关键原则：

1. 草稿数据不能直接进入上级汇总。
2. 未通过校验的数据不能写入正式业务表。
3. 需要人类确认的数据，未确认前不能触发重大任务。
4. 被上级引用的数据不能随意修改，修改应产生新版本或变更记录。
5. 已经生成任务或决策的数据，必须保留来源关系。
6. 归档数据仍可被审计和复盘，但默认不进入高频查询路径。

通过生命周期管理，AutoMage-2 可以避免数据随意生成、随意修改、随意引用的问题，让每一条进入系统的数据都有状态、有来源、有责任人、有后续影响。

5 Staff 每日日报 Schema 设计

5.1 Schema 基本信息

Staff 每日日报 Schema 是 AutoMage-2 MVP 阶段最基础的数据契约，用于记录一线员工每天的工作进展、问题、解决尝试、资源消耗、明日计划和需要上级支持的事项。

该 Schema 的核心作用不是让员工写一篇传统日报，而是将员工当天的工作过程转化为系统可以解析、存储、汇总和追踪的结构化数据。Manager Agent 后续会基于这些 Staff Schema 生成部门级汇总，Executive Agent 再基于部门汇总生成老板侧决策项。

字段	内容
Schema 名称	Staff 每日日报 Schema
Schema ID	`schema_v1_staff`
当前版本	`1.0.0`
使用节点	Staff Agent
数据提交人	一线员工
数据确认人	一线员工本人
主要触发方式	每日下班前 IM 触发
主要写入对象	`work_records`、`work_record_items`
主要读取对象	Manager Agent
核心目标	沉淀员工每日工作记录，为部门汇总和任务闭环提供原始数据

MVP 阶段建议 Staff Schema 采用「员工填写 + Agent 整理 + 员工确认 + 后端校验」的方式生成。也就是说，Agent 可以帮助员工把自然语言内容整理为标准结构，但不能在员工未确认的情况下直接伪造或提交日报。

Staff Schema 的典型流程如下：

```
IM 定时提醒员工
  ↓
员工填写今日工作内容
  ↓
Staff Agent 整理为 schema_v1_staff
  ↓
员工确认
  ↓
后端校验
  ↓
写入 work_records / work_record_items
  ↓
Manager Agent 定时读取并汇总
```

MVP 阶段不要求 Staff Agent 自动监听员工屏幕、自动读取所有工作行为，也不要求完全自动判断员工工作真实性。当前阶段优先保证数据格式稳定、提交流程可跑通、上级节点可读取和可汇总。

5.2 Staff Schema 字段总览

Staff Schema 字段分为五类：基础身份字段、工作内容字段、风险与支持字段、产出物字段、签名与审计字段。

字段名	类型	必填	说明
schema_id	string	是	Schema 标识，固定为 schema_v1_staff
schema_version	string	是	Schema 版本，MVP 阶段默认 1.0.0
timestamp	string	是	提交时间，ISO8601 格式
org_id	string / number	是	所属组织 ID
department_id	string / number	是	所属部门 ID
user_id	string / number	是	员工用户 ID
node_id	string	是	Staff Agent 节点 ID
record_date	string	是	日报对应日期，格式为 YYYY-MM-DD
work_progress	array	是	今日完成事项列表
issues_faced	array	否	今日遇到的问题
solution_attempt	array	否	已尝试的解决方案
need_support	boolean	是	是否需要上级支持
support_detail	string	条件必填	当 need_support = true 时必填
next_day_plan	array	是	明日计划
resource_usage	object	否	工时、工具、预算、资料等资源消耗
artifacts	array	否	今日产出物，如文档、代码、链接、附件
related_task_ids	array	否	关联任务 ID
risk_level	string	是	风险等级，默认 low
employee_comment	string	否	员工补充说明
signature	object	是	员工确认与签名信息
meta	object	否	扩展字段

其中，work_progress、next_day_plan、need_support、risk_level 是 Staff Schema 的核心字段。Manager Agent 后续会重点读取这些字段，用于判断部门进展、阻塞事项和是否存在需要上推的风险。

5.3 字段定义明细

5.3.1 schema_id：Schema 标识

schema_id 用于标识当前数据遵循哪一套 Schema 结构。

MVP 阶段 Staff 日报固定使用：

```
"schema_id": "schema_v1_staff"
```

该字段必须由 Agent 自动填充，不建议让员工手动填写。后端收到数据后，应首先校验 schema_id 是否存在、是否为系统支持的 Schema。

5.3.2 schema_version：Schema 版本

schema_version 用于标识当前 Staff Schema 的具体版本。

MVP 阶段默认值为：

```
"schema_version": "1.0.0"
```

后续如果 Staff Schema 新增字段、调整字段含义或修改校验规则，应通过版本号区分，而不是直接改变原字段含义。

5.3.3 timestamp：提交时间

`timestamp` 表示员工日报被提交或确认的时间，使用 ISO8601 格式。

示例：

```
"timestamp": "2026-05-03T18:12:30+08:00"
```

该字段用于排序、审计和判断日报是否按时提交。后端可以以服务器接收时间为准进行二次记录，但 Schema 中仍应保留 Agent 侧生成时间。

5.3.4 org_id：组织 ID

`org_id` 表示该日报所属组织，用于多租户隔离和数据权限控制。

示例：

```
"org_id": 1
```

所有 Staff Schema 必须绑定组织 ID。Staff Agent 不能提交没有 `org_id` 的数据，Manager Agent 也不能跨组织读取 Staff Schema。

5.3.5 department_id：部门 ID

`department_id` 表示员工所属部门，用于 Manager Agent 按部门读取和汇总数据。

示例：

```
"department_id": 12
```

该字段是 Manager 汇总的核心过滤条件。后端应校验该员工是否确实属于该部门，避免员工伪造部门 ID 或 Agent 错填部门信息。

5.3.6 user_id：员工 ID

`user_id` 表示提交日报的员工用户 ID。

示例：

```
"user_id": 10086
```

该字段必须与当前登录用户或当前 Agent 绑定用户一致。Staff Agent 只能为自己的绑定员工提交日报，不能代替其他员工提交日报。

5.3.7 node_id：Agent 节点 ID

node_id 表示生成该 Schema 的 Staff Agent 节点。

示例：

```
"node_id": "staff_node_10086"
```

该字段用于区分「谁是提交人」和「哪个 Agent 生成了数据」。在后续审计和排查中，系统需要知道是哪个 Agent 版本、哪个节点、哪次运行生成了该数据。

5.3.8 record_date：日报日期

record_date 表示这条日报对应哪一天的工作，而不是单纯表示提交时间。

示例：

```
"record_date": "2026-05-03"
```

如果员工在第二天补交前一天日报，timestamp 是补交时间，record_date 是实际日报日期。二者不能混淆。

5.3.9 work_progress：今日完成事项

work_progress 是 Staff Schema 中最核心的字段，用于记录员工当天完成或推进的具体工作。

建议使用数组结构，每一项代表一件相对独立的工作事项。

示例：

```
"work_progress": [
  {
    "title": " 完成 Staff 日报 Schema 初稿",
    "description": "整理了一线员工日报所需字段，包括工作进展、问题、
    ↪ 明日计划和签名字段。",
    "status": "completed",
    "related_task_id": 201
  },
  {
    "title": " 与后端对齐 work_records 字段映射",
    "description": "确认 Staff Schema 中的固定字段进入 work_records，
    ↪ 弹性字段进入 work_record_items。",
    "status": "in_progress",
    "related_task_id": 202
  }
]
```

建议每条工作事项包含：

子字段	类型	必填	说明
title	string	是	工作事项标题
description	string	是	工作事项说明
status	string	是	完成状态
related_task_id	number / string	否	关联任务 ID

status 建议使用以下枚举：

值	含义
completed	已完成
in_progress	进行中
blocked	阻塞
cancelled	已取消

work_progress 不能为空。即使员工当天主要在处理零散事项，也应至少提交一条有效记录。

5.3.10 issues_faced：遇到的问题

issues_faced 用于记录员工当天遇到的问题、阻塞或不确定事项。

示例：

```
"issues_faced": [
  {
    "title": "Decision 表结构尚未最终确定",
    "description": "当前 DDL 中没有独立 decision_logs 表，Agent mock 代码中已有
    ↪ decision_logs 概念，需要后端确认最终承载方式。",
    "severity": "medium"
  }
]
```

建议子字段包括：

子字段	类型	必填	说明
title	string	是	问题标题
description	string	是	问题说明
severity	string	否	严重程度

severity 建议使用：

值	含义
low	轻微问题
medium	需要关注
high	影响进度
critical	严重影响交付或决策

如果当天没有明显问题，可以提交空数组 []。不建议填「无」「没有」「正常」这类自然语言字符串。

5.3.11 solution_attempt：已尝试的解决方案

solution_attempt 用于记录员工针对问题已经尝试过的处理方式。

示例：

```
"solution_attempt": [
  {
    "issue_title": "Decision 表结构尚未最终确定",
    "attempt": "先在文档中列出 Decision Schema 的最小字段，
    ↪ 并建议后端新增独立决策表。",
    "result": " 待后端确认"
  }
]
```

该字段的价值在于避免上级重复询问「你有没有试过什么方法」。Manager Agent 在汇总时也可以根据这个字段判断问题是否已经被员工处理过，是否确实需要上级介入。

如果 issues_faced 为空，则 solution_attempt 可以为空数组。

5.3.12 need_support：是否需要上级支持

need_support 表示员工是否需要 Manager、其他部门或老板层面的支持。

示例：

```
"need_support": true
```

该字段必须是 boolean，不能使用 " 是"、" 否"、" 需要"、" 不需要" 等字符串。

need_support = true 时，系统应进一步检查 support_detail 是否填写。如果未填写，Agent 应追问员工补充。

5.3.13 support_detail：需要支持的具体内容

当 need_support = true 时，support_detail 必填。

示例：

```
"support_detail": "需要后端确认是否新增 decision_logs 表，以及 Executive
↪ 决策结果是否直接映射到 tasks。"
```

如果 need_support = false，该字段可以为空字符串或不传。

该字段后续可能触发 Incident 或 Task。如果员工提出的是明确待处理事项，Manager Agent 可以将其转换为部门任务；如果涉及超权限决策，可以继续上推给 Executive Agent。

5.3.14 next_day_plan：明日计划

next_day_plan 用于记录员工下一工作日计划完成的事项。

示例：

```
"next_day_plan": [
  {
    "title": " 完善 Manager Schema 字段定义",
    "priority": "high",
    "expected_output": " 形成可提交给后端和 Agent 客制化使用的字段说明"
  },
  {
    "title": " 补充 Staff Schema 错误示例",
    "priority": "medium",
    "expected_output": " 明确 Agent 生成失败和后端校验失败的处理方式"
  }
]
```

建议子字段包括：

子字段	类型	必填	说明
title	string	是	明日计划事项
priority	string	否	优先级
expected_output	string	否	预期产出

priority 建议使用：low / medium / high / urgent。

next_day_plan 是任务下发和次日跟进的重要参考，不建议为空。

5.3.15 resource_usage：资源消耗

resource_usage 用于记录员工当天消耗的主要资源，包括工时、工具、预算、资料、外部协作等。

示例：

```
"resource_usage": {
  "work_hours": 7.5,
  "tools": ["ChatGPT", "PostgreSQL", "Markdown"],
  "budget_used": 0,
  "external_support": [],
  "notes": " 主要消耗在 Schema 字段梳理和文档撰写上"
}
```

建议子字段包括：work_hours、tools、budget_used、external_support、notes（含义见原文表格）。

MVP 阶段该字段可以作为辅助字段，不强制用于绩效计算。后续如果要做资源效率分析，可以逐步规范化。

5.3.16 artifacts：产出物

artifacts 用于记录员工当天产出的文档、代码、表格、图片、链接、附件等。

示例：

```
"artifacts": [
  {
    "title": "AutoMage-2 Staff Schema 设计稿",
    "artifact_type": "document",
    "url": "https://example.com/docs/staff-schema",
    "description": " 用于后端和 Agent 客制化对齐的一线日报 Schema 文档"
  }
]
```

建议子字段：title、artifact_type、url、file_id、description。artifact_type 建议使用 document / code / spreadsheet / image / link / file / other。

如果员工当天没有明确产出物，可以为空数组。

5.3.17 related_task_ids：关联任务 ID

示例："related_task_ids": [201, 202]。若无关联任务，可为空数组。

5.3.18 risk_level：风险等级

示例："risk_level": "medium"。枚举：low / medium / high / critical（含义见原文）。

MVP 阶段可由 Staff Agent 初步判断，但应允许员工修改；最终提交前需员工确认。

5.3.19 employee_comment：员工补充说明

可选字段，不应用于承载关键结构化信息。

5.3.20 signature：数字签名信息

示例：

```
"signature": {
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-03T18:13:00+08:00",
  "payload_hash": "sha256:example_hash",
```

```
  "signature_source": "im"
}
```

MVP 阶段可先采用简化签名机制，重点明确谁确认、何时确认、确认时内容、后续是否变更。若 signature_status != signed，该日报不应进入 Manager 正式汇总。

5.3.21 meta：扩展字段

示例：

```
"meta": {
  "input_channel": "feishu",
  "agent_template_version": "staff_agent_v0.1",
  "locale": "zh-CN"
}
```

meta 只能存放低频扩展数据；高频查询、权限、决策相关字段应提升为正式字段。

5.4 Staff Schema JSON 示例

下面是一个完整 Staff Schema 示例，可作为 Agent 输出和后端联调参考。

```
{
  "schema_id": "schema_v1_staff",
  "schema_version": "1.0.0",
  "timestamp": "2026-05-03T18:12:30+08:00",
  "org_id": 1,
  "department_id": 12,
  "user_id": 10086,
  "node_id": "staff_node_10086",
  "record_date": "2026-05-03",
  "work_progress": [
    {
      "title": " 完成 Staff 日报 Schema 初稿",
      "description": "整理了一线员工日报所需字段，包括工作进展、问题、明日计划、
      ↪  资源消耗和签名字段。",
      "status": "completed",
      "related_task_id": 201
    },
    {
      "title": " 与后端对齐 work_records 字段映射",
      "description": "确认 Staff Schema 中的固定字段进入 work_records，弹性字段进入
      ↪  work_record_items。",
      "status": "in_progress",
      "related_task_id": 202
```

```
    }
  ],
  "issues_faced": [
    {
      "title": "Decision 表结构尚未最终确定",
      "description": "当前 DDL 中没有独立 decision_logs 表，Agent mock 代码中已有
      ↪  decision_logs 概念，需要后端确认最终承载方式。",
      "severity": "medium"
    }
  ],
  "solution_attempt": [
    {
      "issue_title": "Decision 表结构尚未最终确定",
      "attempt": " 先在文档中列出 Decision Schema 的最小字段，并建议后端新增独立决策表。",
      "result": " 待后端确认"
    }
  ],
  "need_support": true,
  "support_detail": "需要后端确认是否新增 decision_logs 表，以及 Executive
  ↪  决策结果是否直接映射到 tasks。",
  "next_day_plan": [
    {
      "title": " 完善 Manager Schema 字段定义",
      "priority": "high",
      "expected_output": " 形成可提交给后端和 Agent 客制化使用的字段说明"
    },
    {
      "title": " 补充 Staff Schema 错误示例",
      "priority": "medium",
      "expected_output": " 明确 Agent 生成失败和后端校验失败的处理方式"
    }
  ],
  "resource_usage": {
    "work_hours": 7.5,
    "tools": ["ChatGPT", "PostgreSQL", "Markdown"],
    "budget_used": 0,
    "external_support": [],
    "notes": " 主要消耗在 Schema 字段梳理和文档撰写上"
  },
  "artifacts": [
    {
      "title": "AutoMage-2 Staff Schema 设计稿",
      "artifact_type": "document",
      "url": "https://example.com/docs/staff-schema",
      "description": " 用于后端和 Agent 客制化对齐的一线日报 Schema 文档"
    }
  ],
```

```
  "related_task_ids": [201, 202],
  "risk_level": "medium",
  "employee_comment": "今天主要时间用于梳理字段和对齐后端表结构，
  ↪  部分内容还需要等产品确认后再定稿。",
  "signature": {
    "signature_required": true,
    "signature_status": "signed",
    "signed_by": 10086,
    "signed_at": "2026-05-03T18:13:00+08:00",
    "payload_hash": "sha256:example_hash",
    "signature_source": "im"
  },
  "meta": {
    "input_channel": "feishu",
    "agent_template_version": "staff_agent_v0.1",
    "locale": "zh-CN"
  }
}
```


5.5 Staff Schema 校验规则

Staff Schema 需要经过 Agent 本地校验和后端最终校验。

5.5.1 基础字段校验

字段	校验规则
schema_id	必须为 schema_v1_staff
schema_version	必须是当前支持版本
timestamp	必须为合法 ISO8601 时间
org_id	必须存在，且当前用户属于该组织
department_id	必须存在，且当前用户属于该部门
user_id	必须与当前提交人一致
node_id	必须与当前 Staff Agent 绑定关系一致
record_date	必须为合法日期

5.5.2 工作内容校验

work_progress 必须：为数组、至少一条、每条含 title、description、status，status 在允许枚举内；单条 description 不宜过长。

next_day_plan 必须：为数组、至少一条、每条含 title；若填 priority 须在允许枚举内。

5.5.3 支持需求校验

need_support 必须为 boolean。若为 true 则必须填写非空 support_detail，否则后端返回 422。

5.5.4 风险等级校验

risk_level 枚举：low / medium / high / critical。内容与等级明显不符时 Agent 可提醒，MVP 阶段不建议强制自动改写。

5.5.5 签名校验

正式写入前校验 signature 存在、signature_status 为 signed、signed_by 等于当前员工、signed_at 与 payload_hash 存在且与内容一致（MVP 可简化哈希）。

5.5.6 权限校验

后端须校验：用户有权提交该 user_id 日报；Agent 绑定；员工属于 department_id 与 org_id；账号有效；日期策略（补交等）。权限校验不能仅靠 Prompt。

5.5.7 重复提交校验

建议唯一键：org_id + user_id + record_date + schema_id。已存在时可「允许更新 + 审计日志」或「修订版本」；MVP 建议允许更新并记审计。

5.6 Staff Schema 必填字段

字段	必填原因
schema_id / schema_version	后端识别结构与校验规则
timestamp	审计与排序
org_id / department_id / user_id / node_id	组织隔离、汇总、身份与节点追踪
record_date	日报归属日期
work_progress	日报核心内容
need_support	是否触发异常或上推
next_day_plan	次日计划与任务衔接
risk_level	部门风险汇总
signature	确认与责任追溯

条件必填：support_detail（当 need_support=true）；issues_faced（当 risk_level 为 high/critical 时建议）；solution_attempt（当存在 issues_faced 时建议）；artifacts（声明有产出物时建议）。

5.7 Staff Schema 可选字段

可选字段：`issues_faced`、`solution_attempt`、`resource_usage`、`artifacts`、`related_task_ids`、`employee_comment`、`meta`。Agent 仍应尽量引导补充以提升 Manager 汇总质量。

5.8 Staff Schema 错误示例

5.8.1 错误示例一：把 boolean 写成字符串

```
{ "need_support": " 需要" }
```

正确：`"need_support": true`。

5.8.2 错误示例二：工作进展不是数组

错误：`work_progress` 为单字符串。正确：为对象数组且含 `title`、`description`、`status`。

5.8.3 错误示例三：需要支持但未填写支持内容

`need_support: true` 且 `support_detail` 为空—应 422。

5.8.4 错误示例四：缺少签名信息

缺少 `signature` 则无法证明员工确认，不应进入 Manager 正式汇总。

5.8.5 错误示例五：把关键数据塞进补充说明

应将内容拆入 `work_progress`、`issues_faced`、`need_support`、`support_detail`、`next_day_plan` 等结构化字段。

5.9 Staff Agent 生成 Schema 的 Prompt 约束

5.9.1 角色约束

Staff Agent 为「岗位执行辅助节点」，非上级、非绩效评估人；不得编造、不得擅自改已确认信息、不得越权读他人数据。

5.9.2 信息收集约束

至少收集：完成事项、进行中事项、问题、是否尝试解决、是否需支持、明日计划、产出物、是否关联任务。回答过简时应追问具体事项名称、结果与状态。

5.9.3 输出格式约束

最终须输出符合 `schema_v1_staff` 的可解析 JSON，不得以 Markdown 表格或自然语言总结作为正式提交体。

5.9.4 不编造约束

缺失字段须追问，不得自动填充例如默认明日计划。

5.9.5 风险提示约束

出现延期、投诉、阻塞等信息时应提醒确认 `risk_level` 与 `need_support`。

5.9.6 员工确认约束

提交前须展示结构化摘要并要求员工确认后再写签名并调接口。

5.10 Staff Agent 提交失败后的重试逻辑

5.10.1 422 Schema 校验失败

读后端返回的具体错误字段，向员工说明缺失或错误内容，只追问需补充字段，重生成 Schema 后再次提交。

日报暂未提交成功，原因是「需要上级支持」已选择是，但没有填写具体支持内容。
请补充需要谁支持、支持什么事项。

5.10.2 401 鉴权失败

勿重复提交；提示重登或重初始化 Agent；记日志；必要时通知管理员。

5.10.3 403 权限不足

停止提交；提示身份与归属不一致；禁止 Agent 自改 `user_id`/`department_id` 重试。

5.10.4 409 重复提交

提示已有日报；选择查看、修改或取消；修改走更新/修订并记审计。

5.10.5 5xx 服务错误

保留草稿；指数退避重试；多次失败提示稍后；通知运维。

5.10.6 网络超时

使用同一幂等键重试，避免重复写入。建议请求体含：`"idempotency_key": "staff_report_10086_2026-05-03"`。

5.11 Staff Schema 与数据库字段映射

MVP 建议映射：work_records、work_record_items、artifacts/artifact_links、incidents、tasks、audit_logs。

5.11.1 映射到 work_records

work_records 存储 Staff 日报主记录。

Staff Schema 字段	数据库字段	说明
org_id	work_records.org_id	组织 ID
department_id	work_records.department_id	部门 ID
user_id	work_records.user_id	员工 ID
record_date	work_records.record_date	日报日期
schema_id / schema_version	work_records.meta	Schema 标识和版本
work_progress 摘要	work_records.title	可生成简短标题,不建议存整段日报
提交状态	work_records.status	草稿、已提交、已确认等
timestamp	work_records.submitted_at	提交时间
来源方式	work_records.source_type	IM、Web、Agent 等
扩展数据	work_records.meta	低频扩展字段

work_records.title 可由 Agent 自动生成，例如：2026-05-03 杨卓每日工作记录。

5.11.2 映射到 work_record_items

work_record_items 存储 Staff Schema 中具体字段内容。

Staff Schema 字段	field_key	field_label
work_progress	work_progress	今日完成事项
issues_faced	issues_faced	遇到的问题
solution_attempt	solution_attempt	已尝试的解决方案
need_support	need_support	是否需要上级支持
support_detail	support_detail	支持需求说明
next_day_plan	next_day_plan	明日计划
resource_usage	resource_usage	资源消耗
risk_level	risk_level	风险等级
employee_comment	employee_comment	员工补充说明
signature	signature	签名信息

结构化对象或数组入 value_json；短文本可入 value_text。同一 work_record_id 下每个 field_key 唯一。

5.11.3 映射到 artifacts

标题、类型、URL/文件与 user_id、work_record_id 关联；链接优先 artifact_links，附件入 artifacts。

5.11.4 映射到 incidents

当 need_support=true、高风险、work_progress 阻塞、严重问题或明确请求介入时可生成；support_detail/问题描述 → 标题与说明；risk_level → severity；来源链关联 work_record_id。

5.11.5 映射到 tasks

明日计划、支持事项、Manager 下发、老板决策等场景；不宜将全部 next_day_plan 自动转正任务，宜确认后生成。

5.11.6 映射到 audit_logs

建议动作：staff_schema_generated、staff_schema_confirmed、staff_schema_submitted、staff_schema_validation_failed、staff_schema_updated、staff_schema_signed、staff_schema_read_by_manager 等；记录操作人、Agent、对象类型与 ID、时间、结果、payload 摘要。

5.11.7 当前数据库映射建议

```
Staff Schema
  ↓
work_records：日报主记录
  ↓
work_record_items：日报字段明细
  ↓
artifacts：产出物
  ↓
incidents：需要支持或高风险事项
  ↓
tasks：确认后的后续任务
  ↓
audit_logs：提交、确认、修改和读取记录
```

高频查询字段（如 risk_level、need_support、work_progress.status）不宜长期仅存 JSON，应适时提升为显式列或索引。

6 Manager 汇总日报 Schema 设计

6.1 Schema 基本信息

Manager 汇总日报 Schema 是 AutoMage-2 MVP 阶段的部门级数据契约，用于承接本部门下属员工的 Staff Schema，并将分散的一线工作记录整理为部门级汇总、风险判断、阻塞事项、待处理问题和需上推老板的决策项。

Manager Schema 的核心价值不是“把员工日报合并成一篇部门日报”，而是完成一次部门级过滤：哪些信息只是普通进展，哪些信息需要部门负责人处理，哪些问题已经超出部门权限，需要进入 Executive Agent 的老板决策流程。

字段	内容
Schema 名称	Manager 汇总日报 Schema
Schema ID	`schema_v1_manager`
当前版本	`1.0.0`
使用节点	Manager Agent
数据生成方	部门级 Manager Agent
数据确认人	部门负责人 / 中层负责人
主要触发方式	每日定时读取部门 Staff Schema
数据来源	本部门下属员工日报、部门任务、异常记录
主要写入对象	`summaries`、`summary_source_links`
主要读取对象	Executive Agent
核心目标	将一线员工数据整理为部门级态势，为老板决策和任务下发提供输入

Manager Schema 的典型流程如下：

```
Manager Agent 定时触发
  ↓
读取本部门 Staff Schema
  ↓
过滤无效或未签名日报
  ↓
汇总部门进展、风险、阻塞和待决策事项
  ↓
生成 schema_v1_manager
  ↓
部门负责人确认
  ↓
写入 summaries / summary_source_links
  ↓
Executive Agent 定时读取
```

MVP 阶段建议 Manager Schema 每天至少生成一次，时间可以设置在员工日报提交截止之后。例如员工 18:00 前提交日报，Manager Agent 在 20:00 或 21:00 进行部门汇总。

6.1.1 Schema ID：schema_v1_manager

Manager 汇总日报固定使用：

```
"schema_id": "schema_v1_manager"
```

该字段用于告诉后端、Agent 和数据库当前数据遵循 Manager 汇总日报结构。后端收到数据后，应根据 `schema_id` 和 `schema_version` 选择对应校验规则。

6.1.2 使用节点：Manager Agent

Manager Schema 由 Manager Agent 生成。

Manager Agent 一般绑定一个部门负责人或部门节点。它只能读取自己权限范围内的数据，即本组织、本部门及下属成员的 Staff Schema、任务、异常和历史汇总。

Manager Agent 不应读取其他部门员工明细，也不应直接修改员工原始日报。

6.1.3 触发方式：每日定时读取部门 Staff Schema

MVP 阶段推荐使用定时任务触发 Manager Schema 生成。

建议触发时间：

时间	动作
18:00	员工日报提交提醒
20:00	Staff Agent 二次催填
21:00	Manager Agent 读取本部门 Staff Schema
21:10	生成部门汇总草稿
21:30	部门负责人确认或自动标记待确认
22:00	Executive Agent 读取部门汇总

具体时间可根据企业作息调整，但流程顺序应保持稳定：先员工日报，后部门汇总，再老板决策。

6.1.4 数据来源：本部门下属员工日报

Manager Schema 的核心输入是本部门员工提交的 Staff Schema。

除 Staff Schema 外，Manager Agent 还可以读取以下数据作为辅助：

1. 本部门未完成任务。
2. 本部门进行中的异常记录。
3. 前一日部门汇总。
4. 老板或上级下发的未完成任务。
5. 与本部门相关的历史决策结果。
6. 当日缺失日报名单。

MVP 阶段不建议 Manager Agent 读取太多外部数据，避免汇总逻辑变重。优先保证基于 Staff Schema 的部门日报可稳定生成。

6.1.5 写入对象：部门汇总表 / summaries

Manager Schema 生成后，建议写入 summaries 表，来源记录写入 summary_source_links。推荐映射方式：

Manager Schema 内容	数据库对象
部门汇总主记录	summaries
引用的员工日报 ID	summary_source_links
需上推老板的事项	Decision 相关表 / audit_logs / tasks
部门内处理事项	tasks / incidents
操作与确认记录	audit_logs

如果当前阶段暂未建立独立 Decision 表，need_executive_decision 可以先存入 summaries.content 或 summaries.meta，同时写入审计日志。后续建议补充独立决策表，方便老板确认和任务下发。

6.2 Manager Schema 字段总览

Manager Schema 字段分为六类：基础身份字段、汇总统计字段、部门态势字段、风险与阻塞字段、决策与调整字段、来源与签名字段。

字段名	类型	是否必填	说明
schema_id	string	是	Schema 标识，固定为 schema_v1_manager
schema_version	string	是	Schema 版本，MVP 阶段默认 1.0.0
timestamp	string	是	汇总生成时间
org_id	string / number	是	所属组织 ID
dept_id	string / number	是	部门 ID
manager_user_id	string / number	是	中层负责人用户 ID
manager_node_id	string	是	Manager Agent 节点 ID
summary_date	string	是	汇总日期，格式为 YYYY-MM-DD
staff_report_count	number	是	已读取的一线日报数量
missing_report_count	number	是	缺失日报数量
missing_staff_ids	array	否	未提交日报的员工 ID
overall_health	string	是	部门整体健康度
aggregated_summary	string	是	部门综合进展摘要
top_3_risks	array	是	三大风险
workforce_efficiency	object	否	部门效能评分与说明
pending_approvals	number	是	待老板或上级决策事项数量
highlight_staff	array	否	表现突出员工
blocked_items	array	否	阻塞事项
manager_decisions	array	否	Manager 权限内已处理决策
need_executive_decision	array	否	需要上推老板的事项
next_day_adjustment	array	是	次日部门调整建议
source_record_ids	array	是	引用的一线日报 ID
related_task_ids	array	否	关联任务 ID
related_incident_ids	array	否	关联异常 ID
signature	object	是	Manager 确认签名
meta	object	否	扩展字段

其中，aggregated_summary、top_3_risks、blocked_items、need_executive_decision、next_day_adjustment 是 Manager Schema 的核心字段。Executive Agent 后续会重点读取这些字段，用于生成老板侧摘要和决策项。

6.3 字段定义明细

6.3.1 timestamp：汇总生成时间

timestamp 表示 Manager Schema 被生成的时间，使用 ISO8601 格式。

示例：

```
"timestamp": "2026-05-03T21:10:00+08:00"
```

该字段用于判断部门汇总是否按时生成，也用于后续审计和排序。需要注意，timestamp 是生成时间，summary_date 是汇总所覆盖的业务日期，二者不能混用。

6.3.2 manager_user_id：中层负责人 ID

manager_user_id 表示该部门汇总对应的负责人用户 ID。

示例：

```
"manager_user_id": 20001
```

该字段用于确认谁对本次部门汇总负责。即使汇总内容由 Manager Agent 自动生成，也需要绑定一个明确的人类负责人。

后端应校验该用户是否具备对应部门的管理权限。

6.3.3 manager_node_id：中层 Agent 节点 ID

manager_node_id 表示生成该 Manager Schema 的 Agent 节点。

示例：

```
"manager_node_id": "manager_node_dept_12"
```

该字段用于追踪是哪一个 Manager Agent 生成了汇总。后续如果不同部门使用不同 Agent 模板，或者 Agent 模板版本发生变化，该字段可以帮助排查问题。

6.3.4 dept_id：部门 ID

dept_id 表示该汇总所属部门。

示例：

```
"dept_id": 12
```

Manager Agent 只能生成自己权限范围内部门的 Manager Schema。后端应校验 manager_user_id、manager_node_id 与 dept_id 之间的绑定关系。

如果一个 Manager Agent 负责多个部门，建议每个部门单独生成一条 Manager Schema，而不是把多个部门混在一条汇总中。

6.3.5 staff_report_count：读取的一线日报数量

staff_report_count 表示本次汇总实际读取并纳入统计的 Staff Schema 数量。

示例：

```
"staff_report_count": 8
```

该字段用于判断部门数据覆盖率。如果部门共有 10 名员工，但只读取到 8 条有效日报，则说明还有 2 人缺失日报，需要在 missing_report_count 中体现。

只应统计已通过校验、已签名或已确认的正式 Staff Schema。草稿、未确认、校验失败的数据不应计入正式汇总。

6.3.6 missing_report_count：缺失日报数量

missing_report_count 表示本部门当日应提交但未提交日报的人数。

示例：

```
"missing_report_count": 2
```

该字段用于评估部门数据完整性。缺失日报不一定代表员工未工作，但会影响 Manager 汇总准确性。

建议同时提供 missing_staff_ids：

```
"missing_staff_ids": [10012, 10017]
```

如果缺失数量较高，Manager Agent 应在 top_3_risks 或 blocked_items 中提示数据不完整风险。

6.3.7 overall_health：部门整体健康度

overall_health 表示 Manager Agent 对部门当天运行状态的整体判断。

示例：

```
"overall_health": "yellow"
```

建议枚举值如下：

值	含义
green	部门整体正常，无明显风险
yellow	有风险或阻塞，但仍可控
red	存在明显风险，可能影响交付或关键目标
gray	数据不足，无法判断

判断规则不宜过度复杂，MVP 阶段可以先基于以下因素综合判断：

1. 缺失日报比例。
2. 高风险 Staff Schema 数量。
3. 阻塞事项数量。
4. 逾期任务数量。
5. 是否存在需上推老板事项。
6. 是否存在关键产出缺失。

如果 missing_report_count 较高，建议优先标记为 gray 或 yellow，而不是强行判断部门正常。

6.3.8 aggregated_summary：部门综合进展摘要

aggregated_summary 是 Manager Schema 中最重要的自然语言摘要字段，用于概括部门当天整体工作进展。

示例：

```
"aggregated_summary": "今日架构组主要完成了 Staff Schema 字段定义、数据库映射梳理和部分 Agent
↪ Mock Flow 对齐。整体进展正常，但 Decision 相关表结构尚未最终确定，
↪ 可能影响老板决策链路和任务下发联调。"
```

该字段应满足以下要求：

1. 语言简洁，不写流水账。
2. 不逐条复制员工日报。
3. 突出部门主要进展。
4. 明确当前风险和阻塞。
5. 能被 Executive Agent 直接读取并进一步压缩。

建议控制在 100 到 300 字之间。过短会丢失信息，过长会增加老板侧摘要成本。

6.3.9 top_3_risks：三大风险

top_3_risks 用于记录部门当天最重要的风险，最多三项。

示例：

```
"top_3_risks": [
  {
    "title": "Decision 表结构未确认",
    "description": "Agent mock 代码中已有 decision_logs 概念，但当前 DDL 中未单独建表，
    ↪ 可能影响 Executive 决策记录和任务生成。",
    "severity": "high",
    "source_record_ids": [301, 302],
    "suggested_action": " 建议后端与架构负责人确认是否新增独立决策表。"
  }
]
```

建议每个风险项包含：

子字段	类型	是否必填	说明
title	string	是	风险标题
description	string	是	风险说明
severity	string	是	风险等级
source_record_ids	array	否	来源 Staff 日报 ID
suggested_action	string	否	建议处理方式

风险等级建议使用：

值	含义
low	轻微风险
medium	需要部门关注
high	可能影响关键节点
critical	需要老板或跨部门立即处理

top_3_risks 不应列出所有问题，只保留最重要的三项。普通问题可以放入 blocked_items 或后续任务中。

6.3.10 workforce_efficiency：部门效能评分

workforce_efficiency 用于表示部门当天的执行效率判断。

示例：

```
"workforce_efficiency": {
  "score": 78,
  "level": "medium",
  "basis": "8 名员工提交日报，6 名员工完成主要任务，2 个事项仍处于阻塞状态，
  ↪ 整体产出符合预期但存在结构设计等待确认。"
}
```

建议子字段包括：

子字段	类型	是否必填	说明
score	number	否	0-100 分
level	string	是	效能等级
basis	string	是	评分依据

level 建议使用：

值	含义
high	效率较高
medium	正常推进
low	推进偏慢
unknown	数据不足

MVP 阶段不建议把该字段直接用于绩效考核。它只是部门态势判断的辅助信息，必须保留依据说明，不能只给一个分数。

6.3.11 pending_approvals：待老板决策事项数量

pending_approvals 表示本部门当天需要上推老板或更高层确认的事项数量。

示例：

```
"pending_approvals": 1
```

该字段应与 need_executive_decision 数组数量保持一致。如果 pending_approvals > 0，Executive Agent 应重点读取这些事项。

需要注意，pending_approvals 不代表所有待处理事项，只代表超出 Manager 权限、需要更高层确认的事项。

6.3.12 highlight_staff：表现突出员工

highlight_staff 用于记录当天有明显贡献、关键产出或主动解决问题的员工。

示例：

```
"highlight_staff": [
  {
    "user_id": 10086,
    "display_name": " 杨卓",
    "reason": " 完成 Staff Schema 字段定义，并主动梳理与数据库表的映射关系。",
    "source_record_ids": [301]
  }
]
```

建议子字段包括：

子字段	类型	是否必填	说明
user_id	number / string	是	员工 ID
display_name	string	否	员工名称
reason	string	是	突出原因
source_record_ids	array	是	来源日报 ID

该字段不是为了做简单排名，而是为了让老板或部门负责人看到具体价值创造。Manager Agent 不应只根据字数多少判断员工表现，而应结合任务完成、产出物、问题解决和关键贡献。

6.3.13 blocked_items：阻塞事项

blocked_items 用于记录部门内当前存在的阻塞事项。

示例：

```
"blocked_items": [
  {
    "title": " 决策记录表结构未定",
    "description": "当前 Agent 侧和数据库侧对 decision_logs 的承载方式不一致，影响后续
    ↪  Executive 决策确认和任务生成链路。",
    "owner_user_id": 20002,
    "severity": "high",
    "need_support": true,
    "suggested_next_step": " 召开一次后端与架构对齐会，确认是否新增 decision_logs 表。"
  }
]
```

建议子字段包括：

子字段	类型	是否必填	说明
`title`	string	是	阻塞事项标题
`description`	string	是	阻塞说明
`owner_user_id`	number / string	否	当前负责人
`severity`	string	是	严重程度
`need_support`	boolean	是	是否需要支持
`suggested_next_step`	string	否	建议下一步

`blocked_items` 与 `top_3_risks` 的区别是：风险可以是潜在问题，阻塞事项是已经影响推进的问题。

6.3.14 manager_decisions：权限内已处理决策

`manager_decisions` 用于记录 Manager Agent 或部门负责人在权限范围内已经处理的事项。

示例：

```
"manager_decisions": [
  {
    "decision_title": " 将 Manager Schema 字段设计任务优先级调高",
    "decision_reason": " 该任务影响 Executive Agent 的输入结构，应优先完成。",
    "action_taken": " 已将任务优先级调整为 high，并安排明日上午完成初稿。",
    "related_task_ids": [401],
    "confirmed_by": 20001
  }
]
```

该字段有两个作用：

第一，避免所有问题都上推老板。部门内可以解决的问题，应在部门层处理。

第二，体现中层管理者的真实价值。系统可以记录哪些事项是中层实际判断和处理过的，而不是只统计会议或转发次数。

Manager 权限内决策可以包括：

1. 部门内任务优先级调整。
2. 部门人员临时协作安排。
3. 部门内阻塞事项处理。
4. 常规资源协调。
5. 对员工日报缺失的催办。
6. 对低中风险异常的处理。

涉及预算、跨部门冲突、战略方向、重大客户、重大延期等事项，不应放入 `manager_decisions`，而应进入 `need_executive_decision`。

6.3.15 need_executive_decision：需上推老板的事项

need_executive_decision 用于记录超出 Manager 权限，需要老板或高管确认的事项。

示例：

```
"need_executive_decision": [
  {
    "decision_title": " 是否新增独立 decision_logs 表",
    "context": "当前 Agent mock 流程中存在 decision_logs，但现有 DDL 未单独建表。若不新增，
    ↪  短期可复用 audit_logs，但后续老板决策追踪和任务来源会不够清晰。",
    "options": [
      {
        "option_id": "A",
        "title": " 新增独立 decision_logs 表",
        "pros": [" 审计清晰", " 便于追踪决策与任务关系", " 后续扩展空间大"],
        "cons": [" 需要增加后端开发工作量"]
      },
      {
        "option_id": "B",
        "title": " 暂时复用 audit_logs 和 tasks",
        "pros": [" 短期实现更快"],
        "cons": [" 决策对象不够独立", " 后续复盘和统计不方便"]
      }
    ],
    "recommended_option": "A",
    "reason": "AutoMage-2 的核心价值在于决策可追溯，独立决策表更符合长期架构。",
    "source_record_ids": [301, 302],
    "urgency": "high"
  }
]
```

建议子字段包括：

子字段	类型	是否必填	说明
decision_title	string	是	决策标题
context	string	是	背景说明
options	array	是	候选方案
recommended_option	string	否	推荐方案
reason	string	否	推荐理由
source_record_ids	array	否	来源记录
urgency	string	是	紧急程度

该字段是 Executive Agent 生成老板决策项的重要输入。Manager Agent 应尽量把问题整理成老板可判断的形式，而不是只写“需要老板决策”。

6.3.16 next_day_adjustment：次日部门调整建议

next_day_adjustment 用于记录 Manager Agent 对部门次日工作的调整建议。

示例：

```
"next_day_adjustment": [
  {
    "title": " 优先确认 Decision 数据结构",
    "reason": " 该事项影响 Executive 决策链路和任务下发闭环。",
    "target_user_ids": [10086, 20002],
    "priority": "high",
    "expected_output": " 形成 decision_logs 是否建表的最终结论和字段草案。"
  }
]
```

建议子字段包括：

子字段	类型	是否必填	说明
title	string	是	调整事项
reason	string	是	调整原因
target_user_ids	array	否	涉及员工
priority	string	是	优先级
expected_output	string	否	预期产出

该字段可以作为次日任务生成的候选输入，但不建议所有调整建议都自动变成任务。是否生成任务，应结合 Manager 确认或后端规则决定。

6.3.17 source_record_ids：引用的一线日报 ID

source_record_ids 表示本次 Manager 汇总引用了哪些 Staff Work Record。

示例：

```
"source_record_ids": [301, 302, 303, 304]
```

这是 Manager Schema 可追溯性的关键字段。Executive Agent 或审计人员后续可以根据该字段追溯部门汇总的来源。

后端应校验这些记录是否都属于同一组织、同一部门，且当前 Manager Agent 有读取权限。

6.3.18 signature：Manager 确认签名

signature 用于记录部门负责人是否确认了本次部门汇总。

示例：

```
"signature": {
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 20001,
  "signed_at": "2026-05-03T21:25:00+08:00",
  "payload_hash": "sha256:manager_summary_example_hash",
  "signature_source": "im"
}
```

MVP 阶段可以支持两种模式：

模式	说明
人工确认模式	Manager Agent 生成汇总后，由部门负责人确认
自动待确认模式	到达截止时间未确认时，系统标记为 `pending_confirm`，但仍可供 Executive Agent 参考

如果 Manager Schema 中存在高风险事项或需上推老板事项，建议必须由部门负责人确认后再进入 Executive 正式决策流程。

6.4 Manager Schema JSON 示例

下面是一个完整 Manager Schema 示例，可作为 Agent 输出和后端联调参考。

```
{
  "schema_id": "schema_v1_manager",
  "schema_version": "1.0.0",
  "timestamp": "2026-05-03T21:10:00+08:00",
  "org_id": 1,
  "dept_id": 12,
  "manager_user_id": 20001,
  "manager_node_id": "manager_node_dept_12",
  "summary_date": "2026-05-03",
  "staff_report_count": 8,
  "missing_report_count": 2,
  "missing_staff_ids": [10012, 10017],
  "overall_health": "yellow",
  "aggregated_summary": "今日架构组主要完成了 Staff Schema 字段定义、数据库映射梳理和部分
  ↪ Agent Mock Flow 对齐。整体进展正常，但 Decision 相关表结构尚未最终确定，
  ↪ 可能影响老板决策链路和任务下发联调。",
  "top_3_risks": [
    {
      "title": "Decision 表结构未确认",
      "description": "Agent mock 代码中已有 decision_logs 概念，但当前 DDL 中未单独建表，
      ↪ 可能影响 Executive 决策记录和任务生成。",
      "severity": "high",
      "source_record_ids": [301, 302],
      "suggested_action": " 建议后端与架构负责人确认是否新增独立决策表。"
    },
    {
      "title": " 部分员工日报缺失",
      "description": " 本部门应提交 10 份日报，当前仅收到 8 份，部门汇总存在数据不完整风险。",
      "severity": "medium",
      "source_record_ids": [],
      "suggested_action": " 建议 Staff Agent 对缺失员工进行二次催填。"
    }
  ],
  "workforce_efficiency": {
```

```
    "score": 78,
    "level": "medium",
    "basis": "8 名员工提交日报，6 名员工完成主要任务，2 个事项仍处于阻塞状态，
    ↪  整体产出符合预期但存在结构设计等待确认。"
  },
  "pending_approvals": 1,
  "highlight_staff": [
    {
      "user_id": 10086,
      "display_name": " 杨卓",
      "reason": " 完成 Staff Schema 字段定义，并主动梳理与数据库表的映射关系。",
      "source_record_ids": [301]
    }
  ],
  "blocked_items": [
    {
      "title": " 决策记录表结构未定",
      "description": "当前 Agent 侧和数据库侧对 decision_logs 的承载方式不一致，影响后续
      ↪  Executive 决策确认和任务生成链路。",
      "owner_user_id": 20002,
      "severity": "high",
      "need_support": true,
      "suggested_next_step": " 召开一次后端与架构对齐会，确认是否新增 decision_logs 表。"
    }
  ],
  "manager_decisions": [
    {
      "decision_title": " 将 Manager Schema 字段设计任务优先级调高",
      "decision_reason": " 该任务影响 Executive Agent 的输入结构，应优先完成。",
      "action_taken": " 已将任务优先级调整为 high，并安排明日上午完成初稿。",
      "related_task_ids": [401],
      "confirmed_by": 20001
    }
  ],
  "need_executive_decision": [
    {
      "decision_title": " 是否新增独立 decision_logs 表",
      "context": "当前 Agent mock 流程中存在 decision_logs，但现有 DDL 未单独建表。若不新增，
      ↪  短期可复用 audit_logs，但后续老板决策追踪和任务来源会不够清晰。",
      "options": [
        {
          "option_id": "A",
          "title": " 新增独立 decision_logs 表",
          "pros": [" 审计清晰", " 便于追踪决策与任务关系", " 后续扩展空间大"],
          "cons": [" 需要增加后端开发工作量"]
        },
        {
```

```
          "option_id": "B",
          "title": " 暂时复用 audit_logs 和 tasks",
          "pros": [" 短期实现更快"],
          "cons": [" 决策对象不够独立", " 后续复盘和统计不方便"]
        }
      ],
      "recommended_option": "A",
      "reason": "AutoMage-2 的核心价值在于决策可追溯，独立决策表更符合长期架构。",
      "source_record_ids": [301, 302],
      "urgency": "high"
    }
  ],
  "next_day_adjustment": [
    {
      "title": " 优先确认 Decision 数据结构",
      "reason": " 该事项影响 Executive 决策链路和任务下发闭环。",
      "target_user_ids": [10086, 20002],
      "priority": "high",
      "expected_output": " 形成 decision_logs 是否建表的最终结论和字段草案。"
    }
  ],
  "source_record_ids": [301, 302, 303, 304, 305, 306, 307, 308],
  "related_task_ids": [401, 402],
  "related_incident_ids": [501],
  "signature": {
    "signature_required": true,
    "signature_status": "signed",
    "signed_by": 20001,
    "signed_at": "2026-05-03T21:25:00+08:00",
    "payload_hash": "sha256:manager_summary_example_hash",
    "signature_source": "im"
  },
  "meta": {
    "input_channel": "scheduled_job",
    "agent_template_version": "manager_agent_v0.1",
    "staff_total_count": 10,
    "summary_window": "daily"
  }
}
```

6.5 Manager Schema 聚合逻辑

Manager Schema 的生成过程应分为“读取、过滤、归类、提炼、判断、输出”六步。

```
读取 Staff Schema
  ↓
```

```
过滤无效数据
  ↓
按事项归类
  ↓
提炼部门进展
  ↓
判断风险和权限边界
  ↓
输出 Manager Schema
```

6.5.1 读取 Staff Schema

Manager Agent 只读取本部门、指定日期、已提交或已确认的 Staff Schema。

推荐读取条件：

```
org_id = 当前组织
department_id = 当前部门
record_date = 汇总日期
status = 已提交 / 已确认
deleted_at is null
```

不应读取：

1. 草稿日报。
2. 校验失败日报。
3. 未签名且需要签名的日报。
4. 其他部门日报。
5. 已删除或无效记录。

6.5.2 过滤无效数据

读取后，Manager Agent 需要过滤不适合进入正式汇总的数据。

需要过滤的数据包括：

1. 空日报。
2. 未确认日报。
3. 重复提交但已被新版本覆盖的日报。
4. 字段明显不完整的日报。
5. 当前 Manager 无权限读取的数据。

过滤结果应影响 `staff_report_count` 和 `missing_report_count`，并在必要时进入风险提示。

6.5.3 按事项归类

Manager Agent 应将 Staff Schema 中的信息归类为以下几类：

类别	来源字段
已完成事项	`work_progress.status = completed`
进行中事项	`work_progress.status = in_progress`
阻塞事项	`work_progress.status = blocked`、`issues_faced`
支持需求	`need_support = true`、`support_detail`
明日计划	`next_day_plan`
产出物	`artifacts`
风险	`risk_level`、`issues_faced.severity`
关联任务	`related_task_ids`

归类后再进行汇总，避免生成一段没有结构的流水账。

6.5.4 提炼部门进展

部门进展应从多个员工日报中提炼出来，不能逐条堆叠。

提炼时应优先关注：

1. 与部门目标相关的事项。
2. 已完成的关键任务。
3. 影响后续链路的关键产出。
4. 多名员工共同推进的事项。
5. 进度明显变化的事项。

示例：

不推荐：

```
张三完成接口文档，李四做了数据库，王五修改了 Prompt，赵六整理了会议纪要。
```

推荐：

```
今日架构组主要推进了 Schema 契约、数据库映射和 Agent Mock Flow 三条线，其中 Staff Schema
↪ 已完成初稿，数据库映射进入对齐阶段，Agent Mock Flow 已具备基本闭环。
```

6.5.5 判断风险和权限边界

Manager Agent 需要判断每个问题属于哪一类：

类型	处理方式
员工个人可解决	记录即可，不生成上推
部门内可解决	进入 `manager_decisions` 或部门任务
需要跨部门支持	进入 `blocked_items`，必要时上推
超出 Manager 权限	进入 `need_executive_decision`
影响关键交付	进入 `top_3_risks`
数据不足	进入风险提示或缺失日报提示

权限边界的判断应保守。凡是涉及预算、战略方向、跨部门资源、重大延期、重大客户、组织结构调整等事项，应优先进入 `need_executive_decision`。

6.5.6 输出 Manager Schema

输出 Manager Schema 前，Agent 应检查：

1. `aggregated_summary` 是否清楚。
2. `top_3_risks` 是否只保留重要风险。
3. `pending_approvals` 是否与 `need_executive_decision` 数量一致。
4. `source_record_ids` 是否完整。
5. `overall_health` 是否与风险情况一致。
6. 是否存在需要部门负责人确认的事项。
7. 是否存在未处理的高风险阻塞。

输出后，应交由部门负责人确认或进入待确认状态。

6.6 Manager Schema 生成 Prompt 约束

Manager Agent 生成 Manager Schema 时，应遵守以下约束。

6.6.1 不复制员工日报

Manager Agent 不应简单拼接员工日报。它需要完成提炼、归类和判断。

错误方式：

```
张三今天做了 A，李四今天做了 B，王五今天做了 C……
```

正确方式：

```
今日部门主要推进了 A、B、C 三类工作，其中 A 已完成，B 仍在进行，C 受外部依赖影响存在延期风险。
```

6.6.2 不编造未出现的信息

Staff Schema 中没有出现的信息，Manager Agent 不得自行补充。

如果员工没有提交产出物，Manager Agent 不能写“已完成文档产出”。如果没有明确风险来源，不能随意生成严重风险。

所有关键判断都应尽量关联 `source_record_ids`。

6.6.3 明确区分事实和判断

Manager Schema 中应区分事实信息和 Agent 判断。

事实包括：

1. 已提交日报数量。
2. 缺失日报数量。
3. 员工填写的完成事项。
4. 已存在任务状态。
5. 已上报异常。

判断包括：

1. 部门健康度。
2. 风险等级。
3. 效能评分。
4. 是否需要上推。
5. 次日调整建议。

判断字段必须尽量给出依据，不要只给结论。

6.6.4 高风险必须说明来源

如果 `overall_health = red`，或某风险 `severity = high / critical`，必须说明来源。

至少应提供：

1. 关联 Staff 日报 ID。
2. 关联任务 ID 或异常 ID。
3. 风险产生原因。
4. 建议下一步动作。

没有来源的高风险判断不应进入正式汇总。

6.6.5 上推事项必须整理成决策题

如果事项进入 `need_executive_decision`，不能只写“需要老板处理”。

应尽量整理为以下结构：

1. 决策标题。
2. 背景说明。
3. 至少两个可选方案。
4. 推荐方案。
5. 推荐理由。
6. 紧急程度。

7. 来源记录。

老板需要的是判断题或选择题，不是开放式问题。

6.6.6 保留中层处理价值

Manager Agent 应记录部门内已经处理的问题，包括调整优先级、安排协作、催办缺失日报、处理轻中度异常等。

这些内容进入 `manager_decisions`，用于体现中层管理者的实际价值。

系统不应只记录“上推给老板”的事项，否则会让中层看起来没有贡献。

6.7 Manager Schema 风险识别规则

Manager Agent 的风险识别应遵循明确规则，避免凭空判断。

6.7.1 风险来源

风险主要来自以下字段或对象：

来源	说明
`Staff Schema.risk_level`	员工自报风险
`Staff Schema.issues_faced`	员工遇到的问题
`Staff Schema.need_support`	员工请求支持
`work_progress.status = blocked`	工作事项阻塞
缺失日报	数据不完整风险
逾期任务	执行风险
未关闭异常	管理风险
重复出现的问题	结构性风险

6.7.2 风险等级判断

建议 Manager Agent 按以下规则判断风险等级：

风险等级	判断标准
`low`	问题轻微，不影响部门计划
`medium`	需要部门负责人关注，但部门内可处理
`high`	可能影响关键任务、交付或跨部门协作
`critical`	已经影响关键交付、客户、预算或老板目标

示例：

情况	建议等级
单个员工日报缺少产出物	low
两名员工任务轻微延期	medium
关键表结构未确认，影响联调	high
客户交付延期或重大事故	critical

6.7.3 风险进入 top_3_risks 的条件

不是所有风险都进入 top_3_risks。

进入条件建议为：

1. 风险等级为 high 或 critical。
2. 影响多个员工或多个任务。
3. 影响 MVP 主链路。
4. 需要跨部门协调。
5. 需要老板或高管决策。
6. 重复出现且长期未解决。

普通问题可以进入 blocked_items，不必进入 Top 风险。

6.7.4 缺失日报风险

如果缺失日报比例较高，Manager Agent 应提示数据完整性风险。

建议规则：

缺失比例	处理方式
0%	正常
1%-20%	轻微提示
20%-40%	标记为 medium 风险
40% 以上	部门健康度建议为 yellow 或 gray

缺失日报不应直接等同于员工未工作，但会影响部门汇总可信度。

6.7.5 风险与异常的转换

如果风险已经影响任务执行，或需要明确负责人处理，应转为 Incident 或 Task。

转换建议：

情况	转换对象
风险仅需关注	留在 top_3_risks
风险需要处理人跟进	生成 incidents
风险已有明确行动	生成 tasks
风险超出部门权限	生成 need_executive_decision

MVP 阶段可以先不自动创建全部 Incident，但必须在 Schema 中保留清晰的风险和来源信息。

6.8 Manager Schema 与 Staff Schema 的关系

Manager Schema 是由多个 Staff Schema 聚合生成的上层结构。

二者关系如下：

```
多个 Staff Schema
  ↓
读取、过滤、归类、提炼
  ↓
一个 Manager Schema
```

6.8.1 字段对应关系

Staff Schema 字段	Manager Schema 字段	说明
work_progress	aggregated_summary	提炼部门整体进展
issues_faced	top_3_risks / blocked_items	提取风险和阻塞
solution_attempt	blocked_items.suggested_next_step	判断是否仍需支持
need_support	blocked_items / need_executive_decision	判断是否需要上级处理
support_detail	blocked_items.description	形成阻塞说明
next_day_plan	next_day_adjustment	提炼次日部门安排
artifacts	highlight_staff / aggregated_summary	标记关键产出
related_task_ids	related_task_ids	关联任务
risk_level	overall_health / top_3_risks	影响部门风险判断
signature	过滤条件	未确认数据不进入正式汇总

6.8.2 一对多与多对一关系

一个 Staff Schema 通常只属于一个员工、一个日期、一个部门。

一个 Manager Schema 通常引用多个 Staff Schema。

```
Staff Schema 1
Staff Schema 2
Staff Schema 3
        ↓
Manager Schema
```

因此，Manager Schema 必须保留 `source_record_ids`，否则后续无法追溯部门汇总来源。

6.8.3 Staff 数据不完整时的处理

如果部分员工未提交日报，Manager Schema 仍然可以生成，但必须标记数据不完整。

处理方式：

1. `missing_report_count` 记录缺失数量。
2. `missing_staff_ids` 记录缺失员工。
3. `overall_health` 可根据缺失比例调整为 `yellow` 或 `gray`。

4. aggregated_summary 中说明数据不完整。
5. 必要时生成催填任务或提醒。

不建议因为少量日报缺失就阻塞整个部门汇总。但如果缺失比例过高，Executive Agent 应看到该风险。

6.8.4 Staff 原始数据不得被覆盖

Manager Agent 可以引用、汇总和判断 Staff Schema，但不能直接覆盖 Staff 原始日报。

如果发现 Staff Schema 有明显错误，应通过以下方式处理：

1. 标记为数据异常。
2. 通知 Staff Agent 或员工修改。
3. 生成审计日志。
4. 等员工重新确认后再进入正式汇总。

Manager Agent 不应直接替员工改日报。

6.9 Manager Schema 与数据库字段映射

Manager Schema 主要映射到 summaries 和 summary_source_links，部分字段可关联 tasks、incidents、audit_logs。

6.9.1 映射到 summaries

summaries 存储部门汇总主记录。

Manager Schema 字段	数据库字段	说明
org_id	summaries.org_id	组织 ID
dept_id	summaries.department_id	部门 ID
manager_user_id	summaries.user_id 或 meta.manager_user_id	中层负责人
summary_date	summaries.summary_date	汇总日期
schema_id / schema_version	summaries.meta	Schema 标识和版本
aggregated_summary	summaries.content	汇总正文
部门汇总标题	summaries.title	可自动生成
summary_type	summaries.summary_type	部门日报类型
scope_type	summaries.scope_type	部门级汇总
staff_report_count	summaries.source_count	来源数量
overall_health	summaries.meta	部门健康度
top_3_risks	summaries.meta	风险列表
signature	summaries.meta	签名信息

标题可以由系统生成，例如：

2026-05-03 架构组部门日报

如果 summaries 当前字段无法承载所有结构化内容，可将完整 Manager Schema 存入 meta，同时将摘要正文存入 content。后续高频查询字段再逐步实体化。

6.9.2 映射到 summary_source_links

summary_source_links 用于记录 Manager Schema 引用了哪些 Staff Work Record。

Manager Schema 字段	数据库字段	说明
source_record_ids[]	summary_source_links.source_id	来源 Staff 日报 ID
固定值	summary_source_links.source_type	来源类型，如 work_record
当前 summary ID	summary_source_links.summary_id	当前部门汇总 ID
org_id	summary_source_links.org_id	组织 ID

该映射非常重要，决定了部门汇总能否追溯到原始员工日报。

6.9.3 映射到 tasks

如果 Manager Schema 中存在次日调整建议或部门内处理事项，可以生成任务。

Manager Schema 字段	数据库字段	说明
next_day_adjustment[].title	tasks.title	任务标题
next_day_adjustment[].expected_output	tasks.description	任务说明
dept_id	tasks.department_id	所属部门
target_user_ids[]	task_assignments.user_id	执行人
priority	tasks.priority	优先级
当前 summary ID	tasks.meta.source_summary_id	来源部门汇总

MVP 阶段建议任务生成需要 Manager 确认，不要把所有建议自动变成任务。

6.9.4 映射到 incidents

如果 Manager Schema 中存在严重阻塞事项，可以生成或更新异常。

Manager Schema 字段	数据库字段	说明
blocked_items[].title	incidents.title	异常标题
blocked_items[].description	incidents.description	异常说明
blocked_items[].severity	incidents.severity	严重程度
dept_id	incidents.department_id	所属部门
owner_user_id	incidents.reporter_user_id 或 meta.owner_user_id	当前负责人
当前 summary ID	incidents.meta.source_summary_id	来源部门汇总

如果阻塞事项已经由 Staff Schema 生成过 Incident，Manager Schema 应关联已有 related_incident_ids，不应重复创建。

6.9.5 映射到决策记录

need_executive_decision 建议映射到独立 Decision 表。若 MVP 阶段暂未建表，可以先存入 summaries.meta.need_executive_decision，并写入 audit_logs。

建议后续新增决策对象时，映射如下：

Manager Schema 字段	决策对象字段	说明
decision_title	decision.title	决策标题
context	decision.context	决策背景
options	decision.options	候选方案
recommended_option	decision.recommended_option	推荐方案
reason	decision.reason	推荐理由
urgency	decision.priority	紧急程度
当前 summary ID	decision.source_summary_id	来源部门汇总

该部分是当前数据库设计需要重点补齐的地方。

6.9.6 映射到 audit_logs

Manager Schema 的关键操作应写入审计日志。

建议记录以下动作：

动作	说明
manager_schema_generated	Manager Agent 生成部门汇总
manager_schema_confirmed	部门负责人确认汇总
manager_schema_submitted	汇总写入后端
manager_schema_validation_failed	汇总校验失败
manager_schema_read_by_executive	Executive Agent 读取部门汇总
manager_decision_created	Manager 生成权限内决策
executive_decision_requested	生成上推老板事项

审计日志应记录操作人、Agent 节点、对象 ID、操作时间和必要的 payload 摘要。

6.10 Manager Agent 读取权限规则

Manager Agent 的读取权限必须由后端控制，不能只依赖 Prompt。

6.10.1 基础读取范围

Manager Agent 只能读取：

1. 当前组织内数据。
2. 当前绑定部门的数据。
3. 本部门员工已提交、已确认的 Staff Schema。
4. 本部门任务。
5. 本部门异常。

6. 本部门历史汇总。
7. 与本部门相关的上级决策和任务。

Manager Agent 不应读取：

1. 其他组织数据。
2. 其他部门员工明细。
3. 老板未授权的组织级敏感数据。
4. 员工未确认草稿。
5. 已删除或无效记录。
6. 无关部门的任务和异常。

6.10.2 读取 Staff Schema 的条件

读取 Staff Schema 时，后端应校验：

条件	说明
`org_id` 一致	防止跨组织读取
`department_id` 在权限范围内	防止跨部门读取
Staff 记录状态有效	只读取正式记录
Staff 签名状态有效	未确认数据不进入正式汇总
当前 Manager Agent 绑定有效	防止伪造节点
请求时间和汇总日期合法	防止异常范围查询

推荐查询条件：

```
org_id = 当前组织
department_id = 当前 Manager 部门
record_date = 指定日期
status in 已提交 / 已确认
deleted_at is null
```

6.10.3 跨部门事项处理

如果某个阻塞事项涉及其他部门，Manager Agent 不能直接读取对方部门明细。

正确方式是：

1. 在 `blocked_items` 中标记跨部门依赖。
2. 生成跨部门协作请求。
3. 由后端或 Executive Agent 判断是否授权。
4. 必要时上推老板或更高层确认。
5. 授权后再读取必要的摘要级数据。

MVP 阶段可以先不做复杂跨部门授权，但必须避免 Manager Agent 默认拥有全公司读取权限。

6.10.4 数据下钻权限

老板或 Executive Agent 可以从部门汇总下钻到来源 Staff 记录，但 Manager Agent 不应跨部门下钻。

Manager Agent 的下钻范围：

```
本部门 Manager Summary
  ↓
本部门 Staff Work Records
  ↓
本部门 Staff Work Record Items
```

禁止范围：

```
其他部门 Manager Summary
其他部门 Staff Work Records
组织级老板决策明细
其他部门员工个人日报
```

6.10.5 权限失败处理

如果 Manager Agent 发起越权读取，后端应返回明确错误。

常见错误包括：

错误类型	建议处理
401 未鉴权	要求重新初始化或登录
403 无权限	拒绝读取，并记录审计
404 数据不存在	返回空结果，不暴露其他部门存在性
422 参数错误	提示 Agent 修正请求参数

Agent 收到权限错误后，不应反复重试，也不应尝试改写部门 ID 继续请求。

6.10.6 Manager 汇总的写入权限

Manager Agent 只能写入自己负责部门的 Manager Schema。

写入时后端应校验：

1. `manager_user_id` 是否为该部门负责人或授权管理者。
2. `manager_node_id` 是否绑定该部门。
3. `dept_id` 是否属于当前组织。
4. `source_record_ids` 是否全部来自该部门。
5. 是否存在未授权来源数据。
6. 签名人是否具备确认权限。

如果 `source_record_ids` 中出现其他部门记录，应拒绝写入。

6.10.7 审计要求

Manager Agent 的读取和写入都应保留审计记录，尤其是以下行为：

1. 读取本部门 Staff 数据。
2. 生成部门汇总。
3. 修改部门汇总。
4. 确认部门汇总。
5. 创建部门任务。
6. 创建上推老板事项。
7. 读取历史汇总。
8. 权限失败请求。

Manager 层是组织数据从一线流向老板的关键中转点，必须保证可追溯。否则老板看到的摘要无法说明来源，员工也无法确认自己的原始信息是否被正确使用。

6.11 本章小结

Manager Schema 是 AutoMage-2 MVP 中承上启下的核心结构。

它向下读取 Staff Schema，向上提供 Executive Agent 可理解的部门态势。它既不能只是员工日报拼接，也不能变成没有来源的主观判断。一个合格的 Manager Schema 应该同时满足四个要求：

1. 能说明部门今天整体推进了什么。
2. 能指出当前最重要的风险和阻塞。
3. 能记录中层已经处理了哪些权限内事项。
4. 能把超权限问题整理成老板可判断的决策项。

MVP 阶段只要 Manager Schema 能稳定完成“读取员工日报 → 生成部门汇总 → 标记风险 → 上推决策 → 写入数据库”这一过程，就可以支撑 Executive Agent 继续生成老板侧摘要和任务下发闭环。

7 Executive 决策 Schema 设计

7.1 Schema 基本信息

Executive 决策 Schema 是 AutoMage-2 MVP 阶段的老板侧数据契约，用于承接各部门 Manager Schema，并将部门汇总、关键风险、上推事项和历史任务状态整理为老板可以直接判断的决策卡片。

Executive Schema 的核心价值不是生成一篇“公司日报”，而是把老板真正需要处理的事情压缩成少量高价值决策项。它要解决的问题是：老板每天不需要看所有一线数据，也不需要逐条读部门日报，而是直接看到“今天公司哪里有风险、什么事情需要我拍板、有哪些方案、确认后会下发什么任务”。

字段	内容
Schema 名称	Executive 决策 Schema
Schema ID	`schema_v1_executive`
当前版本	`1.0.0`
使用节点	Executive Agent
数据生成方	Executive Agent / Dream 机制
数据确认人	老板 / 高管 / 指定决策人
主要触发方式	每日早间 / 定时 Dream / Manager 上推
数据来源	Manager Schema、历史决策、任务状态、异常记录、外部变量
输出对象	老板决策卡片、任务拆解草案、组织级摘要
主要写入对象	决策记录表、`summaries`、`tasks`、`audit_logs`
主要读取对象	老板、高管、任务系统、审计系统
核心目标	将部门汇总转化为老板可确认的决策项，并在确认后生成任务闭环

Executive Schema 的典型流程如下：

```
Executive Agent 定时触发
  ↓
读取各部门 Manager Schema
  ↓
结合历史任务、异常和决策记录
  ↓
生成组织级业务摘要
  ↓
提取关键风险和待决策事项
  ↓
生成 schema_v1_executive
  ↓
推送老板决策卡片
  ↓
老板确认方案
  ↓
系统生成任务并下发
```

↓
记录决策与执行状态

MVP 阶段建议 Executive Schema 每天至少生成一次，优先在早会前推送。例如 Manager Agent 在前一天晚上生成部门汇总，Executive Agent 在第二天早上 8:00 前生成老板侧决策卡片。

7.1.1 Schema ID：schema_v1_executive

Executive 决策 Schema 固定使用：

```
"schema_id": "schema_v1_executive"
```

该字段用于标识当前数据属于老板侧决策结构。后端、Agent 和前端应根据该字段选择对应的校验、展示和存储规则。

7.1.2 使用节点：Executive Agent

Executive Schema 由 Executive Agent 生成。

Executive Agent 一般绑定老板、高管或公司级管理节点。它可以读取组织级汇总数据、部门级 Manager Schema、关键任务和异常记录，但不能随意读取不相关的个人隐私数据。

Executive Agent 的主要工作不是替老板做决定，而是把需要老板判断的事项整理清楚，并在老板确认后推动系统生成任务。

7.1.3 触发方式：每日早间 / 定时 Dream

Executive Schema 可以由两类触发方式生成。

触发方式	说明
每日早间触发	在早会前生成老板侧摘要和决策卡片
定时 Dream 触发	系统按固定时间读取部门汇总，生成组织级判断
Manager 上推触发	部门存在超权限事项时，立即生成待决策项
老板主动查询触发	老板主动询问某部门、某任务或某风险时生成局部摘要

MVP 阶段优先支持“每日早间触发”和“Manager 上推触发”。这样既能保证每天有固定汇总，也能处理临时关键事项。

7.1.4 数据来源：Manager Schema、历史决策、外部变量

Executive Schema 的主要数据来源是各部门的 Manager Schema。

除 Manager Schema 外，Executive Agent 还可以读取以下数据作为辅助：

1. 历史老板决策。
2. 未完成关键任务。
3. 逾期任务。

4. 未关闭异常。
5. 前一日或前一周组织级 Summary。
6. Manager 上推的待决策事项。
7. 老板近期目标或临时指令。
8. 外部变量，例如客户反馈、市场变化、预算限制等。

MVP 阶段不建议引入过多外部数据。优先保证从 Manager Schema 到老板决策项的链路稳定，再逐步扩展外部变量。

7.1.5 输出对象：老板决策卡片、任务拆解

Executive Schema 的输出主要有两类：

输出对象	说明
老板决策卡片	面向老板展示的摘要、风险、决策项和方案
任务拆解草案	老板确认后可生成的执行任务

决策卡片解决“老板怎么快速判断”的问题。任务拆解解决“老板确认后怎么落地”的问题。

Executive Schema 必须同时考虑这两件事。只有摘要没有任务，系统会停留在报告层；只有任务没有决策依据，后续无法审计和复盘。

7.2 Executive Schema 字段总览

Executive Schema 字段分为六类：基础身份字段、组织摘要字段、风险字段、决策字段、任务字段、确认与审计字段。

字段名	类型	是否必填	说明
schema_id	string	是	Schema 标识，固定为 schema_v1_executive
schema_version	string	是	Schema 版本，MVP 阶段默认 1.0.0
timestamp	string	是	生成时间
org_id	string / number	是	组织 ID
executive_user_id	string / number	是	老板或高管用户 ID
executive_node_id	string	是	Executive Agent 节点 ID
summary_date	string	是	摘要对应日期
business_summary	string	是	公司级业务摘要
key_risks	array	是	关键风险
decision_required	boolean	是	是否需要老板决策
decision_items	array	否	待决策事项列表
option_a	object	条件必填	单决策项场景下的方案 A
option_b	object	条件必填	单决策项场景下的方案 B
recommended_option	string	否	Agent 推荐方案
reasoning_summary	string	否	推荐理由摘要
expected_impact	object	否	预期影响
generated_tasks	array	否	老板确认后可下发任务
source_summary_ids	array	是	引用的部门汇总 ID
source_decision_ids	array	否	引用的历史决策 ID
source_incident_ids	array	否	引用的异常 ID
human_confirm_status	string	是	人工确认状态
confirmed_by	string / number	否	确认人
confirmed_at	string	否	确认时间
confirmed_option	string	否	最终确认方案
signature	object	否	老板确认签名
meta	object	否	扩展字段

MVP 阶段建议以 decision_items 作为主要结构，option_a、option_b 可以作为单决策项场景下的快捷字段。后续如果一个 Executive Schema 中包含多个决策项，每个决策项内部应各自包含自己的方案列表。

7.3 字段定义明细

7.3.1 timestamp：生成时间

timestamp 表示 Executive Schema 被生成的时间，使用 ISO8601 格式。

示例：

```
"timestamp": "2026-05-04T08:00:00+08:00"
```

该字段用于判断老板侧摘要是否按时生成，也用于后续审计和任务追踪。

需要注意，timestamp 是生成时间，summary_date 是摘要覆盖的业务日期。比如 5 月 4 日早上生成的摘要，可能覆盖的是 5 月 3 日的业务数据。

7.3.2 executive_user_id：老板/高管用户 ID

executive_user_id 表示该决策卡片面向的老板或高管用户。

示例：

```
"executive_user_id": 30001
```

该字段用于确认谁有权查看和确认该决策。后端应校验当前确认人是否与 executive_user_id 一致，或是否具备代确认权限。

如果一个决策需要多名高管确认，MVP 阶段可先由一个主确认人处理，后续再扩展多签机制。

7.3.3 executive_node_id：老板 Agent 节点 ID

executive_node_id 表示生成该 Executive Schema 的老板侧 Agent 节点。

示例：

```
"executive_node_id": "executive_node_org_1"
```

该字段用于追踪决策项由哪个 Executive Agent 生成。后续如果不同老板或不同业务线使用不同 Executive Agent，该字段可以帮助区分来源。

7.3.4 org_id：组织 ID

org_id 表示该 Executive Schema 所属组织。

示例：

```
"org_id": 1
```

Executive Schema 属于组织级数据，必须绑定组织 ID。Executive Agent 不能跨组织读取或生成决策数据。

7.3.5 business_summary：公司级业务摘要

business_summary 是面向老板的整体业务摘要，用于说明当前组织运行状态。

示例：

```
"business_summary": "昨日各部门整体推进正常，架构组完成 Staff Schema 与 Manager Schema
↪ 的主要字段设计，后端完成部分业务表结构落地，Agent 客制化已跑通 Mock Flow。
↪ 当前主要风险集中在决策记录表结构尚未统一，可能影响 Executive 决策确认和任务下发联调。"
```

该字段应满足以下要求：

1. 不写流水账。
2. 不堆砌各部门所有细节。

3. 只保留老板需要知道的关键信息。
4. 明确整体状态、主要进展和关键问题。
5. 能在 30 秒内读完。

建议控制在 150 到 400 字之间。MVP 阶段老板侧摘要宁可短一些，也不要让老板重新陷入信息过载。

7.3.6 key_risks：关键风险

key_risks 用于记录组织层面最重要的风险。

示例：

```
"key_risks": [
  {
    "title": " 决策记录结构未统一",
    "description": "Agent mock 流程中已存在 decision_logs 概念，但当前数据库 DDL
    ↪  尚未建立独立决策表，可能影响老板确认、任务来源追踪和后续复盘。",
    "severity": "high",
    "affected_departments": [12, 15],
    "source_summary_ids": [801, 802],
    "suggested_action": " 建议尽快确认 Decision Schema 与数据库承载方式。"
  }
]
```

建议每个风险项包含：

子字段	类型	是否必填	说明
title	string	是	风险标题
description	string	是	风险说明
severity	string	是	风险等级
affected_departments	array	否	受影响部门
source_summary_ids	array	否	来源部门汇总
suggested_action	string	否	建议处理方式

风险等级建议使用：

值	含义
low	轻微风险
medium	需要关注
high	可能影响关键链路
critical	需要立即决策或干预

Executive Agent 只应保留组织级关键风险，不应把所有部门风险原样复制上来。

7.3.7 decision_required：是否需要老板决策

decision_required 表示当前 Executive Schema 是否包含需要老板确认的事项。

示例：

```
"decision_required": true
```

如果 decision_required = true，则 decision_items 必须至少包含一项。

如果 decision_required = false，说明当前只是组织级摘要和风险提示，不需要老板立即确认。此时也可以推送给老板，但不应生成任务确认流程。

7.3.8 decision_items：待决策事项列表

decision_items 是 Executive Schema 的核心字段，用于承载老板需要确认的事项。

示例：

```
"decision_items": [
  {
    "decision_id": "decision_tmp_001",
    "title": " 是否新增独立 decision_logs 表",
    "context": "当前 Agent mock 流程中已经存在 decision_logs 概念，但正式 DDL
    ↪  尚未建立独立决策表。如果不新增，短期可用 audit_logs 和 tasks 承载，
    ↪  但后续老板决策追踪和复盘会不够清晰。",
    "decision_level": "L3",
    "urgency": "high",
    "option_a": {
      "option_id": "A",
      "title": " 新增独立 decision_logs 表",
      "description": " 为老板决策、候选方案、确认结果和任务来源建立独立数据表。",
      "pros": [" 审计清晰", " 便于复盘", " 后续扩展空间大"],
      "cons": [" 需要增加后端开发工作量"]
    },
    "option_b": {
      "option_id": "B",
      "title": " 暂时复用 audit_logs 和 tasks",
      "description": " 短期不新增表，先通过审计日志和任务表承载决策结果。",
      "pros": [" 开发更快", "MVP 改动较少"],
      "cons": [" 决策对象不独立", " 后续统计和追踪不方便"]
    },
    "recommended_option": "A",
    "reasoning_summary": "AutoMage-2 的核心价值在于决策可追溯，独立决策对象有利于后续任务生成、
    ↪  复盘和老板侧看板。",
    "expected_impact": {
      "cost": "medium",
      "timeline": " 短期增加 0.5-1 天后端工作量",
      "risk": " 降低后续决策追踪混乱风险",
      "affected_modules": ["backend", "agent", "database"]
    },
    "generated_tasks": [
      {
        "task_title": " 补充 decision_logs 表结构草案",
```

```
        "assignee_role": "backend",
        "priority": "high",
        "due_at": "2026-05-04T18:00:00+08:00"
      }
    ],
    "source_summary_ids": [801, 802]
  }
]
```

每个决策项应尽量包含背景、方案、推荐理由、影响和可生成任务。老板看到后应该可以直接选择，而不是还要再问“具体怎么做”。

7.3.9 option_a：方案 A

option_a 表示某个决策项下的第一个候选方案。

在单决策项场景下，option_a 可以放在 Executive Schema 顶层；在多决策项场景下，建议放在 decision_items[] 内部。

示例：

```
"option_a": {
  "option_id": "A",
  "title": " 新增独立 decision_logs 表",
  "description": " 建立独立表记录老板决策项、候选方案、确认结果和后续任务。",
  "pros": [" 结构清晰", " 便于审计", " 方便后续扩展"],
  "cons": [" 需要增加开发工作量"]
}
```

方案 A 不一定代表推荐方案，只是候选方案之一。

7.3.10 option_b：方案 B

option_b 表示某个决策项下的第二个候选方案。

示例：

```
"option_b": {
  "option_id": "B",
  "title": " 暂时复用 audit_logs 和 tasks",
  "description": "MVP 阶段先不新增决策表，通过审计日志记录确认动作，通过任务表记录执行结果。",
  "pros": [" 实现更快", " 减少短期表结构变更"],
  "cons": [" 长期追踪不清晰", " 后续可能需要迁移数据"]
}
```

MVP 阶段建议每个老板决策项至少提供 A/B 两个方案。后续可以扩展为多方案列表。

7.3.11 recommended_option：Agent 推荐方案

recommended_option 表示 Executive Agent 推荐老板选择的方案。

示例：

```
"recommended_option": "A"
```

推荐方案必须有理由，不能只给结论。推荐理由应写入 reasoning_summary。

如果 Agent 无法明确推荐，应允许该字段为空，并在决策卡片中标注“暂无明确推荐，需要老板判断”。

7.3.12 reasoning_summary：推荐理由摘要

reasoning_summary 用于说明为什么推荐某个方案。

示例：

```
"reasoning_summary": "方案 A 虽然会增加少量后端开发工作量，但能让老板决策、
↪ 任务来源和后续复盘形成独立闭环，更符合 AutoMage-2 长期架构。"
```

该字段不是完整推理过程，也不需要写得很长。老板需要的是清楚的依据，而不是冗长分析。

推荐理由应尽量包含：

1. 与当前目标的关系。
2. 对进度的影响。
3. 对风险的影响。
4. 对后续扩展的影响。
5. 为什么优于其他方案。

7.3.13 expected_impact：预期影响

expected_impact 用于说明某个决策被确认后可能产生的影响。

示例：

```
"expected_impact": {
  "cost": "medium",
  "timeline": " 短期增加 0.5-1 天后端工作量",
  "risk": " 降低后续决策追踪混乱风险",
  "affected_modules": ["backend", "agent", "database"],
  "business_value": " 提升老板决策可追溯性和后续复盘能力"
}
```

建议子字段包括：

子字段	类型	是否必填	说明
cost	string	否	成本影响
timeline	string	否	时间影响
risk	string	否	风险变化
affected_modules	array	否	受影响模块
business_value	string	否	业务价值

MVP 阶段不要求非常精确的量化评估，但必须让老板知道选择不同方案会带来什么后果。

7.3.14 generated_tasks：可下发任务

generated_tasks 表示老板确认后可以自动生成的任务草案。

示例：

```
"generated_tasks": [
  {
    "task_title": " 补充 decision_logs 表结构草案",
    "task_description": "根据 Executive 决策链路设计 decision_logs 表，字段需覆盖决策标题、
    ↪ 背景、候选方案、推荐方案、确认人、确认时间和生成任务。",
    "assignee_user_id": 20002,
    "assignee_role": "backend",
    "department_id": 12,
    "priority": "high",
    "due_at": "2026-05-04T18:00:00+08:00",
    "source_decision_id": "decision_tmp_001"
  }
]
```

该字段在老板确认前只是草案，不应直接写入正式任务表。老板确认后，后端再根据确认方案生成正式 tasks 和 task_assignments。

7.3.15 source_summary_ids：引用的部门汇总 ID

source_summary_ids 表示当前 Executive Schema 引用了哪些 Manager Summary。

示例：

```
"source_summary_ids": [801, 802, 803]
```

这是 Executive Schema 可追溯性的关键字段。老板看到的摘要和决策项必须能追溯到具体部门汇总，再继续追溯到 Staff Work Records。

后端应校验这些 Summary 是否属于同一组织，且 Executive Agent 有读取权限。

7.3.16 human_confirm_status：人工确认状态

human_confirm_status 表示当前 Executive Schema 或其中决策项的人工确认状态。

示例：

```
"human_confirm_status": "pending"
```

建议枚举值如下：

值	含义
draft	草稿，尚未推送老板
pending	已推送，等待老板确认
confirmed	已确认
rejected	已驳回
need_more_info	需要补充信息
expired	超时未确认
cancelled	已取消

MVP 阶段，涉及任务生成的决策必须在 confirmed 后才能继续执行。

7.3.17 confirmed_by：确认人

confirmed_by 表示最终确认该决策的人。

示例：

```
"confirmed_by": 30001
```

该字段在 human_confirm_status = confirmed 时必填。

如果老板未确认，该字段应为空。Agent 不能代替老板填写确认人。

7.3.18 confirmed_at：确认时间

confirmed_at 表示老板确认决策的时间。

示例：

```
"confirmed_at": "2026-05-04T08:16:30+08:00"
```

该字段用于审计和任务生成。后端应以服务器确认时间为准，同时保留 IM 或前端传入的确认来源。

7.4 Executive Schema JSON 示例

下面是一个完整 Executive Schema 示例，可作为 Agent 输出、老板决策卡片和后端联调参考。


```
{
  "schema_id": "schema_v1_executive",
  "schema_version": "1.0.0",
  "timestamp": "2026-05-04T08:00:00+08:00",
  "org_id": 1,
  "executive_user_id": 30001,
  "executive_node_id": "executive_node_org_1",
  "summary_date": "2026-05-03",
  "business_summary": "昨日各部门整体推进正常，架构组完成 Staff Schema 与 Manager Schema
  ↪ 的主要字段设计，后端完成部分业务表结构落地，Agent 客制化已跑通 Mock Flow。
  ↪ 当前主要风险集中在决策记录表结构尚未统一，可能影响 Executive 决策确认和任务下发联调。",
  "key_risks": [
    {
      "title": " 决策记录结构未统一",
      "description": "Agent mock 流程中已存在 decision_logs 概念，但当前数据库 DDL
      ↪ 尚未建立独立决策表，可能影响老板确认、任务来源追踪和后续复盘。",
      "severity": "high",
      "affected_departments": [12, 15],
      "source_summary_ids": [801, 802],
      "suggested_action": " 建议尽快确认 Decision Schema 与数据库承载方式。"
    }
  ],
  "decision_required": true,
  "decision_items": [
    {
      "decision_id": "decision_tmp_001",
      "title": " 是否新增独立 decision_logs 表",
      "context": "当前 Agent mock 流程中已经存在 decision_logs 概念，但正式 DDL
      ↪ 尚未建立独立决策表。如果不新增，短期可用 audit_logs 和 tasks 承载，
      ↪ 但后续老板决策追踪和复盘会不够清晰。",
      "decision_level": "L3",
      "urgency": "high",
      "option_a": {
        "option_id": "A",
        "title": " 新增独立 decision_logs 表",
        "description": " 为老板决策、候选方案、确认结果和任务来源建立独立数据表。",
        "pros": [" 审计清晰", " 便于复盘", " 后续扩展空间大"],
        "cons": [" 需要增加后端开发工作量"]
      },
      "option_b": {
        "option_id": "B",
        "title": " 暂时复用 audit_logs 和 tasks",
        "description": " 短期不新增表，先通过审计日志和任务表承载决策结果。",
```

```
      "pros": [" 开发更快", "MVP 改动较少"],
      "cons": [" 决策对象不独立", " 后续统计和追踪不方便"]
    },
    "recommended_option": "A",
    "reasoning_summary": "AutoMage-2 的核心价值在于决策可追溯,
    ↪  独立决策对象有利于后续任务生成、复盘和老板侧看板。",
    "expected_impact": {
      "cost": "medium",
      "timeline": " 短期增加 0.5-1 天后端工作量",
      "risk": " 降低后续决策追踪混乱风险",
      "affected_modules": ["backend", "agent", "database"],
      "business_value": " 提升老板决策可追溯性和后续复盘能力"
    },
    "generated_tasks": [
      {
        "task_title": " 补充 decision_logs 表结构草案",
        "task_description": "根据 Executive 决策链路设计 decision_logs 表,
        ↪  字段需覆盖决策标题、背景、候选方案、推荐方案、确认人、确认时间和生成任务。",
        "assignee_user_id": 20002,
        "assignee_role": "backend",
        "department_id": 12,
        "priority": "high",
        "due_at": "2026-05-04T18:00:00+08:00",
        "source_decision_id": "decision_tmp_001"
      }
    ],
    "source_summary_ids": [801, 802]
  }
],
"source_summary_ids": [801, 802, 803],
"source_decision_ids": [],
"source_incident_ids": [501],
"human_confirm_status": "pending",
"confirmed_by": null,
"confirmed_at": null,
"confirmed_option": null,
"signature": {
  "signature_required": true,
  "signature_status": "pending",
  "signed_by": null,
  "signed_at": null,
  "payload_hash": "sha256:executive_decision_example_hash",
  "signature_source": "im"
},
"meta": {
  "input_channel": "scheduled_dream",
  "agent_template_version": "executive_agent_v0.1",
```

```
    "summary_window": "daily",
    "push_channel": "feishu"
  }
}
```

7.5 Executive Schema 决策卡片格式

Executive Schema 最终需要以老板容易理解的形式展示出来。MVP 阶段建议优先使用 IM 决策卡片，后续再扩展老板侧 App 或看板。

决策卡片不应展示完整 JSON，而应展示经过整理的关键内容。

推荐格式如下：

```
【今日老板待决策】

日期：2026-05-04
状态：需要确认 1 项决策

一、公司整体情况
昨日各部门整体推进正常，架构组完成 Staff Schema 与 Manager Schema 的主要字段设计，
↪ 后端完成部分业务表结构落地，Agent 客制化已跑通 Mock Flow。
↪ 当前主要风险集中在决策记录表结构尚未统一。

二、关键风险
1. 决策记录结构未统一
影响：可能影响老板确认、任务来源追踪和后续复盘
风险等级：高

三、待决策事项
事项：是否新增独立 decision_logs 表

背景：
当前 Agent mock 流程中已经存在 decision_logs 概念，但正式 DDL 尚未建立独立决策表。

方案 A：新增独立 decision_logs 表
优点：审计清晰、便于复盘、后续扩展空间大
缺点：需要增加后端开发工作量

方案 B：暂时复用 audit_logs 和 tasks
优点：开发更快、MVP 改动较少
缺点：决策对象不独立，后续统计和追踪不方便

推荐：方案 A
理由：AutoMage-2 的核心价值在于决策可追溯，独立决策对象更符合长期架构。

确认后生成任务：
```

```
1. 补充 decision_logs 表结构草案
负责人：后端
截止时间：今日 18:00
```

IM 卡片按钮建议包括：

按钮	系统动作
选择方案 A	记录确认结果，生成 A 对应任务
选择方案 B	记录确认结果，生成 B 对应任务
需要补充信息	状态改为 `need_more_info`
暂不处理	状态改为 `rejected` 或 `cancelled`
查看来源	展示引用的部门汇总和来源记录

老板侧卡片的原则是：短、准、可操作。不建议让老板在卡片中阅读大量原始日报，也不建议只给一段泛泛总结。

7.6 Executive Schema 与 Dream 机制关系

Dream 是 Executive Schema 的主要生成机制之一。它负责在固定时间读取系统中的结构化数据，生成组织级摘要、关键风险和老板决策项。

二者关系如下：

```
Manager Schema
  ↓
Dream 读取和分析
  ↓
Executive Schema
  ↓
老板决策卡片
  ↓
任务下发
```

Dream 的输入主要包括：

输入	说明
Manager Schema	各部门汇总
Summary Schema	历史日报、周报、组织级摘要
Task Schema	未完成任务、逾期任务、关键任务
Incident Schema	未关闭异常、重大风险
Decision Schema	历史老板决策和执行结果
老板目标	临时目标、经营目标、战略重点

Dream 的输出主要包括：

输出	对应 Executive Schema 字段
组织级摘要	`business_summary`
关键风险	`key_risks`
待决策事项	`decision_items`
推荐方案	`recommended_option`
推荐理由	`reasoning_summary`
任务草案	`generated_tasks`
来源引用	`source_summary_ids`

MVP 阶段 Dream 不需要做得很复杂。只要能完成以下动作，就可以支撑主链路：

1. 定时读取部门汇总。
2. 提炼组织级摘要。
3. 识别最重要的风险。
4. 将上推事项整理为老板决策项。
5. 生成 A/B 方案。
6. 在老板确认后生成任务草案。

后续版本可以再扩展长期记忆、跨周期趋势判断、组织效率分析和战略推演。

7.7 Executive Schema 与任务下发关系

Executive Schema 不是任务表，但它是老板任务下发的来源。

任务下发必须发生在老板确认之后。确认前，`generated_tasks` 只是任务草案；确认后，后端才根据确认结果创建正式任务。

推荐流程如下：

```
Executive Schema 生成
  ↓
老板查看决策卡片
  ↓
老板选择方案
  ↓
后端记录确认结果
  ↓
读取对应方案的 generated_tasks
  ↓
创建 tasks / task_assignments
  ↓
推送给 Manager Agent 或 Staff Agent
```

任务下发规则如下：

情况	任务下发方式
任务需要部门拆解	先下发给 Manager Agent
任务已有明确执行人	直接下发给 Staff Agent
任务涉及多个部门	分别下发给多个 Manager Agent
任务需要老板再次确认	保持待确认，不生成正式任务
老板选择“补充信息”	不生成任务，回到信息补全状态

Executive Schema 中的 `generated_tasks` 至少应包含：

1. 任务标题。
2. 任务说明。
3. 负责人或负责人角色。
4. 所属部门。
5. 优先级。
6. 截止时间。
7. 来源决策 ID。
8. 触发方案。

任务一旦创建，必须与决策记录建立来源关系。否则后续无法回答“这个任务为什么产生”。

7.8 Executive Schema 与数据库字段映射

Executive Schema 需要映射到多个数据库对象。当前 DDL 中已有 `summaries`、`tasks`、`task_assignments`、`audit_logs` 等表，但独立 Decision 表仍建议补齐。

7.8.1 映射到组织级 summaries

Executive Schema 中的组织级摘要可以写入 `summaries`。

Executive Schema 字段	数据库字段	说明
`org_id`	`summaries.org_id`	组织 ID
`executive_user_id`	`summaries.user_id` 或 `meta.executive_user_id`	老板/高管 ID
`summary_date`	`summaries.summary_date`	摘要日期
`business_summary`	`summaries.content`	组织级摘要正文
`schema_id` / `schema_version`	`summaries.meta`	Schema 标识和版本
`key_risks`	`summaries.meta.key_risks`	关键风险
`decision_required`	`summaries.meta.decision_required`	是否需要决策
`source_summary_ids`	`summary_source_links`	来源部门汇总
`human_confirm_status`	`summaries.status` 或 `meta`	确认状态

`summaries.scope_type` 应标记为组织级，表示该 Summary 不属于某个单独部门或员工。

7.8.2 映射到 Decision 记录

建议新增独立 Decision 表承载老板决策。若 MVP 阶段暂未新增，可先使用 `audit_logs` + `summaries.meta` 暂存，但这不是长期方案。

建议 Decision 对象字段如下：

Executive Schema 字段	Decision 字段	说明
`decision_items[].title`	`title`	决策标题
`decision_items[].context`	`context`	决策背景
`decision_level`	`decision_level`	决策等级
`option_a` / `option_b`	`options`	候选方案
`recommended_option`	`recommended_option`	推荐方案
`reasoning_summary`	`recommend_reason`	推荐理由
`expected_impact`	`expected_impact`	预期影响
`human_confirm_status`	`status`	确认状态
`confirmed_by`	`confirmed_by`	确认人
`confirmed_at`	`confirmed_at`	确认时间
`confirmed_option`	`confirmed_option`	最终选择
`source_summary_ids`	`source_summary_ids`	来源部门汇总
`generated_tasks`	`meta.generated_tasks`	任务草案

Decision 表是老板决策可追溯的核心。建议后端优先补齐该对象。

7.8.3 映射到 tasks

老板确认后，`generated_tasks` 应写入 `tasks` 和 `task_assignments`。

Executive Schema 字段	数据库字段	说明
`generated_tasks[].task_title`	`tasks.title`	任务标题
`generated_tasks[].task_description`	`tasks.description`	任务说明
`department_id`	`tasks.department_id`	所属部门
`priority`	`tasks.priority`	优先级
`due_at`	`tasks.due_at`	截止时间
`source_decision_id`	`tasks.meta.source_decision_id`	来源决策
`assignee_user_id`	`task_assignments.user_id`	执行人
`assignee_role`	`tasks.meta.assignee_role`	执行角色

如果任务只指定角色而未指定具体人，可以先下发给 Manager Agent，由部门负责人再拆解到具体员工。

7.8.4 映射到 audit_logs

Executive Schema 的关键动作必须写入审计日志。

建议记录以下动作：

动作	说明
executive_schema_generated	Executive Agent 生成老板侧摘要
executive_decision_pushed	决策卡片推送给老板
executive_decision_confirmed	老板确认方案
executive_decision_rejected	老板驳回
executive_decision_need_more_info	老板要求补充信息
executive_task_generated	确认后生成任务
executive_schema_read_source	读取部门汇总来源
executive_decision_expired	决策超时未确认

审计日志应记录操作人、Agent 节点、决策对象、确认方案、确认时间和生成任务。

7.8.5 映射到 summary_source_links

Executive Schema 引用的部门汇总应写入 summary_source_links。

Executive Schema 字段	数据库字段	说明
source_summary_ids[]	summary_source_links.source_id	来源部门 Summary ID
当前组织级 summary ID	summary_source_links.summary_id	当前 Executive Summary ID
固定来源类型	summary_source_links.source_type	来源类型，如 manager_summary
org_id	summary_source_links.org_id	组织 ID

这样可以保证老板侧摘要可以向下追溯到部门汇总，再继续追溯到员工日报。

7.9 老板确认后的状态变化

老板确认是 Executive Schema 从“建议”变成“执行”的关键节点。

确认前，Executive Schema 只是决策草案。确认后，系统才可以记录正式决策，并生成任务。

状态变化建议如下：

```
draft
  ↓
pending
  ↓
confirmed
  ↓
task_generated
  ↓
in_execution
  ↓
completed / closed
```

如果老板驳回或要求补充信息，则进入：

```
pending
  ↓
rejected / need_more_info
```

各状态说明如下：

状态	含义	系统动作
draft	草稿，尚未推送老板	不生成任务
pending	已推送老板，等待确认	允许提醒和查看来源
confirmed	老板已选择方案	记录确认人和确认时间
task_generated	已生成任务	写入任务表和分配表
in_execution	任务执行中	Staff / Manager Agent 跟进
completed	任务已完成	可进入复盘
closed	决策闭环关闭	保留审计记录
rejected	老板驳回	不生成任务
need_more_info	老板要求补充信息	回到 Manager 或相关负责人补充
expired	超时未确认	按规则提醒或降级处理

老板确认后，后端必须执行以下动作：

1. 校验确认人权限。
2. 更新决策状态。
3. 记录 confirmed_by。
4. 记录 confirmed_at。
5. 记录 confirmed_option。
6. 写入审计日志。
7. 根据确认方案生成任务。
8. 将任务下发给对应 Manager 或 Staff。
9. 通知相关人员。
10. 保留来源关系。

Agent 不能在没有后端确认状态的情况下自行声称“老板已确认”。

7.10 未确认时的处理规则

并不是所有决策卡片都会立即被老板确认。MVP 阶段需要明确未确认时的处理规则，避免系统卡死。

7.10.1 等待确认

当决策卡片已推送但老板未操作时，状态为：

```
"human_confirm_status": "pending"
```

在该状态下：

1. 不生成正式任务。
2. 不改变下级任务状态。
3. 可以提醒老板确认。

4. Manager Agent 可以看到该事项处于待确认。
5. 超过一定时间后进入超时规则。

7.10.2 超时未确认

如果老板在设定时间内未确认，系统应按决策等级处理。

建议规则：

决策等级	超时处理
低风险事项	提醒一次后降级为 Manager 处理
中风险事项	持续提醒，并通知相关 Manager
高风险事项	提醒老板，并标记为高优先级待确认
重大事项	不允许自动执行，必须等待人工确认

MVP 阶段建议高风险和重大事项都不自动执行。宁可延迟，也不要让系统绕过老板做关键决策。

7.10.3 老板要求补充信息

如果老板点击“需要补充信息”，状态应变为：

```
"human_confirm_status": "need_more_info"
```

系统应记录老板提出的问题，并将补充请求返回给相关 Manager Agent。

流程如下：

```
老板要求补充信息
  ↓
Executive Agent 记录问题
  ↓
相关 Manager Agent 收到补充请求
  ↓
Manager 补充说明或重新生成汇总
  ↓
Executive Agent 更新决策卡片
  ↓
再次推送老板确认
```

此时不应生成任务，除非老板明确要求先执行部分事项。

7.10.4 老板驳回

如果老板驳回决策项，状态应变为：

```
"human_confirm_status": "rejected"
```

驳回后：

1. 不生成任务。
2. 记录驳回人和驳回时间。
3. 记录驳回原因。
4. 通知相关 Manager Agent。
5. 保留审计日志。
6. 后续如需重新提交，应生成新版本决策项。
   不建议直接覆盖原决策项，否则会丢失决策过程。

7.10.5 决策卡片修改

老板可能不选择方案 A 或 B，而是提出新的方案。此时系统应记录为人工改写。

处理方式：

1. 保留原方案。
2. 记录老板修改内容。
3. 将 `confirmed_option` 标记为 `custom`。
4. 将老板输入写入 `confirmed_custom_plan` 或 `meta`。
5. 根据新方案生成任务。
6. 写入审计日志。
   示例：

```
{
  "confirmed_option": "custom",
  "confirmed_custom_plan": "先新增轻量 decision_logs 表，只保留决策标题、确认人、确认时间、来源
  ↪  summary 和生成任务，复杂字段后续迭代。"
}
```

这种能力对老板侧非常重要，因为真实决策往往不是简单二选一。

7.10.6 未确认数据的可见范围

未确认的 Executive Schema 可以被相关角色查看，但不能被当作最终决策执行。

角色	可见内容
老板	全部决策卡片
Executive Agent	全部生成内容
相关 Manager Agent	与本部门相关的待确认事项
Staff Agent	一般不可见，除非任务已正式下发
审计系统	可见生成、推送和等待确认记录

未确认事项不应下发给一线员工，避免造成“老板还没拍板，下面已经开始执行”的混乱。

7.10.7 未确认事项的复盘

未确认事项也有复盘价值。系统应记录：

1. 什么时间生成。
2. 来自哪些部门汇总。
3. 为什么需要老板确认。
4. 是否被查看。
5. 是否超时。
6. 是否被驳回。
7. 是否被要求补充信息。
8. 最终是否再次提交。

这些数据后续可以帮助判断：哪些事项频繁卡在老板确认环节，哪些部门上推质量较低，哪些决策类型需要调整权限边界。

7.11 本章小结

Executive Schema 是 AutoMage-2 MVP 中连接“部门汇总”和“老板决策”的关键结构。

它向下读取 Manager Schema，向上生成老板可确认的决策卡片。一个合格的 Executive Schema 不应该只是公司日报，也不应该只是 Agent 的主观建议，而应该同时满足四个要求：

1. 能让老板快速了解公司整体状态。
2. 能指出当前最重要的组织级风险。
3. 能把复杂问题整理成可选择、可确认的决策项。
4. 能在老板确认后生成可执行、可追踪的任务。

MVP 阶段只要 Executive Schema 能稳定完成“读取部门汇总 → 生成老板摘要 → 生成决策项 → 老板确认 → 任务下发”这一过程，就可以证明 AutoMage-2 的三级 Agent 决策闭环具备基础可行性。

8 三级节点决策机制设计

8.1 决策机制总览

AutoMage-2 MVP 的决策机制采用“分层处理、逐级上推、人工确认、任务落地”的设计。

系统不要求所有决策都由老板处理，也不允许 Agent 在没有授权的情况下自行处理重大事项。不同层级的 Agent 只在自己的权限范围内完成判断和动作：Staff Agent 处理个人记录和提醒；Manager Agent 处理部门内异常和常规协调；Executive Agent 负责生成老板侧决策项，并在老板确认后推动任务下发。

整体决策链路如下：

```
一线工作数据
  ↓
Staff Agent 判断是否需要支持
  ↓
Manager Agent 判断是否部门内可处理
  ↓
Executive Agent 整理成老板决策项
  ↓
老板确认
  ↓
系统生成任务
  ↓
任务下发到 Manager / Staff
  ↓
执行结果回流为新的 Staff Schema
```

这套机制的重点不在于让 AI “替人拍板”，而是让 AI 把原本散落在日报、聊天、会议和口头沟通中的问题，整理成可判断、可确认、可追踪的决策对象。

MVP 阶段的决策机制应满足四个基本要求：

1. 小事项不打扰上级，Agent 或员工确认后即可处理。
2. 部门内事项优先由 Manager 处理，不直接推给老板。
3. 超出部门权限的事项必须整理后上推 Executive。
4. 老板确认后的结果必须生成任务，不能只停留在文字结论。

决策机制的目标是减少无效沟通，而不是制造更多审批流程。

8.2 决策权限分层原则

AutoMage-2 的决策权限按照组织层级和影响范围划分。权限越低，处理事项越具体、影响范围越小；权限越高，处理事项越关键、影响范围越大。

分层原则如下：

层级	处理主体	典型事项	是否需要人工确认
L0	Agent 自动处理	字段补全提醒、日报催填、格式校验、任务提醒	不需要
L1	员工确认	个人日报提交、个人任务状态更新、个人问题上报	员工确认
L2	Manager 确认	部门任务调整、轻中度异常处理、部门内资源协调	Manager 确认
L3	Executive 确认	跨部门资源、关键架构选择、重大交付风险、老板级任务下发	老板/高管确认
L4	线下会议或正式审批	战略调整、重大预算、人事调整、法律合规、高风险外部承诺	人工流程确认

权限分层要遵守以下原则：

1. **能低层处理的，不向高层上推。**

如果员工自己能补充信息，Staff Agent 不应直接找 Manager；如果 Manager 能在部门内解决，Manager Agent 不应直接找老板。

1. **影响范围越大，确认层级越高。**

只影响个人的事项可以由员工确认；影响部门的事项需要 Manager 确认；影响多个部门、预算、客户或战略的事项需要 Executive 确认。

1. **Agent 可以建议，但关键动作需要人确认。**

MVP 阶段不允许 Agent 绕过人类完成重大决策。Agent 的职责是整理信息、提供方案和触发流程。

1. **所有决策必须有来源。**

任何决策项都应能追溯到 Staff Schema、Manager Schema、Incident、Task 或历史 Summary。

1. **所有越权行为由后端拒绝。**

Prompt 中的限制只是第一层约束，真正的权限控制必须由后端 API 和数据库权限模型保证。

8.3 Staff Agent 的决策边界

Staff Agent 是最低层的岗位执行节点，主要服务于员工个人。它可以辅助员工记录、提醒、整理和上报，但不能代替员工做承诺，也不能越级读取他人数据。

Staff Agent 的决策边界可以概括为：

```
只处理个人范围内的记录、提醒、补全和上报；
不处理部门级判断；
不处理老板级决策；
不读取他人数据。
```

8.3.1 可自主记录

Staff Agent 可以在员工授权和确认的前提下，整理并记录员工个人工作数据。

可自主记录的内容包括：

1. 员工当天完成的工作事项。
2. 员工遇到的问题。
3. 员工已经尝试的解决方案。
4. 员工的明日计划。
5. 员工上传的产出物。
6. 员工关联的任务状态。
7. 员工提出的支持需求。

这里的“自主记录”不是指 Agent 可以凭空编造日报，而是指 Agent 可以把员工输入整理成标准 Staff Schema，并在员工确认后提交。

示例：

```
员工：今天完成了接口字段整理，Decision 表还没确定，明天继续补数据库映射。
Staff Agent：我将其整理为 3 个字段：今日完成事项、遇到的问题、明日计划。请确认是否提交。
```

员工确认后，Staff Agent 才能提交给后端。

8.3.2 可自主提醒

Staff Agent 可以根据时间、任务状态和字段完整度进行提醒。

可自主提醒的场景包括：

1. 下班前提醒员工填写日报。
2. 日报缺少必填字段时提醒补充。
3. 员工选择需要支持但未填写支持内容时提醒补充。
4. 任务截止前提醒员工处理。
5. 任务逾期时提醒员工更新状态。
6. 员工日报长期缺失时提醒提交。
7. 员工工作事项标记为阻塞时提醒是否上报。

这些提醒属于 L0 级动作，不需要 Manager 或老板确认。

提醒内容应尽量具体，不要泛泛地说“请完善”。例如：

你的日报中已选择“需要上级支持”，但还没有填写具体支持内容。请补充需要谁支持、支持什么事项。

8.3.3 不可替员工做重大承诺

Staff Agent 不得替员工做超出个人范围的承诺。

不可做的事项包括：

1. 不得承诺任务一定按时完成。
2. 不得代表员工接受新任务，除非员工确认。
3. 不得承诺客户、合作方或外部对象。
4. 不得替员工确认涉及责任划分的内容。
5. 不得擅自修改员工已经确认的日报。
6. 不得替员工承认错误、承担责任或做绩效相关表态。
7. 不得自行判断员工表现优劣并上报为正式结论。

Staff Agent 可以提出建议，但最终确认必须由员工完成。

示例：

不推荐：

```
Staff Agent: 该任务员工明天一定可以完成。
```

推荐：

```
Staff Agent: 员工计划明天继续推进该任务，预计产出为接口字段草案，仍需后端确认部分字段。
```

MVP 阶段必须避免让 Staff Agent 变成“自动替员工背书”的工具。

8.3.4 不可越级读取他人数据

Staff Agent 只能读取与当前员工本人相关的数据。

可读取范围包括：

1. 员工自己的日报。
2. 员工自己的任务。
3. 员工自己提交的产出物。
4. 员工自己上报的异常。
5. 上级下发给该员工的任务说明。

不可读取范围包括：

1. 其他员工日报。
2. 部门完整汇总。
3. Manager 决策记录。
4. 老板决策卡片。

5. 其他员工任务。
6. 组织级风险和敏感数据。

如果 Staff Agent 需要了解某个任务的背景，只能读取该任务对当前员工开放的部分，而不是读取完整部门数据。

权限由后端强制控制。即使 Staff Agent 在 Prompt 中请求读取其他员工数据，后端也应返回 403。

8.4 Manager Agent 的决策边界

Manager Agent 是部门级节点，负责处理本部门的数据汇总、风险判断、异常处理和部门内任务协调。

它的定位不是“高级日报生成器”，而是部门运行状态的判断节点。它需要区分哪些问题可以在部门内处理，哪些问题需要上推老板。

Manager Agent 的决策边界可以概括为：

```
读取本部门数据；
生成部门汇总；
处理部门内事项；
上推超权限事项；
不越权处理老板级决策。
```

8.4.1 可读取本部门数据

Manager Agent 可以读取本部门范围内的数据。

可读取内容包括：

1. 本部门员工 Staff Schema。
2. 本部门员工任务状态。
3. 本部门异常记录。
4. 本部门历史汇总。
5. 与本部门相关的老板决策和任务。
6. 本部门产出物和工作记录。

读取时必须满足以下条件：

```
org_id 一致
department_id 在权限范围内
数据状态有效
数据未删除
当前 Manager Agent 已绑定该部门
```

Manager Agent 不应读取其他部门员工明细。如果涉及跨部门问题，应通过上推或授权流程处理，而不是默认拥有全公司读取权限。

8.4.2 可生成部门汇总

Manager Agent 可以基于本部门 Staff Schema 生成 Manager Schema。

部门汇总应包含：

1. 部门整体进展。
2. 部门健康度。
3. 关键风险。
4. 阻塞事项。
5. 缺失日报情况。
6. 表现突出员工。
7. 部门内已处理事项。
8. 需上推老板的决策项。
9. 次日调整建议。
10. 来源 Staff 记录。

Manager Agent 生成汇总时，必须保留来源记录，不能把个人日报内容加工成无法追溯的结论。

不推荐：

```
部门整体存在较大风险。
```

推荐：

```
部门整体健康度为 yellow。主要风险来自 2 条 Staff 记录：Decision 表结构未确认、部分日报缺失。
↪ 该风险可能影响 Executive 决策链路。
```

8.4.3 可处理权限内异常

Manager Agent 可以处理部门内权限范围内的异常。

可处理事项包括：

1. 员工日报缺失催办。
2. 部门内任务优先级调整。
3. 部门内人员协作安排。
4. 轻中度任务阻塞处理。
5. 本部门内支持请求。
6. 本部门内资源协调。
7. 部门内低风险异常关闭。
8. 员工任务状态异常提醒。

示例：

```
员工 A 的任务被标记为 blocked，原因是缺少接口字段说明。
Manager Agent 可以将该问题转给负责字段设计的员工 B，并生成部门内协作任务。
```

但如果异常涉及跨部门资源、预算、客户承诺、交付延期或战略方向，Manager Agent 不应自行处理，应整理为 `need_executive_decision` 上推。

8.4.4 可下发部门级任务

Manager Agent 可以在部门权限范围内下发任务。

可下发任务包括：

1. 根据老板决策拆解出的部门内任务。
2. 根据 Staff 日报生成的跟进任务。
3. 根据部门阻塞事项生成的协调任务。
4. 根据缺失日报生成的催办任务。
5. 根据次日调整建议生成的部门任务。
6. 根据异常处理结果生成的补救任务。

任务下发必须满足以下条件：

1. 任务属于本部门范围。
2. 任务不涉及超权限承诺。
3. 任务有明确负责人或负责人角色。
4. 任务有来源对象。
5. 任务有优先级和截止时间。
6. 任务状态进入数据库。

Manager Agent 不应只在 IM 中说“大家明天处理一下”，而应生成正式 Task Schema，并写入任务系统。

8.4.5 超权限事项上推老板

Manager Agent 必须识别超出自己权限的事项，并上推 Executive Agent。

需要上推的典型事项包括：

1. 跨部门资源冲突。
2. 影响关键交付节点的延期。
3. 需要老板确认的架构或产品方向选择。
4. 涉及预算、采购或外部合作的事项。
5. 影响客户承诺的事项。
6. 涉及组织结构或岗位调整的事项。

7. 部门内无法解决的长期阻塞。
8. 多个部门同时出现的系统性风险。

上推时不能只写“请老板处理”，而应整理成决策题。

上推内容至少包括：

1. 事项标题。
2. 背景说明。
3. 影响范围。
4. 可选方案。
5. 推荐方案。
6. 推荐理由。
7. 来源记录。
8. 紧急程度。

这样 Executive Agent 才能进一步生成老板决策卡片。

8.5 Executive Agent 的决策边界

Executive Agent 是老板侧节点，负责读取组织级汇总，生成老板决策项，并在老板确认后推动任务下发。

它拥有最高层级的数据视角，但不等于可以自动执行所有决策。MVP 阶段必须保留老板确认机制。

Executive Agent 的决策边界可以概括为：

```
可读取组织级汇总；
可生成决策方案；
可推荐方案；
老板确认后才可触发重大任务下发。
```

8.5.1 可读取组织级汇总

Executive Agent 可以读取组织级数据和部门汇总数据。

可读取内容包括：

1. 各部门 Manager Schema。
2. 组织级 Summary。
3. 关键任务状态。
4. 关键异常记录。
5. 历史老板决策。
6. 与老板目标相关的数据。

7. 跨部门待决策事项。

Executive Agent 不应默认展示所有一线明细。它可以在有权限的情况下向下追溯来源，但老板侧默认展示应以摘要和决策项为主。

读取原则如下：

1. 默认读取汇总，不默认展开个人明细。
2. 决策需要时可追溯来源。
3. 敏感个人信息按权限过滤。
4. 所有读取行为写入审计日志。

8.5.2 可生成决策方案

Executive Agent 可以将 Manager 上推事项、组织级风险和历史任务状态整理成决策方案。

决策方案应包含：

1. 决策标题。
2. 决策背景。
3. 决策等级。
4. 影响范围。
5. 候选方案。
6. 每个方案的优缺点。
7. 预期影响。
8. 推荐方案。
9. 任务拆解草案。
10. 来源数据。

一个合格的老板决策项应该让老板能在短时间内判断，而不是让老板重新梳理上下文。

不推荐：

```
后端表结构有问题，请老板决策。
```

推荐：

```
事项：是否新增独立 decision_logs 表。
背景：Agent mock 流程已有 decision_logs，但正式 DDL 未单独建表。
方案 A：新增独立表，便于审计和复盘。
方案 B：暂时复用 audit_logs 和 tasks，开发更快但后续追踪弱。
推荐：方案 A。
```

8.5.3 可推荐方案

Executive Agent 可以基于当前数据推荐方案。

推荐时应说明依据，而不是只输出结论。推荐理由可以包括：

1. 哪个方案更符合当前 MVP 目标。
2. 哪个方案风险更低。
3. 哪个方案对后续扩展更友好。
4. 哪个方案对当前工期影响更小。
5. 哪个方案更符合老板历史决策偏好。
6. 哪个方案能减少跨部门协调成本。

推荐方案不是最终决策。老板可以接受、驳回、修改或要求补充信息。

如果信息不足，Executive Agent 应明确说明：

```
当前信息不足，暂不推荐单一方案。建议先补充后端工期评估和数据库改动影响。
```

不应为了显得智能而强行推荐。

8.5.4 不可绕过老板自动执行重大决策

Executive Agent 不得在 MVP 阶段绕过老板执行重大决策。

以下事项必须等待老板或指定高管确认：

1. 影响多个部门的任务调整。
2. 涉及预算、采购、成本支出的事项。
3. 涉及客户承诺或交付周期变化的事项。
4. 涉及组织结构或岗位调整的事项。
5. 涉及产品方向、技术路线或商业策略的事项。
6. 涉及外部合作、合同、法务或合规的事项。
7. 可能造成明显经营风险的事项。

Executive Agent 可以先生成任务草案，但只有确认状态变为 `confirmed` 后，后端才能创建正式任务。

状态必须清楚区分：

```
generated_tasks：任务草案
tasks：正式任务
```

任务草案不等于已下发任务。

8.6 决策等级定义

AutoMage-2 MVP 阶段建议将决策分为 L0 到 L4 五个等级。

等级越低，自动化程度越高；等级越高，人工确认要求越强。

8.6.1 L0：Agent 可自动处理

L0 是最低风险、最低影响范围的事项，可以由 Agent 自动处理。

典型事项包括：

1. 日报填写提醒。
2. 缺失字段追问。
3. 格式错误提示。
4. 任务截止提醒。
5. 日报提交后二次确认提示。
6. 低风险任务状态提醒。
7. 重复提交提示。
8. Schema 校验失败后的补充询问。

L0 事项不改变关键业务状态，不产生重大责任影响，因此不需要人工审批。

但 L0 操作仍应记录必要日志，便于排查 Agent 行为。

8.6.2 L1：员工确认后处理

L1 是影响员工个人数据或个人任务的事项，需要员工本人确认。

典型事项包括：

1. 提交个人日报。
2. 修改个人日报。
3. 更新个人任务状态。
4. 上报个人任务阻塞。
5. 提交个人产出物。
6. 确认个人明日计划。
7. 对 Staff Agent 整理内容进行确认。

L1 的关键原则是：员工个人数据必须由员工本人确认。Staff Agent 可以整理，但不能代替员工确认。

示例：

```
Staff Agent 整理出日报草稿后，员工点击“确认提交”，后端才写入正式 work_records。
```

8.6.3 L2：Manager 确认后处理

L2 是部门级事项，需要 Manager 或部门负责人确认。

典型事项包括：

1. 部门汇总确认。

2. 部门内任务调整。
3. 部门内异常处理。
4. 部门内资源协调。
5. 将事项上推老板。
6. 部门任务优先级调整。
7. 部门内员工协作安排。
8. 部门风险等级确认。

L2 事项影响部门运行，但一般不影响公司整体策略。Manager Agent 可以提出建议，Manager 确认后执行。

如果 L2 事项开始影响其他部门、预算、客户或关键交付，应升级为 L3。

8.6.4 L3：Executive 确认后处理

L3 是老板或高管级事项，需要 Executive 确认。

典型事项包括：

1. 跨部门资源协调。
2. 关键技术路线选择。
3. 重大产品方向调整。
4. 影响交付周期的调整。
5. 重要客户事项处理。
6. 预算或外部合作相关事项。
7. 需要老板拍板的组织级任务。
8. 多部门共同上推的风险处理。

L3 决策必须整理为 Executive Schema 中的 `decision_items`，并通过老板决策卡片确认。确认后，系统才能生成正式任务。

8.6.5 L4：需要人工会议/线下审批

L4 是超出 MVP 自动闭环范围的高风险事项，需要正式会议、线下审批或专门流程处理。

典型事项包括：

1. 公司战略方向重大调整。
2. 大额预算或采购。
3. 人事任免和组织架构变化。
4. 合同、法务和合规事项。
5. 涉及外部法律责任的承诺。
6. 重大客户危机。

7. 安全事故或数据泄露。
8. 资本、并购、融资等事项。

L4 事项中，Executive Agent 可以做三件事：

1. 整理背景。
2. 汇总来源数据。
3. 提醒需要线下会议或正式审批。

但它不能直接生成执行任务，也不能绕过正式流程。

8.7 决策上推规则

决策上推指下级节点发现事项超出自身权限后，将事项整理后提交给上级节点处理。

上推不是简单转发问题，而是把问题整理成上级能判断的结构化对象。

8.7.1 Staff 向 Manager 上推

Staff Agent 在以下情况下应向 Manager Agent 上推：

1. 员工选择 `need_support = true`。
2. 工作事项状态为 `blocked`。
3. 风险等级为 `high` 或 `critical`。
4. 员工明确请求上级介入。
5. 任务逾期且员工无法自行解决。
6. 需要部门内其他人协作。
7. 涉及部门资源调整。

上推内容应包含：

1. 员工 ID。
2. 所属部门。
3. 问题标题。
4. 问题描述。
5. 已尝试方案。
6. 需要支持的内容。
7. 来源 Staff Schema ID。
8. 建议下一步。

不推荐：

```
我这里有问题，需要领导看一下。
```

推荐：

事项：Decision 表结构未确认。
影响：影响 Executive 决策记录和任务下发联调。
已尝试：已在文档中列出最小字段草案。
需要支持：请后端确认是否新增独立 decision_logs 表。
来源：Staff Record 301。

8.7.2 Manager 向 Executive 上推

Manager Agent 在以下情况下应向 Executive Agent 上推：

1. 部门内无法解决。
2. 涉及跨部门资源。
3. 涉及老板目标或公司级优先级。
4. 影响关键交付。
5. 涉及预算或外部合作。
6. 多个部门出现同类风险。
7. 需要老板选择方案。
8. Manager 权限无法确认。

上推内容必须尽量整理成决策项，至少包含：

1. 决策标题。
2. 背景说明。
3. 影响范围。
4. 候选方案。
5. 推荐方案。
6. 推荐理由。
7. 来源部门汇总 ID。
8. 紧急程度。

Manager 上推质量会直接影响老板侧体验。上推内容越结构化，Executive Agent 生成决策卡片越稳定。

8.7.3 上推时机

上推时机分为两类：

类型	说明
实时上推	高风险、阻塞、紧急事项，不等到日报汇总
定时上推	普通待决策事项，进入每日 Manager 汇总后统一上推

MVP 阶段可以先采用定时上推为主，实时上推为辅。

建议规则：

1. `critical` 风险实时上推。
2. `high` 风险可在部门汇总时上推。
3. `medium` 风险优先由 Manager 处理。
4. `low` 风险只记录，不上推。

8.7.4 上推失败处理

如果上推失败，应根据失败原因处理。

失败原因	处理方式
Schema 校验失败	Agent 补全字段后重试
权限不足	停止上推，记录审计
上级节点不可用	暂存待上推队列，稍后重试
决策项缺少方案	返回 Manager Agent 补充
来源数据缺失	补充来源记录或标记来源不足

上推失败不应静默丢弃。系统必须保留失败记录。

8.8 决策下发规则

决策下发指上级确认后，将决策结果转化为任务、提醒或执行动作，下发给对应节点。

下发必须满足一个前提：对应决策已经达到所需确认等级。

8.8.1 Executive 向 Manager 下发

Executive 确认后的部门级任务，应下发给 Manager Agent。

典型场景包括：

1. 需要部门拆解的任务。
2. 涉及多个员工协作的任务。
3. 部门目标调整。
4. 部门风险处理。
5. 部门内任务重新分配。

下发内容应包括：

1. 决策来源。
2. 任务目标。
3. 截止时间。
4. 优先级。
5. 影响范围。
6. 是否需要进一步拆解。
7. 是否需要回传结果。

Manager Agent 收到后，可以继续拆解为 Staff 任务。

8.8.2 Manager 向 Staff 下发

Manager 确认后的执行任务，可以下发给 Staff Agent。

典型场景包括：

1. 员工具体执行任务。
2. 员工补充日报。
3. 员工处理阻塞事项。
4. 员工提交产出物。
5. 员工更新任务状态。
6. 员工执行老板决策拆解后的具体事项。

下发给 Staff 的任务必须足够具体，避免员工收到空泛指令。

不推荐：

```
明天把系统完善一下。
```

推荐：

```
任务：补充 decision_logs 表字段草案。
要求：字段需覆盖决策标题、背景、候选方案、推荐方案、确认人、确认时间和生成任务。
截止时间：明日 12:00。
```

8.8.3 下发前校验

任务下发前，后端应校验：

1. 决策是否已确认。
2. 当前节点是否有权下发任务。
3. 任务是否有明确来源。
4. 任务是否有负责人或负责人角色。
5. 任务优先级是否合法。
6. 截止时间是否合法。
7. 是否重复生成任务。
8. 是否涉及跨部门权限。
9. 是否需要二次确认。

没有来源的任务不应进入正式任务系统。

8.8.4 下发后的状态追踪

任务下发后，应进入任务状态流转。

建议状态如下：

```
待处理
  ↓
进行中
  ↓
阻塞 / 已完成
  ↓
确认关闭
```

任务状态变化应写入 `task_updates`。如果任务由决策产生，应保留 `source_decision_id` 或等价来源字段。

8.9 决策冲突处理

决策冲突指不同节点、不同时间或不同来源产生了相互矛盾的判断或任务。

MVP 阶段必须有基本冲突处理规则，避免多个 Agent 同时下发矛盾任务。

8.9.1 常见冲突类型

常见冲突包括：

冲突类型	示例
任务冲突	同一员工同一时间收到两个高优先级任务
权限冲突	Manager 下发的任务与老板确认的方向不一致
数据冲突	Staff 日报说任务完成，任务状态仍为进行中
风险判断冲突	Staff 标记 low，Manager 判断 high
决策重复	多个 Manager 上推同一事项
方案冲突	老板确认方案 A，但部门按方案 B 执行

8.9.2 冲突处理优先级

建议按照以下优先级处理冲突：

```
老板确认决策
  >
Manager 确认决策
  >
员工确认数据
  >
Agent 自动判断
```

也就是说，人类确认优先于 Agent 自动判断；高层确认优先于低层确认；正式数据库状态优先于聊天上下文。

示例：

如果 Staff Agent 认为风险为 low，但 Manager Agent 基于多条记录判断为 high，应以 Manager 确认后的风险等级为准，同时保留 Staff 原始记录不变。

8.9.3 冲突发现方式

冲突可以由系统规则或 Agent 发现。

发现方式包括：

1. 同一来源生成重复任务。
2. 同一员工任务时间冲突。
3. 同一事项存在多个未关闭决策。
4. 任务状态与日报反馈不一致。
5. 上级决策与下级执行方向不一致。
6. 多个部门上推同一个问题。

发现冲突后，应生成冲突提示或异常记录，而不是直接覆盖其中一方。

8.9.4 冲突处理流程

推荐流程如下：

```
发现冲突
  ↓
标记冲突类型
  ↓
暂停自动下发或自动关闭
  ↓
通知对应 Manager / Executive
  ↓
人工确认处理方案
  ↓
记录处理结果
  ↓
更新任务或决策状态
```

低风险冲突可以由 Manager 处理；高风险冲突应上推 Executive。

8.10 决策撤回机制

决策撤回指已经生成或确认的决策，在执行前或执行中被取消、修改或废止。

MVP 阶段必须支持基本撤回机制，因为真实业务中决策可能基于新信息发生变化。

8.10.1 可撤回范围

不同状态下的撤回规则不同：

状态	是否可撤回	说明
`draft`	可直接删除或修改	尚未推送
`pending`	可撤回	尚未确认
`confirmed`	可撤回但需记录原因	已确认但未生成任务
`task_generated`	可撤回但需同步任务	已生成任务
`in_execution`	谨慎撤回	需通知执行人并处理已完成部分
`completed`	不建议撤回	应通过复盘或新决策修正
`closed`	不可直接撤回	只能追加新记录

撤回不应物理删除原决策，而应更新状态并保留审计记录。

8.10.2 撤回权限

撤回权限应不低于原确认权限。

原决策等级	可撤回人
L1	员工本人或授权 Manager
L2	Manager 或更高层
L3	老板 / 高管 / 授权 Executive
L4	原审批流程或线下会议决定

Agent 不应自行撤回已确认决策，只能提出撤回建议。

8.10.3 撤回流程

推荐撤回流程如下：

```
发起撤回请求
  ↓
校验撤回权限
  ↓
填写撤回原因
  ↓
判断是否已有任务生成
  ↓
更新决策状态
  ↓
同步取消或调整相关任务
  ↓
通知相关人员
  ↓
写入审计日志
```

如果任务已经下发，需要同步处理任务状态：

1. 未开始任务：可取消。

2. 进行中任务：标记暂停或取消，并通知执行人。
3. 已完成任务：不直接删除，进入复盘或新决策调整。
4. 已产生外部影响的任务：必须人工处理。

8.10.4 决策修改与重新提交

如果不是完全撤回，而是修改方案，应生成新版本决策。

不建议覆盖原记录。推荐方式：

```
Decision v1: 已确认，后续撤回
Decision v2: 基于新信息重新生成并确认
```

这样可以保留决策演变过程，方便后续复盘。

8.11 决策审计机制

决策审计用于记录每个决策从生成、推送、确认、下发到执行的全过程。

AutoMage-2 的核心卖点之一是让组织运行可追溯。没有审计，老板无法知道任务从哪里来，员工也无法知道自己为什么收到某项任务。

8.11.1 审计对象

需要审计的对象包括：

1. Staff 上报的问题。
2. Manager 生成的部门决策。
3. Manager 上推的老板事项。
4. Executive 生成的决策项。
5. 老板确认动作。
6. 决策生成的任务。
7. 决策撤回和修改。
8. 决策冲突处理。
9. 权限失败请求。
10. Agent 自动判断结果。

8.11.2 审计事件

建议记录以下事件类型：

事件	说明
decision_generated	决策项生成
decision_pushed	决策项推送给确认人
decision_confirmed	决策被确认
decision_rejected	决策被驳回
decision_need_more_info	决策需要补充信息
decision_escalated	决策上推
decision_delegated	决策下发
decision_task_generated	决策生成任务
decision_revoked	决策撤回
decision_conflict_detected	决策冲突被发现
decision_conflict_resolved	冲突被处理
decision_permission_denied	决策权限不足

8.11.3 审计记录要求

每条审计记录至少应包含：

1. 操作人。
2. 操作 Agent。
3. 所属组织。
4. 目标对象类型。
5. 目标对象 ID。
6. 操作类型。
7. 操作时间。
8. 操作前状态。
9. 操作后状态。
10. 来源对象。
11. 请求渠道。
12. 关键 payload 摘要。

审计记录不一定要保存完整大文本，但必须保存足够信息用于追踪。

8.11.4 审计与原始数据关系

审计日志不替代业务表。它只记录“发生了什么操作”。

例如：

1. 决策正文应存在 Decision 表或 Summary 中。
2. 任务正文应存在 Tasks 表中。
3. 审计日志记录“谁在什么时候确认了该决策，并生成了哪些任务”。

不要把所有业务数据都塞进 `audit_logs.payload`，否则后续查询和复盘会困难。

8.12 决策日志字段定义

MVP 阶段建议尽快补充独立决策日志对象，用于承载决策的完整生命周期。若短期无法新增表，也应先在接口和 Schema 中按此结构设计，后续再落表。

8.12.1 决策日志主字段

建议字段如下：

字段名	类型	是否必填	说明
`decision_id`	string / number	是	决策 ID
`public_id`	string	否	对外展示 ID
`org_id`	number	是	组织 ID
`department_id`	number	否	关联部门
`decision_title`	string	是	决策标题
`decision_context`	string	是	决策背景
`decision_level`	string	是	决策等级，L0-L4
`source_type`	string	是	来源类型
`source_id`	number / string	是	来源对象 ID
`source_summary_ids`	array	否	来源汇总 ID
`source_record_ids`	array	否	来源 Staff 记录 ID
`options`	array	否	候选方案
`recommended_option`	string	否	推荐方案
`recommend_reason`	string	否	推荐理由
`expected_impact`	object	否	预期影响
`status`	string	是	决策状态
`confirm_required`	boolean	是	是否需要人工确认
`confirmed_by`	number	否	确认人
`confirmed_at`	string	否	确认时间
`confirmed_option`	string	否	最终方案
`generated_task_ids`	array	否	生成任务 ID
`created_by_node_id`	string	是	创建决策的 Agent 节点
`signature`	object	否	确认签名
`meta`	object	否	扩展字段

8.12.2 决策状态字段

`status` 建议使用以下枚举：

状态	含义
draft	草稿
pending	待确认
confirmed	已确认
rejected	已驳回
need_more_info	需要补充信息
task_generated	已生成任务
in_execution	执行中
completed	已完成
revoked	已撤回
cancelled	已取消
expired	已超时
closed	已关闭

状态变化必须写入审计日志。

8.12.3 候选方案字段

options 建议结构如下：

```
"options": [
  {
    "option_id": "A",
    "title": " 新增独立 decision_logs 表",
    "description": " 建立独立表记录老板决策项、候选方案、确认结果和后续任务。",
    "pros": [" 审计清晰", " 便于复盘", " 后续扩展空间大"],
    "cons": [" 需要增加后端开发工作量"],
    "estimated_cost": "medium",
    "estimated_time": "0.5-1 day"
  },
  {
    "option_id": "B",
    "title": " 暂时复用 audit_logs 和 tasks",
    "description": "MVP 阶段先不新增表，通过审计日志记录确认动作，通过任务表记录执行结果。",
    "pros": [" 实现更快", " 减少短期改动"],
    "cons": [" 长期追踪不清晰", " 后续可能需要迁移数据"],
    "estimated_cost": "low",
    "estimated_time": "0.2 day"
  }
]
```

8.12.4 签名字段

决策确认时建议记录签名信息：

```
"signature": {
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 30001,
  "signed_at": "2026-05-04T08:16:30+08:00",
  "payload_hash": "sha256:decision_payload_hash",
  "signature_source": "im"
}
```


签名用于证明该决策由谁确认、何时确认、确认时内容是什么。

8.12.5 决策日志 JSON 示例


```
{
  "decision_id": "decision_0001",
  "public_id": "dec_01HXAMPLE000000000001",
  "org_id": 1,
  "department_id": 12,
  "decision_title": " 是否新增独立 decision_logs 表",
  "decision_context": "当前 Agent mock 流程中已存在 decision_logs 概念，但正式 DDL
  ↪ 尚未建立独立决策表。如果不新增，短期可用 audit_logs 和 tasks 承载，
  ↪ 但后续老板决策追踪和复盘会不够清晰。",
  "decision_level": "L3",
  "source_type": "manager_summary",
  "source_id": 801,
  "source_summary_ids": [801, 802],
  "source_record_ids": [301, 302],
  "options": [
    {
      "option_id": "A",
      "title": " 新增独立 decision_logs 表",
      "description": " 建立独立表记录老板决策项、候选方案、确认结果和后续任务。",
      "pros": [" 审计清晰", " 便于复盘", " 后续扩展空间大"],
      "cons": [" 需要增加后端开发工作量"]
    },
    {
      "option_id": "B",
      "title": " 暂时复用 audit_logs 和 tasks",
      "description": "MVP 阶段先不新增表，通过审计日志记录确认动作，通过任务表记录执行结果。",
      "pros": [" 实现更快", " 减少短期改动"],
      "cons": [" 长期追踪不清晰", " 后续可能需要迁移数据"]
    }
  ],
  "recommended_option": "A",
  "recommend_reason": "AutoMage-2 的核心价值在于决策可追溯，独立决策对象有利于后续任务生成、
  ↪ 复盘和老板侧看板。",
  "expected_impact": {
```

```
    "cost": "medium",
    "timeline": " 短期增加 0.5-1 天后端工作量",
    "risk": " 降低后续决策追踪混乱风险",
    "affected_modules": ["backend", "agent", "database"]
  },
  "status": "confirmed",
  "confirm_required": true,
  "confirmed_by": 30001,
  "confirmed_at": "2026-05-04T08:16:30+08:00",
  "confirmed_option": "A",
  "generated_task_ids": [9001],
  "created_by_node_id": "executive_node_org_1",
  "signature": {
    "signature_required": true,
    "signature_status": "signed",
    "signed_by": 30001,
    "signed_at": "2026-05-04T08:16:30+08:00",
    "payload_hash": "sha256:decision_payload_hash",
    "signature_source": "im"
  },
  "meta": {
    "push_channel": "feishu",
    "agent_template_version": "executive_agent_v0.1"
  }
}
```

8.13 本章小结

三级节点决策机制是 AutoMage-2 从“日报汇总系统”走向“组织运行系统”的关键。

Staff Agent 负责个人数据记录和问题上报；Manager Agent 负责部门汇总、异常处理和权限内决策；Executive Agent 负责老板侧决策整理和任务拆解。三者之间通过明确的决策等级和上推下发规则协作，避免所有事项都压到老板，也避免 Agent 越权处理重大事项。

MVP 阶段最重要的不是让 AI 自动决策，而是让决策过程变得清楚：

1. 问题从哪里来。
2. 谁整理了问题。
3. 谁有权限处理。
4. 为什么要上推。
5. 老板确认了什么。
6. 确认后生成了哪些任务。
7. 后续执行结果如何回流。

只要这套机制跑通，AutoMage-2 就具备了最基本的组织级数据闭环和决策闭环。

9 Dream 机制设计

9.1 Dream 的定义

Dream 是 AutoMage-2 中用于周期性整理组织运行数据、生成风险判断、形成老板决策项和次日任务建议的智能分析机制。

它可以理解为系统每天固定时间做的一次“组织级复盘和预推演”。Staff Agent 负责记录一线员工当天做了什么，Manager Agent 负责汇总部门发生了什么，而 Dream 负责站在组织层面判断：这些信息加在一起意味着什么，明天哪些事情最重要，哪些风险需要提前处理，哪些事项需要老板确认。

Dream 不是普通日报生成器，也不是简单的总结 Prompt。它的输入来自系统中已经结构化的数据，包括 Staff Schema、Manager Schema、历史 Summary、历史 Decision、未完成任务、异常记录和外部业务目标。它的输出也必须是结构化结果，包括老板决策项、风险预警、任务拆解、次日重点和部门调整建议。

MVP 阶段的 Dream 不需要实现完整的长期记忆和复杂战略推演，但至少要完成以下能力：

1. 定时读取已确认的部门汇总数据。
2. 识别组织层面的关键风险。
3. 将上推事项整理成老板可确认的决策项。
4. 为每个决策项生成可选方案和推荐理由。
5. 根据确认结果生成任务草案。
6. 将结果写入 Executive Schema 或相关 Summary。

简单来说，Dream 是 AutoMage-2 从“数据汇总”走向“决策驱动”的中间层。

9.2 Dream 在 AutoMage-2 中的作用

Dream 在 AutoMage-2 中承担五个作用。

第一，它负责把多部门汇总变成老板能看懂的组织摘要。Manager Schema 中包含大量部门级信息，如果全部推给老板，老板仍然需要自己筛选。Dream 需要把这些信息进一步压缩成公司级摘要，只保留关键进展、关键风险和关键决策点。

第二，它负责识别跨部门风险。单个 Manager Agent 只能看到本部门数据，但很多风险不是在单个部门内暴露出来的。例如后端表结构未确认，可能同时影响 Agent 客制化、前端联调和老板侧决策卡片。Dream 需要从多个部门汇总中识别这种共同风险。

第三，它负责生成老板决策项。老板不需要看到“有问题，请处理”这种开放式描述，而是需要看到背景、方案、推荐理由和确认动作。Dream 要把模糊问题整理成可选择、可确认、可追踪的决策项。

第四，它负责生成任务拆解草案。决策如果不能落到任务，就无法形成闭环。Dream 需要在决策项中预先生成任务草案，等老板确认后由后端创建正式任务。

第五，它负责形成次日组织运行重点。Dream 应根据当天数据和历史状态，给出第二天的重点事项、部门调整建议和需要关注的风险，为第二天的 Staff Agent 和 Manager Agent 提供输入。

Dream 在系统中的位置如下：

```
Staff Schema
  ↓
Manager Schema
  ↓
Dream
  ↓
Executive Schema
  ↓
老板确认
  ↓
Task Schema
  ↓
Staff / Manager 执行
```

没有 Dream，Executive Agent 容易退化成一个部门日报转发器；有了 Dream，系统才开始具备组织级判断和主动推进能力。

9.3 Dream 的每日运行时间

MVP 阶段建议 Dream 采用定时运行机制，不依赖实时长连接，也不要求全天候频繁推理。

推荐每日运行节奏如下：

时间	动作
18:00	Staff Agent 提醒员工填写日报
20:00	Staff Agent 二次催填缺失日报
21:00	Manager Agent 读取 Staff Schema，生成部门汇总
21:30	部门负责人确认或标记部门汇总
22:00	Dream 读取 Manager Schema，生成夜间组织摘要草稿
次日 08:00 前	Dream 生成老板决策卡片和次日重点
次日 08:30 前	老板确认后，任务下发到 Manager / Staff

MVP 阶段可以先采用一个固定 Dream 批处理任务，例如每天晚上 22:00 或次日早上 08:00 运行。两种方式各有优劣：

运行时间	优点	缺点
前一晚运行	第二天早上可直接推送老板决策卡片	部分部门汇总可能尚未确认
次日早上运行	数据更完整，更适合早会前推送	对运行稳定性和时间要求更高
晚上初稿 + 早上复核	兼顾完整性和及时性	实现稍复杂

推荐 MVP 阶段先采用：

晚上生成草稿，早上生成正式老板卡片。

如果开发时间有限，也可以先只做早上定时运行。关键是保证 Dream 的输入数据已经完成后端校验和权限确认，不要读取员工未确认草稿作为老板决策依据。

9.4 Dream 输入数据

Dream 的输入必须来自系统事实源。IM 聊天记录、Agent 临时上下文、未确认草稿都不能作为正式输入。

MVP 阶段 Dream 主要读取以下六类数据：

1. Staff Schema。
2. Manager Schema。
3. 历史 Summary。
4. 历史 Decision。
5. 未完成任务。
6. 外部业务目标。

其中，Manager Schema 是 Dream 的主输入，其他数据用于补充上下文和判断趋势。

9.4.1 Staff Schema

Staff Schema 是一线执行数据，记录员工当天完成事项、遇到问题、支持需求、明日计划和产出物。

Dream 一般不直接读取全部 Staff Schema 明细，而是通过 Manager Schema 间接获取一线情况。只有在以下场景下，Dream 才需要向下追溯 Staff Schema：

1. 某个老板决策项需要核对来源。
2. Manager Schema 中的风险判断依据不足。
3. 某个员工产出物是关键决策依据。
4. 老板主动要求查看某个事项来源。
5. 系统需要判断某个问题是否多次出现。

读取 Staff Schema 时必须遵守权限规则。Dream 可以追溯来源，但不应默认把所有员工明细展示给老板。

推荐读取字段包括：

字段	用途
`work_progress`	判断实际完成事项
`issues_faced`	识别一线问题
`need_support`	识别需要上级介入的事项
`support_detail`	理解支持需求
`next_day_plan`	判断次日计划
`risk_level`	判断风险等级
`artifacts`	查看关键产出物
`signature`	判断数据是否已确认

未签名、未确认、校验失败的 Staff Schema 不应进入 Dream 正式判断。

9.4.2 Manager Schema

Manager Schema 是 Dream 的核心输入。

Dream 主要读取各部门 Manager Schema，并基于这些汇总生成组织级摘要、关键风险和老板决策项。

推荐读取字段包括：

Manager Schema 字段	Dream 使用方式
`aggregated_summary`	生成公司级业务摘要
`overall_health`	判断部门状态
`top_3_risks`	提炼组织级风险
`blocked_items`	识别阻塞事项
`need_executive_decision`	生成老板决策项
`manager_decisions`	判断部门内已处理事项
`next_day_adjustment`	生成次日重点
`staff_report_count`	判断数据覆盖情况
`missing_report_count`	判断数据完整性风险
`source_record_ids`	追溯来源
`signature`	判断是否已由 Manager 确认

Manager Schema 进入 Dream 前，应满足以下条件：

1. 属于当前组织。
2. 汇总日期在本次运行窗口内。
3. 状态有效。
4. 数据来源可追溯。
5. 必要时已由部门负责人确认。
6. 当前 Executive Agent 有读取权限。

如果某个部门没有提交 Manager Schema，Dream 应在结果中标记数据缺失，而不是假装该部门正常。

9.4.3 历史 Summary

历史 Summary 用于让 Dream 判断趋势，而不是只看当天快照。

例如，某个部门今天出现一个中风险问题，如果昨天、前天也出现类似问题，那么它可能不是偶发问题，而是结构性风险。没有历史 Summary，Dream 很难判断“今天的问题是否正在变严重”。

推荐读取的历史 Summary 包括：

1. 前一日组织级 Summary。
2. 前一周部门 Summary。
3. 最近几天的风险摘要。

4. 最近几天的任务完成摘要。
5. 历史 Dream 输出结果。
6. 关键项目或部门的周期性汇总。

历史 Summary 的用途包括：

用途	示例
趋势判断	某个风险连续三天出现
复盘对比	昨日决策是否改善了今日情况
避免重复提醒	昨天已提醒老板，今天只更新状态
识别长期阻塞	同一问题多日未解决
生成次日重点	根据历史未完成事项排序

MVP 阶段可以先读取最近 3-7 天的 Summary，不需要做长期全量扫描。

9.4.4 历史 Decision

历史 Decision 用于判断当前决策是否与过去老板确认过的方向一致。

如果系统每天都生成新的决策项，但不记得昨天老板已经选过什么，就会反复打扰老板，甚至生成相互冲突的任务。

Dream 应读取以下历史决策信息：

1. 最近已确认的老板决策。
2. 尚未完成的决策。
3. 已驳回的决策。
4. 要求补充信息的决策。
5. 已生成任务的决策。
6. 已撤回或取消的决策。

推荐读取字段包括：

字段	用途
`decision_title`	判断是否重复
`decision_context`	理解决策背景
`confirmed_option`	判断老板已选择方向
`confirmed_at`	判断决策时间
`generated_task_ids`	追踪执行结果
`status`	判断是否完成
`source_summary_ids`	判断来源关系
`expected_impact`	判断后续影响

历史 Decision 的作用主要有三点：

1. 避免重复生成相同决策。
2. 判断新决策是否与旧决策冲突。
3. 跟踪老板决策是否真正产生执行结果。

MVP 阶段如果独立 Decision 表尚未建立，也应至少从 `audit_logs`、`summaries.meta` 或 `tasks.meta` 中读取相关记录。后续建议将 Decision 做成独立对象。

9.4.5 未完成任务

未完成任务是 Dream 判断执行风险的重要输入。

老板已经确认过的决策，如果没有对应任务推进，或者任务长期停留在阻塞状态，Dream 应主动提示。

推荐读取任务范围包括：

1. 高优先级未完成任务。
2. 已逾期任务。
3. 与老板决策相关的任务。
4. 与当前风险相关的任务。
5. 被多次更新为阻塞的任务。
6. 即将到期的重要任务。

推荐读取字段包括：

字段	用途
`title`	任务名称
`status`	当前状态
`priority`	优先级
`due_at`	截止时间
`department_id`	所属部门
`assignee_user_id`	执行人
`source_record_id`	来源日报
`source_decision_id`	来源决策
`result_summary`	任务结果
`task_updates`	判断推进过程

未完成任务进入 Dream 后，可以产生三种结果：

1. 生成风险预警。
2. 生成老板决策项。
3. 生成次日重点或部门调整建议。

9.4.6 外部业务目标

外部业务目标指老板或企业给系统输入的阶段性目标、经营指标、项目目标或临时要求。

例如：

1. 本周必须完成 MVP 联调。
2. 两周内跑通三级 Agent 闭环。
3. 优先保证老板侧决策卡片可演示。

4. 本月重点是媒体演示和试点客户。
5. 某个客户项目必须优先交付。
6. 某项预算不能超过上限。

这些目标不一定来自 Staff 或 Manager Schema，但会影响 Dream 对风险和任务优先级的判断。

MVP 阶段可以先用简单配置方式输入外部目标，例如：

```
{
  "business_goals": [
    {
      "goal_id": "goal_mvp_001",
      "title": " 两周内跑通 AutoMage-2 MVP 三级数据闭环",
      "priority": "high",
      "deadline": "2026-05-15",
      "owner": "project_team"
    }
  ]
}
```

Dream 生成决策和任务时，应参考这些目标。例如，如果 MVP 主链路是最高优先级，那么影响主链路的表结构、API、Schema 和 Agent 联调问题，应优先进入老板侧摘要。

9.5 Dream 输出数据

Dream 的输出不是一段普通总结，而是一组可被 Executive Agent、老板、任务系统和审计系统继续使用的结构化结果。

MVP 阶段 Dream 主要输出五类数据：

1. 老板决策项。
2. 风险预警。
3. 任务拆解。
4. 次日重点。
5. 部门调整建议。

这些输出最终会进入 Executive Schema、Summary、Decision 或 Task。

9.5.1 老板决策项

老板决策项是 Dream 最重要的输出。

一个合格的老板决策项应包含：

1. 决策标题。
2. 背景说明。
3. 影响范围。

4. 决策等级。
5. 候选方案。
6. 推荐方案。
7. 推荐理由。
8. 预期影响。
9. 来源数据。
10. 确认后可生成的任务。

示例：

```
{
  "decision_title": " 是否新增独立 decision_logs 表",
  "context": "当前 Agent mock 流程中已有 decision_logs 概念，但正式 DDL 尚未建立独立决策表。
  ↪ 如果不新增，短期可以复用 audit_logs，但后续老板决策追踪和任务来源会不清晰。",
  "decision_level": "L3",
  "options": [
    {
      "option_id": "A",
      "title": " 新增独立 decision_logs 表",
      "pros": [" 决策对象清晰", " 方便审计", " 便于后续看板和复盘"],
      "cons": [" 需要增加后端开发工作量"]
    },
    {
      "option_id": "B",
      "title": " 暂时复用 audit_logs 和 tasks",
      "pros": [" 短期实现更快"],
      "cons": [" 长期追踪不清晰", " 后续可能需要迁移"]
    }
  ],
  "recommended_option": "A",
  "reasoning_summary": "AutoMage-2 的核心价值在于决策可追溯，独立决策对象更符合长期架构。",
  "source_summary_ids": [801, 802]
}
```

老板决策项不应过多。MVP 阶段建议每天只推送 1-3 个真正重要的决策事项，避免老板侧再次信息过载。

9.5.2 风险预警

风险预警用于提醒老板或 Manager 当前最需要关注的问题。

Dream 的风险预警应与普通部门风险不同。部门风险可以很多，但组织级风险必须经过筛选，只保留影响较大、跨部门、影响主链路或需要高层干预的事项。

风险预警建议包含：

1. 风险标题。

2. 风险说明。
3. 风险等级。
4. 影响范围。
5. 来源部门。
6. 来源 Summary。
7. 建议动作。
8. 是否需要决策。

示例：

```
{
  "risk_title": "MVP 主链路存在决策对象缺口",
  "description": "Staff、Manager、Executive 三层 Schema 已初步定义，但正式 DDL 中尚未建立独立
  ↪  Decision 表，可能影响老板确认、任务来源追踪和审计闭环。",
  "severity": "high",
  "affected_modules": ["database", "backend", "executive_agent"],
  "source_summary_ids": [801],
  "suggested_action": " 建议将 Decision 表是否新增作为老板决策项处理。",
  "decision_required": true
}
```

风险预警的目标不是制造焦虑，而是帮助上级提前看到会影响闭环的关键问题。

9.5.3 任务拆解

任务拆解是 Dream 将决策落地的关键输出。

Dream 不直接创建正式任务，而是生成任务草案。老板确认方案后，后端再根据任务草案创建正式任务。

任务拆解建议包含：

1. 任务标题。
2. 任务说明。
3. 来源决策。
4. 执行人或执行角色。
5. 所属部门。
6. 优先级。
7. 截止时间。
8. 预期产出。
9. 依赖关系。
10. 是否需要 Manager 二次拆解。

示例：

```
{
  "task_title": " 补充 decision_logs 表结构草案",
  "task_description": "根据 Executive 决策链路设计 decision_logs 表，字段需覆盖决策标题、背景、
  ↪ 候选方案、推荐方案、确认人、确认时间和生成任务。",
  "assignee_role": "backend",
  "department_id": 12,
  "priority": "high",
  "due_at": "2026-05-04T18:00:00+08:00",
  "expected_output": " 提交 decision_logs 表 DDL 草案及与 Executive Schema 的映射说明",
  "source_decision_id": "decision_tmp_001"
}
```

任务草案要尽量具体，避免出现“推进项目”“完善系统”这类无法执行、无法验收的描述。

9.5.4 次日重点

次日重点用于指导第二天 Staff Agent 和 Manager Agent 的工作提醒。

它不是正式任务列表，而是组织或部门层面的重点方向。部分次日重点可以在 Manager 或老板确认后转成任务。

次日重点建议包含：

1. 重点事项。
2. 所属部门。
3. 相关负责人。
4. 优先级。
5. 预期产出。
6. 来源原因。
7. 是否建议转任务。

示例：

```
{
  "next_day_focus": [
    {
      "title": " 优先完成 Decision Schema 与数据库映射",
      "department_id": 12,
      "priority": "high",
      "reason": " 该事项影响 Executive 决策卡片和任务下发闭环。",
      "expected_output": " 形成 Decision 字段定义和落表建议",
      "suggest_task_creation": true
    },
    {
      "title": " 补齐缺失日报",
      "department_id": 15,
```

```
      "priority": "medium",
      "reason": " 该部门昨日缺失 3 份 Staff 日报，影响部门汇总可信度。",
      "expected_output": " 完成缺失日报补交或说明",
      "suggest_task_creation": false
    }
  ]
}
```

次日重点可以用于第二天早上推送给 Manager，也可以作为 Staff Agent 任务提醒的输入。

9.5.5 部门调整建议

部门调整建议面向 Manager 或老板，用于提示某个部门第二天或下一周期需要调整的方向。

它和次日重点的区别是：次日重点偏具体事项，部门调整建议偏管理动作。

部门调整建议可以包括：

1. 任务优先级调整。
2. 人员协作调整。
3. 风险处理建议。
4. 缺失日报催办。
5. 跨部门协作建议。
6. 资源支持建议。
7. 阻塞事项处理建议。

示例：

```
{
  "department_adjustments": [
    {
      "department_id": 12,
      "department_name": " 架构组",
      "adjustment_title": " 将决策表结构确认提升为最高优先级",
      "reason": " 该事项影响后端、Agent 和老板侧卡片联调，是当前 MVP 主链路阻塞点。",
      "suggested_action": " 由后端负责人在今日内给出是否新增 decision_logs 表的最终方案。",
      "priority": "high",
      "need_manager_confirm": true
    }
  ]
}
```

部门调整建议不一定直接上推老板。Manager 权限内能处理的，应优先交给 Manager 处理；超出权限的，再进入 Executive 决策项。

9.6 Dream 运行流程

Dream 的运行流程建议分为九步：

```
触发运行
  ↓
读取数据
  ↓
过滤无效数据
  ↓
归并同类事项
  ↓
识别风险
  ↓
生成决策项
  ↓
生成任务草案
  ↓
写入 Executive Schema / Summary
  ↓
推送老板或 Manager
```

9.6.1 触发运行

Dream 可以由定时任务、Manager 上推或老板主动查询触发。

MVP 阶段优先支持定时任务：

```
每天 08:00 生成老板决策卡片
```

如果存在 critical 风险，也可以支持即时触发：

```
Manager Agent 上推 critical 事项
  → 立即触发 Dream 局部分析
  → 生成临时老板决策卡片
```

9.6.2 读取数据

Dream 根据运行窗口读取数据。

常见窗口包括：

1. 昨日所有 Manager Schema。
2. 最近 3 天未关闭风险。
3. 最近 7 天未完成任务。
4. 最近 7 天历史决策。
5. 当前有效外部目标。

读取时必须通过后端 API，不允许直接绕过权限访问数据库。

9.6.3 过滤无效数据

Dream 应过滤以下数据：

1. 草稿状态 Summary。
2. 未确认且需要确认的高风险汇总。
3. 已删除数据。
4. 权限不匹配数据。
5. 重复数据。
6. 明显校验失败数据。
7. 已关闭且无复盘价值的低风险事项。

如果某部门数据缺失，Dream 应在结果中说明，而不是默认该部门没有问题。

9.6.4 归并同类事项

多个部门可能上报同类问题，Dream 应进行归并。

例如：

1. 后端说 Decision 表未确认。
2. Agent 客制化说 decision_logs 接口未定。
3. Executive Schema 说决策确认无法落表。

这三条可能本质上是同一个问题：决策对象缺口。

归并后，Dream 应生成一个组织级风险或老板决策项，而不是重复推送三个相似问题。

9.6.5 识别风险

Dream 根据 Manager Schema、历史任务和历史决策识别风险。

风险判断应优先关注：

1. 是否影响 MVP 主链路。
2. 是否跨部门。
3. 是否多日未解决。
4. 是否影响老板决策。
5. 是否影响任务下发。
6. 是否影响客户或交付。
7. 是否需要高层资源。

低风险事项不应进入老板决策卡片，避免打扰。

9.6.6 生成决策项

对于需要老板确认的事项，Dream 应生成结构化决策项。

每个决策项至少包含：

1. 标题。
2. 背景。
3. 影响。
4. 方案 A。
5. 方案 B。
6. 推荐方案。
7. 推荐理由。
8. 来源 Summary。
9. 确认后任务草案。

如果信息不足，Dream 不应硬生成决策项，而应生成“需要补充信息”的请求。

9.6.7 生成任务草案

Dream 为每个推荐方案生成任务草案。

任务草案不能直接写入正式任务表，除非对应决策不需要人工确认，或者老板已经确认。

任务草案应随 Executive Schema 一起展示，帮助老板理解“确认后会发生什么”。

9.6.8 写入结果

Dream 输出应写入以下对象：

输出	写入对象
组织级摘要	`summaries`
老板决策项	Decision 对象 / Executive Schema
风险预警	`summaries.meta` / `incidents`
任务草案	Executive Schema / Decision meta
审计记录	`audit_logs`

MVP 阶段如果 Decision 表尚未建立，可以先将决策项写入 Executive Schema 的 `decision_items` 中，并在 `audit_logs` 记录生成动作。

9.6.9 推送结果

Dream 结果根据类型推送给不同对象：

结果类型	推送对象
老板决策项	老板 / Executive Agent
高风险预警	老板 + 相关 Manager
部门调整建议	对应 Manager
次日重点	Manager / Staff Agent
任务草案	暂不推员工，等待确认

老板侧推送应尽量少而精。Manager 侧可以更细一些，但也不能把所有明细原样推送。

9.7 Dream Prompt 结构

Dream Prompt 需要保持稳定结构，避免每次输出格式不一致。MVP 阶段建议将 Dream Prompt 拆为六个部分。

9.7.1 角色定义

明确 Dream 当前扮演的角色。

示例：

```
你是 AutoMage-2 的组织级 Dream 分析节点。
你的任务不是写普通日报，而是基于各部门 Manager Schema，生成老板侧摘要、关键风险、
↪ 待决策事项和任务草案。
你不能编造不存在的数据，所有关键判断必须引用来源 Summary。
```

9.7.2 输入说明

列出本次输入数据范围。

示例：

```
本次输入包括：
1. 2026-05-03 各部门 Manager Schema。
2. 最近 7 天未完成任务。
3. 最近 7 天历史决策。
4. 当前 MVP 业务目标。
```

这样可以减少 Dream 混用上下文或引用无关历史信息。

9.7.3 输出目标

明确本次 Dream 需要输出什么。

示例：

```
请输出以下内容：
1. 公司级业务摘要。
2. 关键风险列表。
3. 老板待决策事项。
4. 每个决策项的 A/B 方案。
5. 推荐方案和理由。
6. 老板确认后可生成的任务草案。
7. 次日重点和部门调整建议。
```

9.7.4 约束规则

Dream 必须遵守约束规则。

建议约束包括：

1. 不得编造输入中不存在的事实。
2. 不得把未确认草稿作为正式依据。
3. 不得生成没有来源的高风险判断。
4. 不得直接执行重大决策。
5. 不得把低价值事项推给老板。
6. 每个决策项必须有来源 Summary。
7. 每个任务草案必须有来源决策。
8. 不确定时应标记需要补充信息。

9.7.5 输出格式

Dream 输出必须符合结构化格式，建议直接输出 Executive Schema 所需字段。

推荐输出结构：

```
{
  "business_summary": "...",
  "key_risks": [],
  "decision_required": true,
  "decision_items": [],
  "next_day_focus": [],
  "department_adjustments": [],
  "source_summary_ids": []
}
```

不要输出大段无法解析的 Markdown 作为正式结果。Markdown 可以用于 IM 展示，但系统保存应使用结构化 JSON。

9.7.6 自检要求

Dream 输出前应进行自检。

自检问题包括：

1. 是否引用了来源 Summary？
2. 是否把低风险问题错误上推给老板？
3. 是否有决策项缺少方案？
4. 是否有推荐方案但没有理由？
5. 是否生成了没有来源的任务草案？
6. 是否把未确认数据当作事实？
7. 是否存在和历史决策冲突的内容？

自检不需要展示给老板，但可以作为 Agent 内部校验步骤。

9.8 Dream 结果校验

Dream 结果必须经过校验后才能进入正式流程。校验分为结构校验、来源校验、权限校验、业务校验和人工确认校验。

9.8.1 结构校验

结构校验检查 Dream 输出是否符合 Executive Schema 或指定输出 Schema。

校验项包括：

1. 必填字段是否存在。
2. 字段类型是否正确。
3. 枚举值是否合法。
4. `decision_required = true` 时，`decision_items` 是否非空。
5. 每个决策项是否包含候选方案。
6. 每个任务草案是否包含标题、负责人或角色、优先级和来源。
7. `source_summary_ids` 是否存在。

结构校验失败时，不应推送老板，应返回 Dream 重新生成或进入人工检查。

9.8.2 来源校验

来源校验检查 Dream 判断是否有数据依据。

校验项包括：

1. `source_summary_ids` 是否真实存在。
2. 引用 Summary 是否属于当前组织。

3. 引用 Summary 是否在当前时间窗口内。
4. 引用 Summary 是否状态有效。
5. 高风险判断是否能追溯到 Manager Schema。
6. 决策项是否来自 Manager 上推或风险识别。
   没有来源的决策项不应进入老板卡片。

9.8.3 权限校验

权限校验检查当前 Dream / Executive Agent 是否有权读取输入数据和生成输出结果。
校验项包括：

1. 当前 Agent 是否属于该组织。
2. 当前 Agent 是否具备 Executive 权限。
3. 引用的 Manager Summary 是否在授权范围内。
4. 是否读取了未经授权的员工明细。
5. 是否试图生成超权限执行任务。
   权限失败时，应拒绝生成正式结果，并写入审计日志。

9.8.4 业务校验

业务校验检查 Dream 输出是否符合业务规则。
重点校验：

1. 高风险事项是否错误降级。
2. 低风险事项是否过度上推。
3. 决策项是否缺少明确问题。
4. 任务草案是否过于空泛。
5. 任务截止时间是否合理。
6. 决策项是否和历史已确认决策冲突。
7. 是否重复生成已有未完成决策。
8. 是否将任务草案当作正式任务。
   业务校验可以先由规则 + 人工抽查完成，后续再逐步自动化。

9.8.5 人工确认校验

涉及老板级决策或重大任务下发时，必须校验人工确认状态。
规则如下：

1. `human_confirm_status != confirmed` 时，不创建正式任务。
2. `confirmed_by` 为空时，不视为已确认。

3. `confirmed_at` 为空时，不视为已确认。
4. 确认人无权限时，确认无效。
5. 确认后任务生成必须写入审计日志。

Dream 可以生成建议，但不能替代老板确认。

9.9 Dream 与 Executive Schema 的关系

Dream 是 Executive Schema 的生成逻辑，Executive Schema 是 Dream 的结构化输出。二者关系如下：

```
Dream 读取输入数据
  ↓
Dream 生成组织级分析
  ↓
输出 schema_v1_executive
  ↓
Executive Agent 推送老板决策卡片
```

Dream 输出与 Executive Schema 字段的映射如下：

Dream 输出	Executive Schema 字段
组织级摘要	`business_summary`
关键风险	`key_risks`
是否需要老板确认	`decision_required`
老板决策项	`decision_items`
推荐方案	`recommended_option`
推荐理由	`reasoning_summary`
预期影响	`expected_impact`
任务草案	`generated_tasks`
来源部门汇总	`source_summary_ids`
人工确认状态	`human_confirm_status`

Dream 不应绕开 Executive Schema 直接给老板发一段自由文本。老板侧展示可以是卡片或自然语言，但底层数据必须先进入 Executive Schema，这样后续才能确认、写库、生成任务和审计。

MVP 阶段建议将 Dream 的正式输出统一设为：

```
schema_v1_executive
```

如果后续需要更复杂的 Dream 运行记录，可以新增 `dream_runs` 表记录输入、输出、状态和错误信息。

9.10 Dream 与任务生成的关系

Dream 可以生成任务草案，但不能直接生成正式任务，除非该任务属于低风险自动处理范围。

MVP 阶段的原则是：

```
Dream 生成任务草案；
老板或 Manager 确认后；
后端创建正式任务。
```

9.10.1 任务草案生成

Dream 在以下情况下可以生成任务草案：

1. 老板决策项确认后必然需要执行。
2. 某个风险需要明确负责人处理。
3. 某个部门需要调整次日重点。
4. 某个历史任务长期阻塞，需要补救任务。
5. 某个缺失数据需要催办。

任务草案应包含：

1. 标题。
2. 描述。
3. 所属部门。
4. 负责人或负责人角色。
5. 优先级。
6. 截止时间。
7. 来源决策或来源风险。
8. 预期产出。

9.10.2 正式任务生成

正式任务只能由后端在确认后创建。

创建条件包括：

1. 决策状态为 `confirmed`。
2. 确认人具备权限。
3. 任务草案通过校验。
4. 任务来源明确。
5. 没有重复任务。

6. 任务负责人或负责人角色明确。

创建后应写入：

1. `tasks`。
2. `task_assignments`。
3. `task_updates`。
4. `audit_logs`。

9.10.3 任务分发

任务创建后，根据任务类型分发：

任务类型	分发对象
具体执行任务	Staff Agent
部门拆解任务	Manager Agent
跨部门任务	多个 Manager Agent
老板追踪任务	Executive Agent 保留跟踪
数据补全任务	对应 Staff / Manager Agent

任务分发不应只停留在 IM 通知。正式任务状态必须保存在数据库中。

9.10.4 任务结果回流

任务执行结果最终应回流到 Staff Schema 或 Manager Schema。

例如：

```
老板确认任务
  ↓
任务下发给员工
  ↓
员工执行
  ↓
Staff Agent 在日报中记录任务进展
  ↓
Manager Agent 汇总任务结果
  ↓
Dream 次日判断该决策是否有效
```

这一步是 Dream 闭环的关键。否则 Dream 只会不断生成新建议，却无法判断旧建议是否被执行。

9.11 Dream 失败时的降级策略

Dream 失败不能导致整个系统停止运行。MVP 阶段必须设计降级策略，保证即使 Dream 没有成功生成，员工日报、部门汇总和任务系统仍然可以继续工作。

9.11.1 常见失败类型

Dream 失败可能来自以下原因：

失败类型	示例
输入缺失	部门未生成 Manager Schema
数据质量差	大量员工日报缺失或未确认
Schema 校验失败	Dream 输出不符合 Executive Schema
模型调用失败	LLM 超时或返回异常
权限失败	Executive Agent 无权读取某些汇总
业务冲突	生成的决策与历史决策冲突
任务草案不合格	缺少负责人、截止时间或来源
推送失败	IM 卡片发送失败

9.11.2 降级策略一：生成简版摘要

如果完整 Dream 失败，但 Manager Schema 可读，系统可以生成简版摘要。

简版摘要只包含：

1. 各部门健康度。
2. 关键风险列表。
3. 缺失数据说明。
4. 暂无决策项提示。

示例：

```
今日 Dream 完整分析未完成，已生成简版摘要：
架构组：yellow，主要风险为 Decision 表结构未确认。
后端组：green，暂无高风险。
客制化组：yellow，存在接口契约待确认问题。
```

简版摘要不生成任务，只用于老板了解情况。

9.11.3 降级策略二：只转发 Manager 上推事项

如果 Dream 无法完成综合分析，但 Manager Schema 中已有 `need_executive_decision`，系统可以先将这些事项转成基础老板卡片。

此时不做复杂推荐，只展示：

1. 上推部门。
2. 决策标题。
3. 背景说明。
4. Manager 推荐方案。
5. 来源 Summary。

这样可以保证紧急决策不被 Dream 失败阻塞。

9.11.4 降级策略三：延迟重试

如果失败原因是模型超时、接口异常或 IM 推送失败，可以延迟重试。

建议重试策略：

次数	间隔
第 1 次	1 分钟
第 2 次	5 分钟
第 3 次	15 分钟
第 4 次	通知管理员

重试时必须使用同一个运行 ID 或幂等键，避免重复生成多个老板决策卡片。

9.11.5 降级策略四：人工接管

如果 Dream 连续失败，系统应通知项目管理员或相关负责人。

人工接管内容包括：

1. 查看失败原因。
2. 查看输入数据是否完整。
3. 手动触发重新运行。
4. 手动生成老板摘要。
5. 暂停某些错误输出。
6. 标记本次 Dream 失败原因。

MVP 阶段人工接管是可以接受的。优先保证主链路可演示，不要求所有异常完全自动化。

9.11.6 失败记录

Dream 失败必须写入审计或运行日志。

建议记录：

1. 运行 ID。
2. 运行时间。
3. 输入范围。
4. 失败阶段。
5. 失败原因。
6. 是否重试。
7. 是否降级。
8. 最终状态。
9. 错误摘要。
10. 相关 Agent 节点。

后续优化 Dream 机制时，这些失败记录非常重要。

9.12 Dream 结果的人工确认机制

Dream 结果进入执行链路前，必须区分“可直接参考”和“需要人工确认”。

MVP 阶段建议采用以下原则：

```
摘要可以直接展示；
风险可以直接提醒；
决策必须人工确认；
任务下发必须基于确认状态。
```

9.12.1 不同输出的确认要求

Dream 输出	是否需要确认	说明
公司级摘要	不强制确认	可直接展示给老板
风险预警	不强制确认	可直接提醒，但不自动执行
部门调整建议	Manager 确认后执行	影响部门任务时需确认
老板决策项	必须老板确认	确认后才可执行
任务草案	必须基于确认生成	草案不是正式任务
次日重点	可由 Manager 确认	可作为提醒或任务来源

9.12.2 老板确认

老板确认 Dream 决策项时，应至少记录：

1. 确认人。
2. 确认时间。
3. 选择方案。
4. 是否修改方案。
5. 确认来源。
6. 决策内容哈希。
7. 生成任务列表。

确认后状态从：

```
pending
```

变为：

```
confirmed
```

随后后端才创建正式任务。

9.12.3 Manager 确认

部门调整建议如果影响部门任务，应由 Manager 确认。

例如 Dream 建议“明天优先处理 Decision 表结构”，这可以先推给 Manager。Manager 确认后，再拆成 Staff 任务。

Manager 确认内容包括：

1. 是否接受调整建议。
2. 是否生成任务。
3. 分配给谁。
4. 截止时间。
5. 优先级。

9.12.4 修改确认

老板或 Manager 可以不接受 Dream 的原始建议，而是修改后确认。

系统应保留：

1. Dream 原始建议。
2. 人工修改内容。
3. 最终确认内容。
4. 修改人。
5. 修改时间。
6. 修改原因。

不要覆盖原始 Dream 输出。这样后续才能复盘 Dream 建议质量。

9.12.5 驳回与补充信息

如果老板驳回 Dream 决策项，应记录驳回原因，并通知相关 Manager。

如果老板要求补充信息，系统应将该请求回传给对应 Manager Agent 或 Staff Agent。

流程如下：

```
老板要求补充信息
  ↓
Executive Agent 记录问题
  ↓
相关 Manager Agent 补充来源数据
  ↓
Dream 局部重新生成
  ↓
再次推送老板确认
```

补充信息期间，不生成正式任务。

9.12.6 人工确认后的审计

所有人工确认行为必须写入审计日志。

审计事件包括：

1. `dream_result_generated`
2. `dream_result_pushed`
3. `dream_decision_confirmed`
4. `dream_decision_rejected`
5. `dream_decision_modified`
6. `dream_need_more_info`
7. `dream_task_generated`

审计记录至少包含：

1. 操作人。
2. 操作时间。
3. 操作对象。
4. 原始 Dream 输出。
5. 人工确认内容。
6. 生成任务。
7. 来源 Summary。
8. 渠道信息。

9.13 本章小结

Dream 是 AutoMage-2 MVP 中负责组织级复盘、风险识别和老板决策生成的核心机制。

它向下读取 Staff、Manager、Summary、Decision、Task 等结构化数据，向上输出老板决策项、风险预警、任务拆解和次日重点。它不直接替老板拍板，也不直接绕过确认生成重大任务，而是把复杂的组织信息整理成可判断、可确认、可执行的结构化结果。

MVP 阶段，Dream 只需要先跑通一条最小链路：

```
读取 Manager Schema
  → 生成组织级摘要
  → 识别关键风险
  → 生成老板决策项
  → 老板确认
  → 生成任务草案并下发
```

只要这条链路稳定，AutoMage-2 就具备了每日自动复盘和次日任务驱动的基础能力。后续再在此基础上扩展长期记忆、趋势分析、跨周期复盘、组织诊断和更复杂的战略推演。

10 任务生成与反向下发机制

10.1 任务生成来源

任务是 AutoMage-2 决策闭环的执行载体。系统中所有重要判断、风险处理、老板确认和部门调整，最终都应尽量落到可分配、可执行、可追踪、可关闭的任务上。

如果没有任务机制，AutoMage-2 只能完成“日报收集”和“信息汇总”；有了任务机制，系统才能形成完整闭环：

```
数据输入
  ↓
部门汇总
  ↓
老板决策
  ↓
任务生成
  ↓
执行反馈
  ↓
新的数据输入
```

MVP 阶段任务主要来自四类对象：

任务来源	说明	是否自动生成
Staff 日报	员工日报中出现明确后续事项、支持需求或阻塞问题	条件生成
Manager 汇总	部门汇总中产生部门调整、异常处理或次日重点	Manager 确认后生成
Executive 决策	老板确认某个方案后，需要拆解为执行任务	老板确认后生成
Incident 异常	风险或阻塞需要明确负责人处理	条件生成

任务生成要遵守一个基本原则：**不是所有建议都自动变成任务。**

例如，员工在日报中写“明天继续完善文档”，这可以作为 `next_day_plan` 记录，但不一定要变成正式任务。只有当某个事项需要被追踪、需要分配负责人、需要有截止时间、需要进入执行闭环时，才应生成正式任务。

MVP 阶段建议优先生成以下类型任务：

1. 老板确认后的执行任务。
2. Manager 确认的部门内任务。
3. 高风险或阻塞事项产生的跟进任务。
4. 缺失日报、缺失字段、缺失产出物产生的催办任务。
5. 已逾期任务的补救任务。
6. 跨部门协作任务。

任务生成时必须记录来源。系统需要知道“这个任务为什么产生”，否则后续无法追踪和复盘。

推荐来源结构如下：

```
{
  "source_type": "executive_decision",
  "source_id": "decision_0001",
  "source_summary_id": 801,
  "source_record_id": null
}
```

常见 `source_type` 建议如下：

source_type	含义
`staff_report`	来源于员工日报
`manager_summary`	来源于部门汇总
`executive_decision`	来源于老板决策
`incident`	来源于异常处理
`manual`	人工手动创建
`system`	系统规则触发
`dream`	Dream 机制生成的任务草案

10.2 Staff 日报生成任务

Staff 日报是任务生成的最底层来源。员工在日报中可能会写到未完成事项、明日计划、需要支持的问题和已阻塞任务，这些内容都可能成为任务输入。

但 Staff 日报中的内容不应全部自动转成任务。否则系统会产生大量低价值任务，增加员工和 Manager 的负担。

Staff 日报生成任务的典型场景包括：

场景	是否建议生成任务	说明
明日计划只是普通继续推进	不建议自动生成	保留在 `next_day_plan` 即可
明日计划有明确交付物	可生成任务	例如“提交接口字段文档”
员工选择 `need_support = true`	可生成支持任务或异常	需要 Manager 介入
工作事项状态为 `blocked`	建议生成异常或跟进任务	防止阻塞被日报淹没
员工上传产出物待审核	可生成审核任务	适用于文档、代码、方案评审
员工任务逾期	可生成补救任务	需要重新安排或上推

Staff 日报生成任务时，应优先经过 Staff Agent 整理和员工确认，再由 Manager Agent 或后端规则决定是否转为正式任务。

推荐流程如下：

```
员工填写日报
  ↓
Staff Agent 识别可转任务事项
```

```
  ↓
员工确认日报
  ↓
后端写入 Work Record
  ↓
规则判断是否生成任务草案
  ↓
Manager Agent 确认
  ↓
生成正式任务
```

示例：

员工日报内容：

```
今天完成了 Staff Schema 初稿，但 Decision 表是否新增还未确认，影响后续 Executive 决策链路。
↪  需要后端确认。
```

Staff Agent 可以识别出一个候选任务：

```
{
  "task_title": " 确认是否新增 decision_logs 表",
  "task_description": "当前 Agent mock 流程中存在 decision_logs 概念，但正式 DDL
  ↪  尚未建立独立决策表，需要后端给出结论。",
  "source_type": "staff_report",
  "source_record_id": 301,
  "assignee_role": "backend",
  "priority": "high",
  "confirm_required": true
}
```

该任务不应直接下发给后端负责人，而应先进入 Manager 或项目负责人确认。确认后再写入正式任务表。

Staff 日报生成任务时需要注意三点：

1. 员工个人计划不等于正式任务。
2. 任务必须有明确产出或处理目标。
3. 涉及他人或其他部门时，需要 Manager 确认后再下发。

10.3 Manager 汇总生成任务

Manager 汇总是部门级任务生成的重要来源。Manager Agent 会读取本部门 Staff Schema，识别风险、阻塞、支持需求和次日调整建议，然后将其中需要追踪的事项转化为任务。

Manager 汇总生成任务主要包括四类：

任务类型	说明
部门内执行任务	Manager 可以直接安排给本部门员工
部门协调任务	用于处理本部门员工之间的协作问题
风险处理任务	针对部门内高风险或阻塞事项
上推准备任务	为需要老板决策的事项补充材料

Manager 汇总生成任务的典型来源字段包括：

Manager Schema 字段	可生成任务类型
`blocked_items`	阻塞处理任务
`top_3_risks`	风险跟进任务
`next_day_adjustment`	次日调整任务
`manager_decisions`	部门内执行任务
`need_executive_decision`	上推材料准备任务
`missing_staff_ids`	日报催填任务

示例：

Manager Schema 中存在阻塞事项：

```
{
  "title": " 决策记录表结构未定",
  "description": "当前 Agent 侧和数据库侧对 decision_logs 的承载方式不一致，影响 Executive
  ↪  决策确认和任务生成链路。",
  "severity": "high",
  "need_support": true
}
```

Manager Agent 可生成任务：

```
{
  "task_title": " 输出 decision_logs 是否建表的后端结论",
  "task_description": "确认是否新增独立 decision_logs 表，并说明对 Executive 决策链路、
  ↪  任务生成和审计追踪的影响。",
  "source_type": "manager_summary",
  "source_summary_id": 801,
  "assignee_role": "backend",
  "department_id": 12,
  "priority": "high",
  "due_at": "2026-05-04T18:00:00+08:00"
}
```

Manager 汇总生成任务时，需要区分“部门内可处理”和“需要老板确认”两种情况。

部门内可处理的任务，Manager 确认后可以直接下发。超出部门权限的事项，不能直接下发给员工执行，而应进入 Executive 决策流程。

例如：

事项	处理方式
催员工补交日报	Manager 可直接生成任务
安排员工补充接口文档	Manager 可直接生成任务
决定是否新增核心数据库表	需要上推老板或项目负责人
调整跨部门开发优先级	需要 Executive 确认
涉及预算、客户承诺、交付延期	需要 Executive 确认

Manager 任务生成的核心原则是：**能在部门内处理的，及时处理；超出权限的，整理好再上推。**

10.4 Executive 决策生成任务

Executive 决策是任务生成中最重要的一类来源。老板或高管确认某个方案后，系统需要将确认结果转化为具体执行任务。

Executive 决策生成任务的关键点是：**确认前是任务草案，确认后才是正式任务。**

在 Executive Schema 中，`generated_tasks` 字段表示可下发任务草案。老板未确认前，这些任务不能进入正式执行状态。

流程如下：

```
Executive Agent 生成决策项
  ↓
生成 A/B 方案和任务草案
  ↓
推送老板决策卡片
  ↓
老板确认方案
  ↓
后端记录确认结果
  ↓
根据确认方案创建正式任务
  ↓
分配给 Manager / Staff
```

示例：

Executive 决策项：

```
{
  "decision_title": " 是否新增独立 decision_logs 表",
  "recommended_option": "A",
  "generated_tasks": [
    {
      "task_title": " 补充 decision_logs 表结构草案",
      "task_description": "字段需覆盖决策标题、背景、候选方案、推荐方案、确认人、
      ↪  确认时间和生成任务。",
      "assignee_role": "backend",
      "priority": "high",
```

```
      "due_at": "2026-05-04T18:00:00+08:00"
    }
  ]
}
```

老板确认方案 A 后，系统正式生成任务：

```
{
  "task_title": " 补充 decision_logs 表结构草案",
  "task_description": "根据老板确认的方案 A，设计独立 decision_logs 表结构，字段需覆盖决策标题、
  ↪ 背景、候选方案、推荐方案、确认人、确认时间和生成任务。",
  "source_type": "executive_decision",
  "source_decision_id": "decision_0001",
  "assignee_role": "backend",
  "department_id": 12,
  "priority": "high",
  "due_at": "2026-05-04T18:00:00+08:00",
  "status": "pending"
}
```

Executive 决策生成任务时必须保留以下信息：

1. 来源决策 ID。
2. 老板确认的方案。
3. 确认人。
4. 确认时间。
5. 任务生成时间。
6. 任务负责人或负责人角色。
7. 任务与决策方案之间的关系。

如果老板修改了方案，而不是直接选择 A 或 B，系统应根据修改后的内容生成任务，并保留原方案和人工修改内容。

10.5 老板确认后任务下发

老板确认是任务从“草案”变成“正式任务”的分界点。

老板未确认前：

1. 任务只存在于 Executive Schema 的 `generated_tasks` 中。
2. 不写入正式任务表。
3. 不推送给员工执行。
4. 不影响任务状态统计。
5. 不进入员工每日任务列表。

老板确认后：

1. 后端记录确认结果。
2. 决策状态变为 `confirmed`。
3. 根据确认方案生成正式任务。
4. 写入 `tasks` 和 `task_assignments`。
5. 推送给对应 Manager Agent 或 Staff Agent。
6. 写入审计日志。
7. 进入任务状态流转。

老板确认后的任务下发流程如下：

```
老板点击确认
  ↓
IM / 前端回传确认动作
  ↓
后端校验确认人权限
  ↓
更新 Decision 状态
  ↓
读取对应方案的任务草案
  ↓
创建正式 Task
  ↓
创建 Task Assignment
  ↓
推送给执行节点
  ↓
记录审计日志
```

老板确认动作可能有多种结果：

老板动作	系统处理
选择方案 A	创建方案 A 对应任务
选择方案 B	创建方案 B 对应任务
自定义方案	记录自定义内容后生成任务
要求补充信息	不生成任务，回传 Manager 补充
驳回	不生成任务，关闭或搁置决策
暂缓	保持待确认或延迟提醒

任务下发时，应根据任务粒度选择下发对象：

任务粒度	下发对象
组织级任务	Executive Agent 保留跟踪，Manager 承接
部门级任务	Manager Agent
具体执行任务	Staff Agent
跨部门任务	多个 Manager Agent
需继续拆解任务	Manager Agent 先接收，再拆给 Staff

例如，老板确认“新增 decision_logs 表”后，系统可以生成两个层级任务：

1. 给后端负责人的具体执行任务：输出表结构草案。
2. 给架构负责人的确认任务：审核 Decision Schema 与数据库映射。
   这样既能明确执行人，也能保留管理节点的确认责任。

10.6 任务分配规则

任务分配决定任务由谁执行、谁负责、谁确认。

MVP 阶段建议区分三类角色：

角色	含义
创建方	生成任务的节点或人
执行方	实际完成任务的人或部门
确认方	判断任务是否完成的人或节点

任务分配应遵守以下规则。

10.6.1 优先分配给明确负责人

如果任务中已经有明确 `assignee_user_id`，则直接分配给该用户。

示例：

```
{
  "assignee_user_id": 10086,
  "task_title": " 补充 Staff Schema 错误示例"
}
```

这种任务可以直接推送给对应 Staff Agent。

10.6.2 只有角色时先分配给 Manager

如果任务只指定了角色，例如 `backend`、`frontend`、`agent_customization`，但没有明确用户，则应先分配给对应部门的 Manager Agent。

示例：

```
{
  "assignee_role": "backend",
  "task_title": " 补充 decision_logs 表结构草案"
}
```

此时 Manager Agent 负责将任务拆解给具体员工。

10.6.3 跨部门任务由多个 Manager 承接

如果任务涉及多个部门，不应直接分配给多个员工，而应先分配给相关部门 Manager。

例如：

```
任务：确认 Executive 决策链路接口契约。
涉及：后端、Agent 客制化、前端。
```

系统应生成一个跨部门任务，并分配给多个 Manager Agent，再由各部门拆解为本部门子任务。

10.6.4 老板任务不直接压给一线

老板确认的任务如果影响范围较大，建议先下发给 Manager，再由 Manager 拆解到 Staff。

原因是老板侧任务通常比较粗，如果直接给一线员工，员工可能不知道优先级、背景和交付标准。

例如：

```
老板确认：优先跑通三级 Agent 闭环。
```

不建议直接下发给员工：

```
任务：跑通三级 Agent 闭环。
```

应由 Manager 拆解为：

1. 后端完成 Staff 日报提交接口。
2. Agent 客制化完成 Mock Flow。
3. 架构负责人补充 Schema 映射。
4. 前端或 IM 侧完成老板确认入口。

10.6.5 一人多任务时需要优先级排序

如果同一员工已有多个任务，系统应按优先级和截止时间排序展示，而不是一次性无序推送。

排序建议：

```
urgent + 今日截止
  ↓
high + 今日截止
  ↓
high + 明日截止
  ↓
medium
  ↓
low
```

Staff Agent 展示任务时，应提示最重要的 1-3 项，避免员工早上收到过多信息。

10.6.6 分配失败处理

任务分配可能失败，常见原因包括：

原因	处理方式
没有明确执行人	分配给对应 Manager Agent
找不到对应角色	标记为待分配
用户不属于该部门	拒绝分配并记录审计
执行人已离职或禁用	返回 Manager 重新分配
跨部门权限不足	上推 Executive 或管理员确认
重复任务	合并或提示人工确认

分配失败不能静默丢弃，必须有状态记录。

10.7 任务优先级规则

任务优先级用于决定任务展示顺序、提醒频率和上推策略。

MVP 阶段建议使用四级优先级：

优先级	含义	典型场景
`urgent`	紧急，必须立即处理	影响演示、交付、客户或重大决策
`high`	高优先级，需当天或短期完成	影响 MVP 主链路、阻塞后续联调
`medium`	中优先级，正常排期处理	常规功能、文档补充、非关键优化
`low`	低优先级，可延后	体验优化、非关键补充、后续扩展

优先级判断应参考五个因素：

1. 是否影响 MVP 主链路。
2. 是否影响老板确认或任务下发。
3. 是否阻塞其他部门。
4. 是否有明确截止时间。
5. 是否涉及高风险异常。

常见优先级示例：

任务	建议优先级
修复 Staff Schema 无法写库问题	urgent
确认 decision_logs 是否建表	high
补充 Manager Schema 字段说明	high
完善日报文案体验	medium
增加行业模板字段	low
优化老板侧卡片视觉	medium
修复任务无法下发到员工	urgent

优先级可以由 Agent 初步建议，但重要任务的优先级应由 Manager 或 Executive 确认。

任务优先级变更必须记录到 `task_updates`，避免员工不清楚为什么任务突然变急。

10.8 任务截止时间规则

任务必须尽量有截止时间。没有截止时间的任务很难进入执行闭环，也很难判断是否逾期。

MVP 阶段建议按任务来源设置默认截止时间：

任务来源	默认截止时间
老板确认的 urgent 任务	当日或 24 小时内
老板确认的 high 任务	1-2 个工作日内
Manager 下发的部门任务	当日或次日
Staff 日报中的明日计划	次日工作结束前
缺失日报催办	当日 20:00 前
信息补充任务	半天到 1 天
普通文档补充	1-3 个工作日
后续优化任务	可不强制，但应有排期

截止时间应同时考虑任务复杂度和优先级。不能所有任务都默认“今天完成”，否则系统会失去可信度。

任务截止时间字段建议使用：

```
{
  "due_at": "2026-05-04T18:00:00+08:00"
}
```

对于跨天任务，可以补充计划字段：

```
{
  "start_at": "2026-05-04T10:00:00+08:00",
  "due_at": "2026-05-06T18:00:00+08:00"
}
```

任务截止时间规则：

1. `urgent` 任务必须有明确截止时间。
2. `high` 任务必须有明确截止时间。
3. `medium` 任务建议有截止时间。
4. `low` 任务可进入待排期，但不能长期无状态。
5. 截止时间变更必须记录原因。
6. 逾期任务应进入提醒或风险判断。
7. 多次逾期任务应上报 Manager。
8. 关键任务逾期可能进入 Executive 风险预警。

逾期处理建议如下：

逾期情况	处理方式
轻微逾期	Staff Agent 提醒执行人
逾期超过半天	通知 Manager Agent
高优任务逾期	进入 Manager 汇总风险
关键任务多次逾期	上推 Executive
老板确认任务逾期	Dream 次日重点提示

10.9 任务状态流转

任务状态用于表示任务当前处于哪个阶段。MVP 阶段建议采用六个核心状态：

```
待处理
  ↓
进行中
  ↓
阻塞 / 已完成
  ↓
已关闭

待处理 / 进行中
  ↓
已取消
```

对应状态值建议如下：

中文状态	建议枚举值	说明
待处理	`pending`	已创建，尚未开始
进行中	`in_progress`	执行人已开始处理
阻塞	`blocked`	当前无法继续推进
已完成	`completed`	执行人认为已完成
已关闭	`closed`	确认方验收并关闭
已取消	`cancelled`	不再执行

任务状态变化必须记录到任务更新表或审计日志中。不能只改最终状态，不记录过程。

10.9.1 待处理

`pending` 表示任务已经创建，但执行人尚未开始处理。

典型场景：

1. 老板确认后刚生成任务。
2. Manager 下发任务后员工尚未领取。
3. 系统生成催办任务但员工未查看。

待处理状态下，Staff Agent 可以提醒员工查看任务。如果任务长期处于 `pending`，应根据优先级触发提醒。

建议规则：

优先级	提醒策略
urgent	立即提醒，30 分钟未响应再次提醒
high	当天提醒，半天未响应通知 Manager
medium	每日任务列表中展示
low	列入待办，不频繁提醒

10.9.2 进行中

`in_progress` 表示执行人已经开始处理任务。

进入方式：

1. 员工主动点击“开始处理”。
2. Staff Agent 在日报中识别到该任务正在推进。
3. Manager 手动更新任务状态。

进行中状态应记录开始时间：

```
{
  "status": "in_progress",
  "started_at": "2026-05-04T10:30:00+08:00"
}
```

进行中任务需要定期检查进度。如果到截止时间仍未完成，应进入逾期提醒；如果执行人反馈无法推进，应改为 `blocked`。

10.9.3 阻塞

`blocked` 表示任务当前无法继续推进，需要支持、协调或重新决策。

进入阻塞状态的典型原因包括：

1. 缺少接口或数据。
2. 等待他人确认。
3. 需要跨部门协作。
4. 需求不清楚。
5. 权限不足。
6. 资源不足。
7. 老板决策未确认。
8. 技术方案冲突。

阻塞任务必须填写阻塞原因：

```
{
  "status": "blocked",
  "blocked_reason": "Decision 表结构尚未确认，无法继续完成对应接口映射。",
```

```
  "need_support": true
}
```

阻塞状态的处理规则：

1. Staff 任务阻塞，先通知 Manager。
2. Manager 无法处理，再上推 Executive。
3. 高优先级任务阻塞，应进入部门风险。
4. 老板确认任务阻塞，应进入 Dream 次日分析。
5. 阻塞解除后，可恢复为 `in_progress`。

阻塞不能只作为员工解释延期的标签，必须触发后续处理。

10.9.4 已完成

`completed` 表示执行人认为任务已经完成。

任务完成时，应填写结果摘要和产出物。

示例：

```
{
  "status": "completed",
  "result_summary": "已完成 decision_logs 表结构草案，包含决策标题、背景、候选方案、推荐方案、
  ↪ 确认人、确认时间、来源 Summary 和生成任务字段。",
  "artifacts": [
    {
      "title": "decision_logs DDL 草案",
      "artifact_type": "document",
      "url": "https://example.com/decision-logs-ddl"
    }
  ]
}
```

任务完成不等于关闭。如果任务需要验收，应由确认方检查结果后改为 `closed`。

10.9.5 已关闭

`closed` 表示任务已经完成并通过确认，进入归档状态。

关闭任务的确认方可能是：

1. 员工本人。
2. Manager。
3. Executive。
4. 系统规则。
5. 发起任务的人。

关闭任务时应记录：

1. 关闭人。
2. 关闭时间。
3. 关闭原因。
4. 是否验收通过。
5. 结果摘要。
6. 关联产出物。
7. 是否产生后续任务。

任务关闭后，默认不再进入每日提醒，但可以被 Summary、Dream 和审计系统读取。

10.9.6 已取消

`cancelled` 表示任务不再执行。

取消原因可能包括：

1. 决策撤回。
2. 任务重复。
3. 需求变化。
4. 负责人调整。
5. 上级明确取消。
6. 任务不再有价值。
7. 任务被合并到其他任务。

取消任务时必须记录原因：

```
{
  "status": "cancelled",
  "cancel_reason": " 老板决策已撤回，该任务不再执行。",
  "cancelled_by": 30001,
  "cancelled_at": "2026-05-04T15:00:00+08:00"
}
```

取消不等于删除。取消后的任务仍应保留审计记录，避免后续无法追踪为什么某个决策没有执行。

10.10 任务 Schema 字段定义

MVP 阶段建议定义统一的 Task Schema，用于承载所有来源生成的任务。

10.10.1 Task Schema 字段总览

字段名	类型	是否必填	说明
schema_id	string	是	固定为 schema_v1_task
schema_version	string	是	默认 1.0.0
task_id	string / number	否	任务 ID，创建后生成
org_id	number / string	是	组织 ID
department_id	number / string	否	所属部门
task_title	string	是	任务标题
task_description	string	是	任务说明
source_type	string	是	来源类型
source_id	string / number	是	来源对象 ID
source_record_id	string / number	否	来源 Staff 日报 ID
source_summary_id	string / number	否	来源 Manager / Executive Summary ID
source_decision_id	string / number	否	来源决策 ID
creator_user_id	string / number	是	创建人
created_by_node_id	string	是	创建任务的 Agent 节点
assignee_user_id	string / number	否	执行人
assignee_role	string	否	执行角色
assignment_type	string	否	分配类型
priority	string	是	优先级
status	string	是	任务状态
due_at	string	否	截止时间
started_at	string	否	开始时间
completed_at	string	否	完成时间
closed_at	string	否	关闭时间
result_summary	string	否	结果摘要
artifacts	array	否	任务产出物
confirm_required	boolean	是	是否需要确认关闭
confirmed_by	string / number	否	确认人
confirmed_at	string	否	确认时间
blocked_reason	string	否	阻塞原因
cancel_reason	string	否	取消原因
meta	object	否	扩展字段

10.10.2 Task Schema JSON 示例

```
{
  "schema_id": "schema_v1_task",
  "schema_version": "1.0.0",
  "org_id": 1,
  "department_id": 12,
  "task_title": " 补充 decision_logs 表结构草案",
  "task_description": "根据老板确认的方案 A，设计独立 decision_logs 表结构，字段需覆盖决策标题、
  ↪ 背景、候选方案、推荐方案、确认人、确认时间和生成任务。",
```

```
  "source_type": "executive_decision",
  "source_id": "decision_0001",
  "source_decision_id": "decision_0001",
  "creator_user_id": 30001,
  "created_by_node_id": "executive_node_org_1",
  "assignee_user_id": 20002,
  "assignee_role": "backend",
  "assignment_type": "owner",
  "priority": "high",
  "status": "pending",
  "due_at": "2026-05-04T18:00:00+08:00",
  "confirm_required": true,
  "meta": {
    "confirmed_option": "A",
    "push_channel": "feishu",
    "agent_template_version": "executive_agent_v0.1"
  }
}
```

10.10.3 字段说明

`task_title` 应简短明确，说明要做什么。不推荐写“优化系统”“完善文档”这类模糊标题。

`task_description` 应说明任务背景、执行要求和预期产出。如果任务来自老板决策，应说明确认方案和来源。

`source_type` 和 `source_id` 必填。没有来源的任务不应进入正式任务系统。

`assignee_user_id` 和 `assignee_role` 至少应有一个。如果没有明确用户，可以先分配给角色或 Manager。

`priority` 必须使用统一枚举：`urgent`、`high`、`medium`、`low`。

`status` 必须使用统一枚举：`pending`、`in_progress`、`blocked`、`completed`、`closed`、`cancelled`。

`confirm_required` 表示任务完成后是否需要确认关闭。老板决策产生的任务一般需要确认。

10.11 任务与 Staff Agent 的交互方式

Staff Agent 是员工接收任务、反馈任务和沉淀执行结果的主要入口。

MVP 阶段，员工不需要进入复杂任务系统，也可以通过 Staff Agent 完成基本任务交互。

10.11.1 每日任务推送

每天早上，Staff Agent 应读取员工当天待执行任务，并推送简洁列表。

示例：

```
今日待执行任务：

1. 补充 decision_logs 表结构草案
优先级：高
截止时间：今日 18:00
来源：老板确认的方案 A

2. 完善 Manager Schema 与 Summary 表映射
优先级：中
截止时间：明日 12:00
来源：部门汇总调整建议
```

任务列表不宜过长。默认展示最高优先级的 3-5 个任务，其余任务可以通过“查看全部任务”获取。

10.11.2 任务开始

员工可以通过 Staff Agent 标记任务开始。

示例：

```
员工：开始处理第 1 个任务。
Staff Agent：已将“补充 decision_logs 表结构草案”标记为进行中。
```

后端更新任务状态为 `in_progress`，并写入 `task_updates`。

10.11.3 任务阻塞

员工遇到问题时，可以通过 Staff Agent 标记阻塞。

示例：

```
员工：第 1 个任务阻塞了，还需要确认 Decision 表是否要支持多方案历史版本。
Staff Agent：已记录阻塞原因，是否需要上级支持？
```

如果员工确认需要支持，系统可生成 Incident 或通知 Manager Agent。

阻塞反馈应进入当天 Staff Schema，便于 Manager 汇总时看到。

10.11.4 任务完成

员工完成任务后，可以通过 Staff Agent 提交结果。

示例：

```
员工：第 1 个任务完成了，文档链接是 xxx。
Staff Agent：请确认任务结果：已完成 decision_logs 表结构草案，并提交文档链接。
↪ 确认后将更新任务状态。
```

确认后，任务状态变为 `completed`，并保存结果摘要和产出物。

如果任务需要验收，则等待 Manager 或任务创建人关闭。

10.11.5 任务进入日报

任务执行情况应自动进入员工当日 Staff Schema。

例如员工今天处理了两个任务，Staff Agent 在日报时可以自动带出：

```
今天你处理了以下任务：
1. decision_logs 表结构草案：已完成
2. Manager Schema 映射说明：进行中

请补充每项任务的具体产出和遇到的问题。
```

这样任务执行结果会进入 Staff Schema，再进入 Manager 汇总，形成闭环。

10.11.6 员工拒绝或无法接收任务

如果员工认为任务不属于自己，或者无法执行，应允许反馈。

常见反馈包括：

反馈	处理
不是我的任务	通知 Manager 重新分配
任务描述不清	要求创建方补充信息
截止时间不合理	请求 Manager 调整
缺少权限或资源	标记为阻塞
已经做过	提供产出物并请求关闭
与其他任务冲突	进入冲突处理流程

Staff Agent 不应强制员工接受明显错误的任务，而应把问题反馈给 Manager。

10.12 任务与数据库字段映射

MVP 阶段任务主要映射到以下数据库对象：

1. `tasks`：任务主表。
2. `task_assignments`：任务分配表。

3. task_updates：任务动态表。
4. artifacts / artifact_links：任务产出物。
5. audit_logs：任务相关审计记录。

10.12.1 映射到 tasks

tasks 存储任务主记录。

Task Schema 字段	数据库字段	说明
org_id	tasks.org_id	组织 ID
department_id	tasks.department_id	所属部门
source_record_id	tasks.source_record_id	来源 Staff 日报
creator_user_id	tasks.creator_user_id	创建人
task_title	tasks.title	任务标题
task_description	tasks.description	任务说明
status	tasks.status	任务状态
priority	tasks.priority	优先级
due_at	tasks.due_at	截止时间
started_at	tasks.started_at	开始时间
closed_at	tasks.closed_at	关闭时间
result_summary	tasks.result_summary	结果摘要
source_type / source_id	tasks.meta	来源类型和来源 ID
source_decision_id	tasks.meta.source_decision_id	来源决策
confirm_required	tasks.meta.confirm_required	是否需要确认关闭
created_by_node_id	tasks.meta.created_by_node_id	创建节点

如果当前 tasks 表没有 source_decision_id 显式字段，可以先存入 meta。后续如果老板决策任务成为高频查询对象，建议将其提升为显式字段。

10.12.2 映射到 task_assignments

task_assignments 存储任务与执行人的关系。

Task Schema 字段	数据库字段	说明
org_id	task_assignments.org_id	组织 ID
task_id	task_assignments.task_id	任务 ID
assignee_user_id	task_assignments.user_id	执行人
assignment_type	task_assignments.assignment_type	分配类型
creator_user_id	task_assignments.created_by	分配创建人

如果任务分配给角色而不是具体用户，可以先分配给 Manager，再由 Manager 拆解到具体用户。也可以在 tasks.meta.assignee_role 中保留角色信息。

建议 assignment_type 区分：

assignment_type	含义
`owner`	主负责人
`participant`	协作人
`reviewer`	审核人
`watcher`	关注人

10.12.3 映射到 task_updates

`task_updates` 记录任务状态变化和过程动态。

Task 动作	`task_updates` 内容
创建任务	update_type = created
开始处理	update_type = started
状态更新	update_type = status_changed
标记阻塞	update_type = blocked
补充进展	update_type = comment
提交完成	update_type = completed
确认关闭	update_type = closed
取消任务	update_type = cancelled

建议记录字段：

Task Schema 字段	数据库字段	说明
`task_id`	`task_updates.task_id`	任务 ID
操作人	`task_updates.actor_user_id`	操作用户
更新类型	`task_updates.update_type`	动作类型
更新内容	`task_updates.content`	说明文本
状态前	`task_updates.status_before`	变更前状态
状态后	`task_updates.status_after`	变更后状态
操作时间	`task_updates.event_at`	事件时间
扩展信息	`task_updates.meta`	补充字段

任务状态变化不能只更新 `tasks.status`，必须同步写入 `task_updates`，否则后续无法追踪任务执行过程。

10.12.4 映射到 artifacts

任务产出物写入 `artifacts` 或 `artifact_links`。

Task Schema 字段	数据库字段	说明
`artifacts[].title`	`artifacts.title`	产出物标题
`artifacts[].artifact_type`	`artifacts.artifact_type`	产出物类型
`artifacts[].url`	`artifact_links.url` 或 `artifacts.file_url`	链接或文件
`task_id`	`artifacts.task_id`	关联任务
`assignee_user_id`	`artifacts.user_id`	提交人

任务完成时，如果没有任何结果摘要或产出物，Staff Agent 应提醒员工补充说明。

10.12.5 映射到 audit_logs

任务相关关键行为应写入审计日志。

建议记录以下动作：

动作	说明
task_generated	任务由系统生成
task_assigned	任务被分配
task_started	任务开始处理
task_blocked	任务被标记为阻塞
task_completed	执行人提交完成
task_closed	确认方关闭任务
task_cancelled	任务被取消
task_reassigned	任务重新分配
task_due_changed	截止时间变更
task_priority_changed	优先级变更

审计日志应记录：

1. 操作人。
2. 操作 Agent。
3. 任务 ID。
4. 来源对象。
5. 操作时间。
6. 操作前后状态。
7. 关键 payload 摘要。

10.12.6 当前数据库映射建议

基于当前数据库设计，MVP 阶段任务链路可以按以下方式落地：

```
Executive / Manager / Staff 生成任务草案
  ↓
后端校验来源和权限
  ↓
tasks 写入任务主记录
  ↓
task_assignments 写入执行人
  ↓
task_updates 写入创建动态
  ↓
Staff / Manager Agent 推送任务
  ↓
执行过程继续写入 task_updates
  ↓
完成产出写入 artifacts
```

↓
关键动作写入 `audit_logs`

如果短期还没有独立 Decision 表，老板决策来源可以先放入 `tasks.meta.source_decision_id`，但后续建议补齐独立决策表，避免任务来源长期依赖 JSON 字段。

10.13 本章小结

任务生成与反向下发机制，是 AutoMage-2 从“信息系统”变成“执行系统”的关键。

Staff Schema 让系统知道一线员工做了什么；Manager Schema 让系统知道部门发生了什么；Executive Schema 让老板知道需要决策什么。而任务机制负责把这些判断和决策转化为具体行动。

MVP 阶段任务机制需要先跑通五件事：

1. 能从 Staff、Manager、Executive 三类来源生成任务草案。
2. 能在 Manager 或老板确认后创建正式任务。
3. 能把任务分配给具体员工、角色或部门。
4. 能让 Staff Agent 展示、更新和反馈任务。
5. 能把任务结果重新沉淀到日报、汇总和 Dream 分析中。

只要任务闭环跑通，AutoMage-2 就不再只是“每天总结发生了什么”，而是可以开始推动“明天应该做什么，以及谁来做”。

11 异常与风险识别机制

11.1 异常定义

在 AutoMage-2 中，异常指组织运行过程中已经偏离正常流程、需要被记录、跟进、协调或上推处理的事项。

异常不是普通问题，也不是所有员工提到的困难。只有当某个问题影响任务推进、影响部门汇总质量、影响决策准确性，或者需要其他人介入处理时，才应进入异常机制。

MVP 阶段，异常主要包括以下几类：

异常类型	说明	示例
数据异常	关键数据缺失、格式错误、未确认、无法写入	员工日报缺失，Staff Schema 校验失败
任务异常	任务逾期、阻塞、负责人不明确、任务冲突	老板确认任务未分配执行人
流程异常	某个节点没有按预期产出数据或推进流程	Manager 汇总未生成，Dream 未运行
权限异常	Agent 或用户尝试越权读取或写入	Staff Agent 尝试读取其他员工日报
决策异常	决策缺少来源、方案不完整、确认超时、冲突	老板决策卡片缺少 A/B 方案
业务异常	工作结果影响交付、客户、预算、质量或关键目标	关键接口延期导致 MVP 联调受阻
系统异常	API、数据库、IM、Agent Runtime 等基础能力故障	后端接口 500，IM 推送失败

异常的核心特征是：**它需要被处理。**

如果某个问题只是员工的个人备注，不影响任何任务和流程，可以留在 Staff Schema 中。如果某个问题已经影响执行、汇总、决策或责任划分，就应进入 Incident Schema 或相关异常表。

异常机制的目标不是增加管理负担，而是防止关键问题被埋在日报和聊天记录里。

11.2 风险定义

风险指尚未完全造成损失，但可能影响任务、部门、项目或组织目标的潜在问题。

异常强调“已经发生或已经偏离”；风险强调“可能发生或正在扩大”。两者关系如下：

```
风险未处理
  ↓
形成阻塞
  ↓
升级为异常
  ↓
进入处理流程
  ↓
关闭或上推决策
```

例如：

情况	类型
某员工说明“接口字段还没完全确认”	风险
因接口字段未确认导致任务无法继续	异常
多个部门都在等待该字段确认	组织级风险
该问题影响老板侧决策卡片生成	高级异常 / 需上推决策

风险的价值在于提前预警。异常的价值在于推动处理。

MVP 阶段不需要建立复杂风控系统，但至少需要让 Staff Agent、Manager Agent 和 Executive Agent 能够识别以下风险：

1. 数据不完整。
2. 任务逾期。
3. 工作阻塞。
4. 多部门依赖未确认。
5. 关键决策未确认。
6. 高优任务无人负责。
7. 重复出现的问题。
8. 与 MVP 主链路相关的阻塞。

风险识别后，不一定全部生成异常。低风险可以只进入日报或汇总；中高风险才进入异常处理或决策上推。

11.3 Staff 层异常

Staff 层异常来自一线员工的工作记录、任务执行和日报提交。它是系统最早发现问题的位置。

Staff 层异常的特点是具体、局部、靠近执行现场。很多组织级风险，最早都会以 Staff 层异常的形式出现。

典型 Staff 层异常包括：

异常类型	触发来源	示例
日报缺失	员工未按时提交 Staff Schema	员工 20:00 后仍未提交日报
日报未确认	Staff Agent 生成草稿后员工未确认	日报草稿存在，但未签名
字段缺失	Staff Schema 缺少必填字段	`need_support = true` 但没有 `support_detail`
任务阻塞	员工任务状态为 blocked	等待后端确认表结构
任务逾期	任务超过 `due_at` 未完成	高优先级任务到期未提交
支持请求	员工明确请求 Manager 支持	“需要后端确认接口字段”
产出物缺失	任务要求产出，但员工未提交附件或链接	文档任务完成但无文档链接
权限问题	员工无法访问执行任务所需资源	没有仓库权限、没有接口权限
任务冲突	员工同一时间收到多个高优任务	两个 high 任务都要求今日完成

Staff 层异常的处理原则：

1. 员工能补充的，Staff Agent 先提醒员工补充。
2. 员工自己能解决的，不上推 Manager。
3. 需要他人支持的，生成支持请求或 Incident。
4. 已经影响任务推进的，进入 Manager 汇总。
5. 高风险或多次出现的问题，上推 Manager Agent。

Staff Agent 发现异常后，不应直接向老板上推。除非是系统级紧急安全事件，否则 Staff 层问题应先交由 Manager Agent 判断。

Staff 层异常示例：

```
{
  "incident_title": "Decision 表结构未确认导致任务阻塞",
  "description": "员工在处理 Executive 决策链路映射时，发现当前 DDL 中没有独立 decision_logs
  ↪ 表，无法确定老板确认结果应如何落库。",
  "source_type": "staff_report",
  "source_record_id": 301,
  "reporter_user_id": 10086,
  "department_id": 12,
  "severity": "high",
  "need_escalation": true
}
```

11.4 Manager 层异常

Manager 层异常来自部门汇总、部门任务、部门风险和 Staff 层异常聚合结果。

Manager 层异常的特点是已经不再只是单个员工的问题，而是影响部门运行、部门目标或部门对上汇报质量的问题。

典型 Manager 层异常包括：

异常类型	说明	示例
部门数据缺失	部门内多人未提交日报	10 人中 4 人未提交 Staff Schema
部门任务阻塞	多个员工任务依赖同一未解决问题	多个任务等待 Decision 表结构确认
部门风险升高	Staff 风险聚合后影响部门状态	部门健康度从 green 变为 yellow
跨员工协作失败	员工之间依赖未对齐	前端等后端接口，后端等 Schema 字段
部门内权限不足	Manager 无法解决某项资源问题	需要其他部门提供接口或数据
上推材料不完整	Manager 生成决策项缺少方案或来源	只有问题描述，没有 A/B 方案
任务分配异常	任务没有负责人或分配给错误人员	后端任务分给前端员工
部门汇总失败	Manager Agent 未能生成 Manager Schema	汇总输出格式错误，后端校验失败

Manager 层异常的处理原则：

1. 部门内能解决的，由 Manager 处理。

2. 涉及跨部门依赖的，生成协作任务或上推 Executive。
3. 影响老板决策的数据异常，必须在 Manager Schema 中标记。
4. 多个 Staff 异常指向同一问题时，应合并为部门级异常。
5. 高风险异常应进入 `top_3_risks` 或 `need_executive_decision`。

Manager Agent 不应把所有 Staff 异常都原样上推。它需要做一次过滤和归并，把真正影响部门或公司目标的问题提炼出来。

例如，三个员工都提到“接口字段未定”，Manager Agent 不应生成三条独立异常，而应合并为一条：

```
{
  "incident_title": " 接口字段未定影响多条任务推进",
  "description": "今日共有 3 条 Staff 记录提到接口字段未确认，已影响后端、Agent
  ↪  客制化和文档映射任务。",
  "source_record_ids": [301, 302, 303],
  "severity": "high",
  "need_escalation": true,
  "suggested_action": " 建议确认接口字段责任人，并在今日内输出最终字段清单。"
}
```

11.5 Executive 层异常

Executive 层异常来自组织级汇总、多个部门上推事项、老板决策超时、重大任务逾期或 Dream 分析结果。

Executive 层异常的特点是影响范围大，通常已经不只是某个员工或某个部门的问题，而是影响组织目标、关键交付、跨部门协作或老板决策效率的问题。

典型 Executive 层异常包括：

异常类型	说明	示例
关键决策超时	老板待确认事项长时间未处理	L3 决策超过 24 小时未确认
多部门共性阻塞	多个部门被同一问题阻塞	后端表结构未定影响 Agent、前端、Dream
高优任务逾期	老板确认任务未按时完成	MVP 联调任务逾期
组织级数据缺失	多个部门未生成 Manager Schema	老板早会前缺少部门汇总
Dream 运行失败	无法生成老板决策卡片	Executive Schema 校验失败
决策执行冲突	下发任务与历史决策不一致	老板选 A，但部门按 B 执行
重大交付风险	关键里程碑无法按期完成	两周 MVP 主链路无法联调
外部业务风险	涉及客户、预算、媒体、合规等问题	演示前核心链路不可用

Executive 层异常的处理原则：

1. 必须明确影响范围。
2. 必须有来源数据。
3. 必须整理成老板可理解的风险说明。
4. 能给方案的，应提供 A/B 方案。

5. 需要老板确认的，进入 Executive 决策卡片。
6. 已确认的处理方案，必须生成任务。
7. 未确认的高风险异常，必须持续提醒或进入人工接管。

   Executive 层异常不应直接淹没老板。老板侧只应看到少量真正需要处理的事项。

   推荐展示方式：

```
异常：MVP 主链路存在任务下发阻塞

影响：
当前 Executive 决策可以生成草案，但正式任务来源还未与 Decision 对象打通，可能影响“老板确认 →
↪  任务下发 → Staff 接收”的演示闭环。

建议处理：
方案 A：新增独立 Decision 表，并补齐任务来源映射。
方案 B：短期复用 audit_logs + tasks.meta，先跑通 Demo。

推荐：方案 A。
```

11.6 异常等级

MVP 阶段建议将异常分为四级：low、medium、high、critical。

等级	含义	处理方式
`low`	轻微异常，不影响主流程	记录即可，必要时提醒
`medium`	需要部门关注，但可在部门内处理	Manager 处理或生成部门任务
`high`	影响关键任务、部门目标或 MVP 主链路	Manager 优先处理，必要时上推 Executive
`critical`	影响老板决策、关键交付、客户、预算或系统可用性	立即上推 Executive，必要时人工接管

异常等级判断建议如下：

11.6.1 low

低级异常通常不影响任务主流程。

示例：

1. 员工日报少填了一个可选字段。
2. 普通任务备注不完整。
3. 低优任务延期半天。
4. 非关键产出物链接格式错误。
5. IM 提醒失败一次但后续成功。

   处理方式：

1. Staff Agent 自动提醒。

2. 不进入老板侧。
3. 不进入 Top 风险。
4. 可记录到日志或日报中。

11.6.2 medium

中级异常需要 Manager 关注，但通常可以在部门内解决。

示例：

1. 员工日报缺失。
2. 普通任务逾期。
3. 员工请求部门支持。
4. 部门内任务分配不清。
5. 某个接口字段需要负责人确认。
6. Manager Schema 中存在数据不完整。

处理方式：

1. 进入 Manager Schema。
2. 可生成部门任务。
3. 由 Manager 处理。
4. 必要时进入 `blocked_items`。
5. 暂不上推老板，除非多次重复出现。

11.6.3 high

高级异常可能影响关键链路、部门目标或跨部门协作。

示例：

1. MVP 主链路某个接口未定义。
2. 多个任务同时被同一问题阻塞。
3. 高优任务逾期。
4. Manager 汇总无法生成。
5. Decision 数据结构未确认，影响老板侧闭环。
6. 多部门依赖同一未确认事项。

处理方式：

1. 进入 `top_3_risks`。
2. 生成 Incident。
3. Manager 立即处理。
4. 若超出部门权限，进入 `need_executive_decision`。

5. Dream 运行时应优先读取。

11.6.4 critical

严重异常会影响关键交付、老板决策、客户承诺、系统可用性或组织级目标。

示例：

1. 老板确认任务无法生成。
2. 数据库写入主链路不可用。
3. 大量 Staff Schema 无法提交。
4. Executive 决策卡片无法推送。
5. 关键客户交付风险。
6. 涉及数据泄露、安全、合规或法律问题。
7. 重大预算或外部承诺异常。

处理方式：

1. 立即上推 Executive。
2. 同步通知相关 Manager。
3. 生成高优处理任务。
4. 必要时暂停自动流程。
5. 进入人工接管。
6. 全程写入审计日志。

MVP 阶段如果无法判断是否 critical，应采取保守策略：先上推人工确认，避免 Agent 自行降级。

11.7 异常上报规则

异常上报指下级节点发现异常后，将其提交给上级节点或异常处理系统。

上报不是简单转发，而是要整理成可处理对象。

11.7.1 Staff 向 Manager 上报

Staff Agent 在以下情况下应向 Manager 上报异常：

1. `need_support = true`。
2. 工作事项状态为 `blocked`。
3. 任务已逾期。
4. 员工无法访问必要资源。
5. 员工明确请求上级介入。
6. Staff Schema 校验多次失败。

7. 员工任务与其他任务冲突。
8. 员工无法确认任务归属。

上报内容至少包括：

字段	说明
incident_title	异常标题
description	异常说明
reporter_user_id	上报员工
department_id	所属部门
source_type	来源类型
source_id	来源对象 ID
severity	异常等级
attempted_solution	已尝试方案
need_support	是否需要支持
suggested_action	建议下一步

Staff 层上报时，应尽量附带员工已经尝试过的解决方案，避免 Manager 重复询问。

11.7.2 Manager 向 Executive 上报

Manager Agent 在以下情况下应向 Executive 上报异常：

1. 部门内无法解决。
2. 涉及多个部门。
3. 影响关键交付。
4. 影响老板确认链路。
5. 需要预算、资源或方向确认。
6. 同类异常重复出现。
7. 高风险异常超过处理时限。
8. 异常需要老板选择方案。

上报内容不应只是异常说明，还应尽量整理为决策项。

至少包括：

1. 异常标题。
2. 背景说明。
3. 影响范围。
4. 来源 Staff / Manager 记录。
5. 已尝试处理方式。
6. 可选方案。
7. 推荐方案。
8. 紧急程度。
9. 确认后可生成的任务。

Manager 向 Executive 上报的质量，决定老板侧决策卡片的质量。

11.7.3 自动上报条件

MVP 阶段可以先设置一些简单自动上报规则：

条件	自动动作
`severity = critical`	立即上推 Executive
高优任务逾期超过 1 天	上报 Manager
老板确认任务逾期	进入 Dream 风险
Staff 日报缺失比例超过 40%	Manager Schema 标记部门数据风险
同一异常连续出现 3 天	上推上一级
Manager Schema 连续缺失	Executive 风险预警
Dream 连续失败	通知管理员和 Executive

自动上报必须保留审计记录，并允许人工调整等级。

11.7.4 上报去重

异常上报需要去重，避免同一个问题被多个员工、多个部门重复上报后形成信息噪音。

去重建议参考以下字段：

1. `org_id`
2. `department_id`
3. `source_type`
4. `source_id`
5. `incident_title`
6. `related_task_id`
7. `risk_category`
8. 时间窗口

例如，多个员工都上报“Decision 表结构未确认”，Manager Agent 应合并为一个部门异常，并保留多个来源记录。

合并后的异常应记录：

```
{
  "merged_from_incident_ids": [501, 502, 503],
  "source_record_ids": [301, 302, 303]
}
```

11.8 异常处理流程

异常处理流程应清楚区分“发现异常、确认异常、分配处理、跟进状态、关闭异常”几个阶段。

推荐流程如下：

```
发现异常
  ↓
生成 Incident
  ↓
判断等级
  ↓
分配处理人
  ↓
处理或上推
  ↓
生成任务
  ↓
跟进状态
  ↓
确认关闭
```

11.8.1 发现异常

异常可以由以下方式发现：

1. Staff Agent 从日报中识别。
2. Manager Agent 从部门汇总中识别。
3. Dream 从组织级数据中识别。
4. 后端规则自动发现。
5. 用户主动上报。
6. 系统接口或任务运行失败。

发现异常后，应先生成 Incident 草稿或异常候选项，不一定立即进入正式异常表。

11.8.2 确认异常

异常确认用于判断该问题是否真的需要进入处理流程。

确认人根据层级不同而不同：

异常来源	确认人
Staff 层 low / medium	员工或 Manager
Staff 层 high	Manager
Manager 层 high	Manager 或 Executive
Executive 层 high / critical	老板或高管
系统异常	管理员或技术负责人

MVP 阶段可以允许 high 以下异常自动进入处理流程，但 critical 异常应触发人工确认或人工接管。

11.8.3 分配处理人

异常确认后，需要明确处理人。

处理人可以是：

1. 员工。
2. Manager。
3. 相关部门负责人。
4. 后端负责人。
5. Agent 客制化负责人。
6. 系统管理员。
7. 老板或高管。

如果无法明确处理人，应先分配给对应 Manager，而不是直接下发给一线员工。

11.8.4 生成处理任务

异常一旦需要具体处理，应生成任务。

例如：

异常：

```
Decision 表结构未确认，影响 Executive 决策任务生成。
```

对应任务：

```
输出 decision_logs 是否建表的最终结论，并补充字段草案。
```

异常与任务的关系应明确保存：

```
{
  "incident_id": 501,
  "generated_task_ids": [9001, 9002]
}
```

异常本身记录问题，任务记录行动。二者不能混为一谈。

11.8.5 跟进异常状态

异常处理过程中，应持续更新状态。

建议状态如下：

状态	含义
`open`	已创建，待处理
`triaged`	已确认并分类
`assigned`	已分配处理人
`in_progress`	正在处理
`waiting_decision`	等待上级决策
`waiting_external`	等待外部输入
`resolved`	已解决，待确认
`closed`	已关闭
`cancelled`	已取消
`reopened`	已重新打开

状态变化必须写入 `incident_updates` 或审计日志。

11.8.6 上推处理

如果异常处理过程中发现超出当前节点权限，应上推。

上推条件包括：

1. Manager 无法处理。
2. 需要跨部门资源。
3. 需要老板确认方案。
4. 异常等级升为 high 或 critical。
5. 处理任务多次失败。
6. 异常影响范围扩大。
7. 异常与历史决策冲突。

上推后，异常状态可以变为：

```
waiting_decision
```

等待 Executive 或老板确认。

11.9 异常关闭规则

异常关闭表示该异常已经被处理完成，不再需要继续跟进。

关闭异常不能只看“有人回复了”，而要看是否满足关闭条件。

11.9.1 异常关闭条件

异常关闭至少满足以下条件之一：

1. 阻塞原因已经解除。
2. 对应任务已经完成并确认。
3. 上级决策已经确认并执行。
4. 数据缺失已经补齐。
5. 权限问题已经解决。
6. 异常被证明无效。
7. 相关事项被取消或撤回。
8. 已有新异常替代旧异常。

关闭时应填写关闭说明。

示例：

```
{
  "status": "closed",
  "close_reason": " 后端已确认新增 decision_logs 表，并提交字段草案。相关任务已完成。",
  "closed_by": 20001,
  "closed_at": "2026-05-04T19:00:00+08:00"
}
```

11.9.2 不允许关闭的情况

以下情况不应关闭异常：

1. 只是有人看到了异常，但未处理。
2. 只是生成了任务，但任务未完成。
3. 只是上推老板，但老板未确认。
4. 只是员工说“明天处理”，但没有明确动作。
5. 阻塞原因仍然存在。
6. 相关任务仍处于 blocked。
7. 异常来源数据仍未补齐。
8. 没有关闭说明。

异常关闭必须有结果依据。

11.9.3 关闭权限

关闭权限应与异常等级匹配。

异常等级	可关闭人
low	Staff Agent / 员工 / Manager
medium	Manager
high	Manager 或 Executive
critical	Executive / 老板 / 管理员

如果异常由老板决策产生，关闭时建议由 Executive 或任务确认方确认。

11.9.4 重新打开异常

异常关闭后，如果问题再次出现，应允许重新打开。

重新打开条件包括：

1. 同一问题再次阻塞任务。
2. 关闭依据被证明无效。
3. 老板或 Manager 要求重新处理。
4. 相关任务完成质量不达标。
5. 新数据表明风险仍然存在。

重新打开时，不应覆盖原关闭记录，而应新增状态更新：

```
{
  "status": "reopened",
  "reopen_reason": "任务虽已提交，但 Executive Agent 联调时发现字段仍无法支持多方案决策记录。",
  "reopened_by": 20001,
  "reopened_at": "2026-05-05T10:00:00+08:00"
}
```

11.10 异常 Schema 字段定义

MVP 阶段建议定义统一 Incident Schema，用于记录异常、风险、阻塞和处理过程。

11.10.1 Incident Schema 字段总览

字段名	类型	是否必填	说明
schema_id	string	是	固定为 schema_v1_incident
schema_version	string	是	默认 1.0.0
incident_id	string / number	否	异常 ID，创建后生成
org_id	number / string	是	组织 ID
department_id	number / string	否	所属部门
incident_title	string	是	异常标题
description	string	是	异常说明
incident_type	string	是	异常类型
severity	string	是	异常等级
status	string	是	异常状态
source_type	string	是	来源类型
source_id	string / number	是	来源对象 ID
source_record_ids	array	否	来源 Staff 记录
source_summary_ids	array	否	来源 Summary
related_task_ids	array	否	关联任务
related_decision_ids	array	否	关联决策
reporter_user_id	string / number	否	上报人
owner_user_id	string / number	否	处理负责人
owner_node_id	string	否	处理 Agent 节点
need_escalation	boolean	是	是否需要上推
escalation_level	string	否	上推层级
attempted_solution	string	否	已尝试方案
suggested_action	string	否	建议动作
resolved_at	string	否	解决时间
closed_at	string	否	关闭时间
close_reason	string	否	关闭原因
meta	object	否	扩展字段

11.10.2 异常类型枚举

incident_type 建议使用以下枚举：

值	含义
data_missing	数据缺失
schema_invalid	Schema 校验失败
task_blocked	任务阻塞
task_overdue	任务逾期
permission_issue	权限问题
decision_pending	决策待确认
decision_conflict	决策冲突
workflow_failed	流程失败
system_error	系统错误
business_risk	业务风险
cross_department_dependency	跨部门依赖
other	其他

11.10.3 异常状态枚举

status 建议使用以下枚举：

值	含义
open	已创建
triaged	已分类
assigned	已分配
in_progress	处理中
waiting_decision	等待上级决策
waiting_external	等待外部输入
resolved	已解决，待确认
closed	已关闭
cancelled	已取消
reopened	重新打开

11.10.4 Incident Schema JSON 示例

```
{
  "schema_id": "schema_v1_incident",
  "schema_version": "1.0.0",
  "org_id": 1,
  "department_id": 12,
  "incident_title": "Decision 表结构未确认导致任务阻塞",
  "description": "当前 Agent mock 流程中存在 decision_logs 概念，但正式 DDL 尚未建立独立决策表，
  ↪ 影响 Executive 决策确认和任务生成链路。",
  "incident_type": "task_blocked",
  "severity": "high",
  "status": "open",
  "source_type": "manager_summary",
  "source_id": 801,
```

```
  "source_record_ids": [301, 302],
  "source_summary_ids": [801],
  "related_task_ids": [9001],
  "related_decision_ids": [],
  "reporter_user_id": 20001,
  "owner_user_id": 20002,
  "owner_node_id": "manager_node_dept_12",
  "need_escalation": true,
  "escalation_level": "executive",
  "attempted_solution": "已在 Manager Schema 中列出 A/B 方案，建议确认是否新增独立
  ↪  decision_logs 表。",
  "suggested_action": " 建议 Executive Agent 将该事项整理为老板决策卡片。",
  "resolved_at": null,
  "closed_at": null,
  "close_reason": null,
  "meta": {
    "agent_template_version": "manager_agent_v0.1",
    "input_channel": "scheduled_summary"
  }
}
```

11.11 异常与任务的关系

异常记录问题，任务推动解决。两者不能混用。

异常回答的是：

```
发生了什么问题？
为什么这是问题？
影响范围是什么？
需要谁处理？
```

任务回答的是：

```
接下来要做什么？
谁来做？
什么时候完成？
完成标准是什么？
```

关系如下：

```
Incident
  ↓
生成一个或多个 Task
  ↓
Task 执行
  ↓
```

```
Task 完成
  ↓
Incident 关闭
```

11.11.1 一个异常生成一个任务

简单异常可以生成一个任务。

示例：

异常：

```
员工日报缺失。
```

任务：

```
提醒员工在今日 20:00 前补交日报。
```

11.11.2 一个异常生成多个任务

复杂异常可能需要多个任务协同处理。

示例：

异常：

```
Decision 表结构未确认，影响老板决策和任务下发。
```

可生成任务：

1. 后端补充 decision_logs 表结构草案。
2. 架构负责人确认 Decision Schema 字段。
3. Agent 客制化更新 Executive Skill 中的决策提交逻辑。
4. 后端补充任务来源映射。

11.11.3 多个异常合并为一个任务

如果多个异常本质上指向同一个问题，可以合并处理。

示例：

多个员工上报：

1. 无法提交决策结果。
2. 任务来源缺失。
3. Executive 卡片确认后无法生成任务。

   这些可以合并为一个任务：

```
打通 Executive 决策确认到任务生成链路。
```

合并时应保留所有来源异常 ID，避免丢失上下文。

11.11.4 任务阻塞反向生成异常

任务在执行过程中如果被标记为 blocked，也应生成或更新异常。

流程如下：

```
Task 状态变为 blocked
  ↓
填写 blocked_reason
  ↓
生成 Incident 或更新已有 Incident
  ↓
通知 Manager
  ↓
必要时上推 Executive
```

这样可以避免任务阻塞只停留在任务状态里，而没有进入风险处理流程。

11.11.5 异常关闭依赖任务结果

如果异常生成了任务，异常关闭通常应依赖任务结果。

建议规则：

1. 所有关联任务已完成并确认，异常才可关闭。
2. 如果任务取消，异常不能自动关闭，需要说明原因。
3. 如果任务被撤回，异常应重新判断是否仍存在。
4. 如果任务完成但问题未解决，异常应保持 open 或 reopened。
5. 如果异常被证明无效，可以关闭，但需记录原因。

异常和任务之间必须保留引用关系：

```
{
  "incident_id": 501,
  "related_task_ids": [9001, 9002]
}
```

11.12 异常与审计日志的关系

异常处理过程必须进入审计日志。审计日志不替代异常表，但记录异常从发现到关闭的关键动作。

11.12.1 需要审计的异常事件

建议记录以下事件：

事件	说明
incident_created	异常创建
incident_triaged	异常完成分类
incident_assigned	异常分配处理人
incident_escalated	异常上推
incident_task_generated	异常生成任务
incident_status_changed	异常状态变化
incident_resolved	异常标记解决
incident_closed	异常关闭
incident_reopened	异常重新打开
incident_cancelled	异常取消
incident_merged	异常合并
incident_permission_denied	异常处理权限不足

11.12.2 审计记录内容

每条异常审计日志至少应包含：

1. 操作人。
2. 操作 Agent。
3. 所属组织。
4. 异常 ID。
5. 操作类型。
6. 操作时间。
7. 操作前状态。
8. 操作后状态。
9. 来源对象。
10. 相关任务。
11. 相关决策。
12. 操作渠道。
13. 关键 payload 摘要。

示例：

```
{
  "event_type": "incident_escalated",
  "org_id": 1,
  "actor_user_id": 20001,
  "actor_node_id": "manager_node_dept_12",
```

```
  "target_type": "incident",
  "target_id": 501,
  "status_before": "assigned",
  "status_after": "waiting_decision",
  "source_type": "manager_summary",
  "source_id": 801,
  "related_task_ids": [9001],
  "related_decision_ids": [],
  "event_at": "2026-05-04T10:30:00+08:00",
  "payload_summary": "Decision 表结构未确认，Manager 判断超出部门权限，上推 Executive。"
}
```

11.12.3 审计与责任划分

异常审计的价值不只是排查系统问题，也用于责任划分。

它可以回答：

1. 异常是谁发现的？
2. 什么时候发现的？
3. 由谁判断等级？
4. 是否及时上推？
5. 分配给谁处理？
6. 是否生成任务？
7. 任务是否完成？
8. 谁确认关闭？
9. 是否反复出现？
10. 是否存在越权处理？

对于 AutoMage-2 这种组织级系统，异常审计是建立企业信任的基础能力。

11.12.4 审计与复盘

Dream 和 Executive Agent 后续可以读取异常审计结果，用于复盘组织运行质量。

例如：

1. 哪类异常最常出现。
2. 哪个部门异常关闭最慢。
3. 哪些异常经常被重新打开。
4. 哪些任务最容易阻塞。
5. 哪些决策经常超时。
6. 哪些异常上推质量较低。

7. 哪些 Agent 经常生成无效异常。

这些数据后续可以用于组织诊断，但 MVP 阶段只需要先把审计数据记录下来，不必立刻做复杂分析。

11.13 本章小结

异常与风险识别机制，是 AutoMage-2 保证组织数据闭环不失真的关键。

Staff Agent 负责发现一线问题，Manager Agent 负责归并部门异常，Executive Agent 和 Dream 负责识别组织级风险。异常不是为了增加流程，而是为了让真正影响任务、决策和交付的问题不被日报和聊天记录淹没。

MVP 阶段需要先跑通五件事：

1. Staff 层能上报阻塞和支持需求。
2. Manager 层能合并部门异常并判断是否上推。
3. Executive 层能识别组织级风险并生成决策项。
4. 异常能生成任务，任务结果能反向关闭异常。
5. 异常全过程能被审计和追溯。

只要异常机制跑通，AutoMage-2 就不只是记录“大家做了什么”，还可以主动发现“哪里卡住了、谁该处理、什么时候必须解决”。

12 数字签名与数据可信机制

12.1 为什么需要数字签名

AutoMage-2 的核心不是普通聊天，也不是简单日报工具，而是要让 Agent 参与企业组织运行。只要系统开始记录员工日报、部门汇总、老板决策、任务下发和异常处理，就必须解决一个基础问题：**这些数据是谁确认的，确认时内容是什么，后续有没有被改过。**

如果没有数字签名或等价的确认机制，系统会出现几个问题：

1. 员工可能认为日报内容是 Agent 自动整理的，不能代表本人意见。
2. Manager 可能认为部门汇总是系统自动生成的，不应由自己负责。
3. 老板确认过的决策，如果没有记录确认动作，后续任务来源无法追溯。
4. 任务执行人可能不清楚任务是由谁下发、基于什么决策产生的。
5. 异常处理过程中，如果状态被修改，无法判断是谁改的、何时改的。
6. 后续复盘时，只能看到最终结果，看不到责任链路。

因此，数字签名机制的目的不是为了增加复杂流程，而是为了让关键数据有明确责任边界。

在 AutoMage-2 中，Agent 可以辅助生成内容，但不能替人承担责任。数字签名解决的就是这条边界：

```
Agent 负责整理；
人负责确认；
后端负责记录；
数据库负责留痕。
```

MVP 阶段的数字签名不必一开始就做到复杂密码学级别，但必须先建立“确认、哈希、留痕、校验”的基本机制。否则后续系统无法被企业信任，也很难做审计和责任划分。

12.2 MVP 阶段的数字签名定义

MVP 阶段的数字签名，指系统在关键数据被确认时，记录确认人、确认时间、确认对象、确认内容哈希、确认来源和校验状态的一组机制。

它不等同于完整法律意义上的电子签章，也不要求一开始就实现 CA 证书、私钥签名、时间戳服务等复杂能力。MVP 阶段更关注的是组织内部可信闭环。

MVP 阶段数字签名至少要证明四件事：

1. **谁确认了这份数据。**

   例如某员工确认了自己的日报，某 Manager 确认了部门汇总，某老板确认了决策方案。

1. **什么时候确认的。**

   确认时间必须由后端记录，不能只依赖 Agent 或前端传入。

1. **确认时的数据内容是什么。**

   通过 `payload_hash` 记录确认时的内容摘要，防止后续内容被悄悄修改。

1. **确认动作来自哪里。**
   例如来自 IM 按钮、Web 页面、Agent 对话确认、后台操作等。
   MVP 阶段的签名可以理解为：

```
用户身份
+
确认动作
+
确认时间
+
数据内容哈希
+
审计日志
```

对应到数据结构上，签名对象应至少包含：

```
{
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-03T18:13:00+08:00",
  "payload_hash": "sha256:xxxxx",
  "signature_source": "im",
  "verify_status": "verified"
}
```

MVP 阶段应明确一点：**签名不是装饰字段。**

只要某个数据对象要求签名，但当前签名状态不是 `signed` 或 `verified`，该对象就不应进入正式汇总、正式决策或正式任务下发流程。

12.3 数字签名适用对象

数字签名不需要覆盖所有系统数据。普通提醒、临时草稿、系统自动日志不一定需要签名。但凡涉及责任归属、决策确认、任务执行和异常关闭的对象，都应进入签名机制。

MVP 阶段建议优先覆盖五类对象：

1. Staff 日报。
2. Manager 汇总。
3. Executive 决策。
4. 任务确认。
5. 异常处理。

12.3.1 Staff 日报

Staff 日报是员工个人工作记录，必须有员工本人确认。

Staff Agent 可以帮助员工整理日报，但不能替员工确认日报。员工确认前，日报只能是草稿；员工确认后，才进入正式 Work Record，并可被 Manager Agent 汇总。

Staff 日报签名需要记录：

字段	说明
signed_by	员工本人用户 ID
signed_at	员工确认时间
payload_hash	日报内容哈希
signature_source	确认来源，如 IM、Web
signature_status	签名状态
verify_status	后端校验状态

Staff 日报签名流程：

```
员工填写日报
  ↓
Staff Agent 整理 Staff Schema
  ↓
员工查看结构化内容
  ↓
员工点击确认
  ↓
后端生成 payload_hash
  ↓
写入签名信息
  ↓
日报进入正式汇总范围
```

如果员工未确认，Manager Agent 可以看到“该员工日报未确认”这个状态，但不应将其作为正式数据参与部门汇总。

12.3.2 Manager 汇总

Manager 汇总是部门级判断结果，需要部门负责人确认或至少标记确认状态。

Manager Agent 可以自动读取 Staff Schema 并生成部门汇总，但部门健康度、Top 风险、待上推事项、部门调整建议等内容会影响老板判断，因此需要明确责任主体。

Manager 汇总签名需要记录：

字段	说明
signed_by	部门负责人或授权 Manager
signed_at	汇总确认时间
payload_hash	Manager Schema 内容哈希
source_record_hash	可选，来源 Staff 记录集合摘要
signature_source	确认来源
signature_status	签名状态

Manager 汇总可以支持两种状态：

状态	含义
signed	部门负责人已确认
pending_confirm	系统已生成，但负责人未确认

对于普通汇总，MVP 阶段可以允许 pending_confirm 状态进入 Executive 参考，但应明确标记“未确认”。对于包含高风险、上推老板决策或部门任务下发的 Manager Schema，建议必须签名后才能进入正式流程。

12.3.3 Executive 决策

Executive 决策是签名机制中最关键的对象。

老板确认某个方案后，系统会生成任务并影响组织执行。如果没有签名，后续无法证明老板选择了哪个方案，也无法解释任务为什么产生。

Executive 决策签名需要记录：

字段	说明
signed_by	老板或授权高管
signed_at	决策确认时间
confirmed_option	选择的方案
payload_hash	确认时决策内容哈希
signature_source	IM、Web、后台等
generated_task_ids	确认后生成的任务
verify_status	后端校验状态

Executive 决策签名流程：

```
Executive Agent 生成决策卡片
  ↓
老板查看 A/B 方案
  ↓
老板选择方案或自定义方案
  ↓
后端校验老板身份和权限
  ↓
记录 confirmed_option
  ↓
生成 payload_hash
  ↓
写入签名和审计日志
  ↓
生成正式任务
```

没有老板签名，generated_tasks 只能是任务草案，不能变成正式任务。

12.3.4 任务确认

任务确认包括任务接收、任务完成、任务关闭等动作。不是每一步都必须强签名，但关键任务状态变化必须留痕。

MVP 阶段建议以下动作需要确认记录：

1. 任务创建。
2. 任务分配。
3. 执行人确认开始。
4. 执行人提交完成。
5. 确认方关闭任务。
6. 任务取消。
7. 任务截止时间变更。
8. 任务优先级变更。

任务确认签名可以根据任务等级区分：

任务类型	签名要求
普通个人任务	记录操作人和时间即可
Manager 下发任务	Manager 或执行人确认
老板决策生成任务	必须保留来源决策签名
高优任务关闭	需要确认方确认
跨部门任务	需要相关 Manager 确认

任务完成签名示例：

```
{
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-04T18:30:00+08:00",
  "payload_hash": "sha256:task_result_hash",
  "signature_source": "im",
  "verify_status": "verified"
}
```

任务关闭时，系统应记录关闭人，而不是只记录执行人提交完成。

12.3.5 异常处理

异常处理涉及责任划分和风险处置，因此关键状态变化需要签名或确认记录。

需要签名或确认的异常动作包括：

1. 异常创建。
2. 异常分级。
3. 异常上推。

4. 异常处理任务生成。
5. 异常解决。
6. 异常关闭。
7. 异常重新打开。
8. 异常取消。

不同等级异常的签名要求不同：

异常等级	签名要求
low	可以由 Agent 自动记录，必要时员工确认
medium	Manager 确认处理结果
high	Manager 或 Executive 确认
critical	Executive、老板或管理员确认

异常关闭尤其需要注意。如果异常只是生成了任务，但任务没有完成，不应签名关闭。关闭签名必须基于处理结果。

异常关闭签名示例：

```
{
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 20001,
  "signed_at": "2026-05-04T19:00:00+08:00",
  "payload_hash": "sha256:incident_close_hash",
  "signature_source": "web",
  "verify_status": "verified"
}
```

12.4 签名字段设计

MVP 阶段建议所有需要签名的对象统一使用一组签名字段，避免 Staff、Manager、Executive、Task、Incident 各自定义一套字段。

统一签名对象建议如下：

```
{
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-03T18:13:00+08:00",
  "payload_hash": "sha256:xxxxx",
  "signature_type": "simple_hash",
  "signature_source": "im",
  "verify_status": "verified",
  "signature_version": "1.0.0"
}
```

字段说明如下：

字段名	类型	是否必填	说明
signature_required	boolean	是	当前对象是否需要签名
signature_status	string	是	签名状态
signed_by	string / number	条件必填	签名人用户 ID
signed_at	string	条件必填	签名时间
payload_hash	string	条件必填	签名内容哈希
signature_type	string	是	签名类型
signature_source	string	是	签名来源
verify_status	string	是	后端校验状态
signature_version	string	否	签名机制版本
extra	object	否	扩展信息

signature_status 建议枚举如下：

值	含义
not_required	不需要签名
pending	等待签名
signed	已签名
rejected	拒绝签名
expired	签名超时
invalid	签名无效
revoked	签名已撤回

verify_status 建议枚举如下：

值	含义
not_checked	尚未校验
verified	校验通过
failed	校验失败
payload_changed	内容已变更
permission_denied	签名人无权限
expired	校验超时

signature_type MVP 阶段建议先使用：

值	含义
simple_hash	简化哈希签名
account_confirm	企业账号确认
im_action	IM 按钮确认
web_action	Web 操作确认
system	系统自动记录
external_signature	后续外部电子签能力

signature_source 建议使用：

值	含义
`im`	IM 确认
`web`	Web 页面确认
`agent_chat`	Agent 对话确认
`backend_admin`	后台管理员确认
`scheduled_job`	定时任务生成
`api`	API 调用确认

签名字段应尽量放在业务对象中，同时关键动作写入审计日志。业务对象保存当前签名状态，审计日志保存签名动作过程。

12.5 签名生成流程

签名生成流程应由后端控制，Agent 和前端不能自行伪造签名结果。

MVP 阶段推荐签名生成流程如下：

```
生成待确认内容
  ↓
展示给用户
  ↓
用户执行确认动作
  ↓
后端校验身份和权限
  ↓
后端规范化 payload
  ↓
生成 payload_hash
  ↓
写入签名字段
  ↓
写入审计日志
  ↓
返回签名结果
```

12.5.1 生成待确认内容

Agent 先生成待确认内容，例如 Staff Schema、Manager Schema、Executive 决策卡片、任务完成结果或异常关闭说明。

这些内容在用户确认前应处于草稿或待确认状态。

示例状态：

对象	确认前状态
Staff 日报	`draft`
Manager 汇总	`pending_confirm`
Executive 决策	`pending`
任务完成	`completed_pending_confirm`
异常关闭	`resolved`

12.5.2 展示给用户

系统必须把需要确认的关键内容展示给用户。

不能让用户只点击一个“确认”按钮，却不知道确认的内容是什么。

展示内容至少包括：

1. 对象标题。
2. 核心内容摘要。
3. 关键字段。
4. 来源信息。
5. 确认后影响。
6. 是否会生成任务。
7. 是否会进入上级汇总或审计。

例如老板确认决策时，卡片应明确显示：

1. 决策事项。
2. 方案 A。
3. 方案 B。
4. 推荐方案。
5. 确认后生成的任务。

12.5.3 用户执行确认动作

确认动作可以来自：

1. IM 按钮。
2. Web 页面按钮。
3. Agent 对话中的明确确认。
4. 后台管理操作。
5. API 调用。

MVP 阶段建议优先使用 IM 按钮或 Web 按钮，因为它们比自然语言“嗯”“可以”“就这样”更清晰。

如果使用 Agent 对话确认，应要求明确语句，例如：

而不是将模糊回复自动识别为确认。

12.5.4 后端校验身份和权限

用户确认后，后端必须校验：

1. 当前用户是否登录。
2. 当前用户是否有权确认该对象。
3. 当前对象是否处于可确认状态。
4. 当前对象是否属于该组织。
5. 当前 Agent 是否具备对应节点权限。
6. 是否存在重复确认。
7. 是否已过确认有效期。

例如：

* Staff 日报只能由员工本人确认。* Manager 汇总只能由部门负责人或授权 Manager 确认。* Executive 决策只能由老板、高管或授权确认人确认。* 任务关闭只能由任务确认方或上级节点确认。* critical 异常关闭不能由普通员工确认。

权限校验失败时，不生成签名。

12.5.5 后端规范化 payload

生成 `payload_hash` 前，需要先规范化内容，避免同一份数据因为字段顺序、空格、换行差异导致哈希不同。

规范化建议：

1. 移除不参与签名的临时字段。
2. 字段按固定顺序序列化。
3. 时间统一格式。
4. 空值统一处理。
5. 数组顺序保持业务顺序。
6. JSON 序列化规则固定。
7. 只对确认内容本身做哈希，不对签名字段本身做哈希。

不参与签名的字段可以包括：

1. `signature` 自身。
2. 临时展示字段。
3. 前端 UI 字段。
4. 非业务 debug 信息。

5. 可变的请求追踪 ID。

12.5.6 生成 payload_hash

MVP 阶段建议使用 SHA-256 对规范化后的 payload 生成哈希。

示例：

```
payload_hash = sha256(canonical_payload)
```

存储格式建议：

```
"payload_hash": "sha256:8d969eef6ecad3c29a3a629280e686cf..."
```

后续如果需要接入更严格的电子签机制，可以在 `signature_type` 中区分。

12.5.7 写入签名字段

签名生成后，写入业务对象。

示例：

```
{
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 30001,
  "signed_at": "2026-05-04T08:16:30+08:00",
  "payload_hash": "sha256:decision_payload_hash",
  "signature_type": "simple_hash",
  "signature_source": "im",
  "verify_status": "verified",
  "signature_version": "1.0.0"
}
```

`signed_at` 应以后端服务器时间为准。

12.5.8 写入审计日志

签名生成后，必须写入审计日志。

审计日志应记录：

1. 操作人。
2. 操作 Agent。
3. 操作对象。
4. 签名前状态。
5. 签名后状态。

6. payload_hash。
7. 签名来源。
8. 请求渠道。
9. 操作时间。
10. 关键 payload 摘要。

签名字段说明“当前对象已经确认”，审计日志说明“确认动作是如何发生的”。

12.6 签名校验流程

签名校验用于判断签名后的数据是否仍然可信。

校验流程如下：

```
读取业务对象
  ↓
检查签名字段
  ↓
检查签名人权限
  ↓
重新计算 payload_hash
  ↓
对比已保存 hash
  ↓
更新 verify_status
  ↓
返回校验结果
```

12.6.1 检查签名字段

系统首先检查签名字段是否完整。

必查字段包括：

1. signature_required
2. signature_status
3. signed_by
4. signed_at
5. payload_hash
6. signature_type
7. signature_source
8. verify_status

如果 signature_required = true，但 signature_status != signed，该对象不能被视为正式确认数据。

12.6.2 检查签名人权限

系统需要检查签名人在签名时是否具备权限。

例如：

1. Staff 日报的 `signed_by` 必须等于日报对应员工。
2. Manager 汇总的 `signed_by` 必须是该部门负责人或授权用户。
3. Executive 决策的 `signed_by` 必须是老板、高管或授权确认人。
4. 任务关闭的 `signed_by` 必须是确认方。
5. critical 异常关闭的 `signed_by` 必须具备高层或管理员权限。

权限校验失败时，`verify_status` 应为：

```
permission_denied
```

12.6.3 重新计算 payload_hash

校验时，后端应对当前业务内容重新进行规范化，并计算新的 hash。

然后与签名时保存的 `payload_hash` 对比。

如果一致，说明签名后内容没有变化。如果不一致，说明签名后内容被修改过，必须标记为 `payload_changed`。

12.6.4 校验结果处理

校验结果建议如下：

校验结果	处理方式
`verified`	可进入正式流程
`payload_changed`	需要重新签名
`permission_denied`	签名无效
`failed`	不进入正式流程
`expired`	需要重新确认
`not_checked`	不应作为最终可信数据

例如，Manager 汇总签名后，如果有人修改了 `top_3_risks`，系统应检测到 hash 不一致，并要求重新签名。

12.6.5 何时触发校验

签名校验应在以下场景触发：

1. Staff 日报进入 Manager 汇总前。
2. Manager 汇总进入 Executive 分析前。
3. Executive 决策生成任务前。

4. 任务关闭前。
5. 异常关闭前。
6. 审计或复盘查询时。
7. 对象被修改后。
8. 导出报表或生成老板看板前。

MVP 阶段可以先在关键流程触发校验，不需要每次读取都强制重新计算，但进入上级流程前必须校验。

12.7 签名失败处理

签名失败不能静默忽略。不同失败原因需要不同处理方式。

12.7.1 未签名

情况：

```
signature_required = true
signature_status = pending
```

处理方式：

1. 对象保持待确认状态。
2. 不进入正式上级汇总。
3. 通知对应确认人。
4. 到期后触发提醒或标记超时。

示例：

员工日报未签名，则不进入正式 Manager 汇总，只在缺失或未确认列表中展示。

12.7.2 签名人无权限

情况：

```
verify_status = permission_denied
```

处理方式：

1. 拒绝确认动作。
2. 不写入正式签名。
3. 记录审计日志。
4. 提示用户无权限。
5. 必要时通知管理员。

示例：

普通员工尝试确认老板决策，应返回权限错误。

12.7.3 内容被修改

情况：

```
verify_status = payload_changed
```

说明签名后对象内容发生变化。

处理方式：

1. 标记原签名失效。
2. 对象状态回到待确认。
3. 记录修改人和修改内容。
4. 要求责任人重新确认。
5. 修改前后版本都保留审计记录。

示例：

Manager 已确认部门汇总后，`need_executive_decision` 被修改，则必须重新签名，不能沿用旧签名。

12.7.4 签名超时

某些确认动作应有有效期。例如老板决策卡片推送后 24 小时未确认，可以标记超时。

处理方式：

1. 状态变为 `expired`。
2. 不生成任务。
3. 通知相关 Manager 或 Executive。
4. 必要时重新生成决策卡片。
5. 保留原卡片审计记录。

超时不等于驳回，只代表该确认动作没有在有效时间内完成。

12.7.5 签名被拒绝

用户可以拒绝签名。

例如：

1. 员工认为 Staff Agent 整理的日报不准确。
2. Manager 不认可部门汇总。
3. 老板驳回决策方案。
4. 任务确认方认为任务未完成。
5. 异常关闭人认为异常尚未解决。

处理方式：

1. 状态变为 `rejected` 或对应业务状态。

2. 记录拒绝人和拒绝原因。
3. 返回给生成方修改。
4. 不进入正式流程。
5. 保留审计日志。

拒绝签名是正常流程，不应被视为系统错误。

12.7.6 签名系统异常

如果签名过程本身出现系统错误，例如 hash 生成失败、数据库写入失败、审计日志失败，应中断正式流程。

处理方式：

1. 不标记为已签名。
2. 返回系统错误。
3. 暂存待确认内容。
4. 通知管理员或重试。
5. 避免生成正式任务或上级汇总。

签名失败时宁可阻断关键流程，也不要让未确认数据继续执行。

12.8 签名与责任划分

数字签名的核心价值是责任划分。它要明确区分 Agent、员工、Manager、老板和后端系统各自承担什么责任。

12.8.1 Agent 的责任

Agent 的责任是整理、提示、生成草案和调用接口。

Agent 负责：

1. 按 Schema 整理数据。
2. 标记字段缺失。
3. 提醒用户确认。
4. 生成摘要和建议。
5. 保留来源引用。
6. 调用后端接口。

Agent 不负责：

1. 代替用户确认。
2. 代替老板决策。
3. 承担人类承诺。

4. 绕过后端生成签名。
5. 修改已签名内容而不留痕。

如果 Agent 生成了错误草案，但用户没有确认，该草案不应成为正式业务数据。如果用户确认了草案，则用户对确认内容承担责任，系统仍需保留 Agent 生成记录。

12.8.2 员工的责任

员工对自己确认的 Staff 日报、任务状态和产出物负责。

员工确认后，表示：

1. 日报内容基本代表本人工作情况。
2. 任务状态由本人提交。
3. 提交的产出物链接或附件有效。
4. 上报的问题和支持需求真实存在。
5. 对确认时的内容承担个人确认责任。

员工不应为 Agent 未经确认生成的草稿负责。

12.8.3 Manager 的责任

Manager 对已确认的部门汇总、部门内决策和部门任务分配负责。

Manager 确认后，表示：

1. 部门汇总基本反映部门情况。
2. 风险判断和阻塞事项已经过确认。
3. 部门内任务调整是授权行为。
4. 上推老板的事项具备必要背景和来源。
5. 部门内异常处理结论可被追溯。

如果 Manager Schema 是 Agent 自动生成但未确认，应标记为待确认，不能直接当作 Manager 责任结论。

12.8.4 Executive / 老板的责任

老板或高管对自己确认的 Executive 决策负责。

老板确认后，表示：

1. 已选择某个方案或给出自定义方案。
2. 允许系统基于该方案生成任务。
3. 该决策进入组织执行链路。
4. 后续任务来源可以追溯到该确认动作。

老板未确认的决策草案，不应生成正式任务，也不应被视为老板意见。

12.8.5 后端系统的责任

后端负责确保签名可信、权限正确、数据不被无痕修改。

后端负责：

1. 鉴权。
2. 权限校验。
3. Schema 校验。
4. payload hash 生成。
5. 签名字段写入。
6. 签名校验。
7. 审计日志记录。
8. 状态流转控制。
9. 防止未签名数据进入正式流程。

后端不负责替业务判断内容是否“聪明”，但必须保证确认流程本身可靠。

12.8.6 责任边界示例

以员工日报为例：

```
Agent 整理日报草稿
  → Agent 对整理过程负责
员工确认日报
  → 员工对确认内容负责
后端生成签名
  → 后端对签名记录负责
Manager 读取汇总
  → Manager 对部门汇总判断负责
```

以老板决策为例：

```
Executive Agent 生成 A/B 方案
  → Agent 对方案整理负责
老板选择方案 A
  → 老板对选择动作负责
后端创建任务
  → 后端对任务来源和状态负责
员工执行任务
  → 员工对执行结果负责
```

这种责任划分是 AutoMage-2 进入企业场景的基础。

12.9 签名与审计日志

签名字段记录当前对象是否被确认，审计日志记录确认动作发生的过程。

二者不能互相替代。

12.9.1 签名字段记录当前状态

业务对象中的签名字段回答的是：

1. 当前是否需要签名？
2. 当前是否已签名？
3. 谁签的？
4. 什么时候签的？
5. 签名内容 hash 是什么？
6. 校验状态是什么？

它适合用于业务流程判断。例如：未签名的 Staff 日报不进入 Manager 汇总；未确认的 Executive 决策不生成任务。

12.9.2 审计日志记录过程

审计日志回答的是：

1. 谁发起了确认？
2. 通过什么渠道确认？
3. 确认前对象是什么状态？
4. 确认后对象是什么状态？
5. 是否有失败或重试？
6. 签名是否被撤回？
7. 内容是否后来被修改？
8. 修改后是否重新签名？

审计日志适合用于排查、复盘和责任追踪。

12.9.3 需要写入审计的签名事件

建议记录以下签名事件：

事件	说明
signature_requested	请求签名
signature_signed	完成签名
signature_rejected	拒绝签名
signature_expired	签名超时
signature_invalid	签名无效
signature_revoked	签名撤回
signature_payload_changed	签名后内容变更
signature_verified	签名校验通过
signature_verify_failed	签名校验失败

12.9.4 审计日志字段建议

签名相关审计日志建议包含：

字段	说明
event_type	审计事件类型
org_id	组织 ID
actor_user_id	操作人
actor_node_id	操作 Agent
target_type	目标对象类型
target_id	目标对象 ID
signature_status_before	操作前签名状态
signature_status_after	操作后签名状态
payload_hash	内容哈希
signature_source	签名来源
ip_address	可选，请求 IP
user_agent	可选，客户端信息
event_at	操作时间
payload_summary	内容摘要

示例：

```
{
  "event_type": "signature_signed",
  "org_id": 1,
  "actor_user_id": 30001,
  "actor_node_id": "executive_node_org_1",
  "target_type": "decision",
  "target_id": "decision_0001",
  "signature_status_before": "pending",
  "signature_status_after": "signed",
  "payload_hash": "sha256:decision_payload_hash",
  "signature_source": "im",
  "event_at": "2026-05-04T08:16:30+08:00",
  "payload_summary": " 老板确认方案 A: 新增独立 decision_logs 表。"
}
```

12.9.5 签名后修改的审计要求

已签名对象如果被修改，必须记录修改动作，并使原签名失效或生成新版本。

处理方式：

1. 保存原版本。
2. 记录修改人。
3. 记录修改字段。
4. 标记原签名 `payload_changed` 或 `revoked`。
5. 要求重新确认。
6. 写入审计日志。

不允许直接覆盖已签名内容。

示例：

```
Manager 汇总已确认
  ↓
有人修改 top_3_risks
  ↓
原 payload_hash 不再匹配
  ↓
signature.verify_status = payload_changed
  ↓
Manager 需要重新确认
```

这是保证数据可信的关键规则。

12.10 MVP 阶段可接受的简化实现

MVP 阶段不需要一次性实现完整电子签章系统，但必须实现可用的简化签名闭环。

建议先做“账号确认 + 内容哈希 + 审计日志”的轻量版本。

12.10.1 简化实现目标

MVP 阶段签名机制至少要做到：

1. 用户必须登录或具备明确身份。
2. 关键对象确认前后状态清楚。
3. 确认时生成 `payload_hash`。
4. 记录确认人和确认时间。
5. 记录确认来源。
6. 签名后内容变更可检测。

7. 未签名数据不能进入关键流程。
8. 所有确认动作写入审计日志。

不要求一开始实现：

1. CA 证书。
2. 法律电子签章。
3. 用户私钥。
4. 第三方时间戳服务。
5. 硬件密钥。
6. 复杂多方签署。
7. 区块链存证。

12.10.2 推荐简化方案

MVP 阶段推荐如下方案：

```
企业账号登录
  +
IM / Web 明确确认动作
  +
后端生成 SHA-256 payload_hash
  +
业务对象写入 signature
  +
audit_logs 写入签名事件
```

该方案开发成本低，足够支撑 MVP 内部演示和团队联调。

12.10.3 最小落地字段

如果时间非常紧，最小字段可以先保留以下几个：

字段	说明
signature_status	签名状态
signed_by	签名人
signed_at	签名时间
payload_hash	内容哈希
signature_source	签名来源

最小示例：

```
{
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-03T18:13:00+08:00",
```

```
  "payload_hash": "sha256:xxxxx",
  "signature_source": "im"
}
```

后续再扩展 `verify_status`、`signature_type`、`signature_version` 等字段。

12.10.4 MVP 阶段签名规则建议

MVP 阶段建议先锁定以下规则：

对象	MVP 签名规则
Staff 日报	员工确认后才进入正式汇总
Manager 汇总	普通汇总可待确认，高风险和上推事项必须 Manager 确认
Executive 决策	老板确认后才生成任务
任务完成	执行人提交完成，确认人关闭任务
异常关闭	根据异常等级由对应权限人确认关闭

12.10.5 不同阶段的演进路线

签名机制可以分阶段演进。

阶段	能力
v0.1	账号确认 + payload_hash + 审计日志
v0.2	签名校验、签名失效、重新签名
v0.3	多角色确认、任务关闭确认、异常关闭确认
v0.4	签名版本管理、签名有效期、签名撤回
v1.0	企业级电子签、外部存证、合规审计能力

MVP 阶段优先完成 v0.1 和部分 v0.2 能力即可。

12.10.6 MVP 不应省略的底线

虽然 MVP 可以简化，但以下能力不建议省略：

1. Staff 日报必须有员工确认。
2. 老板决策必须有老板确认。
3. 确认动作必须写入审计日志。
4. 确认内容必须生成 hash。
5. 签名后修改必须重新确认。
6. 未确认老板决策不能生成正式任务。
7. 签名人权限必须由后端校验。
8. 关键状态不能只存在 IM 聊天记录中。

如果这些底线被省略，系统演示可能仍然能跑，但后续很难进入真实企业场景。

12.11 本章小结

数字签名与数据可信机制，是 AutoMage-2 能否被企业接受的基础能力。

Staff Agent 可以整理日报，但员工必须确认；Manager Agent 可以生成部门汇总，但部门负责人需要对关键判断负责；Executive Agent 可以生成老板决策项，但重大任务必须在老板确认后才能下发。签名机制把这些确认动作变成可记录、可校验、可追溯的数据。

MVP 阶段不需要一开始就实现复杂电子签章，但必须跑通最小可信闭环：

```
用户确认
  ↓
后端校验权限
  ↓
生成内容哈希
  ↓
写入签名字段
  ↓
写入审计日志
  ↓
进入正式流程
```

只要这条链路稳定，AutoMage-2 的日报、汇总、决策、任务和异常处理就不再只是 Agent 生成的文本，而是具备责任边界和审计价值的组织数据。

13 前端 / IM / Agent 校验逻辑

13.1 校验逻辑总览

AutoMage-2 MVP 中，数据从用户输入到最终写入数据库，至少会经过四层校验：

```
前端 / IM 校验
  ↓
Agent 生成前校验
  ↓
Agent 生成后校验
  ↓
后端 Schema / 权限 / 签名校验
  ↓
写入数据库
```

这四层校验的职责不同。

前端和 IM 校验主要负责提升用户体验，尽早发现明显缺失和格式错误；Agent 校验主要负责保证生成内容符合业务语义和 Schema 结构；后端校验负责最终兜底，决定数据是否可以进入正式业务流程。

需要明确一点：**前端、IM 和 Agent 的校验都不能代替后端校验。**

原因很简单。前端可能被绕过，IM 消息可能不完整，Agent 输出可能不稳定。最终能否写入数据库、能否进入 Manager 汇总、能否生成老板决策和任务，必须由后端统一判断。

整体校验职责建议如下：

校验层	主要职责	是否最终可信
前端表单校验	检查用户输入是否完整、格式是否明显错误	否
IM 交互校验	引导用户补全关键信息，避免空输入	否
Agent 生成前校验	判断输入是否足够生成 Schema	否
Agent 生成后校验	检查输出是否符合 Schema 结构	否
后端 Schema 校验	校验字段、类型、枚举、权限、签名、状态	是
数据库约束	保证唯一性、外键、状态一致性	是

MVP 阶段建议采用“前面尽量拦，后端必须拦”的策略。前面拦是为了减少用户反复提交失败；后端拦是为了保证系统数据可信。

13.2 前端表单校验

前端表单校验适用于 Web、App、小程序或 IM 表单场景。它的目标是让用户在提交前就知道哪些信息缺失，避免把明显错误的数据交给 Agent 或后端。

前端表单校验主要包括：

1. 必填字段是否为空。
2. 字段长度是否超过限制。

3. 数字、日期、布尔值格式是否正确。
4. 枚举选项是否在允许范围内。
5. 条件必填字段是否已填写。
6. 文件、链接、附件格式是否合理。
7. 用户是否进行了确认动作。

以前端 Staff 日报表单为例，至少应校验：

字段	前端校验规则
今日完成事项	至少填写一项
明日计划	至少填写一项
是否需要支持	必须选择是或否
支持详情	当选择“需要支持”时必填
风险等级	必须从枚举中选择
产出物链接	如果填写，必须是合法 URL
确认提交	必须由员工主动点击确认

前端展示文案可以面向用户优化，但提交字段必须与 Schema 保持一致。

例如，页面上可以展示为：

```
今日完成事项
```

但提交给后端时应使用：

```
{
  "work_progress": []
}
```

前端校验不应做过重的业务判断。例如，前端不需要判断某个风险是否应该上推老板，也不需要判断 Manager 是否有权读取某部门数据。这些应交给 Agent 和后端处理。

前端校验失败时，应提示用户具体字段，而不是只提示“提交失败”。

不推荐：

```
提交失败，请检查。
```

推荐：

```
请补充“需要支持的具体内容”。你已选择需要上级支持，但尚未说明需要谁支持、支持什么事项。
```

13.3 Agent 生成前校验

Agent 生成前校验发生在 Agent 准备把用户输入转成 Schema 之前。

它的目标是判断当前输入是否足够生成结构化数据。如果信息明显不足，Agent 应先追问，而不是强行生成一个看似完整但实际编造的 Schema。

以 Staff Agent 为例，员工可能只输入：

今天正常推进，明天继续。

这类输入不足以生成正式 Staff Schema。Agent 应追问：

请补充至少一项今日完成事项，并说明当前状态是已完成、进行中还是阻塞。

Agent 生成前需要检查：

检查项	说明
输入是否为空	空消息不能生成 Schema
是否包含核心业务内容	不能只有“正常”“继续”“OK”
是否缺少必填字段	缺字段先追问
是否存在明显矛盾	例如说无问题，但又说需要支持
是否缺少确认动作	生成前可整理，提交前必须确认
是否越权请求	用户要求查看无权数据时拒绝
是否需要更多上下文	缺少来源任务、部门、日期时补问

不同 Agent 的生成前校验重点不同。

Staff Agent 重点检查员工输入是否足够生成日报。Manager Agent 重点检查是否已经读取到足够的 Staff Schema。Executive Agent 重点检查是否有可用的 Manager Schema 和来源数据。Dream 重点检查输入窗口内的数据是否完整、是否已确认、是否可用于老板决策。

Agent 生成前校验的原则是：**不够就问，不清楚就标记，不编造。**

13.4 Agent 生成后校验

Agent 生成后校验发生在 Agent 已经输出 Schema，但尚未调用后端接口之前。

这个阶段主要检查 Agent 输出是否符合预期结构，减少后端 422 错误。

Agent 生成后至少需要校验：

1. 是否为合法 JSON。
2. 是否包含 `schema_id`。
3. 是否包含 `schema_version`。
4. 必填字段是否存在。
5. 字段类型是否正确。
6. 枚举值是否在允许范围内。
7. 条件必填字段是否满足。
8. 来源 ID 是否存在。
9. 签名状态是否合理。
10. 是否存在多余且危险的字段。

例如 Staff Schema 生成后，Agent 应检查：

```
need_support = true 时，support_detail 是否填写
work_progress 是否为数组
risk_level 是否属于 low / medium / high / critical
signature 是否为 pending 或 signed
```

如果 Agent 生成后发现错误，应优先在本地修正或追问用户，而不是直接提交后端。

示例：

```
我整理出的日报中缺少“明日计划”。请补充至少一项明天准备完成的事项。
```

Agent 生成后校验还应防止自由文本污染正式 JSON。正式提交给后端的数据不能夹杂解释性文字。

不推荐提交：

```
下面是整理好的 JSON:
{
  ...
}
请查收。
```

推荐提交：

```
{
  "schema_id": "schema_v1_staff",
  "schema_version": "1.0.0"
}
```

Agent 可以在聊天里展示摘要，但调用 API 时必须只传结构化 JSON。

13.5 后端 Schema 校验

后端 Schema 校验是最终校验层。所有正式进入数据库、参与上级汇总、生成决策和任务的数据，都必须通过后端校验。

后端校验至少包括：

1. Schema ID 是否存在。
2. Schema 版本是否支持。
3. 必填字段是否完整。
4. 字段类型是否正确。
5. 枚举值是否合法。
6. 条件必填是否满足。
7. 字段长度是否超限。
8. 来源对象是否存在。
9. 当前用户是否有权限。
10. 当前 Agent 是否有权限。
11. 签名状态是否满足要求。
12. 是否重复提交。

13. 状态流转是否合法。
14. 幂等键是否重复。

    后端收到数据后，不应因为 Agent “看起来生成得对”就直接写库。

    推荐校验流程如下：

```
接收请求
  ↓
鉴权
  ↓
识别 schema_id / schema_version
  ↓
加载对应校验规则
  ↓
字段和类型校验
  ↓
业务规则校验
  ↓
权限校验
  ↓
签名校验
  ↓
幂等校验
  ↓
写入数据库
  ↓
写入审计日志
```

后端校验失败时，应返回明确错误结构，方便 Agent 或前端修正。

推荐错误格式：

```
{
  "code": "SCHEMA_VALIDATION_FAILED",
  "message": "Schema 校验失败",
  "errors": [
    {
      "field": "support_detail",
      "reason": "need_support 为 true 时，support_detail 必填"
    }
  ]
}
```

这样 Agent 可以准确追问用户，而不是让用户重新填写整份内容。

13.6 字段类型校验

字段类型校验用于保证每个字段的数据类型稳定。类型不稳定会导致后端解析、数据库存储、Agent 汇总和前端展示全部变复杂。

MVP 阶段常见字段类型包括：

类型	示例
string	`"schema_v1_staff"`
number	`10086`
boolean	`true`
array	`[301, 302]`
object	`{ "score": 78 }`
datetime	`"2026-05-04T18:00:00+08:00"`
date	`"2026-05-04"`

类型校验要求如下：

13.6.1 string 字段

字符串字段必须是字符串，不能传对象或数组。

示例：

```
{
  "risk_level": "high"
}
```

不允许：

```
{
  "risk_level": ["high"]
}
```

13.6.2 boolean 字段

布尔字段必须是 `true` 或 `false`，不能用中文或数字代替。

正确：

```
{
  "need_support": true
}
```

错误：

```
{
  "need_support": " 需要"
}
```

13.6.3 array 字段

数组字段必须是数组。即使只有一项，也应使用数组。

正确：

```
{
  "source_record_ids": [301]
}
```

错误：

```
{
  "source_record_ids": 301
}
```

13.6.4 object 字段

对象字段必须是结构化对象，不能直接塞入长文本。

正确：

```
{
  "signature": {
    "signature_status": "signed",
    "signed_by": 10086
  }
}
```

错误：

```
{
  "signature": " 员工已确认"
}
```

13.6.5 datetime 字段

时间字段建议统一使用 ISO8601 格式。

正确：

```
{
  "timestamp": "2026-05-04T18:00:00+08:00"
}
```

错误：

```
{
  "timestamp": " 今天下午六点"
}
```

类型校验失败应返回 422，不应由后端自动猜测和转换。例如 `"true"` 不应自动转成 `true`，否则容易隐藏前端或 Agent 的问题。

13.7 必填字段校验

必填字段校验用于保证每个 Schema 具备最小可用数据。

不同 Schema 的必填字段不同，但都应包含基础身份、来源、核心内容和状态信息。

13.7.1 Staff Schema 必填字段

字段	说明
schema_id	标识 Schema 类型
schema_version	标识版本
timestamp	提交时间
org_id	组织 ID
department_id	部门 ID
user_id	员工 ID
node_id	Staff Agent 节点
record_date	日报日期
work_progress	今日完成事项
need_support	是否需要支持
next_day_plan	明日计划
risk_level	风险等级
signature	员工确认信息

13.7.2 Manager Schema 必填字段

字段	说明
schema_id	标识 Schema 类型
schema_version	标识版本
timestamp	汇总生成时间
org_id	组织 ID
dept_id	部门 ID
manager_user_id	部门负责人
manager_node_id	Manager Agent 节点
summary_date	汇总日期
staff_report_count	读取日报数量
missing_report_count	缺失日报数量
overall_health	部门健康度
aggregated_summary	部门摘要
top_3_risks	主要风险
pending_approvals	待上推事项数量
next_day_adjustment	次日调整建议
source_record_ids	来源日报
signature	Manager 确认信息

13.7.3 Executive Schema 必填字段

字段	说明
schema_id	标识 Schema 类型
schema_version	标识版本
timestamp	生成时间
org_id	组织 ID
executive_user_id	老板 / 高管 ID
executive_node_id	Executive Agent 节点
summary_date	摘要日期
business_summary	组织级摘要
key_risks	关键风险
decision_required	是否需要老板确认
source_summary_ids	来源部门汇总
human_confirm_status	人工确认状态

必填字段缺失时，后端返回 422。Agent 收到后应只追问缺失字段，不要要求用户重填全部内容。

13.8 枚举值校验

枚举值校验用于保证状态、等级、类型等字段使用统一值，避免同一个意思出现多种写法。

例如风险等级应统一为：

```
low / medium / high / critical
```

不能出现：

```
低 / 中 / 高 / 严重 / 紧急 / 很危险
```

MVP 阶段建议统一以下枚举。

13.8.1 风险等级

值	含义
low	低风险
medium	中风险
high	高风险
critical	严重风险

13.8.2 部门健康度

值	含义
green	正常
yellow	有风险但可控
red	高风险
gray	数据不足

13.8.3 任务优先级

值	含义
——	–
urgent	紧急
high	高
medium	中
low	低

13.8.4 任务状态

值	含义
pending	待处理
in_progress	进行中
blocked	阻塞
completed	已完成
closed	已关闭
cancelled	已取消

13.8.5 决策状态

值	含义
draft	草稿
pending	待确认
confirmed	已确认
rejected	已驳回
need_more_info	需要补充信息
task_generated	已生成任务
expired	已超时
closed	已关闭

13.8.6 签名状态

值	含义
`not_required`	不需要签名
`pending`	等待签名
`signed`	已签名
`rejected`	拒绝签名
`expired`	签名超时
`invalid`	签名无效
`revoked`	签名撤回

枚举值校验失败时，后端不应自动转换。例如收到 `" 高"`，应返回错误，让 Agent 或前端转换为 `"high"` 后重新提交。

13.9 权限校验

权限校验决定当前用户或 Agent 是否有权读取、创建、修改、确认某个对象。

权限校验必须由后端执行，不能只依赖前端隐藏按钮，也不能只依赖 Agent Prompt。

MVP 阶段权限校验至少覆盖四个维度：

1. 组织维度。
2. 部门维度。
3. 用户维度。
4. Agent 节点维度。

13.9.1 Staff Agent 权限

Staff Agent 只能操作当前员工个人范围内的数据。

可操作：

1. 提交本人 Staff Schema。
2. 查询本人任务。
3. 更新本人任务状态。
4. 上报本人异常。
5. 查看本人历史日报。

不可操作：

1. 读取其他员工日报。
2. 读取部门汇总。
3. 查看老板决策卡片。
4. 替其他员工确认日报。
5. 下发部门任务。

13.9.2 Manager Agent 权限

Manager Agent 只能操作本部门范围内的数据。

可操作：

1. 读取本部门 Staff Schema。
2. 生成本部门 Manager Schema。
3. 创建本部门任务。
4. 处理本部门异常。
5. 上推需老板决策事项。

不可操作：

1. 读取其他部门员工明细。
2. 修改员工原始日报。
3. 确认老板决策。
4. 越权生成跨部门任务。
5. 删除历史审计记录。

13.9.3 Executive Agent 权限

Executive Agent 可以读取组织级汇总和部门汇总。

可操作：

1. 读取 Manager Schema。
2. 生成 Executive Schema。
3. 生成老板决策卡片。
4. 根据老板确认生成任务。
5. 追踪组织级风险。

不可操作：

1. 绕过老板确认执行重大决策。
2. 随意读取无关个人隐私明细。
3. 修改员工原始日报。
4. 无审计删除决策或任务。

权限校验失败时：

错误码	含义
401	未认证或登录失效
403	已认证但无权限
404	数据不存在或不可见
422	参数不合法

建议对于跨部门敏感数据，返回 404 或脱敏结果，避免暴露其他部门数据是否存在。

13.10 签名校验

签名校验用于判断某个对象是否已经被有权限的人确认，确认后内容是否发生变化。

签名校验至少包含：

1. 是否需要签名。
2. 签名状态是否为 `signed`。
3. 签名人是否有权限。
4. 签名时间是否存在。
5. `payload_hash` 是否存在。
6. 当前内容重新计算 hash 后是否一致。
7. 签名是否过期。
8. 签名后内容是否被修改。

不同对象的签名要求不同。

对象	签名要求
Staff 日报	员工确认后进入正式汇总
Manager 汇总	高风险和上推事项必须 Manager 确认
Executive 决策	老板确认后才能生成任务
任务完成	执行人提交，确认人关闭
异常关闭	根据异常等级由对应权限人确认

签名校验失败时的处理：

失败原因	处理
未签名	保持待确认，不进入正式流程
签名人无权限	拒绝操作，记录审计
内容被修改	原签名失效，要求重新确认
签名超时	状态置为 expired，重新推送
hash 缺失	拒绝进入正式流程
校验异常	阻断关键流程，通知管理员

尤其需要注意：**老板决策未签名，不能生成正式任务。**

Executive Agent 可以生成任务草案，但后端只有在 `human_confirm_status = confirmed` 且签名校验通过后，才能创建正式任务。

13.11 重试机制

重试机制用于处理网络波动、模型超时、接口短暂失败等问题。

MVP 阶段需要区分“可以重试”和“不应该重试”的错误。

13.11.1 可以重试的场景

场景	处理
网络超时	使用同一幂等键重试
后端 5xx	延迟重试
IM 推送失败	重试推送
Agent 模型调用超时	重新调用或降级
Dream 运行超时	延迟重试或生成简版摘要

13.11.2 不建议自动重试的场景

场景	处理
401 未认证	重新登录或初始化
403 无权限	停止请求，记录审计
422 Schema 错误	补充或修正字段后再提交
409 重复提交	走更新或幂等返回
签名人无权限	不重试
状态流转非法	不重试，提示人工处理

重试必须配合幂等键，避免重复写入。

例如 Staff 日报提交可以使用：

```
staff_report:{org_id}:{user_id}:{record_date}
```

任务生成可以使用：

```
task_from_decision:{decision_id}:{option_id}
```

Dream 运行可以使用：

```
dream_run:{org_id}:{summary_date}:{run_type}
```

推荐重试策略：

次数	间隔
第 1 次	1 秒
第 2 次	3 秒
第 3 次	10 秒
第 4 次	30 秒
仍失败	标记失败并通知管理员

重试过程中不应重复通知用户，避免用户收到多条相同消息。

13.12 422 错误处理

422 表示请求格式或业务字段不符合 Schema 要求。这是 Agent 和前端最常遇到的错误。

常见 422 场景包括：

1. 缺少必填字段。
2. 字段类型错误。
3. 枚举值非法。
4. 条件必填未满足。
5. 字符串长度超限。
6. 日期格式错误。
7. 来源对象不存在。
8. 状态流转不合法。
9. `decision_required = true` 但 `decision_items` 为空。
10. `need_support = true` 但 `support_detail` 为空。

后端返回 422 时，应提供可机器读取的错误详情。

示例：

```
{
  "code": "SCHEMA_VALIDATION_FAILED",
  "message": " 提交内容不符合 Staff Schema",
  "errors": [
    {
      "field": "support_detail",
      "reason": "need_support 为 true 时，support_detail 必填",
      "suggestion": " 请补充需要谁支持、支持什么事项"
    }
  ]
}
```

Agent 收到 422 后，应根据 `errors` 精准追问。

示例：

```
日报暂未提交成功。你选择了“需要上级支持”，但没有填写具体支持内容。请补充需要谁支持、
↪ 支持什么事项。
```

不应让用户重新填写整份日报。

对于多字段错误，Agent 可以一次性列出需要补充的字段：

```
还需要补充 2 项信息：
1. 明日计划。
2. 需要支持的具体内容。
```

422 不是系统故障，而是正常的数据修正流程。MVP 阶段要把 422 处理好，否则用户会觉得系统“不稳定”。

13.13 401/403 权限错误处理

401 和 403 都与权限相关，但含义不同。

错误码	含义
401	未认证、登录失效、Token 无效
403	已认证，但无权访问或操作

13.13.1 401 处理

401 常见场景：

1. 用户未登录。
2. Token 过期。
3. Agent 初始化失败。
4. IM 用户未绑定系统账号。
5. API 凭证错误。

处理方式：

1. 停止当前写入操作。
2. 提示用户重新登录或绑定账号。
3. 暂存当前草稿。
4. 不重复提交。
5. 记录错误日志。

提示示例：

当前账号状态已失效，日报暂未提交。我已保留本次填写内容，请重新登录后继续提交。

13.13.2 403 处理

403 常见场景：

1. Staff Agent 读取他人日报。
2. Manager Agent 读取其他部门数据。
3. 普通员工确认老板决策。
4. 未授权用户关闭高风险异常。
5. Agent 尝试创建跨部门任务。
6. 用户提交不属于自己的日报。

处理方式：

1. 停止操作。
2. 不自动改写用户、部门或组织 ID。
3. 提示无权限。

4. 写入审计日志。
5. 必要时通知管理员或上级。

提示示例：

```
当前账号没有权限确认该决策。该操作需要老板或授权高管确认。
```

403 不能通过重试解决。Agent 收到 403 后，不应尝试换一个 ID 或请求更大范围数据。

13.14 超时与幂等处理

超时和重复请求是 MVP 联调中非常常见的问题。尤其是 IM、Agent、后端、数据库多方协作时，一次操作可能因为网络、模型响应或接口延迟被重复触发。

因此，关键写入接口必须支持幂等。

13.14.1 为什么需要幂等

没有幂等时，可能出现：

1. 员工一条日报被提交两次。
2. 老板确认一次，系统生成两组任务。
3. Dream 重试后生成多个重复决策卡片。
4. IM 推送失败重试后，用户收到多条相同消息。
5. 任务状态被重复更新。
6. 审计日志出现重复关键事件。

幂等的目标是：**同一业务动作重复请求多次，最终只产生一次业务结果。**

13.14.2 推荐幂等键设计

不同场景建议使用不同幂等键。

场景	幂等键
Staff 日报提交	`staff_report:{org_id}:{user_id}:{record_date}`
Manager 汇总生成	`manager_summary:{org_id}:{dept_id}:{summary_date}`
Executive 摘要生成	`executive_summary:{org_id}:{summary_date}`
老板确认决策	`decision_confirm:{decision_id}:{confirmed_by}`
决策生成任务	`task_from_decision:{decision_id}:{confirmed_option}`
异常生成任务	`task_from_incident:{incident_id}`
Dream 运行	`dream_run:{org_id}:{summary_date}:{run_type}`
IM 消息推送	`im_push:{target_type}:{target_id}:{event_type}`

幂等键应由后端校验和保存，不能只在 Agent 本地判断。

13.14.3 超时处理

如果请求超时，调用方不能立即认为失败，也不能无脑生成新请求。

推荐流程：

```
请求超时
  ↓
使用同一幂等键查询结果
  ↓
如果已成功，返回成功结果
  ↓
如果未成功，使用同一幂等键重试
  ↓
多次失败后标记待处理
```

例如老板确认决策时，如果前端超时，但后端其实已经创建了任务，再次点击确认不应重复生成任务。

13.14.4 幂等返回

如果后端发现同一幂等键已经处理过，应返回原处理结果。

示例：

```
{
  "code": "IDEMPOTENT_REPLAY",
  "message": " 该请求已处理，返回原结果",
  "data": {
    "task_ids": [9001, 9002]
  }
}
```

这样 Agent 或前端可以继续展示正确结果，而不是提示用户失败。

13.14.5 重复提交处理

重复提交不一定都是错误。系统应区分：

情况	处理
同一请求重复发送	幂等返回原结果
用户修改后重新提交	创建新版本或更新原记录
老板重复点击同一确认按钮	返回已确认结果
Dream 重复运行同一日期	返回已有运行记录或生成新版本
任务重复生成	合并或返回已有任务

对于 Staff 日报，可以采用：

```
同一员工同一天一条主记录；
修改走 update 或 revision；
重复提交走幂等返回。
```

对于老板决策任务，必须防止重复生成任务：

```
同一 decision_id + confirmed_option 只能生成一次正式任务集。
```

13.14.6 超时后的用户提示

超时提示要谨慎，不要让用户以为数据一定丢了。

不推荐：

```
提交失败，请重新提交。
```

推荐：

```
提交结果暂时未确认，我会继续查询处理结果。请不要重复提交，避免产生重复记录。
```

如果最终确认失败，再提示用户重试或联系管理员。

13.15 本章小结

前端、IM、Agent 和后端校验逻辑，是 AutoMage-2 MVP 稳定运行的基础。

前端和 IM 负责让用户少犯明显错误；Agent 负责把自然语言整理成合格的 Schema；后端负责最终校验字段、权限、签名、状态和幂等；数据库负责保证事实数据一致。任何一层缺失，都会导致数据不可用、任务重复、权限失控或决策无法追溯。

MVP 阶段至少要先跑通以下校验闭环：

```
用户输入
  ↓
前端 / IM 基础校验
  ↓
Agent 追问和结构化
  ↓
Agent 本地 Schema 校验
  ↓
后端字段 / 权限 / 签名校验
  ↓
幂等写库
  ↓
错误可追踪、可修正
```

只要这套校验链路稳定，Staff 日报、Manager 汇总、Executive 决策、任务下发和异常处理才能真正进入可联调、可演示、可扩展的状态。

14 API 契约设计

14.1 API 设计原则

AutoMage-2 MVP 的 API 不是普通 CRUD 接口，而是 Agent、IM、前端、后端和数据库之间的统一协作契约。

API 的核心作用有三个：

1. 接收 Agent 生成的结构化 Schema。
2. 对 Schema 做鉴权、校验、签名、幂等和写库。
3. 为上级 Agent 提供可追溯、可权限控制的数据读取能力。

MVP 阶段 API 设计应遵守以下原则。

14.1.1 Schema 优先

所有核心写入接口都应以 Schema 为核心输入，而不是让每个接口自定义一套字段。

例如 Staff 日报提交接口应接收 `schema_v1_staff`，Manager 汇总提交接口应接收 `schema_v1_manager`，Executive 决策提交接口应接收 `schema_v1_executive`。

这样做的好处是：

1. Agent 输出结构稳定。
2. 后端校验规则清晰。
3. 前后端字段一致。
4. 数据库映射可追踪。
5. 后续 Schema 版本可以演进。

14.1.2 后端最终校验

Agent 可以做本地校验，前端可以做表单校验，IM 可以做交互校验，但所有正式写入必须经过后端校验。

后端至少负责：

1. 用户鉴权。
2. Agent 鉴权。
3. Schema 字段校验。
4. 字段类型校验。
5. 枚举值校验。
6. 权限校验。
7. 签名校验。
8. 幂等校验。

9. 状态流转校验。
10. 审计日志记录。

不允许 Agent 直接写数据库，也不允许前端绕过 API 写入正式业务表。

14.1.3 数据库是事实源

API 写入成功后，数据库中的数据才是正式事实。

IM 聊天记录、Agent 上下文、前端临时状态都不能作为最终事实源。

例如：

```
员工在 IM 中说“确认提交”

日报已正式提交
```

只有当 Staff Agent 调用 `/api/v1/report/staff` 成功，后端完成校验、签名和写库后，该日报才算正式进入系统。

14.1.4 幂等优先

所有关键写入接口必须支持幂等，避免重复提交、重复任务、重复决策和重复审计。

需要支持幂等的典型接口包括：

1. Staff 日报提交。
2. Manager 汇总提交。
3. Executive 决策提交。
4. 老板确认决策。
5. 任务生成。
6. 异常上报。
7. Dream 运行结果写入。

建议统一使用请求头：

```
Idempotency-Key: staff_report:1:10086:2026-05-03
```

后端发现同一幂等键已处理时，应返回原结果，而不是重复写入。

14.1.5 错误可被 Agent 理解

API 错误不能只返回“失败”。错误必须明确到字段、原因和建议修正方式，这样 Agent 才能继续追问用户或重新生成 Schema。

推荐错误格式：

```
{
  "success": false,
  "code": "SCHEMA_VALIDATION_FAILED",
  "message": "Schema 校验失败",
  "errors": [
    {
      "field": "support_detail",
      "reason": "need_support 为 true 时，support_detail 必填",
      "suggestion": " 请补充需要谁支持、支持什么事项"
    }
  ],
  "request_id": "req_01HXAMPLE"
}
```

14.1.6 权限由后端强制执行

权限不能只写在 Prompt 中。

即使 Agent 请求越权数据，后端也必须拒绝。例如：

1. Staff Agent 不能读取其他员工日报。
2. Manager Agent 不能读取其他部门员工明细。
3. 普通员工不能确认老板决策。
4. 未授权 Agent 不能创建跨部门任务。
5. 未签名老板决策不能生成正式任务。

14.1.7 接口版本化

MVP 阶段统一使用：

```
/api/v1
```

后续如果 Schema、权限或任务状态流转发生破坏性变化，应新增版本，而不是直接修改旧接口含义。

14.2 统一响应格式

所有 API 建议使用统一响应结构，方便前端、IM 和 Agent Runtime 统一处理。

14.2.1 成功响应格式

```
{
  "success": true,
  "code": "OK",
```

```
  "message": "success",
  "data": {},
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

字段说明：

字段	类型	说明
success	boolean	是否成功
code	string	业务状态码
message	string	简短说明
data	object	业务数据
request_id	string	请求 ID，用于排查
server_time	string	服务端时间

14.2.2 失败响应格式

```
{
  "success": false,
  "code": "SCHEMA_VALIDATION_FAILED",
  "message": " 提交内容不符合 Staff Schema",
  "errors": [
    {
      "field": "support_detail",
      "reason": "need_support 为 true 时，support_detail 必填",
      "suggestion": " 请补充需要谁支持、支持什么事项"
    }
  ],
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

字段说明：

字段	类型	说明
success	boolean	固定为 false
code	string	错误码
message	string	错误摘要
errors	array	字段级错误列表
request_id	string	请求 ID
server_time	string	服务端时间

14.2.3 分页响应格式

读取列表类接口使用统一分页结构。

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
    "items": [],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total": 128,
      "has_more": true
    }
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

14.2.4 幂等重放响应格式

如果请求已经处理过，后端应返回原结果，并明确标记为幂等重放。

```
{
  "success": true,
  "code": "IDEMPOTENT_REPLAY",
  "message": " 该请求已处理，返回原结果",
  "data": {
    "work_record_id": 301
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

14.3 统一错误码

MVP 阶段建议先定义一组稳定错误码，供 Agent、前端和后端联调使用。

HTTP 状态码	错误码	含义	处理方式
200	OK	成功	正常处理
200	IDEMPOTENT_REPLAY	幂等重放	使用原结果
400	BAD_REQUEST	请求格式错误	修正请求结构
401	UNAUTHORIZED	未认证或登录失效	重新登录 / 重新初始化
403	FORBIDDEN	无权限	停止操作，记录审计
404	NOT_FOUND	数据不存在或不可见	返回空或提示无数据
409	CONFLICT	数据冲突或重复提交	走幂等、更新或人工处理
422	SCHEMA_VALIDATION_FAILED	Schema 校验失败	Agent 追问补全
422	FIELD_TYPE_INVALID	字段类型错误	修正字段类型
422	ENUM_VALUE_INVALID	枚举值非法	使用标准枚举
422	REQUIRED_FIELD_MISSING	缺少必填字段	补充字段
422	SIGNATURE_REQUIRED	缺少签名	请求确认
422	SIGNATURE_INVALID	签名无效	重新签名
422	STATE_TRANSITION_INVALID	状态流转非法	人工检查
429	RATE_LIMITED	请求过快	延迟重试
500	INTERNAL_ERROR	服务端异常	重试或通知管理员
503	SERVICE_UNAVAILABLE	服务不可用	延迟重试

Agent 处理错误时应遵守以下规则：

1. 422：根据字段错误追问用户。
2. 401：提示重新登录或重新绑定。
3. 403：停止请求，不要尝试绕过。
4. 409：查询已有结果或走更新流程。
5. 5xx：使用幂等键延迟重试。

14.4 Agent 初始化接口

14.4.1 POST /api/v1/agent/init

该接口用于初始化 Agent 节点，获取当前用户、组织、部门、节点类型、权限范围和 Schema 版本信息。

Staff Agent、Manager Agent、Executive Agent 启动时都应先调用该接口。

14.4.2 请求参数

```
{
  "org_id": 1,
  "user_id": 10086,
  "agent_type": "staff",
  "client_type": "im",
  "client_user_id": "feishu_open_id_xxx",
```

```
  "agent_template_version": "staff_agent_v0.1"
}
```

字段说明：

字段	类型	是否必填	说明
org_id	number / string	是	组织 ID
user_id	number / string	否	系统用户 ID
agent_type	string	是	Agent 类型
client_type	string	是	客户端类型，如 im、web
client_user_id	string	否	IM 或外部系统用户 ID
agent_template_version	string	否	Agent 模板版本

agent_type 建议枚举：

值	含义
staff	Staff Agent
manager	Manager Agent
executive	Executive Agent
dream	Dream 运行节点
admin	管理节点

14.4.3 响应参数

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
    "agent_session_id": "agent_sess_01HXAMPLE",
    "node_id": "staff_node_10086",
    "agent_type": "staff",
    "org_id": 1,
    "user_id": 10086,
    "department_id": 12,
    "role": "staff",
    "permissions": {
      "can_read_own_reports": true,
      "can_submit_staff_report": true,
      "can_read_department_reports": false,
      "can_generate_manager_summary": false,
      "can_generate_executive_decision": false
    },
    "supported_schemas": [
      {
        "schema_id": "schema_v1_staff",
        "schema_version": "1.0.0"
      }
```

```
    ],
    "expires_at": "2026-05-04T23:59:59+08:00"
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

14.4.4 错误码

错误码	场景
UNAUTHORIZED	用户未登录或 IM 账号未绑定
FORBIDDEN	用户无权初始化该类型 Agent
NOT_FOUND	组织、用户或部门不存在
AGENT_TYPE_INVALID	Agent 类型非法
INTERNAL_ERROR	初始化失败

14.5 Staff 日报提交接口

14.5.1 POST /api/v1/report/staff

该接口用于提交员工每日 Staff Schema。

Staff Agent 在员工确认日报后调用该接口。后端完成 Schema 校验、权限校验、签名校验、幂等校验后，将数据写入 `work_records` 和 `work_record_items`。

14.5.2 请求参数

请求头建议：

```
Authorization: Bearer <token>
Idempotency-Key: staff_report:1:10086:2026-05-03
Content-Type: application/json
```

请求体：

```
{
  "schema": {
    "schema_id": "schema_v1_staff",
    "schema_version": "1.0.0",
    "timestamp": "2026-05-03T18:12:30+08:00",
    "org_id": 1,
    "department_id": 12,
    "user_id": 10086,
    "node_id": "staff_node_10086",
    "record_date": "2026-05-03",
```

```
"work_progress": [
  {
    "title": " 完成 Staff 日报 Schema 初稿",
    "description": " 整理一线员工日报字段、签名字段和数据库映射关系。",
    "status": "completed",
    "related_task_id": 201
  }
],
"issues_faced": [
  {
    "title": "Decision 表结构尚未最终确定",
    "description": " 当前 DDL 中没有独立 decision_logs 表，需要后端确认承载方式。",
    "severity": "medium"
  }
],
"solution_attempt": [
  {
    "issue_title": "Decision 表结构尚未最终确定",
    "attempt": " 先在文档中列出 Decision Schema 最小字段。",
    "result": " 待后端确认"
  }
],
"need_support": true,
"support_detail": " 需要后端确认是否新增 decision_logs 表。",
"next_day_plan": [
  {
    "title": " 完善 Manager Schema 字段定义",
    "priority": "high",
    "expected_output": " 形成可提交后端联调的字段说明"
  }
],
"resource_usage": {
  "work_hours": 7.5,
  "tools": ["ChatGPT", "PostgreSQL", "Markdown"]
},
"artifacts": [],
"related_task_ids": [201],
"risk_level": "medium",
"employee_comment": " 部分内容需要等后端确认后定稿。",
"signature": {
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-03T18:13:00+08:00",
  "payload_hash": "sha256:example_hash",
  "signature_source": "im"
},
```

```
    "meta": {
      "input_channel": "feishu",
      "agent_template_version": "staff_agent_v0.1"
    }
  }
}
```

14.5.3 响应参数

```
{
  "success": true,
  "code": "OK",
  "message": "Staff 日报提交成功",
  "data": {
    "work_record_id": 301,
    "record_date": "2026-05-03",
    "status": "submitted",
    "created_incident_ids": [501],
    "created_task_ids": [],
    "audit_log_id": 7001
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-03T18:13:05+08:00"
}
```

字段说明：

字段	说明
work_record_id	写入后的日报主记录 ID
record_date	日报日期
status	写入状态
created_incident_ids	因风险或支持需求生成的异常
created_task_ids	因明确事项生成的任务
audit_log_id	审计日志 ID

14.5.4 Schema 校验规则

后端至少校验：

1. schema_id = schema_v1_staff。
2. schema_version 为支持版本。
3. org_id、department_id、user_id 与当前身份匹配。
4. node_id 与当前 Staff Agent 绑定。
5. work_progress 至少一项。

6. next_day_plan 至少一项。
7. need_support 必须是 boolean。
8. need_support = true 时，support_detail 必填。
9. risk_level 必须属于 low / medium / high / critical。
10. signature.signed_by 必须等于 user_id。
11. 同一用户同一日期重复提交时走幂等或修订流程。

14.6 Manager 汇总提交接口

14.6.1 POST /api/v1/report/manager

该接口用于提交部门级 Manager Schema。

Manager Agent 读取本部门 Staff Schema 后，生成部门汇总，并在 Manager 确认或系统标记后调用该接口。

14.6.2 请求参数

请求头：

```
Authorization: Bearer <token>
Idempotency-Key: manager_summary:1:12:2026-05-03
Content-Type: application/json
```

请求体：

```
{
  "schema": {
    "schema_id": "schema_v1_manager",
    "schema_version": "1.0.0",
    "timestamp": "2026-05-03T21:10:00+08:00",
    "org_id": 1,
    "dept_id": 12,
    "manager_user_id": 20001,
    "manager_node_id": "manager_node_dept_12",
    "summary_date": "2026-05-03",
    "staff_report_count": 8,
    "missing_report_count": 2,
    "missing_staff_ids": [10012, 10017],
    "overall_health": "yellow",
    "aggregated_summary": "今日架构组主要完成 Staff Schema 字段定义和数据库映射梳理。Decision
    ↪ 相关表结构尚未最终确定，可能影响 Executive 决策链路。",
    "top_3_risks": [
      {
        "title": "Decision 表结构未确认",
```

```
    "description": "当前 DDL 中未单独建立 decision_logs 表，可能影响老板决策记录和任务生成。
    ↪ ",
    "severity": "high",
    "source_record_ids": [301, 302],
    "suggested_action": " 建议后端与架构负责人确认是否新增独立决策表。"
  }
],
"workforce_efficiency": {
  "score": 78,
  "level": "medium",
  "basis": "8 名员工提交日报，6 名员工完成主要任务，2 个事项仍处于阻塞状态。"
},
"pending_approvals": 1,
"highlight_staff": [
  {
    "user_id": 10086,
    "display_name": " 杨卓",
    "reason": " 完成 Staff Schema 字段定义，并主动梳理数据库映射。",
    "source_record_ids": [301]
  }
],
"blocked_items": [],
"manager_decisions": [],
"need_executive_decision": [
  {
    "decision_title": " 是否新增独立 decision_logs 表",
    "context": " 当前 Agent mock 流程中存在 decision_logs，但正式 DDL 尚未建表。",
    "options": [
      {
        "option_id": "A",
        "title": " 新增独立 decision_logs 表"
      },
      {
        "option_id": "B",
        "title": " 暂时复用 audit_logs 和 tasks"
      }
    ],
    "recommended_option": "A",
    "reason": " 独立决策表更利于审计和任务来源追踪。",
    "source_record_ids": [301, 302],
    "urgency": "high"
  }
],
"next_day_adjustment": [
  {
    "title": " 优先确认 Decision 数据结构",
    "reason": " 该事项影响 Executive 决策链路。",
```

```
        "target_user_ids": [10086, 20002],
        "priority": "high",
        "expected_output": " 形成 decision_logs 是否建表的最终结论和字段草案。"
      }
    ],
    "source_record_ids": [301, 302, 303, 304],
    "related_task_ids": [401],
    "related_incident_ids": [501],
    "signature": {
      "signature_required": true,
      "signature_status": "signed",
      "signed_by": 20001,
      "signed_at": "2026-05-03T21:25:00+08:00",
      "payload_hash": "sha256:manager_summary_hash",
      "signature_source": "im"
    },
    "meta": {
      "input_channel": "scheduled_job",
      "agent_template_version": "manager_agent_v0.1"
    }
  }
}
```

14.6.3 响应参数


```
{
  "success": true,
  "code": "OK",
  "message": "Manager 汇总提交成功",
  "data": {
    "summary_id": 801,
    "summary_type": "department_daily_summary",
    "summary_date": "2026-05-03",
    "status": "submitted",
    "created_decision_candidate_ids": ["decision_tmp_001"],
    "created_task_ids": [],
    "audit_log_id": 7002
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-03T21:25:05+08:00"
}
```


后端至少校验：

1. Manager Agent 是否有权读取 `source_record_ids`。
2. `source_record_ids` 是否全部属于该部门。

3. `pending_approvals` 是否与 `need_executive_decision.length` 一致。
4. `overall_health` 是否属于合法枚举。
5. 高风险和上推事项是否包含来源。
6. 签名人是否为部门负责人或授权 Manager。

14.7 Executive 决策提交接口

14.7.1 POST /api/v1/decision/commit

该接口用于提交老板确认动作。它不是生成 Executive Schema 的接口，而是老板对某个决策项做出确认、驳回、补充信息或自定义方案后的提交接口。

Executive Agent 生成决策卡片后，老板通过 IM 或前端确认，前端 / IM 将确认结果提交到该接口。

14.7.2 请求参数

请求头：

```
Authorization: Bearer <token>
Idempotency-Key: decision_confirm:decision_0001:30001
Content-Type: application/json
```

请求体：

```
{
  "decision_id": "decision_0001",
  "org_id": 1,
  "executive_user_id": 30001,
  "action": "confirm",
  "confirmed_option": "A",
  "confirmed_custom_plan": null,
  "confirm_source": "im",
  "comment": " 选择方案 A，新增独立 decision_logs 表。",
  "signature": {
    "signature_required": true,
    "signature_source": "im"
  }
}
```

字段说明：

字段	类型	是否必填	说明
decision_id	string / number	是	决策 ID
org_id	number / string	是	组织 ID
executive_user_id	number / string	是	确认人 ID
action	string	是	确认动作
confirmed_option	string	条件必填	选择的方案
confirmed_custom_plan	string	否	自定义方案
confirm_source	string	是	确认来源
comment	string	否	补充说明
signature	object	是	签名请求信息

action 建议枚举：

值	含义
confirm	确认
reject	驳回
need_more_info	需要补充信息
cancel	取消
custom_confirm	自定义方案确认

14.7.3 响应参数

```
{
  "success": true,
  "code": "OK",
  "message": " 决策确认成功",
  "data": {
    "decision_id": "decision_0001",
    "status": "confirmed",
    "confirmed_option": "A",
    "confirmed_by": 30001,
    "confirmed_at": "2026-05-04T08:16:30+08:00",
    "generated_task_ids": [9001, 9002],
    "signature": {
      "signature_status": "signed",
      "payload_hash": "sha256:decision_payload_hash",
      "verify_status": "verified"
    },
    "audit_log_id": 7003
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T08:16:30+08:00"
}
```

后端至少校验：

1. 决策是否存在。
2. 决策是否属于当前组织。

3. 当前用户是否有权确认。
4. 决策状态是否为 pending。
5. confirmed_option 是否存在于候选方案中。
6. 如果是 custom_confirm，是否填写自定义方案。
7. 是否已经确认过。
8. 是否可以生成任务。
9. 生成任务是否满足幂等。

14.8 任务获取接口

14.8.1 GET /api/v1/tasks

该接口用于获取任务列表。Staff Agent 用它获取员工每日任务，Manager Agent 用它获取部门任务，Executive Agent 用它跟踪老板决策产生的关键任务。

14.8.2 请求参数

Query 参数：

参数	类型	是否必填	说明
org_id	number / string	是	组织 ID
assignee_user_id	number / string	否	执行人
department_id	number / string	否	部门 ID
status	string	否	任务状态
priority	string	否	优先级
source_type	string	否	来源类型
source_decision_id	string / number	否	来源决策
due_from	string	否	截止时间起点
due_to	string	否	截止时间终点
page	number	否	页码
page_size	number	否	每页数量

示例：

```
GET /api/v1/tasks?org_id=1&assignee_user_id=10086&status=pending&page=1&page_size=20
```

14.8.3 响应参数

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
```

```
    "items": [
      {
        "task_id": 9001,
        "org_id": 1,
        "department_id": 12,
        "task_title": " 补充 decision_logs 表结构草案",
        "task_description": " 根据老板确认的方案 A，设计独立 decision_logs 表结构。",
        "source_type": "executive_decision",
        "source_decision_id": "decision_0001",
        "assignee_user_id": 20002,
        "assignee_role": "backend",
        "priority": "high",
        "status": "pending",
        "due_at": "2026-05-04T18:00:00+08:00",
        "confirm_required": true,
        "created_at": "2026-05-04T08:16:35+08:00"
      }
    ],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total": 1,
      "has_more": false
    }
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```


权限规则：

1. Staff Agent 只能查询本人任务。
2. Manager Agent 只能查询本部门任务。
3. Executive Agent 可以查询组织级关键任务。
4. 跨部门任务按授权范围返回。
5. 无权限任务不返回明细。

14.9 汇总读取接口

14.9.1 GET /api/v1/summaries

该接口用于读取 Staff、Manager、Executive 或 Dream 生成的汇总数据。

Manager Agent 读取 Staff 汇总时，优先通过日报接口或 Work Record 读取；Executive Agent 读取部门汇总时，使用该接口读取 Manager Summary；老板看板也可以使用该接口读取组织级 Summary。

14.9.2 请求参数

参数	类型	是否必填	说明
org_id	number / string	是	组织 ID
scope_type	string	否	汇总范围
scope_id	number / string	否	用户、部门或组织 ID
summary_type	string	否	汇总类型
summary_date	string	否	汇总日期
date_from	string	否	起始日期
date_to	string	否	结束日期
department_id	number / string	否	部门 ID
status	string	否	状态
page	number	否	页码
page_size	number	否	每页数量

示例：

```
GET /api/v1/summaries?org_id=1&summary_type=department_daily_summary&summary_date=2026-05-03
```

14.9.3 响应参数


```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
    "items": [
      {
        "summary_id": 801,
        "org_id": 1,
        "department_id": 12,
        "summary_type": "department_daily_summary",
        "scope_type": "department",
        "scope_id": 12,
        "summary_date": "2026-05-03",
        "title": "2026-05-03 架构组部门日报",
        "content": " 今日架构组主要完成 Staff Schema 字段定义和数据库映射梳理。",
        "source_count": 8,
        "status": "submitted",
        "meta": {
          "schema_id": "schema_v1_manager",
          "schema_version": "1.0.0",
          "overall_health": "yellow",
          "top_3_risks": []
        },
        "created_at": "2026-05-03T21:25:00+08:00"
      }
```

```
    ],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total": 1,
      "has_more": false
    }
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

14.9.4 来源追溯接口

如需查看某个 Summary 的来源，可补充：

```
GET /api/v1/summaries/{summary_id}/sources
```

响应：

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
    "summary_id": 801,
    "sources": [
      {
        "source_type": "work_record",
        "source_id": 301
      },
      {
        "source_type": "work_record",
        "source_id": 302
      }
    ]
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

14.10 异常上报接口

14.10.1 POST /api/v1/incidents

该接口用于创建异常或风险记录。Staff Agent、Manager Agent、Dream、后端规则都可以在权限范围内调用。

14.10.2 请求参数

请求头：

```
Authorization: Bearer <token>
Idempotency-Key: incident:manager_summary:801:decision_schema_missing
Content-Type: application/json
```

请求体：

```
{
  "schema": {
    "schema_id": "schema_v1_incident",
    "schema_version": "1.0.0",
    "org_id": 1,
    "department_id": 12,
    "incident_title": "Decision 表结构未确认导致任务阻塞",
    "description": "当前 Agent mock 流程中存在 decision_logs 概念，但正式 DDL
    ↪  尚未建立独立决策表，影响 Executive 决策确认和任务生成链路。",
    "incident_type": "task_blocked",
    "severity": "high",
    "status": "open",
    "source_type": "manager_summary",
    "source_id": 801,
    "source_record_ids": [301, 302],
    "source_summary_ids": [801],
    "related_task_ids": [9001],
    "related_decision_ids": [],
    "reporter_user_id": 20001,
    "owner_user_id": 20002,
    "owner_node_id": "manager_node_dept_12",
    "need_escalation": true,
    "escalation_level": "executive",
    "attempted_solution": " 已在 Manager Schema 中列出 A/B 方案。",
    "suggested_action": " 建议 Executive Agent 将该事项整理为老板决策卡片。",
    "meta": {
      "input_channel": "scheduled_summary"
    }
  }
}
```

14.10.3 响应参数

```
{
  "success": true,
  "code": "OK",
  "message": " 异常上报成功",
```

```
  "data": {
    "incident_id": 501,
    "status": "open",
    "severity": "high",
    "created_task_ids": [],
    "escalation_required": true,
    "audit_log_id": 7004
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:30:00+08:00"
}
```

14.10.4 异常状态更新接口

`PATCH /api/v1/incidents/{incident_id}`

请求体示例：

```
{
  "status": "in_progress",
  "owner_user_id": 20002,
  "comment": " 后端负责人已开始确认 decision_logs 表结构。",
  "updated_by": 20001
}
```

用于更新异常处理状态、负责人、关闭原因、重新打开原因等。

14.11 审计日志接口

14.11.1 GET /api/v1/audit-logs

该接口用于读取审计日志。主要供管理员、Executive Agent、调试工具和复盘系统使用。

14.11.2 请求参数

参数	类型	是否必填	说明
org_id	number / string	是	组织 ID
actor_user_id	number / string	否	操作人
actor_node_id	string	否	操作 Agent
target_type	string	否	目标对象类型
target_id	string / number	否	目标对象 ID
event_type	string	否	事件类型
date_from	string	否	起始时间
date_to	string	否	结束时间
page	number	否	页码
page_size	number	否	每页数量

示例：

```
GET /api/v1/audit-logs?org_id=1&target_type=decision&target_id=decision_0001
```

14.11.3 响应参数


```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
    "items": [
      {
        "audit_log_id": 7003,
        "event_type": "executive_decision_confirmed",
        "org_id": 1,
        "actor_user_id": 30001,
        "actor_node_id": "executive_node_org_1",
        "target_type": "decision",
        "target_id": "decision_0001",
        "status_before": "pending",
        "status_after": "confirmed",
        "source_type": "executive_schema",
        "source_id": "exec_schema_001",
        "payload_summary": " 老板确认方案 A: 新增独立 decision_logs 表。",
        "event_at": "2026-05-04T08:16:30+08:00"
      }
    ],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total": 1,
      "has_more": false
    }
  },
  "request_id": "req_01HXAMPLE",
  "server_time": "2026-05-04T10:00:00+08:00"
}
```

14.11.4 审计写入规则

审计日志一般由后端内部写入，不建议开放普通外部写接口。

必须写审计的事件包括：

1. Staff 日报提交。

2. Manager 汇总生成和确认。
3. Executive 决策生成、推送和确认。
4. 任务创建、分配、状态变化和关闭。
5. 异常创建、上推、关闭和重新打开。
6. 签名生成、失败、撤回和内容变更。
7. 权限失败。
8. 幂等重放。
9. Dream 运行成功或失败。

14.12 API 与 Schema 的映射关系

API 与 Schema 的映射关系如下：

API	输入 Schema	输出对象	说明
`POST /api/v1/agent/init`	无	Agent Session	初始化 Agent 节点和权限
`POST /api/v1/report/staff`	`schema_v1_staff`	Work Record	提交员工日报
`POST /api/v1/report/manager`	`schema_v1_manager`	Summary	提交部门汇总
`POST /api/v1/decision/commit`	决策确认动作	Decision / Task	老板确认决策并生成任务
`GET /api/v1/tasks`	查询参数	Task 列表	获取任务
`GET /api/v1/summaries`	查询参数	Summary 列表	读取汇总
`GET /api/v1/summaries/{id}/sources`	路径参数	来源记录	追溯 Summary 来源
`POST /api/v1/incidents`	`schema_v1_incident`	Incident	创建异常
`PATCH /api/v1/incidents/{id}`	状态更新	Incident	更新异常状态
`GET /api/v1/audit-logs`	查询参数	Audit Log 列表	读取审计日志

更完整的关系可以表示为：

```
schema_v1_staff
  → POST /api/v1/report/staff
  → work_records / work_record_items

schema_v1_manager
  → POST /api/v1/report/manager
  → summaries / summary_source_links

schema_v1_executive
```

```
  → Dream / Executive Agent 生成
  → summaries / decision candidate

decision confirm action
  → POST /api/v1/decision/commit
  → decisions / tasks / task_assignments

schema_v1_task
  → 后端任务创建逻辑
  → tasks / task_assignments / task_updates

schema_v1_incident
  → POST /api/v1/incidents
  → incidents / incident_updates
```

需要注意，`schema_v1_executive` 通常由 Dream 或 Executive Agent 生成，并用于老板卡片展示；老板确认动作本身通过 `/api/v1/decision/commit` 提交。二者不要混淆。

14.13 API 与数据库的映射关系

API 最终需要落到数据库事实表中。MVP 阶段建议按以下方式映射。

14.13.1 Staff 日报提交映射

接口：

```
POST /api/v1/report/staff
```

数据库映射：

API / Schema 内容	数据库对象	说明
Staff Schema 主信息	work_records	日报主记录
work_progress	work_record_items	字段明细
issues_faced	work_record_items	字段明细
solution_attempt	work_record_items	字段明细
need_support	work_record_items	字段明细
support_detail	work_record_items	字段明细
next_day_plan	work_record_items	字段明细
resource_usage	work_record_items	字段明细
artifacts	artifacts / artifact_links	产出物
高风险或支持需求	incidents	条件生成
明确后续行动	tasks	条件生成
提交与签名行为	audit_logs	审计记录

14.13.2 Manager 汇总提交映射

接口：

```
POST /api/v1/report/manager
```

数据库映射：

API / Schema 内容	数据库对象	说明
部门汇总主信息	summaries	部门日报
source_record_ids	summary_source_links	来源 Staff 记录
top_3_risks	summaries.meta / incidents	风险信息
blocked_items	incidents	条件生成异常
manager_decisions	Decision 相关表 / audit_logs	部门内决策
need_executive_decision	Decision 候选对象 / summaries.meta	上推老板事项
next_day_adjustment	tasks	Manager 确认后生成
签名与确认	audit_logs	审计记录

14.13.3 Executive 决策确认映射

接口：

```
POST /api/v1/decision/commit
```

数据库映射：

API 内容	数据库对象	说明
决策确认结果	Decision 表 / summaries.meta	记录老板选择
confirmed_option	Decision 字段	最终方案
confirmed_by	Decision 字段	确认人
confirmed_at	Decision 字段	确认时间
签名字段	Decision 字段 / meta	签名信息
generated_tasks	tasks	创建正式任务
任务分配	task_assignments	分配执行人
任务创建动态	task_updates	创建记录
确认行为	audit_logs	审计记录

如果 MVP 阶段暂未建立独立 Decision 表，可以先用 summaries.meta + audit_logs + tasks.meta.source_decision_id 承载，但后续应补齐独立 Decision 对象。

14.13.4 任务读取映射

接口：

```
GET /api/v1/tasks
```

数据库映射：

API 查询参数	数据库对象	说明
assignee_user_id	task_assignments.user_id	查询个人任务
department_id	tasks.department_id	查询部门任务
status	tasks.status	按状态过滤
priority	tasks.priority	按优先级过滤
source_decision_id	tasks.meta.source_decision_id	按来源决策过滤
任务动态	task_updates	可选展开

14.13.5 汇总读取映射

接口：

```
GET /api/v1/summaries
GET /api/v1/summaries/{summary_id}/sources
```

数据库映射：

API 内容	数据库对象	说明
汇总列表	summaries	读取部门或组织摘要
来源记录	summary_source_links	追溯来源
来源 Staff 日报	work_records	下钻读取
来源字段明细	work_record_items	下钻读取
汇总元数据	summaries.meta	风险、Schema、签名等

14.13.6 异常上报映射

接口：

```
POST /api/v1/incidents
PATCH /api/v1/incidents/{incident_id}
```

数据库映射：

API / Schema 内容	数据库对象	说明
Incident 主信息	incidents	异常主记录
状态变化	incident_updates	异常动态
生成任务	tasks	条件创建
关联任务	incidents.meta.related_task_ids 或关系表	追踪处理任务
关联决策	incidents.meta.related_decision_ids	追踪上推决策
异常操作	audit_logs	审计记录

14.13.7 审计日志读取映射

接口：

```
GET /api/v1/audit-logs
```

数据库映射：

API 查询参数	数据库字段
org_id	audit_logs.org_id
actor_user_id	audit_logs.actor_user_id
actor_node_id	audit_logs.actor_node_id
target_type	audit_logs.target_type
target_id	audit_logs.target_id
event_type	audit_logs.event_type
date_from / date_to	audit_logs.event_at

14.14 本章小结

API 契约是 AutoMage-2 MVP 中连接 Agent、IM、前端、后端和数据库的核心接口层。

它的重点不是接口数量多，而是每个接口都要有清楚的边界：谁可以调用，输入什么 Schema，后端校验什么，写入哪些表，失败时如何返回，重复调用如何处理，后续如何审计。

MVP 阶段至少要先跑通以下接口链路：

```
Agent 初始化
  ↓
Staff 日报提交
  ↓
Manager 汇总提交
  ↓
Executive 决策确认
  ↓
任务获取
  ↓
异常上报
  ↓
审计追踪
```

只要这组 API 稳定，AutoMage-2 的核心闭环就可以进入真实联调：

```
员工日报
→ 部门汇总
→ 老板决策
→ 任务下发
→ 异常处理
→ 审计复盘
```

后续无论接入 IM、Web 看板、小程序、行业模板还是更复杂的 Dream 机制，都应建立在这套 API 契约之上。

15 数据库映射设计

15.1 数据库映射总览

AutoMage-2 MVP 的数据库映射目标，是把 Staff、Manager、Executive、Task、Incident、Artifact、Audit 等 Schema 稳定落到当前业务表中，并保证每条数据都能被追溯、校验和继续汇总。

数据库在系统中的定位不是“存一下 Agent 生成的文本”，而是整个组织运行数据的事实源。Agent 可以生成内容，IM 可以承载交互，前端可以展示结果，但最终可信数据必须落到数据库表中。

MVP 阶段整体映射关系如下：

```
Staff Schema
  → work_records
  → work_record_items
  → artifacts / incidents / tasks
  → audit_logs

Manager Schema
  → summaries
  → summary_source_links
  → incidents / tasks / decision candidate
  → audit_logs

Executive Schema
  → summaries
  → decisions
  → tasks / task_assignments
  → audit_logs

Task Schema
  → tasks
  → task_assignments
  → task_updates
  → artifacts
  → audit_logs

Incident Schema
  → incidents
  → incident_updates
  → tasks
  → audit_logs
```

当前 DDL 已经具备 MVP 主链路的大部分基础表，包括：

能力	当前已有表
组织与用户	organizations、users、roles、user_roles
部门与成员	departments、department_members
表单 / Schema 模板	form_templates
员工日报	work_records、work_record_items
产出物	artifacts、artifact_links
任务	tasks、task_assignments、task_updates
异常	incidents、incident_updates
汇总	summaries、summary_source_links
审计	audit_logs

但当前 DDL 仍有几个明显缺口：

1. 缺少独立 Agent 节点表，无法正式管理 Staff / Manager / Executive Agent 的绑定关系。
2. 缺少独立 Decision 记录表，老板决策只能临时放入 `summaries.meta` 或 `audit_logs.payload`。
3. 缺少统一数字签名表或显式签名字段，签名信息只能暂放在 `meta`。
4. 缺少 Dream Run 表，无法记录 Dream 每次运行的输入、输出和失败原因。
5. 缺少 Schema Contract 表，Schema 版本、字段规则和校验规则目前只能依赖代码或 `form_templates.schema_json`。
6. 部分多来源关系只能放在 `meta`，长期看不利于追溯和查询。

MVP 阶段可以先采用“现有表承载 + meta 兜底 + 后续补表”的方式推进。但涉及老板决策、签名、Dream 运行和 Schema 版本管理的部分，建议尽快补齐独立表，避免后续迁移成本过高。

15.2 Staff Schema 与 ‘work_records‘ 的映射

`work_records` 用于保存 Staff 日报主记录。它不保存所有字段明细，而是保存一条日报的身份、归属、日期、状态、标题、提交时间和元信息。

Staff Schema 与 `work_records` 的映射如下：

Staff Schema 字段	work_records 字段	说明
org_id	org_id	组织 ID
department_id	department_id	员工所属部门
user_id	user_id	提交日报的员工
record_date	record_date	日报对应日期
schema_id / schema_version	meta.schema_id / meta.schema_version	Schema 标识与版本
node_id	meta.node_id	Staff Agent 节点 ID
自动生成标题	title	日报标题
提交状态	status	草稿、已提交、已确认等
timestamp	submitted_at	日报提交时间
来源渠道	source_type / meta.input_channel	IM、Web、Agent 等
signature	meta.signature	MVP 阶段暂存签名信息
meta	meta	扩展信息

标题建议由系统自动生成：

```
2026-05-03 杨卓 每日工作记录
```

work_records.template_id 当前为必填字段，因此 Staff 日报落库时需要先准备对应的 form_templates 记录，用于表示 Staff 日报模板。MVP 阶段可以为 Staff、Manager、Executive 分别创建固定模板：

模板	用途
Staff Daily Report Template	员工日报
Manager Daily Summary Template	部门汇总
Executive Daily Decision Template	老板决策摘要

Staff 日报写入 work_records 时，建议状态映射如下：

业务状态	work_records.status 建议值	说明
草稿	0	Agent 已整理但员工未确认
已提交	1	员工已确认并提交
已汇总	2	已被 Manager Schema 引用
已作废	9	被修订或取消

具体 smallint 枚举可以由后端统一定义，文档中只要求语义保持稳定。

15.3 Staff Schema 与 ‘work_record_items‘ 的映射

work_record_items 用于保存 Staff Schema 的字段明细。它适合承载结构化字段和可扩展字段，避免 work_records 主表变得过宽。

Staff Schema 字段与 work_record_items 的映射如下：

Staff Schema 字段	field_key	field_label	建议存储位置
work_progress	work_progress	今日完成事项	value_json
issues_faced	issues_faced	遇到的问题	value_json
solution_attempt	solution_attempt	已尝试方案	value_json
need_support	need_support	是否需要支持	value_json 或 value_text
support_detail	support_detail	支持需求说明	value_text
next_day_plan	next_day_plan	明日计划	value_json
resource_usage	resource_usage	资源消耗	value_json
artifacts	artifacts	产出物摘要	value_json
related_task_ids	related_task_ids	关联任务	value_json
risk_level	risk_level	风险等级	value_text
employee_comment	employee_comment	员工补充说明	value_text
signature	signature	签名信息	value_json

work_record_items 已有唯一约束：

```
work_record_id + field_key 唯一
```

这意味着同一条日报中，同一个字段只能保存一次。数组型字段，例如 work_progress，应整体存入一条 value_json，不要把每个工作事项拆成多条 work_record_items。

示例：

```
{
  "field_key": "work_progress",
  "field_label": " 今日完成事项",
  "field_type": "array",
  "value_json": [
    {
      "title": " 完成 Staff Schema 初稿",
      "description": " 整理员工日报字段和数据库映射",
      "status": "completed"
    }
  ]
}
```

MVP 阶段该设计足够灵活。后续如果某些字段进入高频查询，例如 risk_level、need_support、work_progress.status，不建议长期只放在 JSON 中，应提升为显式列或建立专门索引。

15.4 Manager Schema 与 ‘summaries‘ 的映射

Manager Schema 应落到 summaries 表。summary_type 表示汇总类型，scope_type 表示汇总范围。

Manager 汇总属于部门级汇总，因此建议：

```
summary_type = department_daily_summary
scope_type = department
department_id = 当前部门 ID
```

由于当前 DDL 中 summary_type 和 scope_type 是 smallint，后端需要维护枚举映射。例如：

字段	建议值	含义
summary_type	1	员工日报摘要
summary_type	2	部门每日汇总
summary_type	3	组织级老板摘要
scope_type	1	用户级
scope_type	2	部门级
scope_type	3	组织级

Manager Schema 与 summaries 的映射如下：

Manager Schema 字段	summaries 字段	说明
org_id	org_id	组织 ID
dept_id	department_id	部门 ID
manager_user_id	user_id 或 meta.manager_user_id	部门负责人
summary_date	summary_date	汇总日期
自动生成标题	title	部门日报标题
aggregated_summary	content	部门汇总正文
staff_report_count	source_count	来源日报数量
schema_id / schema_version	meta.schema_id / meta.schema_version	Schema 信息
manager_node_id	meta.manager_node_id	Manager Agent 节点
overall_health	meta.overall_health	部门健康度
top_3_risks	meta.top_3_risks	三大风险
workforce_efficiency	meta.workforce_efficiency	部门效能
pending_approvals	meta.pending_approvals	待上推数量
highlight_staff	meta.highlight_staff	突出员工
blocked_items	meta.blocked_items	阻塞事项
manager_decisions	meta.manager_decisions	部门内已处理决策
need_executive_decision	meta.need_executive_decision	待老板决策事项
next_day_adjustment	meta.next_day_adjustment	次日调整建议
signature	meta.signature	Manager 确认信息

来源 Staff 日报不建议只放在 meta，应写入 summary_source_links。

Manager Schema 与 summary_source_links 的映射如下：

Manager Schema 字段	summary_source_links 字段	说明
当前 summary_id	summary_id	Manager 汇总 ID
source_record_ids[]	source_id	Staff Work Record ID
固定类型	source_type	来源类型，如 work_record
org_id	org_id	组织 ID

这样 Executive Agent 或审计人员后续可以从部门汇总追溯到员工日报。

15.5 Executive Schema 与决策表的映射

Executive Schema 有两类数据需要落库：

1. 组织级摘要。
2. 老板待决策事项。

组织级摘要可以先写入 `summaries`。老板决策事项建议写入独立 `decisions` 表。

当前 DDL 中没有独立 Decision 表，因此 MVP 阶段有两种落地方式。

15.5.1 MVP 临时映射方式

在未新增 Decision 表前，可以先这样承载：

Executive Schema 内容	临时存储位置	说明
`business_summary`	`summaries.content`	组织级摘要
`key_risks`	`summaries.meta.key_risks`	关键风险
`decision_items`	`summaries.meta.decision_items`	老板待决策事项
`human_confirm_status`	`summaries.meta.human_confirm_status`	确认状态
`confirmed_by`	`summaries.meta.confirmed_by`	确认人
`confirmed_at`	`summaries.meta.confirmed_at`	确认时间
`generated_tasks`	`summaries.meta.generated_tasks`	任务草案
确认动作	`audit_logs`	审计记录
生成任务	`tasks.meta.source_decision_id`	任务来源

这种方式可以支撑 Demo，但不适合长期使用。主要问题是：

1. 决策状态难以独立流转。
2. 决策与任务关系不清晰。
3. 老板确认历史不易查询。
4. 多个决策项放在一个 Summary 的 JSON 中，后续更新和审计复杂。
5. 难以做决策看板和复盘分析。

15.5.2 建议正式映射方式

建议新增 `decisions` 表，专门承载老板决策和 Manager 上推决策。

Executive Schema 与 `decisions` 的映射如下：

Executive Schema 字段	decisions 字段	说明
org_id	org_id	组织 ID
决策关联部门	department_id	可为空，组织级决策为空
decision_items[].title	title	决策标题
decision_items[].context	context	决策背景
decision_level	decision_level	L0-L4
option_a / option_b	options	候选方案 JSON
recommended_option	recommended_option	推荐方案
reasoning_summary	recommend_reason	推荐理由
expected_impact	expected_impact	预期影响
human_confirm_status	status	决策状态
confirmed_by	confirmed_by	确认人
confirmed_at	confirmed_at	确认时间
confirmed_option	confirmed_option	最终选择
generated_tasks	meta.generated_tasks	任务草案
source_summary_ids	decision_source_links	来源汇总
signature	signature 或 meta.signature	签名信息

推荐状态流转：

```
draft
  ↓
pending
  ↓
confirmed / rejected / need_more_info
  ↓
task_generated
  ↓
in_execution
  ↓
completed / closed
```

如果只保留 summaries.meta.decision_items，这条状态流转会很难维护。因此 Decision 表建议作为 15 章中优先级最高的新增表之一。

15.6 Task Schema 与 ‘tasks‘ 的映射

tasks 存储任务主记录。Task Schema 与 tasks 的映射如下：

Task Schema 字段	tasks 字段	说明
org_id	org_id	组织 ID
department_id	department_id	所属部门
source_record_id	source_record_id	来源 Staff 日报
creator_user_id	creator_user_id	创建人
task_title	title	任务标题
task_description	description	任务说明
status	status	任务状态
priority	priority	优先级
due_at	due_at	截止时间
started_at	started_at	开始时间
closed_at	closed_at	关闭时间
result_summary	result_summary	结果摘要
source_type	meta.source_type	来源类型
source_id	meta.source_id	来源对象 ID
source_summary_id	meta.source_summary_id	来源 Summary
source_decision_id	meta.source_decision_id	来源决策
created_by_node_id	meta.created_by_node_id	创建任务的 Agent 节点
assignee_role	meta.assignee_role	执行角色
confirm_required	meta.confirm_required	是否需要确认关闭
signature	meta.signature	任务确认签名

当前 tasks 表已经支持任务主链路，但有三个需要注意的点：

1. source_record_id 只能直接关联 Staff 日报，不能直接关联 Decision 或 Summary。
2. source_decision_id 暂时只能放在 meta，长期建议提升为显式字段。
3. status 和 priority 是 smallint，后端需要维护枚举映射。

任务状态建议映射：

业务状态	smallint 建议值
pending	0
in_progress	1
blocked	2
completed	3
closed	4
cancelled	9

任务优先级建议映射：

业务优先级	smallint 建议值
urgent	4
high	3
medium	2
low	1

具体值可以由后端统一定义，关键是不要在不同模块中使用不同枚举。

15.7 Task Assignment 与 ‘task_assignments‘ 的映射

task_assignments 用于记录任务与执行人的关系。一个任务可以有一个主负责人，也可以有多个协作人、审核人或关注人。

Task Assignment 与 task_assignments 的映射如下：

Task Schema 字段	task_assignments 字段	说明
org_id	org_id	组织 ID
task_id	task_id	任务 ID
assignee_user_id	user_id	执行人
assignment_type	assignment_type	分配类型
creator_user_id	created_by	分配创建人

建议 assignment_type smallint 枚举如下：

业务含义	smallint 建议值
owner 主负责人	1
participant 协作人	2
reviewer 审核人	3
watcher 关注人	4

当前表中 user_id 必填，因此如果任务只指定了 assignee_role，但还没有明确执行人，建议先采用以下方式之一：

1. 分配给部门 Manager，由 Manager 再拆给具体员工。
2. 在 tasks.meta.assignee_role 中保留角色，任务状态标记为待分配。
3. 后续新增角色级任务分配表。

MVP 阶段推荐第一种方式：角色不明确时先给 Manager。

任务过程动态写入 task_updates。

任务动作	task_updates.update_type	说明
创建任务	created	系统生成任务
分配任务	assigned	写入执行人
开始处理	started	状态变为进行中
标记阻塞	blocked	记录阻塞原因
提交完成	completed	执行人提交结果
确认关闭	closed	确认方关闭
取消任务	cancelled	任务取消

由于当前 update_type 是 smallint，后端同样需要维护枚举映射。

15.8 Incident Schema 与 ‘incidents‘ 的映射

incidents 用于存储异常主记录。Incident Schema 与 incidents 的映射如下：

Incident Schema 字段	incidents 字段	说明
org_id	org_id	组织 ID
department_id	department_id	所属部门
source_record_id	source_record_id	来源 Staff 日报
related_task_ids[0]	related_task_id	当前表只支持一个关联任务
reporter_user_id	reporter_user_id	上报人
incident_title	title	异常标题
description	description	异常说明
severity	severity	异常等级
status	status	异常状态
resolved_at	resolved_at	解决时间
closed_at	closed_at	关闭时间
incident_type	meta.incident_type	异常类型
source_type	meta.source_type	来源类型
source_id	meta.source_id	来源对象 ID
source_record_ids	meta.source_record_ids	多来源 Staff 记录
source_summary_ids	meta.source_summary_ids	来源 Summary
related_task_ids	meta.related_task_ids	多关联任务
related_decision_ids	meta.related_decision_ids	关联决策
owner_user_id	meta.owner_user_id	处理负责人
owner_node_id	meta.owner_node_id	处理 Agent
need_escalation	meta.need_escalation	是否需要上推
escalation_level	meta.escalation_level	上推层级
attempted_solution	meta.attempted_solution	已尝试方案
suggested_action	meta.suggested_action	建议处理方式
close_reason	meta.close_reason	关闭原因

当前 incidents 表已经能支撑异常主流程，但存在两个限制：

1. source_record_id 只能直接关联一条 Staff 日报。
2. related_task_id 只能直接关联一个任务。

如果多个 Staff 异常合并为一个部门异常，或者一个异常生成多个任务，就需要暂时把数组放在 meta 中。长期建议新增异常来源和异常任务关联表。

异常状态 smallint 建议映射：

业务状态	smallint 建议值
open	0
triaged	1
assigned	2
in_progress	3
waiting_decision	4
resolved	5
closed	6
cancelled	9
reopened	10

异常更新写入 incident_updates。任何状态变化、处理人变化、上推、关闭、重新打开，都应写入更新表和审计日志。

15.9 Artifact 与 ‘artifacts‘ 的映射

artifacts 用于保存员工、任务和异常产生的文档、代码、链接、文件等产出物。artifact_links 用于保存外部链接。

Artifact Schema 与 artifacts 的映射如下：

Artifact 字段	artifacts 字段	说明
org_id	org_id	组织 ID
work_record_id	work_record_id	关联日报
task_id	task_id	关联任务
incident_id	incident_id	关联异常
user_id	user_id	提交人
artifact_type	artifact_type	产出物类型
title	title	产出物标题
storage_type	storage_type	存储方式
file_name	file_name	文件名
file_url	file_url	文件地址
mime_type	mime_type	文件类型
file_size	file_size	文件大小
description	meta.description	产出物说明
source_type	meta.source_type	来源类型

当前 artifacts 表有约束：

```
work_record_id / task_id / incident_id 至少一个不为空
```

这符合 MVP 需求。产出物必须挂到日报、任务或异常上，不能无来源地孤立存在。

如果产出物是外部链接，建议先创建一条 artifacts 主记录，再在 artifact_links 中保存链接。

artifact_links 映射如下：

Artifact 字段	artifact_links 字段	说明
当前 Artifact ID	artifact_id	所属产出物
link_type	link_type	链接类型
title	title	链接标题
url	url	链接地址
created_by	created_by	创建人

artifact_type 和 storage_type 当前都是 smallint，建议后端维护统一枚举。

示例：

artifact_type	含义
————:	–
1	文档
2	代码
3	表格
4	图片
5	链接
9	其他

15.10 Audit 与 ‘audit_logs‘ 的映射

`audit_logs` 用于记录关键操作。它不替代业务表，只记录“谁在什么时候对什么对象做了什么”。

当前 `audit_logs` 字段较轻量：

字段	说明
`org_id`	组织 ID
`actor_user_id`	操作人
`target_type`	目标对象类型
`target_id`	目标对象 ID
`action`	操作动作
`summary`	操作摘要
`payload`	扩展内容
`event_at`	事件时间

各类 Schema 与 `audit_logs` 的映射如下：

业务动作	`target_type`	`action`	说明
Staff 日报提交	`work_record`	`staff_report_submitted`	员工提交日报
Staff 日报签名	`work_record`	`signature_signed`	员工确认日报
Manager 汇总生成	`summary`	`manager_summary_generated`	生成部门汇总
Manager 汇总确认	`summary`	`manager_summary_confirmed`	部门负责人确认
Executive 摘要生成	`summary`	`executive_schema_generated`	生成老板摘要
老板决策确认	`decision`	`executive_decision_confirmed`	老板确认方案
任务生成	`task`	`task_generated`	决策或异常生成任务
任务状态变化	`task`	`task_status_changed`	任务流转
异常创建	`incident`	`incident_created`	异常上报
异常上推	`incident`	`incident_escalated`	上推处理
异常关闭	`incident`	`incident_closed`	异常关闭
Dream 运行	`dream_run`	`dream_run_completed`	Dream 完成
权限失败	对应对象	`permission_denied`	越权访问
幂等重放	对应对象	`idempotent_replay`	重复请求返回原结果

当前 `audit_logs` 缺少 `actor_node_id`、`status_before`、`status_after`、`source_type`、`source_id` 等显式字段。MVP 阶段可以先放入 `payload`，例如：

```
{
```

```
  "actor_node_id": "manager_node_dept_12",
  "status_before": "pending",
  "status_after": "confirmed",
  "source_type": "manager_summary",
  "source_id": 801,
  "payload_hash": "sha256:xxx"
}
```

长期建议将高频审计字段提升为显式列，便于查询和追责。

15.11 当前 DDL 缺口说明

当前 DDL 能支撑基础日报、汇总、任务、异常和审计，但要完整支撑 AutoMage-2 的三级 Agent 决策闭环，还需要补齐几个关键对象。

15.11.1 Agent 节点表

当前 DDL 中没有独立的 Agent 节点表。

问题表现：

1. `node_id` 只能放在各业务表的 `meta` 中。
2. 无法统一管理 Staff / Manager / Executive Agent 的绑定关系。
3. 无法判断某个 Agent 是否有权读取某个用户或部门的数据。
4. 无法记录 Agent 模板版本、运行状态和权限范围。
5. Agent 初始化接口缺少稳定落库对象。

建议新增：

```
agent_nodes
```

用于记录每个 Agent 节点。

建议字段包括：

字段	说明
`id`	内部主键
`public_id`	对外 ID
`org_id`	组织 ID
`node_code`	节点编码
`agent_type`	staff / manager / executive / dream
`bound_user_id`	绑定用户
`bound_department_id`	绑定部门
`template_version`	Agent 模板版本
`status`	节点状态
`permissions`	权限配置
`meta`	扩展信息

15.11.2 Decision 记录表

当前 DDL 中没有独立决策表，这是当前最重要的缺口。

问题表现：

1. 老板决策只能放在 `summaries.meta` 或 `audit_logs.payload` 中。
2. 决策状态无法独立流转。
3. 决策确认后生成的任务缺少正式来源对象。
4. 决策项很难做列表、筛选、统计和复盘。
5. 多个决策项放在一个 Executive Summary 中，不方便单独确认或驳回。

建议新增：

```
decisions
decision_source_links
```

`decisions` 保存决策主记录。`decision_source_links` 保存来源 Summary、Work Record、Incident 等对象。

15.11.3 Digital Signature 字段

当前 DDL 中没有统一数字签名表，也没有在主业务表中显式签名字段。

问题表现：

1. 签名只能放在 `meta.signature`。
2. 无法统一查询哪些对象已签名、待签名或签名失效。
3. 签名后内容变更的校验不够方便。
4. 多次签名、撤回、重新签名难以管理。
5. 签名审计只能依赖 `audit_logs.payload`。

MVP 阶段可以先用 `meta.signature`，但建议尽快新增：

```
digital_signatures
```

它可以统一挂到 Work Record、Summary、Decision、Task、Incident 等对象上。

15.11.4 Dream Run 表

当前 DDL 中没有 Dream 运行记录表。

问题表现：

1. 无法记录 Dream 每次读取了哪些输入。
2. 无法记录 Dream 生成了哪些输出。
3. 无法追踪 Dream 失败原因。
4. 无法做重试、降级和幂等。

5. 无法对比不同 Dream 版本输出质量。

建议新增：

```
dream_runs
```

用于记录每次 Dream 运行。

建议记录：

1. 运行日期。
2. 运行类型。
3. 输入 Summary。
4. 输出 Executive Schema。
5. 生成的决策项。
6. 生成的任务草案。
7. 运行状态。
8. 失败原因。
9. Agent 模板版本。
10. 幂等键。

15.11.5 Schema Contract 表

当前 DDL 中有 `form_templates.schema_json`，可以暂时存 Staff 日报表单结构，但它还不等同于完整 Schema Contract。

Schema Contract 不只是表单结构，还应包含：

1. Schema ID。
2. Schema 版本。
3. 字段定义。
4. 必填规则。
5. 枚举规则。
6. 条件必填规则。
7. 签名要求。
8. 权限要求。
9. Agent 输出约束。
10. 后端校验规则。

如果没有 Schema Contract 表，Schema 版本管理会依赖代码和文档，不利于 Agent、前端、后端统一升级。

建议新增：

schema_contracts

15.12 建议新增表结构清单

MVP 阶段建议按优先级新增以下表。不是所有表都必须第一天完成，但 Decision、Agent Node、Digital Signature 至少应尽早确定方案。

15.12.1 agent_nodes

用于管理 Agent 节点。

```
create table if not exists agent_nodes (
  id bigint generated by default as identity primary key,
  public_id varchar(26) not null unique,
  org_id bigint not null references organizations(id),
  node_code varchar(128) not null,
  agent_type smallint not null,
  bound_user_id bigint null references users(id),
  bound_department_id bigint null references departments(id),
  template_version varchar(64) null,
  status smallint not null default 1,
  permissions jsonb not null default '{}'::jsonb,
  last_active_at timestamptz null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz null,
  created_by bigint null references users(id),
  updated_by bigint null references users(id),
  meta jsonb not null default '{}'::jsonb,
  constraint uq_agent_nodes_org_code unique (org_id, node_code)
);
```

建议索引：

```
create index if not exists idx_agent_nodes_org_type_status
  on agent_nodes (org_id, agent_type, status, id)
  where deleted_at is null;

create index if not exists idx_agent_nodes_bound_user
  on agent_nodes (bound_user_id)
  where deleted_at is null and bound_user_id is not null;

create index if not exists idx_agent_nodes_bound_department
  on agent_nodes (bound_department_id)
  where deleted_at is null and bound_department_id is not null;
```

15.12.2 decisions

用于保存老板决策、Manager 上推决策和重要系统决策。

```
create table if not exists decisions (
  id bigint generated by default as identity primary key,
  public_id varchar(26) not null unique,
  org_id bigint not null references organizations(id),
  department_id bigint null references departments(id),
  title varchar(255) not null,
  context text not null,
  decision_level smallint not null default 3,
  options jsonb not null default '[]'::jsonb,
  recommended_option varchar(64) null,
  recommend_reason text null,
  expected_impact jsonb not null default '{}'::jsonb,
  status smallint not null default 0,
  confirm_required boolean not null default true,
  confirmed_by bigint null references users(id),
  confirmed_at timestamptz null,
  confirmed_option varchar(64) null,
  generated_task_ids jsonb not null default '[]'::jsonb,
  created_by_node_id bigint null references agent_nodes(id),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz null,
  created_by bigint null references users(id),
  updated_by bigint null references users(id),
  meta jsonb not null default '{}'::jsonb
);
```

建议索引：

```
create index if not exists idx_decisions_org_status_created
  on decisions (org_id, status, created_at desc, id desc)
  where deleted_at is null;

create index if not exists idx_decisions_department_status_created
  on decisions (department_id, status, created_at desc, id desc)
  where deleted_at is null and department_id is not null;

create index if not exists idx_decisions_confirmed_by_created
  on decisions (confirmed_by, confirmed_at desc, id desc)
  where deleted_at is null and confirmed_by is not null;
```

15.12.3 decision_source_links

用于记录决策来源。

```
create table if not exists decision_source_links (
  id bigint generated by default as identity primary key,
  org_id bigint not null references organizations(id),
  decision_id bigint not null references decisions(id) on delete cascade,
  source_type smallint not null,
  source_id bigint not null,
  created_at timestamptz not null default now(),
  constraint uq_decision_source_links unique (decision_id, source_type, source_id)
);
```

建议索引：

```
create index if not exists idx_decision_source_links_source
  on decision_source_links (source_type, source_id, decision_id);
```

15.12.4 digital_signatures

用于统一保存签名记录。

```
create table if not exists digital_signatures (
  id bigint generated by default as identity primary key,
  public_id varchar(26) not null unique,
  org_id bigint not null references organizations(id),
  target_type varchar(64) not null,
  target_id bigint not null,
  signature_status smallint not null default 0,
  signed_by bigint null references users(id),
  signed_at timestamptz null,
  payload_hash varchar(128) null,
  signature_type smallint not null default 1,
  signature_source smallint not null default 1,
  verify_status smallint not null default 0,
  signature_version varchar(32) not null default '1.0.0',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  meta jsonb not null default '{}'::jsonb
);
```

建议索引：

```
create index if not exists idx_digital_signatures_target
  on digital_signatures (target_type, target_id, created_at desc, id desc);

create index if not exists idx_digital_signatures_org_status_created
  on digital_signatures (org_id, signature_status, created_at desc, id desc);
```

15.12.5 dream_runs

用于记录 Dream 每次运行。

```
create table if not exists dream_runs (
  id bigint generated by default as identity primary key,
  public_id varchar(26) not null unique,
  org_id bigint not null references organizations(id),
  run_type smallint not null default 1,
  run_date date not null,
  status smallint not null default 0,
  input_summary_ids jsonb not null default '[]'::jsonb,
  input_decision_ids jsonb not null default '[]'::jsonb,
  input_task_ids jsonb not null default '[]'::jsonb,
  output_summary_id bigint null references summaries(id),
  output_decision_ids jsonb not null default '[]'::jsonb,
  output_task_drafts jsonb not null default '[]'::jsonb,
  error_code varchar(128) null,
  error_message text null,
  started_at timestamptz null,
  finished_at timestamptz null,
  created_by_node_id bigint null references agent_nodes(id),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  meta jsonb not null default '{}'::jsonb,
  constraint uq_dream_runs_org_date_type unique (org_id, run_date, run_type)
);
```

建议索引：

```
create index if not exists idx_dream_runs_org_status_created
  on dream_runs (org_id, status, created_at desc, id desc);
```

15.12.6 schema_contracts

用于保存 Schema 契约和版本。

```
create table if not exists schema_contracts (
  id bigint generated by default as identity primary key,
  public_id varchar(26) not null unique,
  org_id bigint null references organizations(id),
  schema_id varchar(128) not null,
  schema_version varchar(32) not null,
  schema_name varchar(128) not null,
  schema_type smallint not null,
  status smallint not null default 1,
  contract_json jsonb not null default '{}'::jsonb,
```

```
  validation_rules jsonb not null default '{}'::jsonb,
  prompt_constraints jsonb not null default '{}'::jsonb,
  effective_from timestamptz null,
  effective_to timestamptz null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz null,
  created_by bigint null references users(id),
  updated_by bigint null references users(id),
  meta jsonb not null default '{}'::jsonb,
  constraint uq_schema_contracts_schema_version unique (org_id, schema_id, schema_version)
);
```

建议索引：

```
create index if not exists idx_schema_contracts_org_status
  on schema_contracts (org_id, status, schema_id, schema_version)
  where deleted_at is null;
```

15.12.7 可选补充表：incident_source_links

如果后续异常合并和来源追溯变复杂，建议新增异常来源表。

```
create table if not exists incident_source_links (
  id bigint generated by default as identity primary key,
  org_id bigint not null references organizations(id),
  incident_id bigint not null references incidents(id) on delete cascade,
  source_type smallint not null,
  source_id bigint not null,
  created_at timestamptz not null default now(),
  constraint uq_incident_source_links unique (incident_id, source_type, source_id)
);
```

15.12.8 可选补充表：incident_task_links

如果一个异常经常生成多个任务，建议新增异常任务关联表。

```
create table if not exists incident_task_links (
  id bigint generated by default as identity primary key,
  org_id bigint not null references organizations(id),
  incident_id bigint not null references incidents(id) on delete cascade,
  task_id bigint not null references tasks(id) on delete cascade,
  link_type smallint not null default 1,
  created_at timestamptz not null default now(),
  created_by bigint null references users(id),
  constraint uq_incident_task_links unique (incident_id, task_id, link_type)
);
```

15.12.9 新增表优先级建议

建议按以下优先级推进：

优先级	表	原因
P0	decisions	老板决策闭环必须有独立对象
P0	agent_nodes	Agent 权限和节点绑定必须落库
P0	digital_signatures	关键确认动作需要可信记录
P1	dream_runs	Dream 运行、失败和幂等需要追踪
P1	schema_contracts	Schema 版本和校验规则需要统一管理
P2	decision_source_links	决策来源追溯需要结构化
P2	incident_source_links	异常多来源合并需要结构化
P2	incident_task_links	异常与多个任务关系需要结构化

如果时间紧张，最小改造建议如下：

```
先新增 decisions 和 agent_nodes;
签名暂放 meta + audit_logs;
Dream 暂放 summaries.meta;
Schema Contract 暂用 form_templates.schema_json;
异常多来源暂放 incidents.meta。
```

但这只是短期方案。只要 MVP 主链路开始稳定运行，digital_signatures、dream_runs、schema_contracts 应尽快补齐。

15.13 本章小结

数据库映射设计决定了 AutoMage-2 的数据能否真正成为组织资产。

当前 DDL 已经具备基础业务闭环：员工日报可以落到 work_records 和 work_record_items，部门和组织汇总可以落到 summaries，任务可以落到 tasks，异常可以落到 incidents，产出物和审计也有对应表支撑。

但要完整支撑 AutoMage-2 的三级 Agent 架构，还需要补齐几个关键对象：

```
Agent 节点
+
Decision 决策记录
+
Digital Signature 签名
+
Dream Run 运行记录
+
Schema Contract 契约版本
```

MVP 阶段可以先用现有表和 meta 字段跑通 Demo，但不应长期把老板决策、签名、Dream 运行和 Schema 契约都塞进 JSON。凡是会进入高频查询、权限判断、状态流转、审计追踪和任务来源分析的字段，都应逐步提升为显式表或显式字段。

最终目标是形成一条稳定的数据链路：

```
员工日报可追溯
→ 部门汇总可追溯
→ 老板决策可追溯
→ 任务来源可追溯
→ 异常处理可追溯
→ 审计责任可追溯
```

只要这条数据库链路设计清楚，AutoMage-2 后续无论接入 IM、Web 看板、行业模板、自动采集还是更复杂的 Dream 机制，都有稳定的事实源可以依赖。

16 权限与数据隔离设计

16.1 权限设计原则

AutoMage-2 MVP 的权限设计目标，是保证不同组织、部门、用户和 Agent 节点只能访问自己被授权的数据。系统可以让 Agent 帮助员工记录、帮助 Manager 汇总、帮助老板决策，但不能因为引入 Agent 而放松数据边界。

权限设计必须遵守一个基本原则：

```
Agent 能看什么，不由 Prompt 决定；
Agent 能做什么，必须由后端权限决定。
```

Prompt 可以告诉 Staff Agent 不要读取他人日报，但真正的限制必须落在 API 和数据库查询上。否则只要 Agent 输出错误请求，或者接口被外部调用，就可能造成越权访问。

MVP 阶段权限设计应遵守以下原则：

1. **组织隔离优先。**

不同企业、团队或租户之间的数据必须完全隔离。任何查询和写入都必须带 `org_id`，且后端校验当前用户是否属于该组织。

1. **最小权限原则。**

每个用户和 Agent 只拥有完成当前任务所需的最小权限。Staff 不看部门全量数据，Manager 不看其他部门明细，Executive 默认看汇总而不是所有个人明细。

1. **按层级逐步开放。**

下级数据可以在授权后被上级汇总读取，但上级决策不能自动暴露给所有下级。员工只看到与自己任务相关的部分。

1. **所有权限由后端判断。**

前端隐藏按钮、IM 不展示入口、Agent Prompt 限制，都只是体验层约束。真正是否允许读写，由后端统一判断。

1. **越权请求必须记录审计。**

如果 Staff Agent 尝试读取其他员工日报，或者 Manager Agent 尝试读取其他部门明细，后端应拒绝并写入审计日志。

1. **汇总可见，明细受控。**

老板可以看到组织级和部门级摘要，但是否下钻到个人明细，需要按组织配置和场景控制。默认展示汇总，必要时再下钻。

1. **权限变更可追溯。**

用户角色变更、部门负责人变更、Agent 节点绑定变更，都应写入审计日志，避免后续无法解释数据访问范围变化。

权限体系可以概括为四层：

```
组织级隔离
  ↓
部门级隔离
```

```
  ↓
用户级隔离
  ↓
Agent 节点级隔离
```

只有这四层都稳定，AutoMage-2 才能从 Demo 进入真实企业场景。

16.2 组织级隔离

组织级隔离是最高优先级的数据边界。

AutoMage-2 中的所有核心业务数据都必须归属于一个组织，包括用户、部门、日报、任务、异常、汇总、决策、签名、审计和 Agent 节点。

所有核心表都应包含：

```
org_id
```

组织级隔离要求如下：

对象	隔离要求
用户	用户必须属于某个组织，不能跨组织默认可见
部门	部门必须归属于组织
Staff 日报	只能被同组织内授权节点读取
Manager 汇总	只能被同组织内授权 Manager / Executive 读取
Executive 决策	只能被同组织内老板或授权高管读取
任务	只能被同组织内相关负责人、Manager、Executive 读取
异常	只能被同组织内相关处理人和授权管理者读取
审计日志	只能被同组织管理员或授权角色读取
Agent 节点	只能绑定同组织用户或部门

组织级权限校验的基本规则：

```
请求中的 org_id
=
当前登录用户所属 org_id
=
目标数据所属 org_id
```

如果三者不一致，后端应拒绝访问。

示例：

```
用户 A 属于 org_id = 1
请求读取 org_id = 2 的日报
→ 后端返回 403 或 404
```

对于多租户部署，组织级隔离必须作为数据库查询的第一条件。不推荐先查出数据再在代码层过滤，正确方式是在查询时就带上 `org_id` 条件。

推荐查询方式：

```
select *
from work_records
where org_id = :current_org_id
  and user_id = :target_user_id
  and deleted_at is null;
```

不推荐：

```
先查 work_records，再在应用层判断 org_id。
```

组织级隔离失败属于严重安全问题，应进入高优先级异常和审计。

16.3 部门级隔离

部门级隔离用于控制 Manager、部门成员和跨部门场景的数据访问范围。

部门是 AutoMage-2 中承上启下的关键边界。Staff 日报通常归属于员工所在部门，Manager 汇总只覆盖本部门，跨部门事项需要经过授权或上推。

部门级隔离的基本原则：

```
Manager 只能默认读取本部门数据；
跨部门数据默认不可见；
需要跨部门协作时，通过任务、异常或 Executive 授权完成。
```

部门级数据包括：

1. 本部门员工日报。
2. 本部门任务。
3. 本部门异常。
4. 本部门 Manager 汇总。
5. 本部门产出物。
6. 本部门内部处理决策。
7. 与本部门相关的老板下发任务。

Manager Agent 默认可读取范围：

数据类型	是否可读	说明
本部门 Staff 日报	可读	只读正式提交和已确认数据
本部门任务	可读	包括分配给本部门的任务
本部门异常	可读	包括 Staff 上报异常
本部门历史汇总	可读	用于趋势判断
其他部门 Staff 日报	不可读	默认禁止
其他部门任务明细	不可读	除非跨部门任务授权
其他部门 Manager 汇总	条件可读	一般由 Executive 统一协调
老板全部决策	不可读	只读与本部门相关的部分

跨部门事项处理时，不应让 Manager Agent 直接读取对方部门明细。推荐流程是：

```
本部门发现跨部门依赖
  ↓
Manager Agent 生成 blocked_items 或 incident
  ↓
标记涉及部门
  ↓
上推 Executive 或创建跨部门协作任务
  ↓
相关部门 Manager 接收授权范围内的信息
  ↓
各部门分别处理
```

例如，架构组需要后端确认 `decision_logs` 表结构。架构组 Manager 不应直接读取后端组所有员工日报，而应生成跨部门协作任务或上推 Executive，由后端 Manager 接收任务后安排负责人处理。

部门级隔离的后端校验应包括：

1. 当前用户是否属于该部门。
2. 当前用户是否为该部门 Manager。
3. 当前 Agent 节点是否绑定该部门。
4. 目标数据是否属于该部门。
5. 目标数据是否处于可读取状态。
6. 是否存在跨部门授权或上级下发任务。

如果部门负责人发生变化，新的 Manager 应获得本部门管理权限，原 Manager 应失去默认读取权限。该变更必须写入审计日志。

16.4 用户级隔离

用户级隔离用于保护员工个人工作记录、任务、产出物和确认动作。

Staff Agent 默认只服务于一个员工，因此它的读取和写入范围必须非常明确。

员工可访问的数据包括：

1. 自己的 Staff 日报。
2. 自己提交的产出物。
3. 分配给自己的任务。
4. 自己上报的异常。
5. 与自己有关的任务反馈。
6. 上级下发给自己的任务说明。
7. 自己的历史确认记录。

员工不可默认访问的数据包括：

1. 其他员工日报。

2. 其他员工任务。
3. 部门完整汇总。
4. Manager 内部判断。
5. 老板决策卡片。
6. 组织级风险摘要。
7. 其他员工绩效或表现标记。
8. 跨部门敏感数据。

用户级隔离的基本规则：

```
Staff 用户只能读写 user_id = 当前用户 的个人数据。
```

例如，员工提交日报时，后端应校验：

校验项	说明
`schema.user_id`	必须等于当前登录用户 ID
`schema.org_id`	必须等于当前用户所属组织
`schema.department_id`	必须等于当前用户所属部门
`signature.signed_by`	必须等于当前用户 ID
`node_id`	必须是绑定该用户的 Staff Agent 节点

如果员工尝试替别人提交日报，应直接拒绝。

任务读取时，也需要区分“创建人”和“执行人”。员工不应因为自己创建过某个任务，就自动看到任务下游所有人的执行明细，除非他也是任务确认方或被授权查看。

用户级隔离还涉及数据展示。例如 Manager 汇总中可能包含 `highlight_staff`。这个字段可以被老板和 Manager 看到，但不一定应该默认推给所有员工。MVP 阶段建议：

1. Staff 只看自己的任务和日报。
2. Manager 看本部门员工摘要和必要明细。
3. Executive 看部门汇总和关键来源。
4. 全员榜单、表现排名等功能暂不进入 MVP。

这样可以减少组织阻力，也避免系统过早被理解成“监控员工”的工具。

16.5 Agent 节点级隔离

Agent 节点级隔离是 AutoMage-2 特有的权限边界。

系统中不仅用户有权限，Agent 节点本身也必须有权限。因为同一个用户可能有多个 Agent，同一个部门可能有多个 Manager Agent，不同 Agent 模板也可能拥有不同能力。

Agent 节点级隔离的目标是回答：

```
这个 Agent 是谁？
绑定哪个用户或部门？
属于哪个组织？
能读哪些数据？
```

```
能写哪些对象？
能调用哪些接口？
```

MVP 阶段建议新增或维护 `agent_nodes` 表，用于记录 Agent 节点绑定关系。

典型 Agent 节点包括：

Agent 类型	绑定对象	默认权限
Staff Agent	员工用户	读写本人日报、本人任务
Manager Agent	部门 / Manager 用户	读取本部门数据，生成部门汇总
Executive Agent	老板 / 组织	读取组织汇总，生成决策项
Dream Agent	组织 / Executive 节点	读取授权汇总，生成组织摘要
Admin Agent	管理员	配置组织、用户、模板和权限

Agent 节点初始化时，后端应返回权限范围：

```
{
  "node_id": "manager_node_dept_12",
  "agent_type": "manager",
  "org_id": 1,
  "bound_department_id": 12,
  "bound_user_id": 20001,
  "permissions": {
    "can_read_department_staff_reports": true,
    "can_generate_manager_summary": true,
    "can_create_department_tasks": true,
    "can_read_other_department_reports": false,
    "can_confirm_executive_decision": false
  }
}
```

Agent 调用 API 时，后端需要同时校验两层身份：

1. 当前用户身份。
2. 当前 Agent 节点身份。

例如 Manager Agent 提交部门汇总时，后端应校验：

```
用户是该部门 Manager
+
Agent 节点绑定该部门
+
请求数据属于该部门
```

不能只校验用户，也不能只校验 Agent。因为如果某个 Agent Session 被错误复用，可能导致越权访问。

Agent 节点级隔离还要控制 Agent 的能力范围：

能力	Staff Agent	Manager Agent	Executive Agent
提交 Staff 日报	是	否	否
读取本人任务	是	否	否
读取部门 Staff 数据	否	是	条件可追溯
生成 Manager 汇总	否	是	否
生成 Executive 决策	否	否	是
确认老板决策	否	否	需老板用户确认
创建部门任务	否	是	是
查看审计日志	否	条件可见	是

Agent 节点失效、模板下线、权限变更时，应立即让相关 session 失效或重新初始化。

16.6 Staff 权限

Staff 权限面向一线员工和 Staff Agent。

Staff 的权限范围应尽量简单，避免员工在 MVP 阶段接触过多管理功能。

Staff 可执行操作：

操作	权限说明
填写本人日报	可创建和提交自己的 Staff Schema
查看本人日报	可查看自己的历史日报
修改本人草稿	可修改未签名或未提交的草稿
提交确认	可签名确认自己的日报
查看本人任务	可读取分配给自己的任务
更新本人任务状态	可标记进行中、阻塞、完成
提交产出物	可上传与本人日报或任务相关的产出物
上报异常	可上报本人任务或工作中的异常
查看本人异常	可查看自己上报或负责处理的异常

Staff 不可执行操作：

操作	原因
查看其他员工日报	保护个人工作数据
查看部门完整汇总	该数据属于 Manager 视角
修改他人日报	防止责任混乱
替他人确认日报	签名必须本人完成
创建老板决策	超出权限
确认老板决策	超出权限
创建跨部门任务	超出权限
关闭高风险异常	超出权限
查看全量审计日志	超出权限

Staff 权限的典型查询限制：

```
select *
from work_records
where org_id = :current_org_id
  and user_id = :current_user_id
  and deleted_at is null;
```

Staff 查询任务时，需要通过 `task_assignments` 限制：

```
select t.*
from tasks t
join task_assignments ta on ta.task_id = t.id
where t.org_id = :current_org_id
  and ta.user_id = :current_user_id
  and t.deleted_at is null;
```

Staff Agent 如果收到类似“帮我看看李四今天写了什么”的请求，应拒绝，并说明只能查看本人数据。但最终拒绝应由后端权限兜底。

16.7 Manager 权限

Manager 权限面向部门负责人和 Manager Agent。

Manager 是部门级数据的主要处理者，可以读取本部门员工日报摘要和必要明细，生成部门汇总，处理部门内异常，下发部门任务，并将超权限事项上推 Executive。

Manager 可执行操作：

操作	权限说明
读取本部门 Staff 日报	只读本部门正式提交数据
读取本部门任务	包括部门任务和员工任务摘要
读取本部门异常	包括 Staff 上报异常
生成本部门 Manager Schema	生成部门汇总
确认部门汇总	对高风险和上推事项签名
创建部门任务	分配给本部门员工
调整部门任务优先级	限本部门任务
处理本部门异常	包括分配、上推、关闭
上推老板决策事项	生成 `need_executive_decision`
查看本部门历史汇总	用于趋势判断

Manager 不可执行操作：

操作	原因
读取其他部门员工日报	跨部门越权
修改员工原始日报	员工个人签名数据不可覆盖
替员工确认日报	签名边界不允许
确认老板决策	超出权限
下发跨部门任务	需要 Executive 或授权
删除审计日志	审计不可篡改
修改老板确认结果	超出权限
查看无关部门敏感异常	超出权限

Manager 查询本部门日报时，后端应基于部门成员关系过滤：

```
select wr.*
from work_records wr
where wr.org_id = :current_org_id
```

```
and wr.department_id = :manager_department_id
and wr.status in (:submitted_status, :confirmed_status)
and wr.deleted_at is null;
```

Manager 写入部门汇总时，后端必须校验：

1. `manager_user_id` 是否有该部门管理权限。
2. `manager_node_id` 是否绑定该部门。
3. `dept_id` 是否等于 Manager 授权部门。
4. `source_record_ids` 是否全部属于该部门。
5. 是否存在未签名或无效 Staff 记录。
6. 高风险和上推事项是否有来源。

如果 Manager 需要跨部门协作，应通过任务或异常上推实现，而不是直接读取其他部门数据。

16.8 Executive 权限

Executive 权限面向老板、高管和 Executive Agent。

Executive 拥有组织级视角，可以读取部门汇总、关键风险、老板决策、关键任务和审计摘要。但即使是 Executive，也不应默认无限制展示所有个人明细。老板侧默认应看汇总，需要追溯时再下钻。

Executive 可执行操作：

操作	权限说明
读取组织级 Summary	查看公司整体摘要
读取各部门 Manager Summary	查看部门汇总
读取关键风险	查看组织级风险
读取待决策事项	查看 Manager 上推事项
生成 Executive Schema	生成老板决策卡片
确认老板决策	老板或授权高管确认
生成任务	基于确认后的决策生成
查看关键任务状态	跟踪老板决策执行
查看异常上推记录	查看高风险异常
查看审计摘要	用于复盘和责任追踪

Executive 默认不应做的事情：

操作	原因
随意修改员工原始日报	原始日报属于员工签名数据
绕过老板确认自动执行重大决策	MVP 阶段必须人工确认
无审计删除任务或决策	破坏追溯链路
直接替 Manager 修改部门汇总	应生成修订或要求 Manager 重新确认
默认展示所有个人细节	容易造成隐私和组织阻力

Executive 下钻权限应分级：

下钻层级	默认策略
组织级 Summary	默认可见
部门 Manager Summary	默认可见
部门风险和上推事项	默认可见
Staff 日报摘要	条件可见
Staff 原始明细	需有来源关联或明确授权
员工个人敏感信息	默认不可见或脱敏

例如，老板看到“Decision 表结构未确认”这个风险，可以下钻到来源 Manager Summary，再根据需要查看相关 Staff Record 的问题摘要。但不应默认展示所有员工完整日报。

Executive 确认决策时，后端应校验：

1. 当前用户是否为老板、高管或授权确认人。
2. 决策是否属于当前组织。
3. 决策状态是否为 `pending`。
4. 决策内容是否未被修改或签名可用。
5. 当前确认动作是否重复。
6. 生成任务是否通过权限和幂等校验。

Executive 权限越高，审计要求越高。老板确认、驳回、修改方案、生成任务，都必须写入审计日志。

16.9 跨级读取规则

跨级读取指上级节点读取下级数据，或下级节点请求读取上级数据。

AutoMage-2 的跨级读取必须遵守方向和范围限制。

推荐规则如下：

```
上级可按权限读取下级汇总和必要来源；
下级默认不可读取上级决策明细；
下级只接收与自己有关的任务和反馈。
```

16.9.1 Staff → Manager

Staff Agent 不读取 Manager Schema。员工只接收与自己有关的任务、提醒和反馈。

Staff 可以看到：

1. Manager 下发给自己的任务。
2. 与自己日报相关的补充要求。
3. 与自己任务相关的异常反馈。
4. 自己任务的确认结果。

Staff 不应看到：

1. 部门完整 Manager Summary。
2. 其他员工表现。
3. 部门整体风险明细。
4. Manager 内部判断。
5. 老板决策卡片。

16.9.2 Manager → Staff

Manager Agent 可以读取本部门 Staff Schema，但必须满足条件：

1. Staff 日报已提交或已确认。
2. 数据属于 Manager 所管部门。
3. 查询范围在汇总日期或授权周期内。
4. 读取动作写入审计日志。
5. 不直接修改 Staff 原始日报。

Manager 可以下钻 Staff 数据，但应优先读取结构化字段，而不是读取所有聊天上下文。

16.9.3 Executive → Manager

Executive Agent 可以读取 Manager Schema，并基于 Manager Schema 生成组织摘要和老板决策。

Executive 默认读取：

1. 部门汇总正文。
2. 部门健康度。
3. Top 风险。
4. 待上推决策。
5. 阻塞事项。
6. 来源 Summary ID。
7. 关键任务状态。

Executive 不应默认修改 Manager Summary。如果老板认为部门汇总有问题，应生成补充请求或要求 Manager 重新确认，而不是直接覆盖原汇总。

16.9.4 Executive → Staff

Executive 可以在必要时追溯 Staff 来源数据，但不建议默认全量读取。

允许下钻场景：

1. 老板决策项需要核对来源。
2. 高风险异常需要确认原始记录。

3. Manager Summary 引用依据不足。
4. 审计或复盘需要查看来源。
5. 老板主动要求查看某个任务的原始执行记录。

限制：

1. 只能读取与当前决策、异常、任务相关的 Staff 记录。
2. 默认不展示与该事项无关的个人内容。
3. 读取行为写入审计。
4. 敏感字段可脱敏或摘要化展示。

16.9.5 下级读取上级数据

下级节点默认不能读取上级完整数据。

请求方	可读上级数据	不可读内容
Staff	分配给自己的任务说明	部门完整汇总、老板决策
Manager	与本部门相关的老板任务和决策摘要	其他部门老板决策细节
Staff Agent	自己任务来源的必要说明	Executive Schema 原文
Manager Agent	本部门相关的决策结果	老板全部决策列表

任务下发时可以附带必要背景，但不应把完整老板决策卡片向所有执行人展开。

16.10 跨部门读取规则

跨部门读取是 MVP 阶段最容易出现越权的地方。

系统必须区分三种情况：

1. 跨部门协作。
2. 跨部门汇总。
3. 跨部门明细读取。

这三者权限不同。

16.10.1 跨部门协作

跨部门协作可以通过任务或异常实现。

例如架构组需要后端支持，正确方式是：

```
架构组 Manager 生成协作任务
  ↓
任务分配给后端 Manager
  ↓
后端 Manager 再分配给后端员工
```

这种情况下，架构组 Manager 可以看到任务状态和结果，但不应看到后端员工全部日报。

16.10.2 跨部门汇总

Executive Agent 可以读取多个部门的 Manager Summary，用于生成组织级摘要。

Manager Agent 默认不能读取其他部门汇总，除非满足以下条件：

1. 该汇总被标记为跨部门协作可见。
2. 上级已授权。
3. 与当前任务或异常直接相关。
4. 只返回摘要级数据，不返回员工明细。

例如，后端组的“接口字段已确认”可以作为摘要返回给架构组，但不需要暴露后端组所有员工日报。

16.10.3 跨部门明细读取

跨部门员工明细默认禁止。

只有在极少数情况下允许：

1. 老板或授权 Executive 下钻。
2. 审计人员按权限查看。
3. 跨部门重大异常需要核查来源。
4. 法务、合规或安全事件需要正式授权。
5. 系统管理员在排查问题时读取，且必须审计。

Manager 不应因为参与跨部门任务，就自动获得对方部门员工日报明细。

16.10.4 跨部门任务权限

跨部门任务应拆成两个层级：

```
跨部门父任务
  ↓
各部门子任务
```

父任务由 Executive 或项目负责人创建，分配给多个 Manager。子任务由各部门 Manager 分配给本部门 Staff。

这样可以避免一个 Manager 直接管理其他部门员工。

示例：

```
父任务：打通 Executive 决策确认到任务生成链路
  - 后端子任务：实现 decision/commit 接口
  - Agent 子任务：更新 Executive Skill 调用逻辑
  - 前端子任务：实现老板确认卡片按钮
```

每个 Manager 只管理本部门子任务。

16.10.5 跨部门授权记录

任何跨部门读取或协作授权，都应记录：

1. 授权人。
2. 授权对象。
3. 授权范围。
4. 授权时间。
5. 失效时间。
6. 关联任务或异常。
7. 读取的数据类型。
8. 审计日志 ID。

MVP 阶段如果暂不做复杂授权表，也应至少在 `audit_logs.payload` 中记录跨部门授权原因。

16.11 权限失败处理

权限失败是正常流程的一部分，不应被隐藏，也不应让 Agent 反复尝试绕过。

权限失败主要包括 401、403 和部分 404。

16.11.1 401 未认证

401 表示当前请求没有有效身份。

常见原因：

1. 用户未登录。
2. Token 过期。
3. IM 用户未绑定系统账号。
4. Agent Session 失效。
5. API 凭证错误。

处理方式：

1. 停止当前操作。
2. 暂存用户输入或 Agent 草稿。
3. 提示用户重新登录或绑定账号。
4. 不写入正式数据。
5. 记录失败日志。

提示示例：

当前账号状态已失效，本次内容暂未提交。我已保留草稿，请重新登录后继续提交。

16.11.2 403 无权限

403 表示用户已认证，但没有权限执行该操作。

常见原因：

1. Staff Agent 读取他人日报。
2. Manager Agent 读取其他部门数据。
3. 普通员工确认老板决策。
4. 未授权 Manager 创建跨部门任务。
5. 非负责人关闭 high 异常。
6. Agent 节点与请求部门不匹配。

处理方式：

1. 立即拒绝。
2. 不自动重试。
3. 不更换参数重试。
4. 写入审计日志。
5. 必要时通知管理员或上级。
6. 给出清晰提示。

提示示例：

```
当前账号没有权限读取该部门数据。你只能查看自己负责部门范围内的汇总和任务。
```

Agent 收到 403 后，禁止尝试修改 `department_id`、`user_id` 或 `org_id` 再次请求。

16.11.3 404 不存在或不可见

对于敏感资源，后端可以返回 404，而不是 403，避免暴露数据是否存在。

适用场景：

1. 查询其他组织数据。
2. 查询无权限部门明细。
3. 查询无权限员工日报。
4. 查询已删除或不可见对象。

例如：

```
Manager A 查询其他部门某员工日报
→ 返回 404
```

这样可以避免系统泄露“该员工日报确实存在”。

16.11.4 权限失败的 Agent 处理

Agent 应根据错误类型处理：

错误	Agent 行为
401	提示重新登录或绑定
403	停止操作，说明无权限
404	提示未找到可访问数据
422	修正字段，不属于权限失败
5xx	可使用幂等键重试

Agent 不应把权限失败描述为系统故障。例如 403 应明确是“没有权限”，不是“系统出错”。

16.11.5 权限失败与用户体验

权限提示要简洁，不要暴露过多内部规则。

不推荐：

```
你不是 department_id=13 的 manager_user_id，因此无法访问 work_records 表。
```

推荐：

```
你没有权限查看该部门的员工日报。如需跨部门协作，请发起协作任务或联系上级授权。
```

对内部开发调试接口，可以返回更详细字段；对普通用户界面，应只展示必要说明。

16.12 权限审计

权限审计用于记录谁在什么时候访问了什么数据、是否成功、是否越权。

在 AutoMage-2 中，权限审计非常重要，因为 Agent 会自动读取、汇总、上推和下发任务。如果没有权限审计，后续很难排查数据是否被越权访问。

16.12.1 需要审计的权限事件

MVP 阶段建议至少记录以下事件：

事件	说明
permission_check_passed	权限校验通过
permission_denied	权限校验失败
agent_initialized	Agent 初始化成功
agent_init_failed	Agent 初始化失败
cross_department_access_requested	请求跨部门访问
cross_department_access_granted	跨部门访问被授权
cross_department_access_denied	跨部门访问被拒绝
sensitive_data_read	读取敏感数据
executive_drilldown_staff_record	老板下钻员工来源记录
role_changed	用户角色变更
department_manager_changed	部门负责人变更
agent_permission_changed	Agent 权限变更
unauthorized_decision_confirm_attempt	未授权决策确认尝试

16.12.2 权限审计字段

权限审计建议记录以下字段：

字段	说明
org_id	组织 ID
actor_user_id	操作用户
actor_node_id	操作 Agent
actor_role	操作角色
target_type	目标对象类型
target_id	目标对象 ID
target_org_id	目标组织
target_department_id	目标部门
action	请求动作
permission_result	通过或拒绝
deny_reason	拒绝原因
request_channel	IM / Web / API / Agent
request_id	请求 ID
event_at	事件时间
payload_summary	摘要说明

示例：

```
{
  "event_type": "permission_denied",
  "org_id": 1,
  "actor_user_id": 10086,
  "actor_node_id": "staff_node_10086",
  "actor_role": "staff",
  "target_type": "work_record",
  "target_id": 302,
  "target_department_id": 12,
```

```
  "action": "read",
  "permission_result": "denied",
  "deny_reason": "Staff 用户不能读取他人日报",
  "request_channel": "agent",
  "request_id": "req_01HXAMPLE",
  "event_at": "2026-05-04T10:20:00+08:00",
  "payload_summary": "Staff Agent 尝试读取非本人 work_record。"
}
```

16.12.3 审计粒度

不是所有读取都必须写详细审计，否则日志量会过大。

建议分级记录：

事件类型	审计粒度
普通本人任务读取	可简化记录或不记录详细 payload
Staff 日报提交	必须记录
Manager 读取部门 Staff 数据	建议记录批量读取摘要
Executive 下钻员工明细	必须记录
跨部门读取	必须记录
权限失败	必须记录
决策确认	必须记录
角色和权限变更	必须记录

Manager Agent 每日读取本部门 Staff 数据，可以记录为一条批量审计：

```
{
  "event_type": "manager_read_staff_records",
  "department_id": 12,
  "record_date": "2026-05-03",
  "record_count": 8,
  "source_record_ids": [301, 302, 303, 304]
}
```

这样既保留追溯能力，也避免每条记录都产生大量日志。

16.12.4 权限变更审计

权限变更比普通访问更敏感，必须记录。

需要记录的变更包括：

1. 新增用户角色。
2. 删除用户角色。
3. 修改部门负责人。
4. 调整员工所属部门。

5. 新增 Agent 节点。
6. 修改 Agent 绑定用户或部门。
7. 修改 Agent 权限配置。
8. 授权跨部门访问。
9. 撤销跨部门访问。
10. 授权某人确认老板决策。

权限变更审计应包含变更前和变更后：

```
{
  "event_type": "agent_permission_changed",
  "target_type": "agent_node",
  "target_id": "manager_node_dept_12",
  "before": {
    "can_read_other_department_reports": false
  },
  "after": {
    "can_read_other_department_reports": true
  },
  "changed_by": 30001,
  "reason": " 临时授权处理跨部门联调任务"
}
```

如果没有变更审计，后续很难解释为什么某个 Agent 曾经能读取某些数据。

16.12.5 权限审计与异常联动

某些权限事件应自动生成异常或安全提示。

建议规则：

条件	动作
同一 Agent 多次 403	生成权限异常
Staff Agent 请求他人日报	记录审计，必要时提示管理员
Manager Agent 请求其他部门明细	记录跨部门越权尝试
普通用户尝试确认老板决策	记录高风险权限事件
Agent 节点权限突然扩大	提醒管理员复核
Executive 下钻大量员工明细	记录敏感读取审计

MVP 阶段不一定要做复杂安全告警，但至少应把这些事件记录下来，便于后续排查。

16.12.6 权限审计的查询入口

权限审计可以通过 `audit_logs` 查询，也可以后续建立专门的权限审计看板。

MVP 阶段建议管理员至少能按以下条件查询：

1. 操作人。

2. Agent 节点。
3. 目标对象。
4. 部门。
5. 事件类型。
6. 是否失败。
7. 时间范围。
8. 请求渠道。

这样在出现数据争议时，可以快速回答：

```
谁看过这条日报？
谁确认了这个决策？
哪个 Agent 生成了这个任务？
为什么这个 Manager 能看到这个部门汇总？
是否存在越权访问？
```

16.13 本章小结

权限与数据隔离设计，是 AutoMage-2 能否进入企业真实场景的底座。

Agent 能够自动读取、汇总和生成建议，但它不能天然获得无限权限。Staff Agent 只能处理个人数据；Manager Agent 只能处理本部门数据；Executive Agent 默认读取组织级汇总和关键风险；跨部门、跨级和个人明细读取都必须经过后端权限控制和审计记录。

MVP 阶段至少要先做到以下几点：

```
组织必须隔离；
部门必须隔离；
员工个人数据必须隔离；
Agent 节点必须绑定权限；
跨部门读取必须受控；
老板确认必须校验身份；
权限失败必须拒绝并审计。
```

只要这套权限边界清楚，AutoMage-2 的三级 Agent 才能在组织中稳定协作：

```
Staff 记录个人事实
  ↓
Manager 汇总部门事实
  ↓
Executive 判断组织事实
  ↓
老板确认关键决策
  ↓
任务按权限下发
```

权限机制不是附加功能，而是整个系统可信运行的前提。

17 MVP 核心 Demo 流程

17.1 Demo 目标

MVP 核心 Demo 的目标，是用一条最短但完整的链路证明 AutoMage-2 的三级 Agent 架构可以跑通：

```
员工提交工作数据
  ↓
Staff Agent 结构化
  ↓
Manager Agent 部门汇总
  ↓
Executive Agent / Dream 生成老板决策
  ↓
老板确认方案
  ↓
系统生成任务
  ↓
任务下发给执行人
```

本 Demo 不追求覆盖所有企业管理场景，也不追求界面完整，而是验证 AutoMage-2 最核心的产品假设：

1. 一线员工的自然语言工作记录可以被整理成 Staff Schema。
2. Staff Schema 可以被后端校验并写入数据库。
3. Manager Agent 可以读取本部门数据并生成部门汇总。
4. Executive Agent 可以基于部门汇总生成老板决策项。
5. 老板确认后，系统可以自动生成正式任务。
6. Staff Agent 可以读取并展示下发给员工的任务。
7. 整个过程有来源、有状态、有签名、有审计记录。

   Demo 的重点不是“AI 会不会写总结”，而是证明系统可以形成组织级闭环：

```
事实数据 → 汇总判断 → 老板决策 → 任务执行 → 数据回流
```

只要这条链路跑通，AutoMage-2 就具备了继续扩展行业模板、自动采集、复杂 Dream、老板看板和组织诊断的基础。

17.2 Demo 角色

本次 Demo 建议使用 4 类角色，尽量减少人员复杂度。

角色	示例身份	主要动作
一线员工	杨卓	提交日报、确认 Staff Schema、接收任务
部门负责人	架构组 Manager	查看部门汇总、确认上推事项
老板 / 高管	Executive 用户	接收决策卡片、选择方案
系统管理员 / 演示者	Demo 操作人	初始化数据、触发定时任务、检查数据库

对应 Agent 节点如下：

Agent 节点	绑定对象	Demo 职责
Staff Agent	杨卓	整理员工日报，提交 Staff Schema，查询任务
Manager Agent	架构组	读取部门 Staff Schema，生成 Manager Schema
Executive Agent	老板	读取部门汇总，生成老板决策项
Dream	组织级节点	生成组织摘要、风险和任务草案

Demo 中不需要真实接入全公司员工。可以只模拟一个部门、一个员工、一个 Manager、一个老板，但系统数据结构应按正式链路设计，避免做成一次性假数据演示。

17.3 Demo 前置数据

Demo 前需要准备一组最小可运行数据。

17.3.1 组织与用户数据

至少准备：

数据	示例
组织	AutoMage Demo 公司
部门	架构组
员工	杨卓
Manager	架构组负责人
Executive	老板 / 高管用户

示例：

```
org_id = 1
department_id = 12
staff_user_id = 10086
manager_user_id = 20001
executive_user_id = 30001
```

17.3.2 Agent 节点数据

至少准备三个 Agent 节点：

```
staff_node_10086
manager_node_dept_12
executive_node_org_1
```

对应绑定关系：

node_id	Agent 类型	绑定对象
`staff_node_10086`	Staff Agent	杨卓
`manager_node_dept_12`	Manager Agent	架构组
`executive_node_org_1`	Executive Agent	组织 / 老板

如果当前还没有 `agent_nodes` 表，可以先在 Mock 配置或 `meta` 中维护节点信息。但接口返回和日志中应保持节点 ID 一致，方便后续迁移到正式表。

17.3.3 Schema 与模板数据

至少准备以下 Schema：

Schema	用途
`schema_v1_staff`	员工日报
`schema_v1_manager`	部门汇总
`schema_v1_executive`	老板决策
`schema_v1_task`	任务
`schema_v1_incident`	异常

如果当前使用 `form_templates` 承载模板，建议创建对应模板记录：

模板	说明
Staff Daily Report Template	员工日报模板
Manager Daily Summary Template	部门汇总模板
Executive Decision Template	老板决策模板

17.3.4 Demo 业务场景数据

建议 Demo 使用一个清晰、具体、能体现三级决策价值的场景：

```
当前 Agent mock 流程中已经存在 decision_logs 概念，
但正式 DDL 尚未建立独立决策表。
如果不确认该结构，Executive 决策确认和任务下发链路会受影响。
```

这个场景适合 Demo 的原因是：

1. 它来自一线开发工作。
2. 它会影响部门汇总。
3. 它需要老板或项目负责人确认。
4. 它可以整理成 A/B 方案。
5. 它能在确认后生成明确任务。

6. 它能体现数据库、Agent、任务和审计之间的闭环。

老板决策项可以设计为：

```
是否新增独立 decision_logs 表？
```

方案：

方案	内容
方案 A	新增独立 decision_logs / decisions 表
方案 B	暂时复用 audit_logs 和 tasks.meta

推荐方案：

```
方案 A
```

17.4 Demo 流程总览

完整 Demo 分为 12 步：

```
Step 1: Staff Agent 初始化
Step 2: 员工提交每日工作 Schema
Step 3: Staff Schema 写入数据库
Step 4: Manager Agent 定时读取部门数据
Step 5: Manager 生成部门汇总 Schema
Step 6: Executive Agent 读取部门汇总
Step 7: Dream 生成老板决策项
Step 8: IM 推送老板决策卡片
Step 9: 老板选择方案
Step 10: 系统自动生成任务
Step 11: 任务下发到 Staff Agent
Step 12: 员工查询今日任务
```

Demo 主链路图如下：

```
杨卓
  ↓
Staff Agent
  ↓
schema_v1_staff
  ↓
/api/v1/report/staff
  ↓
work_records / work_record_items
  ↓
Manager Agent
  ↓
schema_v1_manager
  ↓
```

```
/api/v1/report/manager
  ↓
summaries / summary_source_links
  ↓
Executive Agent / Dream
  ↓
schema_v1_executive
  ↓
老板决策卡片
  ↓
/api/v1/decision/commit
  ↓
decisions / tasks / task_assignments
  ↓
Staff Agent 查询任务
```

17.5 Step 1：Staff Agent 初始化

17.5.1 目标

验证 Staff Agent 可以正确识别当前用户、组织、部门和权限范围。

17.5.2 调用接口

```
POST /api/v1/agent/init
```

请求示例：

```
{
  "org_id": 1,
  "user_id": 10086,
  "agent_type": "staff",
  "client_type": "im",
  "client_user_id": "feishu_open_id_yangzhuo",
  "agent_template_version": "staff_agent_v0.1"
}
```

响应示例：

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
    "agent_session_id": "agent_sess_001",
    "node_id": "staff_node_10086",
```

```
    "agent_type": "staff",
    "org_id": 1,
    "user_id": 10086,
    "department_id": 12,
    "role": "staff",
    "permissions": {
      "can_read_own_reports": true,
      "can_submit_staff_report": true,
      "can_read_department_reports": false,
      "can_generate_manager_summary": false
    },
    "supported_schemas": [
      {
        "schema_id": "schema_v1_staff",
        "schema_version": "1.0.0"
      }
    ]
  }
}
```

17.5.3 成功表现

Staff Agent 能明确知道：

1. 当前用户是杨卓。
2. 当前组织是 AutoMage Demo 公司。
3. 当前部门是架构组。
4. 当前节点是 `staff_node_10086`。
5. 当前 Agent 只能提交本人日报和查询本人任务。
6. 当前 Agent 不能读取部门汇总和他人日报。

Demo 展示时，可以让 Staff Agent 返回一句：

杨卓，你好。我已初始化为你的 Staff Agent。今天我会协助你提交工作记录，并在任务下发后提醒你处理。

17.6 Step 2：员工提交每日工作 Schema

17.6.1 目标

验证 Staff Agent 可以把员工自然语言输入整理为标准 Staff Schema，并让员工确认。

17.6.2 员工输入示例

今天完成了 Staff 日报 Schema 的字段整理，补充了签名字段和数据库映射说明。
遇到的问题是 Decision 表结构还没确认，目前 DDL 里没有独立 decision_logs 表。

我已经先在文档里列了最小字段草案，但需要后端确认是否新增独立表。
明天计划继续完善 Manager Schema 和 Executive 决策链路。

17.6.3 Staff Agent 整理结果

Staff Agent 应整理出如下结构：


```
{
  "schema_id": "schema_v1_staff",
  "schema_version": "1.0.0",
  "timestamp": "2026-05-03T18:12:30+08:00",
  "org_id": 1,
  "department_id": 12,
  "user_id": 10086,
  "node_id": "staff_node_10086",
  "record_date": "2026-05-03",
  "work_progress": [
    {
      "title": " 完成 Staff 日报 Schema 字段整理",
      "description": " 补充 Staff 日报字段、签名字段和数据库映射说明。",
      "status": "completed"
    }
  ],
  "issues_faced": [
    {
      "title": "Decision 表结构尚未确认",
      "description": "当前 DDL 中没有独立 decision_logs 表，影响 Executive
      ↪ 决策确认和任务来源追踪。",
      "severity": "medium"
    }
  ],
  "solution_attempt": [
    {
      "issue_title": "Decision 表结构尚未确认",
      "attempt": " 先在文档中列出 Decision 最小字段草案。",
      "result": " 等待后端确认"
    }
  ],
  "need_support": true,
  "support_detail": " 需要后端确认是否新增独立 decision_logs 表。",
  "next_day_plan": [
    {
      "title": " 完善 Manager Schema 和 Executive 决策链路",
      "priority": "high",
      "expected_output": " 形成可用于联调的 Schema 与数据库映射说明"
    }
  ],
  "risk_level": "medium",
```

```
  "employee_comment": "Decision 表结构需要后端确认后才能定稿。",
  "signature": {
    "signature_required": true,
    "signature_status": "pending",
    "signature_source": "im"
  },
  "meta": {
    "input_channel": "feishu",
    "agent_template_version": "staff_agent_v0.1"
  }
}
```

17.6.4 员工确认

Staff Agent 展示简化确认内容：

```
我整理成以下日报，请确认：

今日完成：
- 完成 Staff 日报 Schema 字段整理，补充签名字段和数据库映射说明。

遇到问题：
- Decision 表结构尚未确认，当前 DDL 中没有独立 decision_logs 表。

已尝试：
- 已先在文档中列出 Decision 最小字段草案。

需要支持：
- 需要后端确认是否新增独立 decision_logs 表。

明日计划：
- 完善 Manager Schema 和 Executive 决策链路。

风险等级：medium

请确认是否提交。
```

员工回复：

```
确认提交
```

确认后，Staff Agent 将 `signature.signature_status` 更新为 `signed`，并准备提交后端。

17.7 Step 3：Staff Schema 写入数据库

17.7.1 目标

验证 Staff Schema 可以通过 API 校验、签名、幂等并写入数据库。

17.7.2 调用接口

```
POST /api/v1/report/staff
```

请求头：

```
Idempotency-Key: staff_report:1:10086:2026-05-03
```

请求体：


```
{
  "schema": {
    "schema_id": "schema_v1_staff",
    "schema_version": "1.0.0",
    "timestamp": "2026-05-03T18:12:30+08:00",
    "org_id": 1,
    "department_id": 12,
    "user_id": 10086,
    "node_id": "staff_node_10086",
    "record_date": "2026-05-03",
    "work_progress": [
      {
        "title": " 完成 Staff 日报 Schema 字段整理",
        "description": " 补充 Staff 日报字段、签名字段和数据库映射说明。",
        "status": "completed"
      }
    ],
    "issues_faced": [
      {
        "title": "Decision 表结构尚未确认",
        "description": "当前 DDL 中没有独立 decision_logs 表，影响 Executive
        ↪  决策确认和任务来源追踪。",
        "severity": "medium"
      }
    ],
    "solution_attempt": [
      {
        "issue_title": "Decision 表结构尚未确认",
        "attempt": " 先在文档中列出 Decision 最小字段草案。",
        "result": " 等待后端确认"
      }
    ],
    "need_support": true,
    "support_detail": " 需要后端确认是否新增独立 decision_logs 表。",
    "next_day_plan": [
      {
        "title": " 完善 Manager Schema 和 Executive 决策链路",
        "priority": "high",
```

```
        "expected_output": " 形成可用于联调的 Schema 与数据库映射说明"
      }
    ],
    "risk_level": "medium",
    "employee_comment": "Decision 表结构需要后端确认后才能定稿。",
    "signature": {
      "signature_required": true,
      "signature_status": "signed",
      "signed_by": 10086,
      "signed_at": "2026-05-03T18:13:00+08:00",
      "payload_hash": "sha256:staff_report_demo_hash",
      "signature_source": "im"
    },
    "meta": {
      "input_channel": "feishu",
      "agent_template_version": "staff_agent_v0.1"
    }
  }
}
```


17.7.3 后端处理

后端应完成：

1. 校验 Staff Agent 权限。
2. 校验 `schema_id = schema_v1_staff`。
3. 校验 `user_id = 当前用户`。
4. 校验 `department_id = 当前用户部门`。
5. 校验必填字段。
6. 校验 `need_support = true` 时 `support_detail` 必填。
7. 校验签名人等于员工本人。
8. 校验幂等键。
9. 写入 `work_records`。
10. 写入 `work_record_items`。
11. 记录审计日志。
12. 由于存在支持需求，可生成异常候选或 Incident。

17.7.4 响应示例


```
{
  "success": true,
  "code": "OK",
  "message": "Staff 日报提交成功",
```

```
  "data": {
    "work_record_id": 301,
    "record_date": "2026-05-03",
    "status": "submitted",
    "created_incident_ids": [501],
    "created_task_ids": [],
    "audit_log_id": 7001
  }
}
```

17.7.5 成功表现

数据库中应能看到：

表	结果
work_records	新增一条员工日报主记录
work_record_items	新增多条字段明细
incidents	可选新增一条支持需求或阻塞异常
audit_logs	记录 Staff 日报提交和签名动作

17.8 Step 4：Manager Agent 定时读取部门数据

17.8.1 目标

验证 Manager Agent 可以读取本部门 Staff Schema，但不能读取其他部门或未确认数据。

17.8.2 触发方式

Demo 中可以用按钮或脚本模拟定时任务：

```
触发 Manager Agent 汇总架构组 2026-05-03 日报
```

17.8.3 Manager Agent 读取条件

Manager Agent 应读取：

```
org_id = 1
department_id = 12
record_date = 2026-05-03
status = submitted / confirmed
deleted_at is null
```

至少读取到刚才的 `work_record_id = 301`。

17.8.4 读取结果示例

```
{
  "records": [
    {
      "work_record_id": 301,
      "user_id": 10086,
      "record_date": "2026-05-03",
      "work_progress": [
        {
          "title": " 完成 Staff 日报 Schema 字段整理",
          "status": "completed"
        }
      ],
      "issues_faced": [
        {
          "title": "Decision 表结构尚未确认",
          "severity": "medium"
        }
      ],
      "need_support": true,
      "support_detail": " 需要后端确认是否新增独立 decision_logs 表。",
      "risk_level": "medium"
    }
  ]
}
```

17.8.5 成功表现

Manager Agent 应能说明：

```
已读取架构组 2026-05-03 的 1 条有效 Staff 日报。
其中 1 条包含支持需求，主要问题为 Decision 表结构尚未确认。
```

同时，后端应拒绝 Manager Agent 读取其他部门员工明细。这个权限点可以作为可选演示，不建议占用主 Demo 时间。

17.9 Step 5：Manager 生成部门汇总 Schema

17.9.1 目标

验证 Manager Agent 可以将 Staff Schema 聚合为 Manager Schema，并识别出需要上推老板的事项。

17.9.2 Manager Schema 示例

```
{
```

```
"schema_id": "schema_v1_manager",
"schema_version": "1.0.0",
"timestamp": "2026-05-03T21:10:00+08:00",
"org_id": 1,
"dept_id": 12,
"manager_user_id": 20001,
"manager_node_id": "manager_node_dept_12",
"summary_date": "2026-05-03",
"staff_report_count": 1,
"missing_report_count": 0,
"missing_staff_ids": [],
"overall_health": "yellow",
"aggregated_summary": "今日架构组主要完成 Staff 日报 Schema 字段整理，
↪  并补充签名字段和数据库映射说明。当前主要阻塞点是 Decision 表结构尚未确认，可能影响
↪  Executive 决策确认、任务来源追踪和审计闭环。",
"top_3_risks": [
  {
    "title": "Decision 表结构未确认",
    "description": "当前 DDL 中尚未建立独立 decision_logs 表，
    ↪  可能影响老板决策确认和任务生成链路。",
    "severity": "high",
    "source_record_ids": [301],
    "suggested_action": " 建议确认是否新增独立 Decision 记录表。"
  }
],
"workforce_efficiency": {
  "score": 80,
  "level": "medium",
  "basis": " 今日有效日报 1 条，主要工作按计划完成，但存在影响主链路的结构确认问题。"
},
"pending_approvals": 1,
"highlight_staff": [
  {
    "user_id": 10086,
    "display_name": " 杨卓",
    "reason": " 完成 Staff Schema 字段整理，并主动提出 Decision 表结构缺口。",
    "source_record_ids": [301]
  }
],
"blocked_items": [
  {
    "title": "Decision 表结构未定",
    "description": "当前无法确定老板确认结果应落入独立决策表，还是暂时复用 audit_logs 和
    ↪  tasks.meta。",
    "owner_user_id": 20002,
    "severity": "high",
    "need_support": true,
```

```
    "suggested_next_step": " 由老板或项目负责人确认是否新增独立 Decision 表。"
  }
],
"manager_decisions": [],
"need_executive_decision": [
  {
    "decision_title": " 是否新增独立 Decision 记录表",
    "context": "当前 Agent mock 流程中已经存在 decision_logs 概念，但正式 DDL
    ↪ 尚未建立独立决策表。如果不新增，短期可用 audit_logs 和 tasks.meta 承载，
    ↪ 但后续老板决策追踪和复盘会不够清晰。",
    "options": [
      {
        "option_id": "A",
        "title": " 新增独立 decisions / decision_logs 表",
        "pros": [" 决策对象清晰", " 便于审计", " 方便任务来源追踪"],
        "cons": [" 需要增加后端开发工作量"]
      },
      {
        "option_id": "B",
        "title": " 暂时复用 audit_logs 和 tasks.meta",
        "pros": [" 短期开发更快"],
        "cons": [" 决策对象不独立", " 后续统计和复盘不方便"]
      }
    ],
    "recommended_option": "A",
    "reason": "AutoMage-2 的核心价值在于决策可追溯，独立决策对象更符合长期架构。",
    "source_record_ids": [301],
    "urgency": "high"
  }
],
"next_day_adjustment": [
  {
    "title": " 优先确认 Decision 数据结构",
    "reason": " 该事项影响 Executive 决策确认和任务下发闭环。",
    "target_user_ids": [10086, 20002],
    "priority": "high",
    "expected_output": " 形成是否新增 Decision 表的最终结论和字段草案。"
  }
],
"source_record_ids": [301],
"related_task_ids": [],
"related_incident_ids": [501],
"signature": {
  "signature_required": true,
  "signature_status": "signed",
  "signed_by": 20001,
  "signed_at": "2026-05-03T21:25:00+08:00",
```

```
    "payload_hash": "sha256:manager_summary_demo_hash",
    "signature_source": "im"
  },
  "meta": {
    "input_channel": "scheduled_job",
    "agent_template_version": "manager_agent_v0.1"
  }
}
```


17.9.3 提交接口

```
POST /api/v1/report/manager
```

请求头：

```
Idempotency-Key: manager_summary:1:12:2026-05-03
```

17.9.4 响应示例


```
{
  "success": true,
  "code": "OK",
  "message": "Manager 汇总提交成功",
  "data": {
    "summary_id": 801,
    "summary_type": "department_daily_summary",
    "summary_date": "2026-05-03",
    "status": "submitted",
    "created_decision_candidate_ids": ["decision_tmp_001"],
    "created_task_ids": [],
    "audit_log_id": 7002
  }
}
```


17.9.5 成功表现

数据库中应能看到：

表	结果
summaries	新增部门汇总
summary_source_links	记录 summary 801 来源于 work_record 301
audit_logs	记录 Manager 汇总生成和确认
incidents	可关联更新 Decision 表结构异常
决策候选对象	可进入 Executive / Dream 后续处理

17.10 Step 6：Executive Agent 读取部门汇总

17.10.1 目标

验证 Executive Agent 可以读取 Manager Summary，并基于部门汇总生成组织级视角。

17.10.2 读取接口

```
GET /api/v1/summaries?org_id=1&summary_type=department_daily_summary&summary_date=2026-05-03
```

17.10.3 返回数据

至少返回：

```
{
  "summary_id": 801,
  "org_id": 1,
  "department_id": 12,
  "summary_type": "department_daily_summary",
  "scope_type": "department",
  "summary_date": "2026-05-03",
  "title": "2026-05-03 架构组部门日报",
  "content": "今日架构组主要完成 Staff 日报 Schema 字段整理，并补充签名字段和数据库映射说明。
  ↪ 当前主要阻塞点是 Decision 表结构尚未确认。",
  "source_count": 1,
  "status": "submitted",
  "meta": {
    "schema_id": "schema_v1_manager",
    "overall_health": "yellow",
    "top_3_risks": [
      {
        "title": "Decision 表结构未确认",
        "severity": "high"
      }
    ],
    "need_executive_decision": [
      {
        "decision_title": " 是否新增独立 Decision 记录表",
        "recommended_option": "A"
      }
    ]
  }
}
```

17.10.4 成功表现

Executive Agent 应能输出：

已读取 1 条部门汇总。架构组整体状态为 yellow，当前最关键风险是 Decision 表结构未确认。该事项已由
↪ Manager 上推为老板待决策事项。

17.11 Step 7：Dream 生成老板决策项

17.11.1 目标

验证 Dream 可以基于 Manager Schema 生成 Executive Schema 和老板决策卡片。

17.11.2 Executive Schema 示例


```
{
  "schema_id": "schema_v1_executive",
  "schema_version": "1.0.0",
  "timestamp": "2026-05-04T08:00:00+08:00",
  "org_id": 1,
  "executive_user_id": 30001,
  "executive_node_id": "executive_node_org_1",
  "summary_date": "2026-05-03",
  "business_summary": "昨日架构组完成 Staff 日报 Schema 字段整理和数据库映射说明，整体推进正常。
  ↪ 当前最主要风险是 Decision 表结构尚未确认，可能影响老板决策确认、
  ↪ 任务来源追踪和后续审计闭环。",
  "key_risks": [
    {
      "title": "Decision 表结构未确认",
      "description": "当前 DDL 尚未建立独立 Decision 记录表，若继续复用 audit_logs 和
      ↪ tasks.meta，短期可跑通，但长期追踪和复盘不清晰。",
      "severity": "high",
      "affected_departments": [12],
      "source_summary_ids": [801],
      "suggested_action": " 建议老板确认是否新增独立 Decision 表。"
    }
  ],
  "decision_required": true,
  "decision_items": [
    {
      "decision_id": "decision_0001",
      "title": " 是否新增独立 Decision 记录表",
      "context": "当前 Agent mock 流程中已经存在 decision_logs 概念，但正式 DDL
      ↪ 尚未建立独立决策表。如果不新增，短期可用 audit_logs 和 tasks.meta 承载，
      ↪ 但后续老板决策追踪和复盘会不够清晰。",
      "decision_level": "L3",
      "urgency": "high",
      "option_a": {
        "option_id": "A",
        "title": " 新增独立 decisions / decision_logs 表",
```

```
        "description": " 为老板决策、候选方案、确认结果和任务来源建立独立数据对象。",
        "pros": [" 决策对象清晰", " 便于审计", " 方便任务来源追踪"],
        "cons": [" 需要增加后端开发工作量"]
      },
      "option_b": {
        "option_id": "B",
        "title": " 暂时复用 audit_logs 和 tasks.meta",
        "description": "MVP 阶段不新增表，先用审计日志和任务 meta 承载决策结果。",
        "pros": [" 短期开发更快", " 减少表结构变更"],
        "cons": [" 决策对象不独立", " 后续统计和复盘不方便"]
      },
      "recommended_option": "A",
      "reasoning_summary": "AutoMage-2 的核心价值在于决策可追溯，独立 Decision
      ↪  表更利于后续任务生成、老板看板和审计复盘。",
      "expected_impact": {
        "cost": "medium",
        "timeline": " 短期增加 0.5-1 天后端工作量",
        "risk": " 降低后续决策追踪混乱风险",
        "affected_modules": ["database", "backend", "executive_agent"]
      },
      "generated_tasks": [
        {
          "task_title": " 补充 decisions 表结构草案",
          "task_description": "根据老板确认的方案 A，设计独立 decisions 表，字段需覆盖决策标题、
          ↪  背景、候选方案、推荐方案、确认人、确认时间和生成任务。",
          "assignee_role": "backend",
          "department_id": 12,
          "priority": "high",
          "due_at": "2026-05-04T18:00:00+08:00",
          "source_decision_id": "decision_0001"
        }
      ],
      "source_summary_ids": [801]
    }
  ],
  "source_summary_ids": [801],
  "human_confirm_status": "pending",
  "confirmed_by": null,
  "confirmed_at": null,
  "confirmed_option": null,
  "signature": {
    "signature_required": true,
    "signature_status": "pending",
    "signature_source": "im"
  }
}
```

17.11.3 成功表现

系统应生成：

对象	结果
Executive Schema	包含组织摘要、关键风险、决策项
Decision 候选对象	`decision_0001`，状态为 `pending`
任务草案	存在于决策项中，但尚未正式下发
审计日志	记录 Dream / Executive 生成决策项

注意：此时任务还不能正式创建，因为老板尚未确认。

17.12 Step 8：IM 推送老板决策卡片

17.12.1 目标

验证 Executive Agent 可以将 Executive Schema 转换为老板可读的 IM 决策卡片。

17.12.2 决策卡片内容

```
【今日老板待决策】

日期：2026-05-04
状态：需要确认 1 项决策

一、公司整体情况
昨日架构组完成 Staff 日报 Schema 字段整理和数据库映射说明，整体推进正常。当前主要风险是
↪ Decision 表结构尚未确认，可能影响老板决策确认、任务来源追踪和审计闭环。

二、关键风险
风险：Decision 表结构未确认
等级：高
影响：可能影响 Executive 决策确认、任务下发和后续复盘。

三、待决策事项
事项：是否新增独立 Decision 记录表

背景：
当前 Agent mock 流程中已经存在 decision_logs 概念，但正式 DDL 尚未建立独立决策表。

方案 A：新增独立 decisions / decision_logs 表
优点：决策对象清晰、便于审计、方便任务来源追踪
缺点：需要增加后端开发工作量

方案 B：暂时复用 audit_logs 和 tasks.meta
优点：短期开发更快、减少表结构变更
缺点：决策对象不独立，后续统计和复盘不方便
```

```
推荐：方案 A
理由：AutoMage-2 的核心价值在于决策可追溯，独立 Decision 表更利于后续任务生成、
↪  老板看板和审计复盘。

确认后将生成任务：
- 补充 decisions 表结构草案
- 负责人角色：后端
- 截止时间：今日 18:00
```

17.12.3 卡片按钮

建议按钮：

按钮	动作
选择方案 A	`action = confirm，confirmed_option = A`
选择方案 B	`action = confirm，confirmed_option = B`
需要补充信息	`action = need_more_info`
暂不处理	`action = reject` 或 `cancel`
查看来源	展示 `source_summary_ids = [801]`

17.12.4 成功表现

老板能在 IM 中看到：

1. 为什么需要决策。
2. 有哪些方案。
3. Agent 推荐哪个方案。
4. 推荐理由是什么。
5. 确认后会生成什么任务。

17.13 Step 9：老板选择方案

17.13.1 目标

验证老板确认动作可以被后端记录，并触发决策状态变化。

17.13.2 老板操作

老板点击：

```
选择方案 A
```

17.13.3 调用接口

```
POST /api/v1/decision/commit
```

请求头：

```
Idempotency-Key: decision_confirm:decision_0001:30001
```

请求体：

```
{
  "decision_id": "decision_0001",
  "org_id": 1,
  "executive_user_id": 30001,
  "action": "confirm",
  "confirmed_option": "A",
  "confirmed_custom_plan": null,
  "confirm_source": "im",
  "comment": " 选择方案 A，新增独立 Decision 记录表。",
  "signature": {
    "signature_required": true,
    "signature_source": "im"
  }
}
```

17.13.4 后端处理

后端应完成：

1. 校验老板身份。
2. 校验 decision_id 存在。
3. 校验决策状态为 pending。
4. 校验方案 A 存在。
5. 记录 confirmed_option = A。
6. 记录 confirmed_by = 30001。
7. 记录 confirmed_at。
8. 生成 payload_hash。
9. 更新签名状态。
10. 写入审计日志。
11. 准备生成任务。

17.13.5 响应示例

```
{
  "success": true,
  "code": "OK",
  "message": " 决策确认成功",
  "data": {
```

```
    "decision_id": "decision_0001",
    "status": "confirmed",
    "confirmed_option": "A",
    "confirmed_by": 30001,
    "confirmed_at": "2026-05-04T08:16:30+08:00",
    "generated_task_ids": [9001],
    "signature": {
      "signature_status": "signed",
      "payload_hash": "sha256:decision_demo_hash",
      "verify_status": "verified"
    },
    "audit_log_id": 7003
  }
}
```

17.14 Step 10：系统自动生成任务

17.14.1 目标

验证老板确认后，系统可以根据确认方案生成正式任务。

17.14.2 任务生成规则

老板选择方案 A 后，系统读取方案 A 对应的 `generated_tasks`，创建正式 Task。

生成任务示例：


```
{
  "schema_id": "schema_v1_task",
  "schema_version": "1.0.0",
  "org_id": 1,
  "department_id": 12,
  "task_title": " 补充 decisions 表结构草案",
  "task_description": "根据老板确认的方案 A，设计独立 decisions 表，字段需覆盖决策标题、背景、
  ↪ 候选方案、推荐方案、确认人、确认时间、来源 Summary 和生成任务。",
  "source_type": "executive_decision",
  "source_id": "decision_0001",
  "source_decision_id": "decision_0001",
  "creator_user_id": 30001,
  "created_by_node_id": "executive_node_org_1",
  "assignee_user_id": 20002,
  "assignee_role": "backend",
  "assignment_type": "owner",
  "priority": "high",
  "status": "pending",
  "due_at": "2026-05-04T18:00:00+08:00",
  "confirm_required": true,
```

```
  "meta": {
    "confirmed_option": "A",
    "source_summary_ids": [801],
    "push_channel": "feishu"
  }
}
```

17.14.3 写入数据库

系统应写入：

表	结果
tasks	创建任务 9001
task_assignments	分配给后端负责人或 Manager
task_updates	记录任务创建动态
audit_logs	记录老板决策生成任务

17.14.4 幂等要求

同一个决策和同一个确认方案不能重复生成任务。

幂等键建议：

```
task_from_decision:decision_0001:A
```

如果老板重复点击方案 A，系统应返回已有任务：

```
{
  "success": true,
  "code": "IDEMPOTENT_REPLAY",
  "message": " 该决策已确认并生成任务，返回原任务结果",
  "data": {
    "generated_task_ids": [9001]
  }
}
```

17.15 Step 11：任务下发到 Staff Agent

17.15.1 目标

验证任务可以推送给执行节点，并能被 Staff Agent 或 Manager Agent 接收。

17.15.2 下发逻辑

如果任务有明确 `assignee_user_id`，直接下发给对应 Staff Agent。如果只有 `assignee_role`，先下发给对应 Manager Agent，由 Manager 分配给具体员工。

Demo 中可以采用两种方式之一：

方式	说明
简化方式	直接分配给杨卓，方便演示 Staff Agent 查询
正式方式	先分配给后端 Manager，再拆给后端负责人

为了 Demo 流程更直观，建议直接分配给一个可展示的执行人，例如杨卓或后端负责人。

17.15.3 Staff Agent 收到任务通知

```
你收到 1 个新任务：

任务：补充 decisions 表结构草案
来源：老板确认的方案 A
优先级：高
截止时间：今日 18:00
要求：字段需覆盖决策标题、背景、候选方案、推荐方案、确认人、确认时间、来源 Summary 和生成任务。
```

17.15.4 成功表现

任务下发后：

1. `tasks.status = pending`。
2. `task_assignments` 中有执行人。
3. Staff Agent 可以通过任务接口查询到。
4. 任务来源可以追溯到 `decision_0001`。
5. `decision_0001` 可以追溯到 `summary_id = 801`。
6. `summary_id = 801` 可以追溯到 `work_record_id = 301`。
   这条追溯链是 Demo 的重点。

17.16 Step 12：员工查询今日任务

17.16.1 目标

验证 Staff Agent 可以读取员工自己的任务，并以自然语言展示给员工。

17.16.2 调用接口

```
GET /api/v1/tasks?org_id=1&assignee_user_id=10086&status=pending&page=1&page_size=20
```

17.16.3 响应示例

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {
```

```
    "items": [
      {
        "task_id": 9001,
        "org_id": 1,
        "department_id": 12,
        "task_title": " 补充 decisions 表结构草案",
        "task_description": " 根据老板确认的方案 A，设计独立 decisions 表结构。",
        "source_type": "executive_decision",
        "source_decision_id": "decision_0001",
        "assignee_user_id": 10086,
        "assignee_role": "backend",
        "priority": "high",
        "status": "pending",
        "due_at": "2026-05-04T18:00:00+08:00",
        "confirm_required": true,
        "created_at": "2026-05-04T08:16:35+08:00"
      }
    ],
    "pagination": {
      "page": 1,
      "page_size": 20,
      "total": 1,
      "has_more": false
    }
  }
}
```


17.16.4 Staff Agent 展示

```
今日待处理任务：

1. 补充 decisions 表结构草案
优先级：高
截止时间：今日 18:00
来源：老板确认的方案 A
要求：设计独立 decisions 表，字段需覆盖决策标题、背景、候选方案、推荐方案、确认人、确认时间、来源
↪  Summary 和生成任务。

你可以回复“开始处理第 1 个任务”来更新状态。
```

17.16.5 可选演示：员工开始处理

员工回复：

```
开始处理第 1 个任务
```

系统更新任务状态：

```
pending → in_progress
```

写入 task_updates 和 audit_logs。

17.17 Demo 成功判定标准

Demo 是否成功，不看界面是否精美，而看核心数据闭环是否真实跑通。

17.17.1 主链路成功标准

必须满足以下条件：

检查项	成功标准
Staff Agent 初始化	能返回用户、部门、节点和权限
Staff 日报提交	schema_v1_staff 成功写入 work_records
Staff 字段明细	work_record_items 中存在日报字段
员工签名	Staff 日报有签名状态和确认人
Manager 汇总生成	schema_v1_manager 成功写入 summaries
来源追溯	summary_source_links 能追溯到 Staff 日报
Executive 决策生成	能生成老板决策项和 A/B 方案
老板确认	决策状态变为 confirmed
任务生成	老板确认后生成正式任务
任务分配	task_assignments 中存在执行人
Staff 查询任务	员工能通过 Staff Agent 看到任务
审计记录	关键动作均有 audit_logs

17.17.2 数据追溯成功标准

Demo 最后应能展示一条完整来源链：

```
任务 9001
  来源于 decision_0001
    来源于 Manager Summary 801
      来源于 Staff Work Record 301
        来源于杨卓确认提交的 Staff Schema
```

如果能展示这条链，说明 AutoMage-2 的核心价值已经表达出来：

```
老板任务不是凭空来的；
它来自员工真实工作记录；
经过部门汇总；
经过老板确认；
最后下发给执行人。
```

17.17.3 权限成功标准

至少验证以下权限点：

1. Staff Agent 只能提交本人日报。
2. Staff Agent 只能查询本人任务。
3. Manager Agent 只能读取本部门 Staff 数据。
4. Executive Agent 读取的是部门汇总，而不是默认读取所有员工明细。
5. 老板决策必须由 Executive 用户确认。
6. 未确认决策不能生成正式任务。

Demo 中不一定逐条展示失败场景，但后端接口必须具备这些校验。

17.17.4 签名成功标准

至少验证：

1. 员工日报有员工确认记录。
2. Manager 汇总有 Manager 确认或待确认状态。
3. 老板决策有老板确认记录。
4. 确认动作有 `payload_hash` 或等价内容哈希。
5. 签名动作写入审计日志。
6. 未签名老板决策不能生成任务。

17.17.5 幂等成功标准

至少验证：

1. 员工重复提交同一天日报不会产生重复主记录。
2. 老板重复点击确认不会生成重复任务。
3. Dream 或任务生成重试不会产生重复决策卡片。
4. 幂等重放能返回已有结果。

17.17.6 演示话术建议

Demo 结束时可以这样总结：

```
这条 Demo 链路证明了 AutoMage-2 不是普通日报系统。

员工只提交一次工作记录，系统会自动转成结构化 Staff Schema；
Manager Agent 基于真实日报生成部门汇总；
Executive Agent 和 Dream 把部门风险整理成老板可判断的 A/B 决策；
老板确认后，系统自动生成任务并下发给执行人；
```

```
最后任务还能追溯回最初的员工日报。

这就是 AutoMage-2 MVP 要验证的核心闭环：
一线事实 → 部门判断 → 老板决策 → 任务执行。
```

17.18 本章小结

MVP Demo 的核心不在于展示所有功能，而在于用一条清晰链路证明系统成立：

```
Staff Agent
  → Manager Agent
  → Executive Agent / Dream
  → 老板确认
  → 任务下发
  → Staff Agent 接收
```

只要 Demo 能稳定跑通以下结果，MVP 就具备可交付价值：

1. 员工日报能结构化。
2. 部门汇总能自动生成。
3. 老板决策能被整理成 A/B 方案。
4. 老板确认能生成正式任务。
5. 任务能下发到执行人。
6. 全链路能追溯来源、签名和审计。

后续所有复杂能力，包括自动采集、行业模板、老板看板、长期 Dream、跨部门协作和组织诊断，都可以建立在这条最小闭环之上。

18 MVP 验收标准

18.1 Schema 验收标准

Schema 是 AutoMage-2 MVP 的数据契约。只要 Schema 不稳定，Agent、API、数据库、前端和 IM 都会被迫各自理解一套字段，后续联调会非常痛苦。

MVP 阶段至少需要完成以下 Schema：

Schema	用途	验收要求
`schema_v1_staff`	员工日报	可表达员工工作进展、问题、支持需求、明日计划、签名
`schema_v1_manager`	部门汇总	可表达部门摘要、风险、阻塞、待上推事项、来源记录
`schema_v1_executive`	老板决策	可表达组织摘要、关键风险、A/B 决策项、任务草案、确认状态
`schema_v1_task`	任务	可表达任务来源、负责人、状态、优先级、截止时间、结果
`schema_v1_incident`	异常	可表达异常来源、等级、处理状态、负责人、上推关系

Schema 验收标准如下：

验收项	通过标准
Schema ID 清晰	每类 Schema 有固定 `schema_id`，不能混用
版本可识别	每类 Schema 包含 `schema_version`
必填字段明确	每个 Schema 有明确必填字段清单
字段类型稳定	string、number、boolean、array、object 不混用
枚举值统一	风险、状态、优先级等字段使用统一枚举
来源可追溯	Staff、Manager、Executive、Task、Incident 都能保留来源 ID
签名字段统一	涉及确认的数据有统一 `signature` 结构
可被后端校验	后端能根据 Schema 规则返回明确 422 错误
可被数据库承载	每个核心字段能映射到表字段或 `meta`
可被 Agent 稳定生成	Agent 连续生成多次，结构不应大幅漂移

不通过情况包括：

1. 同一个字段在不同示例中类型不一致。
2. 同一个状态出现多套写法，例如 `high`、高、严重混用。
3. Staff Schema 不能追溯到员工和日期。
4. Manager Schema 不能追溯到来源 Staff 记录。
5. Executive Schema 不能追溯到来源 Manager Summary。
6. Task 没有来源对象。
7. Incident 不能关联任务或来源数据。
8. 签名字段只是文案，没有签名状态、确认人和时间。

Schema 最小验收结论应为：

```
Staff、Manager、Executive、Task、Incident 五类 Schema 均已定义；
字段结构稳定；
能通过后端校验；
能落到现有数据库或明确的新增表；
能支撑 Demo 主链路。
```

18.2 Agent 验收标准

Agent 是 AutoMage-2 MVP 的执行入口。MVP 阶段不要求 Agent 拥有完整长期记忆和复杂自主规划能力，但必须能在各自边界内稳定完成结构化、汇总、决策整理和任务读取。

需要验收的 Agent 包括：

Agent	核心职责
Staff Agent	整理员工日报、提醒补全、提交 Staff Schema、读取个人任务
Manager Agent	读取本部门日报、生成部门汇总、识别风险、上推老板事项
Executive Agent	读取部门汇总、生成老板决策卡片、处理确认结果
Dream	定时生成组织摘要、风险、决策项和任务草案

18.2.1 Staff Agent 验收标准

验收项	通过标准
初始化	能通过 `/api/v1/agent/init` 获取用户、部门、节点和权限
日报整理	能把员工自然语言整理为 `schema_v1_staff`
缺失追问	必填字段缺失时能精准追问
员工确认	提交前必须让员工确认
权限边界	只能读取和提交本人数据
任务读取	能读取并展示分配给员工自己的任务
状态更新	能将任务更新为进行中、阻塞、已完成
异常上报	能识别 `need_support` 和 `blocked` 并上报

不通过情况包括：

1. 员工未确认就提交日报。
2. Staff Agent 替员工编造日报内容。
3. Staff Agent 读取其他员工日报。
4. `need_support = true` 但不追问支持详情。
5. 任务来源、截止时间、优先级展示不清楚。

18.2.2 Manager Agent 验收标准

验收项	通过标准
部门读取	能读取本部门已提交 Staff Schema
数据过滤	不读取未确认、已删除或无权限数据
部门汇总	能生成 `schema_v1_manager`
风险识别	能从 Staff 记录中识别部门风险
缺失统计	能统计日报数量和缺失数量
来源引用	`source_record_ids` 准确
上推事项	能生成 `need_executive_decision`
权限边界	不能读取其他部门员工明细

不通过情况包括：

1. Manager Schema 没有来源 Staff 记录。
2. 部门风险只是空泛总结，没有证据。
3. 直接把所有员工日报原文拼接给老板。
4. 把部门内可解决事项全部上推老板。
5. 读取其他部门员工明细。

18.2.3 Executive Agent 验收标准

验收项	通过标准
汇总读取	能读取组织内 Manager Summary
决策生成	能生成老板可判断的决策项
方案表达	每个关键决策至少有 A/B 方案
推荐理由	推荐方案必须有简明理由
任务草案	确认后可生成任务草案
卡片展示	能生成 IM 决策卡片
确认处理	老板确认后调用决策提交接口
权限边界	不能绕过老板确认生成重大任务

不通过情况包括：

1. 只生成一段普通日报，没有决策项。
2. 决策项没有方案。
3. 推荐方案没有理由。
4. 老板未确认就创建正式任务。
5. 无法追溯决策来源。

18.2.4 Dream 验收标准

验收项	通过标准
定时运行	能按指定时间或手动触发运行
输入读取	能读取 Manager Schema、历史 Summary、任务和决策
风险归并	能合并同类风险，避免重复推送
输出结构	输出能映射到 Executive Schema
降级处理	输入不足或运行失败时能生成简版摘要或失败记录
幂等处理	同一日期重复运行不生成重复决策
审计记录	每次运行有运行记录或审计日志

不通过情况包括：

1. Dream 输出自由文本，无法写库。
2. 重复生成同一老板决策项。
3. 不引用来源 Summary。
4. 模型失败后没有记录。
5. 低风险事项大量推给老板。

18.3 API 验收标准

API 是 Agent、IM、前端和数据库之间的正式契约。MVP 阶段至少要保证主链路 API 可用、可校验、可幂等、可追踪。

必须验收的接口如下：

接口	用途
`POST /api/v1/agent/init`	Agent 初始化
`POST /api/v1/report/staff`	Staff 日报提交
`POST /api/v1/report/manager`	Manager 汇总提交
`POST /api/v1/decision/commit`	老板决策确认
`GET /api/v1/tasks`	任务获取
`GET /api/v1/summaries`	汇总读取
`POST /api/v1/incidents`	异常上报
`GET /api/v1/audit-logs`	审计日志读取

API 验收标准如下：

验收项	通过标准
统一响应格式	所有接口返回 `success/code/message/data/request_id`
统一错误格式	失败时返回明确错误码和字段级错误
鉴权可用	未登录或 Token 无效返回 401
权限可用	越权访问返回 403 或 404
Schema 校验	字段缺失、类型错误、枚举错误返回 422
幂等支持	关键写入接口支持 `Idempotency-Key`
审计写入	关键操作写入 `audit_logs`
错误可修复	Agent 能根据错误信息追问用户或重试
状态可控	非法状态流转会被拒绝
数据可追踪	返回结果中包含写入对象 ID

关键接口验收重点如下：

18.3.1 Agent 初始化接口

必须返回：

1. 当前用户。
2. 当前组织。
3. 当前部门。
4. Agent 节点 ID。
5. Agent 类型。
6. 权限范围。
7. 支持的 Schema 版本。

18.3.2 Staff 日报提交接口

必须支持：

1. Staff Schema 校验。
2. 员工本人权限校验。
3. 员工签名校验。
4. 幂等提交。
5. 写入 `work_records` 和 `work_record_items`。

18.3.3 Manager 汇总提交接口

必须支持：

1. Manager Schema 校验。
2. 本部门权限校验。
3. 来源 Staff 记录校验。
4. 写入 `summaries`。

5. 写入 `summary_source_links`。

18.3.4 决策确认接口

必须支持：

1. 决策存在性校验。
2. 老板确认权限校验。
3. 确认方案校验。
4. 签名生成。
5. 任务生成。
6. 幂等防重复任务。

API 不通过情况包括：

1. 错误只返回“系统错误”。
2. Staff 能替别人提交日报。
3. Manager 能读取其他部门明细。
4. 老板重复确认生成多组任务。
5. 422 错误没有字段级提示。
6. 关键写入成功但没有审计日志。

18.4 数据库验收标准

数据库是 AutoMage-2 的事实源。MVP 验收时，不能只看 Agent 是否生成了文字，还要检查数据是否真实落表、关系是否完整、来源是否可追溯。

数据库验收分为两类：

1. 当前已有表是否能支撑主链路。
2. 关键缺口是否有明确补齐方案。

18.4.1 主链路落表验收

数据对象	目标表	通过标准
Staff 日报主记录	work_records	能写入员工、日期、部门、状态、模板
Staff 字段明细	work_record_items	能保存工作进展、问题、支持需求、明日计划
Manager 汇总	summaries	能保存部门摘要、风险、健康度
Summary 来源	summary_source_links	能追溯到 Staff 日报
任务主记录	tasks	能保存任务标题、描述、状态、优先级、截止时间
任务分配	task_assignments	能关联任务和执行人
任务动态	task_updates	能记录状态变化
异常主记录	incidents	能保存异常标题、等级、状态
异常动态	incident_updates	能记录异常处理过程
产出物	artifacts	能关联日报、任务或异常
审计日志	audit_logs	能记录关键动作

18.4.2 数据追溯验收

MVP 必须能追溯以下链路：

```
task_id
  → source_decision_id
  → source_summary_ids
  → source_record_ids
  → staff user_id
```

如果还没有独立 Decision 表，至少要能通过 `tasks.meta.source_decision_id`、`summaries.meta.decision_items`、`summary_source_links` 追溯。

最低通过标准：

```
任务可以追溯到老板决策；
老板决策可以追溯到 Manager Summary；
Manager Summary 可以追溯到 Staff Work Record；
Staff Work Record 可以追溯到员工本人确认。
```

18.4.3 数据库缺口验收

当前 DDL 中建议补齐的对象包括：

缺口	MVP 验收要求
Agent 节点表	至少有 Mock 配置或 agent_nodes 表方案
Decision 表	至少有临时承载方式，最好新增 decisions
Digital Signature	至少能在 meta.signature 中保存签名字段
Dream Run 表	至少有运行日志或审计记录
Schema Contract	至少有文档化 Schema 和后端校验规则

不通过情况包括：

1. 只有 IM 消息，没有数据库记录。
2. Manager 汇总无法追溯来源日报。
3. 老板决策无法追溯来源 Summary。
4. 任务不知道由哪个决策产生。
5. 审计日志无法说明关键状态变化。
6. 签名信息完全没有落库。

18.5 权限验收标准

权限验收的重点是证明系统不会因为引入 Agent 而破坏组织、部门和用户边界。

权限验收至少覆盖四层：

组织级隔离
部门级隔离
用户级隔离
Agent 节点级隔离

18.5.1 组织级权限

验收项	通过标准
org_id 校验	所有核心查询和写入都校验 `org_id`
跨组织读取	用户不能读取其他组织数据
跨组织写入	用户不能向其他组织写数据
审计记录	跨组织越权尝试有审计日志

18.5.2 用户级权限

验收项	通过标准
本人日报	Staff 只能提交本人日报
本人任务	Staff 只能查询分配给自己的任务
本人签名	Staff 日报签名人必须是本人
他人数据	Staff 不能读取其他员工日报

18.5.3 部门级权限

验收项	通过标准
本部门读取	Manager 可读取本部门 Staff 数据
跨部门限制	Manager 默认不能读取其他部门员工明细
部门汇总	Manager 只能提交本部门汇总
部门任务	Manager 只能下发或管理本部门任务

18.5.4 Executive 权限

验收项	通过标准
组织汇总	Executive 可读取组织级和部门级汇总
老板确认	只有老板或授权高管能确认 L3 决策
下钻控制	Executive 默认看汇总，必要时可追溯来源
任务生成	未确认决策不能生成正式任务

18.5.5 Agent 节点权限

验收项	通过标准
节点绑定	Staff / Manager / Executive Agent 有明确绑定对象
初始化权限	`/agent/init` 返回准确权限
API 校验	后端同时校验用户和 Agent 节点
节点越权	Agent 越权请求被拒绝

权限不通过情况包括：

1. Staff Agent 能查询其他员工日报。
2. Manager Agent 能读取其他部门明细。
3. 普通员工能确认老板决策。
4. 未初始化 Agent 能调用核心写入接口。
5. Agent 修改 `org_id` 或 `department_id` 后能绕过权限。
6. 权限失败没有审计日志。

18.6 决策机制验收标准

决策机制是 AutoMage-2 区别于普通日报系统的核心。MVP 验收时，要重点检查决策是否能从 Staff 问题逐步上推到老板确认，并在确认后下发任务。

18.6.1 决策分层验收

决策等级	验收标准
L0	Agent 可自动提醒和补全，不改变重大业务状态
L1	员工确认后提交个人日报和任务状态
L2	Manager 确认部门汇总、部门任务和部门异常
L3	老板确认后才能执行组织级决策
L4	高风险事项能标记为线下会议或人工审批

18.6.2 上推机制验收

验收项	通过标准
Staff 上推	Staff 的阻塞和支持需求能进入 Manager 视角
Manager 上推	Manager 能生成 `need_executive_decision`
来源完整	上推事项包含来源记录
方案完整	老板决策项包含 A/B 方案
推荐明确	Agent 推荐方案有理由
权限正确	超权限事项不会被下级直接执行

18.6.3 老板确认验收

验收项	通过标准
决策卡片	老板能看到背景、方案、推荐理由和任务草案
确认动作	老板可选择方案 A / B / 补充信息 / 驳回
签名记录	确认动作记录确认人、时间、来源和 hash
状态变化	决策从 `pending` 变为 `confirmed`
审计记录	决策确认写入审计日志
任务触发	确认后生成正式任务

不通过情况包括：

1. Manager 直接执行老板级决策。
2. 老板卡片没有方案，只是“请老板处理”。
3. 老板未确认，系统已生成任务。
4. 决策没有来源 Summary。
5. 决策确认后没有记录签名和审计。
6. 决策结果无法追溯到任务。

18.7 Dream 机制验收标准

Dream 机制的验收重点，不是判断它是否“聪明”，而是判断它是否能稳定完成组织级摘要、风险识别、决策生成和任务草案输出。

18.7.1 Dream 输入验收

输入类型	通过标准
Manager Schema	能读取指定日期部门汇总
历史 Summary	能读取近几天相关摘要
历史 Decision	能判断是否重复决策
未完成任务	能识别高优、逾期、阻塞任务
外部目标	能参考当前 MVP 目标或演示目标

MVP 最低要求是能读取 Manager Schema。其他输入可以先作为增强项。

18.7.2 Dream 输出验收

输出类型	通过标准
组织摘要	生成 `business_summary`
关键风险	生成 `key_risks`，且有来源
决策项	生成 `decision_items`
A/B 方案	每个关键决策至少两个方案
推荐理由	`reasoning_summary` 不为空
任务草案	`generated_tasks` 可转成正式任务
次日重点	可生成 `next_day_focus` 或部门调整建议

18.7.3 Dream 运行验收

验收项	通过标准
定时运行	能按时间或手动触发
输出结构	输出能映射到 `schema_v1_executive`
来源引用	每个高风险和决策项有来源 Summary
幂等	同一运行窗口不重复生成同一决策
失败记录	运行失败有错误记录
降级策略	失败时可生成简版摘要或转发上推事项
审计	运行成功、失败、重试都有记录

不通过情况包括：

1. Dream 输出无法解析。
2. Dream 编造不存在的数据。
3. Dream 不引用来源 Summary。
4. Dream 每次运行都生成重复决策。
5. Dream 失败没有记录。
6. Dream 直接绕过老板确认生成重大任务。

18.8 任务闭环验收标准

任务闭环证明系统不只是汇总信息，而是能推动执行。

MVP 阶段必须跑通：

```
老板确认
  → 任务生成
  → 任务分配
  → Staff Agent 展示
  → 员工更新状态
  → 任务结果回流
```

18.8.1 任务生成验收

验收项	通过标准
来源明确	每个任务有 `source_type` 和 `source_id`
决策任务	老板确认后才能创建
任务字段	标题、描述、负责人、优先级、截止时间完整
任务写库	写入 `tasks`
任务分配	写入 `task_assignments`
任务动态	创建时写入 `task_updates`
幂等	同一决策不重复生成任务

18.8.2 任务分配验收

验收项	通过标准
明确执行人	有 `assignee_user_id` 时直接分配
角色任务	只有角色时先给 Manager 分配
跨部门任务	可由多个 Manager 承接
权限校验	不能把任务分配给无关组织或部门
分配失败	任务进入待分配状态或返回明确错误

18.8.3 任务状态验收

任务至少支持以下状态：

状态	通过标准
`pending`	任务已创建但未开始
`in_progress`	员工开始处理
`blocked`	员工说明阻塞原因
`completed`	员工提交完成结果
`closed`	确认方关闭任务
`cancelled`	任务被取消并记录原因

状态变化必须写入 `task_updates`，关键动作写入 `audit_logs`。

18.8.4 Staff Agent 任务交互验收

验收项	通过标准
查询任务	Staff Agent 能获取本人任务
展示任务	展示标题、来源、优先级、截止时间
开始任务	员工可将任务改为进行中
标记阻塞	员工可说明阻塞原因
提交完成	员工可提交结果摘要和产出物
回流日报	当日任务状态可进入 Staff Schema

不通过情况包括：

1. 任务没有来源。
2. 老板未确认就生成任务。
3. 员工看不到自己的任务。
4. 任务完成没有结果摘要。
5. 任务状态变化没有记录。
6. 重复点击老板确认生成重复任务。

18.9 异常处理验收标准

异常处理机制用于证明系统能识别阻塞和风险，而不是只记录顺利完成的事项。

18.9.1 异常识别验收

异常来源	通过标准
Staff 日报	能从 `need_support`、`blocked`、高风险字段识别异常
Manager 汇总	能合并多个 Staff 异常
Task	任务逾期或阻塞能生成异常
Dream	组织级风险能生成异常或决策项
系统规则	Schema 校验失败、推送失败、接口失败能记录异常

18.9.2 异常字段验收

Incident 至少包含：

1. `incident_title`
2. `description`
3. `incident_type`
4. `severity`
5. `status`
6. `source_type`
7. `source_id`
8. `reporter_user_id`
9. `owner_user_id`
10. `need_escalation`

18.9.3 异常流转验收

异常至少支持以下流转：

```
open
  → assigned
  → in_progress
  → resolved
  → closed
```

涉及老板决策的异常可进入：

```
waiting_decision
```

异常处理必须满足：

验收项	通过标准
创建	Staff / Manager / 系统规则能创建异常
分级	异常有 low / medium / high / critical
分配	异常能指定负责人
上推	high 或 critical 可上推 Executive
生成任务	需要处理的异常可生成任务
关闭	关闭必须有原因和确认人
重新打开	问题复现时可 reopened
审计	状态变化写入审计

18.9.4 异常与任务关系验收

验收项	通过标准
异常生成任务	一个异常可生成一个或多个任务
任务关联异常	任务能追溯到异常
任务阻塞反向异常	任务 blocked 可生成或更新异常
异常关闭依赖任务	关联任务未完成时，不应直接关闭异常
多来源合并	多个 Staff 问题可合并为一个部门异常

不通过情况包括：

1. 员工标记阻塞后 Manager 看不到。
2. 异常没有来源。
3. 异常只创建不处理。
4. 异常生成任务后无法追踪。
5. 任务未完成但异常已关闭。
6. critical 异常没有上推和审计。

18.10 演示验收标准

演示验收是最终验收。它不要求所有边缘能力完美，但必须证明主链路真实可用。

MVP Demo 必须完整跑通以下 12 步：

```
Step 1: Staff Agent 初始化
Step 2: 员工提交每日工作 Schema
Step 3: Staff Schema 写入数据库
Step 4: Manager Agent 定时读取部门数据
Step 5: Manager 生成部门汇总 Schema
Step 6: Executive Agent 读取部门汇总
Step 7: Dream 生成老板决策项
Step 8: IM 推送老板决策卡片
Step 9: 老板选择方案
Step 10: 系统自动生成任务
Step 11: 任务下发到 Staff Agent
Step 12: 员工查询今日任务
```

18.10.1 Demo 主链路验收

步骤	成功标准
Staff 初始化	返回用户、部门、Agent 节点和权限
员工填报	自然语言被整理成 Staff Schema
员工确认	Staff 日报有签名信息
日报写库	`work_records` 和 `work_record_items` 有记录
Manager 读取	能读取本部门 Staff 数据
Manager 汇总	`summaries` 有部门汇总
来源追溯	`summary_source_links` 指向 Staff 日报
Executive 读取	能读取 Manager Summary
Dream 输出	能生成老板决策项
IM 卡片	老板能看到 A/B 方案
老板确认	决策状态变为 confirmed
任务生成	`tasks` 和 `task_assignments` 有记录
Staff 查询	员工能看到下发任务

18.10.2 Demo 数据追溯验收

Demo 最后必须能展示一条完整链路：

```
任务 9001
  → 来源于老板决策 decision_0001
  → 来源于 Manager Summary 801
  → 来源于 Staff Work Record 301
  → 来源于员工杨卓确认提交的 Staff Schema
```

这条链路是 MVP 的核心验收点。如果追溯链断了，说明系统只是“生成了内容”，还没有形成可信组织数据闭环。

18.10.3 Demo 签名与审计验收

至少要展示以下确认和审计：

动作	验收标准
员工确认日报	有 `signed_by`、`signed_at`、`payload_hash`
Manager 确认汇总	有确认状态或签名记录
老板确认决策	有确认人、确认时间、确认方案
任务生成	有审计日志
权限校验	越权操作会被拒绝
幂等重放	重复确认不会重复生成任务

18.10.4 Demo 失败判定

出现以下任一情况，应判定 Demo 未通过：

1. 员工日报没有真实写库。
2. Manager 汇总无法追溯 Staff 日报。
3. 老板决策没有 A/B 方案。
4. 老板未确认就生成正式任务。
5. 老板重复确认生成重复任务。
6. Staff Agent 看不到下发任务。
7. 关键动作没有审计记录。
8. Staff 能读取他人日报。
9. Manager 能越权读取其他部门明细。
10. 任务无法追溯来源决策。

18.10.5 Demo 最终通过标准

MVP Demo 最终通过标准可以写成一句话：

```
系统能从员工确认的 Staff 日报出发，经 Manager 汇总、Dream 分析、老板确认，
↪ 生成正式任务并下发给执行人；同时全链路具备来源追溯、权限校验、签名确认、幂等控制和审计记录。
```

只要达到这个标准，AutoMage-2 MVP 就可以认为完成了第一阶段核心闭环。

18.11 本章小结

MVP 验收不应只看“功能有没有页面”，而应看 AutoMage-2 的组织闭环是否成立。

本阶段最重要的验收结果是：

```
Schema 能稳定表达业务；
Agent 能按边界生成和读取数据；
API 能校验、写库、返回错误；
数据库能支撑来源追溯；
权限能阻止越权访问；
决策能从下级上推到老板；
Dream 能生成老板可判断的决策项；
任务能在确认后下发并回流；
异常能被发现、处理和关闭；
Demo 能完整跑通主链路。
```

MVP 通过后，系统应具备一条最小但完整的运行链路：

```
员工事实
  → 部门判断
  → 老板决策
  → 任务执行
  → 异常处理
  → 审计复盘
```

这条链路跑通，才说明 AutoMage-2 不只是一个日报生成器，而是具备了企业级 AI 中枢的基础形态。

19 风险与待确认问题

19.1 Schema 是否过于通用

当前 MVP 设计采用通用 Schema 作为主线，包括 Staff Schema、Manager Schema、Executive Schema、Task Schema、Incident Schema 等。这样做的好处是开发链路统一，Agent、API、数据库和前端可以围绕一套稳定结构联调，不需要一开始就为不同行业分别设计字段。

但风险也很明显：如果 Schema 过于通用，业务表达会变弱。员工填报时可能觉得字段不贴合自己的岗位，Manager 汇总时也可能只能生成比较宽泛的部门摘要，难以体现具体行业或岗位差异。

例如，研发岗位关心接口、代码、Bug、联调、DDL；销售岗位关心客户、商机、跟进阶段、成交概率；客服岗位关心工单、满意度、投诉风险。如果全部用一套字段表达，MVP 可以跑通，但长期会影响使用体验和数据分析价值。

MVP 阶段建议采用以下策略：

```
先通用 Schema，后行业模板 Schema。
```

也就是说，当前版本先保证所有岗位都能用同一套基础结构表达：

1. 今天做了什么。
2. 遇到什么问题。
3. 是否需要支持。
4. 明天计划做什么。
5. 是否存在风险。
6. 有哪些产出物。
7. 是否需要上级决策。

后续再在通用 Schema 外层增加行业模板或岗位模板，例如：

模板类型	示例
研发岗位模板	任务、代码提交、接口、Bug、联调风险
销售岗位模板	客户、商机、报价、合同、回款风险
客服岗位模板	工单、投诉、满意度、升级处理
运营岗位模板	活动、数据、投放、转化、异常
管理岗位模板	团队状态、风险、资源、决策事项

建议本阶段不要为了行业适配过早拆分大量 Schema。否则会导致接口、数据库、Agent Prompt 和前端表单快速复杂化，影响 MVP 主链路交付。

本问题的推荐结论：

```
MVP 阶段保留通用 Schema；
只为 Demo 场景补充少量岗位字段示例；
行业模板放到下一阶段。
```

19.2 员工填报真实性问题

员工填报真实性是 AutoMage-2 进入真实企业场景后必须面对的问题。

Staff Agent 可以帮助员工整理日报，但不能保证员工输入一定真实。如果员工随便填写、漏填、过度美化，系统后续的 Manager 汇总、Executive 决策和 Dream 分析都会受到影响。

这个问题不能只靠 AI 解决，也不能简单通过“监控员工”解决。MVP 阶段应采用低侵入、可解释、可追溯的方式提高数据可信度。

主要风险包括：

1. 员工填写过于简单，例如“正常推进”。
2. 员工不填写真实阻塞，导致风险无法提前暴露。
3. 员工夸大完成情况，任务状态与实际不一致。
4. 员工忘记填写，导致 Manager 汇总数据缺失。
5. Agent 过度润色，使原始问题看起来不明显。
6. 员工认为系统是考核工具，从而不愿真实表达问题。

MVP 阶段建议采用以下处理方式：

19.2.1 强制结构化，不强制复杂化

员工不需要写长篇日报，但必须填写几个关键字段：

1. 今日完成事项。
2. 遇到的问题。
3. 是否需要支持。
4. 明日计划。
5. 风险等级。
6. 确认提交。

如果员工只输入“今天正常”，Staff Agent 应追问：

```
请至少补充一项今日完成事项，并说明明天计划继续推进什么。
```

19.2.2 员工确认后再入库

Agent 生成的 Staff Schema 必须经过员工确认。未确认内容只能是草稿，不能进入 Manager 汇总。

这样可以明确责任边界：

```
Agent 负责整理；
员工负责确认。
```

19.2.3 任务状态与日报交叉校验

如果员工日报说任务完成，但任务系统中仍为 `in_progress`，系统应提示员工确认是否更新任务状态。

如果任务已逾期，但员工日报未提到风险，Staff Agent 应提醒：

```
你有一个高优先级任务已接近截止时间，是否需要在今天日报中说明当前进度或阻塞原因？
```

19.2.4 Manager 汇总中标记数据完整度

Manager Schema 应包含：

1. `staff_report_count`
2. `missing_report_count`
3. `missing_staff_ids`
4. 数据可信度说明

这样老板看到部门汇总时，可以知道数据是否完整，而不是默认所有部门信息都可靠。

19.2.5 不把系统做成强监控工具

MVP 阶段不建议一开始引入屏幕监控、键盘记录、自动截图等强感知能力。这会显著增加员工抵触，也会带来隐私、合规和解释成本。

推荐先用“员工主动确认 + 任务状态回流 + Manager 审核 + 审计追踪”的方式提高可信度。

本问题的推荐结论：

```
MVP 阶段不追求完全自动验证真实性；
先通过结构化填报、员工确认、任务交叉校验和 Manager 审核提高可信度。
```

19.3 屏幕记忆方案是否进入 MVP

屏幕记忆指系统通过截图、窗口记录、操作轨迹、文档访问记录等方式，自动采集员工工作过程，再由 Agent 整理为工作记忆或日报内容。

这个方向长期有价值，因为它能减少员工填报负担，也能让系统更接近真实工作现场。但它不适合过早进入 MVP 主链路。

原因有四点。

第一，技术复杂度高。屏幕记忆涉及客户端采集、权限授权、数据压缩、OCR、多模态理解、隐私过滤、存储成本和检索能力，远超当前 MVP 的核心验证范围。

第二，隐私和合规风险高。屏幕可能包含客户信息、个人聊天、密码、财务数据、合同内容等敏感信息。如果没有完善的本地过滤和权限控制，企业很难接受。

第三，员工心理阻力大。如果 MVP 一开始就强调屏幕采集，员工很容易把系统理解为监控工具，而不是协作工具。

第四，它会分散主链路。AutoMage-2 当前最需要验证的是：

```
Staff → Manager → Executive → Dream → Task
```

屏幕记忆属于输入增强，不是主链路本身。主链路没有跑通之前，不应把复杂采集能力放入核心交付。

MVP 阶段建议采用替代方案：

能力	MVP 做法
工作记录	员工主动输入或 IM 表单
产出物	员工主动上传链接或文件
任务进展	从任务状态和员工反馈中读取
真实性辅助	通过产出物、任务状态、Manager 确认交叉判断
自动采集	暂不进入主链路

可以在文档中保留屏幕记忆作为后续方向：

```
v0.1：IM 表单填报
v0.2：任务与产出物自动回填
v0.3：文档、代码、工单系统接入
v0.4：可授权的屏幕记忆 / 工作流记忆
```

本问题的推荐结论：

```
屏幕记忆不进入 MVP 主链路；
MVP 先用 IM 表单、任务状态和产出物链接完成数据输入。
```

19.4 Agent 是否允许自动决策

这是 AutoMage-2 最重要的边界问题之一。

AutoMage-2 的目标不是让 Agent 直接替老板、Manager 或员工做所有决定，而是让 Agent 负责整理事实、识别风险、生成方案、推动确认和记录执行。

如果 MVP 阶段允许 Agent 自动处理重大决策，会带来明显风险：

1. 责任边界不清。
2. 老板不信任系统。
3. 员工可能收到未经确认的任务。
4. Manager 的管理权限被绕过。
5. 决策错误后难以追责。
6. 企业难以接受系统自动改变组织执行方向。

因此，MVP 阶段应明确采用分级决策机制。

决策等级	是否允许 Agent 自动处理
L0	允许，例如提醒、字段补全、格式校验
L1	员工确认后处理
L2	Manager 确认后处理
L3	老板 / Executive 确认后处理
L4	不允许系统闭环，进入线下会议或正式审批

Agent 可以自动做的事情包括：

1. 提醒员工填写日报。
2. 提醒缺失字段。
3. 根据员工输入整理 Staff Schema。
4. 根据本部门数据生成 Manager Schema 草稿。
5. 识别风险并提出建议。
6. 生成老板决策卡片草案。
7. 生成任务草案。
8. 在确认后调用后端创建正式任务。

Agent 不应自动做的事情包括：

1. 替员工确认日报。
2. 替 Manager 确认部门风险。
3. 替老板选择方案。
4. 自动下发重大任务。
5. 自动承诺客户或外部合作方。
6. 自动调整跨部门优先级。
7. 自动关闭高风险异常。
8. 自动修改已签名数据。

MVP 阶段最关键的一条规则是：

```
Agent 可以生成建议，但重大动作必须由人确认。
```

本问题的推荐结论：

```
MVP 阶段只允许 Agent 自动处理 L0 低风险事项；
L1-L3 必须经过对应人确认；
L4 只做提醒和材料整理。
```

19.5 Manager 权限边界是否清晰

Manager 是 AutoMage-2 中最容易出现权限边界模糊的角色。

如果 Manager 权限太小，部门汇总和异常处理做不起来；如果权限太大，又容易越权读取其他部门或员工敏感信息。

MVP 阶段需要把 Manager 的权限边界写清楚。

Manager 可以做：

1. 读取本部门员工已提交的 Staff Schema。
2. 读取本部门任务和异常。
3. 生成本部门 Manager Schema。

4. 确认部门汇总。
5. 处理部门内异常。
6. 创建本部门任务。
7. 将超权限事项上推 Executive。
8. 查看本部门历史汇总。

Manager 不可以做：

1. 读取其他部门员工日报明细。
2. 修改员工原始日报。
3. 替员工签名确认日报。
4. 确认老板决策。
5. 直接下发跨部门任务给其他部门员工。
6. 删除审计日志。
7. 修改老板已确认的决策。
8. 查看无关部门敏感异常。

最容易模糊的是跨部门协作。例如架构组需要后端支持时，架构组 Manager 不应直接读取后端员工日报，而应通过以下方式处理：

```
架构组 Manager 生成协作任务 / 异常
  ↓
上推 Executive 或分配给后端 Manager
  ↓
后端 Manager 再分配给后端员工
```

Manager 的默认权限应是“管本部门”，不是“看全公司”。

本问题的推荐结论：

```
MVP 阶段 Manager 只默认拥有本部门权限；
跨部门事项通过任务、异常或 Executive 授权处理。
```

19.6 老板侧是否必须做独立 App

老板侧是否做独立 App，关系到 MVP 的开发范围和演示成本。

从产品体验看，老板侧独立 App 长期有价值。它可以承载老板看板、组织健康度、关键风险、决策历史、任务追踪、趋势分析和复盘报告。

但从 MVP 交付看，不建议一开始就做独立 App。原因如下：

1. 老板侧核心动作很少，主要是看摘要和确认决策。
2. IM 卡片足够支撑 MVP 演示。
3. 独立 App 会增加前端开发、登录、权限、消息同步和看板设计成本。

4. 过早做 App 容易把重点从“决策闭环”转移到“界面展示”。
5. 当前最关键的是数据链路，而不是复杂 UI。

MVP 阶段老板侧建议采用：

```
IM 决策卡片 + 简单 Web 调试页 / 管理页
```

IM 决策卡片负责：

1. 展示公司摘要。
2. 展示关键风险。
3. 展示 A/B 决策方案。
4. 老板点击确认。
5. 老板要求补充信息。
6. 老板查看来源摘要。

简单 Web 调试页负责：

1. 查看 Schema JSON。
2. 查看任务列表。
3. 查看数据库记录。
4. 查看审计日志。
5. 支持内部联调。

后续再演进为老板 App：

阶段	老板侧形态
MVP	IM 卡片
v0.2	Web 看板
v0.3	老板侧工作台
v1.0	独立 App / 多端看板

本问题的推荐结论：

```
老板侧 MVP 不做独立 App；
先用 IM 决策卡片完成确认闭环，Web 页面只做联调和看板雏形。
```

19.7 Dream 机制 MVP 阶段做到什么程度

Dream 是 AutoMage-2 的核心概念之一，但 MVP 阶段不能把 Dream 做得过重。

完整 Dream 可能包含长期记忆、跨周期趋势分析、战略推演、组织诊断、外部数据输入、多 Agent 协同等能力。这些能力有价值，但不是 MVP 第一阶段必须全部完成。

MVP 阶段 Dream 只需要完成最小可用闭环：

```
读取 Manager Schema
  ↓
```

```
生成组织级摘要
  ↓
识别关键风险
  ↓
生成老板决策项
  ↓
生成任务草案
  ↓
等待老板确认
```

建议 MVP Dream 能力边界如下：

能力	MVP 是否需要
读取当日 Manager Summary	需要
读取最近几天历史 Summary	可选
读取历史 Decision	可选，但建议保留接口
读取未完成任务	可选
生成组织级摘要	需要
生成关键风险	需要
生成 A/B 决策项	需要
生成任务草案	需要
长期趋势分析	暂不做
复杂战略推演	暂不做
多轮自我反思	暂不做
自动执行决策	不允许

Dream 的输出必须结构化，不能只是一段自然语言。MVP 阶段建议统一输出到 `schema_v1_executive`，并保留 `source_summary_ids`。

Dream 失败时，也要有降级方案：

1. 生成简版摘要。
2. 只转发 Manager 上推事项。
3. 延迟重试。
4. 记录失败原因。
5. 通知管理员或演示者。

本问题的推荐结论：

```
MVP 阶段 Dream 只做每日组织摘要、风险识别、老板决策项和任务草案；
长期记忆、趋势分析和战略推演后置。
```

19.8 后端是否新增 Decision 表

是否新增 Decision 表，是当前数据库设计中最关键的待确认问题之一。

从短期看，不新增 Decision 表也能勉强跑通 Demo。可以把老板决策暂时放在：

1. `summaries.meta.decision_items`

2. `audit_logs.payload`
3. `tasks.meta.source_decision_id`

这种方式开发快，但长期问题非常明显。

如果没有独立 Decision 表，会出现：

1. 决策状态无法独立流转。
2. 一个 Executive Summary 中多个决策项不好分别确认。
3. 老板确认历史不好查询。
4. 决策与任务的来源关系不清楚。
5. 决策撤回、修改、补充信息不好实现。
6. 后续老板看板无法方便展示决策列表。
7. 审计和复盘会依赖复杂 JSON 查询。

而 AutoMage-2 的核心卖点之一就是“决策可追溯”。如果决策对象本身没有独立表，系统架构会显得不够扎实。

因此，建议新增独立 Decision 表，至少覆盖以下字段：

1. 决策标题。
2. 决策背景。
3. 决策等级。
4. 候选方案。
5. 推荐方案。
6. 推荐理由。
7. 预期影响。
8. 决策状态。
9. 确认人。
10. 确认时间。
11. 最终方案。
12. 来源 Summary。
13. 生成任务。
14. 签名信息。
15. 扩展 meta。

如果时间非常紧，可以采用折中方案：

```
Demo 前先用 summaries.meta 临时承载；
同时在文档和 DDL 中明确 decisions 表；
Demo 后第一优先级补表。
```

但从架构完整性看，推荐尽早加。

本问题的推荐结论：

建议 MVP 阶段新增 decisions 表；
如果开发排期无法支持，也必须保留清晰临时映射和后续迁移方案。

19.9 数字签名是否本阶段落地

数字签名是数据可信机制的基础，但 MVP 阶段不需要一次性实现完整电子签章。

这里需要区分两个概念：

1. 企业级法律电子签。
2. MVP 内部确认签名。

MVP 阶段不需要做 CA、私钥、第三方存证、法律电子签章。但必须做最小确认签名，否则 Staff 日报、Manager 汇总和老板决策都缺少责任边界。

MVP 阶段至少要做到：

1. 记录谁确认。
2. 记录确认时间。
3. 记录确认来源。
4. 记录确认内容 hash。
5. 写入审计日志。
6. 签名后内容变更需重新确认。
7. 未确认老板决策不能生成正式任务。

建议最小字段：

```
{
  "signature_status": "signed",
  "signed_by": 10086,
  "signed_at": "2026-05-03T18:13:00+08:00",
  "payload_hash": "sha256:xxxxx",
  "signature_source": "im"
}
```

这些字段可以先放在业务对象的 `meta.signature` 中，后续再抽成 `digital_signatures` 表。

不同对象的要求如下：

对象	MVP 签名要求
Staff 日报	员工确认后进入正式汇总
Manager 汇总	高风险和上推事项建议 Manager 确认
Executive 决策	老板确认后才能生成任务
任务完成	执行人提交完成，确认方关闭
异常关闭	根据等级由对应权限人确认

如果本阶段完全不做签名，至少会出现三个问题：

1. 员工可能不认可 Agent 整理的日报。

2. 老板确认无法成为任务来源。
3. 审计链路无法证明关键动作由谁完成。

本问题的推荐结论：

```
MVP 阶段必须落地轻量数字签名；
不做法律电子签，但要做账号确认、payload_hash 和审计日志。
```

19.10 IM 平台优先接飞书还是企业微信

IM 平台选择会影响 Demo 速度、企业客户覆盖和后续集成成本。

飞书和企业微信各有优势。

平台	优势	风险
飞书	卡片能力强，开放接口体验好，适合研发团队和 Demo	部分传统企业不一定使用
企业微信	国内企业覆盖广，客户接受度高	卡片和接口体验相对更复杂，调试成本可能更高

从 MVP 演示角度看，建议优先飞书。原因是：

1. 卡片交互更适合老板决策。
2. 研发团队接入和调试速度较快。
3. 适合展示 A/B 方案、按钮确认、来源查看。
4. 能更快证明产品主链路。
5. 对内部 Demo 和技术联调更友好。

从商业落地角度看，企业微信后续必须考虑。尤其是面向传统企业、制造业、销售组织和国内中小企业时，企业微信覆盖面更广。

建议路线：

```
MVP：优先飞书
v0.2：抽象 IM Adapter
v0.3：接企业微信
v0.4：支持钉钉 / Slack / Webhook
```

关键是不要把业务逻辑写死在某一个 IM 平台里。应该抽象出统一 IM 消息层：

1. 发送文本。
2. 发送卡片。
3. 接收按钮事件。
4. 接收用户回复。
5. 绑定用户身份。
6. 回调后调用内部 API。

这样后续切换平台时，不需要重写 Staff / Manager / Executive 的业务逻辑。

本问题的推荐结论：

```
MVP 优先接飞书；
同时在架构上预留 IM Adapter，避免绑定单一平台。
```

19.11 Schema 版本如何管理

Schema 版本管理如果一开始不设计清楚，后续会严重影响 Agent、API、数据库和前端联调。

MVP 阶段至少会存在以下 Schema：

1. `schema_v1_staff`
2. `schema_v1_manager`
3. `schema_v1_executive`
4. `schema_v1_task`
5. `schema_v1_incident`

每个 Schema 都应有：

```
{
  "schema_id": "schema_v1_staff",
  "schema_version": "1.0.0"
}
```

版本管理需要解决几个问题：

1. Agent 应该按哪个版本输出。
2. 后端支持哪些版本。
3. 前端表单对应哪个版本。
4. 数据库里存的是哪个版本。
5. 老数据如何兼容。
6. 新字段如何灰度上线。
7. 废弃字段如何处理。

MVP 阶段建议采用以下规则：

19.11.1 Schema ID 表示类型

例如：

```
schema_v1_staff
schema_v1_manager
schema_v1_executive
```

它表示这是哪类 Schema。

19.11.2 schema_version 表示版本

例如：

```
1.0.0
1.1.0
2.0.0
```

推荐语义：

版本变化	含义
patch	修正文案、说明，不破坏结构
minor	新增可选字段，兼容旧版本
major	删除字段、修改类型、破坏兼容

19.11.3 后端维护支持版本

后端应知道每个接口支持哪些版本：

```
schema_v1_staff: 1.0.0, 1.1.0
schema_v1_manager: 1.0.0
```

不支持版本应返回明确错误。

19.11.4 数据库保存版本信息

所有核心对象都应保存：

1. `schema_id`
2. `schema_version`
   如果没有显式字段，至少保存在 `meta` 中。

19.11.5 Schema Contract 后续落表

MVP 阶段可以先用文档和代码维护 Schema，后续建议新增 `schema_contracts` 表，用于保存：

1. 字段定义。
2. 必填规则。
3. 枚举规则。
4. 签名规则。
5. Prompt 约束。
6. 校验规则。
7. 生效时间。
8. 废弃时间。

本问题的推荐结论：

```
MVP 阶段采用 schema_id + schema_version 管理；
后端显式维护支持版本；
后续新增 schema_contracts 表统一管理。
```

19.12 产品侧需要补充的问题清单

以下问题需要产品侧进一步确认。它们不一定阻塞技术方案设计，但会影响 MVP 范围、Demo 话术、交互优先级和后续开发排期。

19.12.1 MVP 场景边界

1. MVP 首个 Demo 面向内部演示，还是面向外部客户？
2. Demo 场景是研发团队，还是更通用的企业管理场景？
3. 第一版是否只做一个部门，还是至少两个部门？
4. 是否必须展示跨部门协作？
5. Demo 是否需要真实 IM 回调，还是可以用模拟按钮？
6. MVP 是否要求真实多用户登录？
7. MVP 是否要求部署到可访问环境，还是本地联调即可？

19.12.2 员工侧体验

1. 员工日报是 IM 对话式，还是表单式？
2. 员工每天必须填报，还是任务触发时填报？
3. 员工是否可以修改已提交日报？
4. 修改后是否需要重新签名？
5. 员工是否能看到 Manager 对自己的评价？
6. 员工是否能看到自己被标记为 highlight_staff？
7. 员工是否可以拒绝或申诉任务分配？
8. 员工是否能关闭自己的任务，还是必须 Manager 关闭？

19.12.3 Manager 侧体验

1. Manager 汇总是否必须人工确认？
2. Manager 是否可以编辑 Agent 生成的部门汇总？
3. Manager 修改汇总后是否重新签名？

4. Manager 能否看到员工原文，还是只看结构化字段？
5. Manager 是否可以直接创建部门任务？
6. Manager 是否可以把任务分配给其他部门？
7. Manager 上推老板事项是否必须提供 A/B 方案？
8. Manager 是否需要每日固定收到部门汇总卡片？

19.12.4 老板侧体验

1. 老板每天最多接收几条决策卡片？
2. 老板是否接受 IM 卡片作为 MVP 入口？
3. 老板是否需要 Web 看板？
4. 老板是否能一键选择方案 A/B？
5. 老板是否可以输入自定义方案？
6. 老板点击“需要补充信息”后，信息回到谁那里？
7. 老板是否可以查看来源 Staff 日报？
8. 老板是否需要决策历史列表？
9. 老板是否需要每周复盘报告？

19.12.5 Dream 机制边界

1. Dream 每天什么时候运行？
2. Dream 是晚上生成草稿，还是早上生成正式卡片？
3. Dream 是否需要读取历史任务？
4. Dream 是否需要读取历史决策？
5. Dream 是否允许生成多个老板决策项？
6. 每天最多向老板推送多少个风险？
7. Dream 失败时是否通知老板，还是只通知管理员？
8. Dream 生成内容是否需要人工预审？

19.12.6 数据可信与签名

1. Staff 日报是否必须员工签名？
2. Manager 汇总是否必须 Manager 签名？
3. 老板确认是否必须生成 payload_hash？
4. 签名后内容是否允许修改？

5. 修改后是否保留旧版本？
6. 是否需要导出审计日志？
7. 是否需要对外解释签名不是法律电子签？
8. 是否需要未来接第三方电子签平台？

19.12.7 权限与隐私

1. 老板是否默认可查看所有员工日报明细？
2. Manager 是否能查看员工完整日报原文？
3. 员工是否能看到部门整体健康度？
4. Staff Agent 是否允许查询他人任务状态？
5. 跨部门任务是否允许双方 Manager 互看进度？
6. 权限失败是否展示具体原因？
7. 是否需要敏感字段脱敏？
8. 是否有企业客户对数据隔离的特殊要求？

19.12.8 数据库与后端范围

1. MVP 是否新增 `decisions` 表？
2. MVP 是否新增 `agent_nodes` 表？
3. MVP 是否新增 `digital_signatures` 表？
4. Dream Run 是否单独建表？
5. Schema Contract 是否进入本期？
6. 当前 DDL 是否允许调整？
7. 是否接受部分字段先放 `meta`？
8. 哪些字段必须显式建列方便查询？

19.12.9 IM 平台与交互

1. MVP 首发飞书、企业微信还是 Web？
2. 是否需要支持 IM 用户绑定系统账号？
3. 决策卡片按钮是否必须真实可点？
4. IM 卡片是否需要展示来源链接？
5. 员工日报是否通过 IM 对话填写？
6. 是否允许员工在 IM 中上传产出物链接？

7. IM 推送失败如何提示？
8. 是否需要支持消息撤回或更新卡片？

19.12.10 验收与交付

1. MVP 验收以 Demo 跑通为准，还是以接口联调完成为准？
2. 是否需要自动化测试脚本？
3. 是否需要提供种子数据？
4. 是否需要提供 Postman / Apifox 接口集合？
5. 是否需要提供数据库初始化 SQL？
6. 是否需要录屏演示？
7. 是否需要准备老板侧演示话术？
8. 是否需要输出下一阶段路线图？

19.13 风险优先级与建议处理顺序

为了避免所有问题同时讨论，建议将待确认问题按优先级分层处理。

优先级	问题	建议处理方式
P0	是否新增 Decision 表	尽快确认，影响决策闭环
P0	数字签名是否落地	必须做轻量版，影响可信链路
P0	Agent 是否允许自动决策	必须明确不允许自动重大决策
P0	Manager 权限边界	必须明确，影响数据安全
P0	Demo 使用哪个 IM 平台	影响开发排期和联调方式
P1	Dream MVP 做到什么程度	控制范围，避免过度开发
P1	Schema 版本如何管理	影响后续迭代
P1	老板侧是否做 App	建议后置
P1	员工填报真实性	先用确认和交叉校验解决
P2	屏幕记忆是否进入 MVP	建议后置
P2	行业模板 Schema	后续版本展开
P2	复杂权限授权模型	MVP 先做基础隔离

建议本阶段优先开会确认以下 6 个结论：

```
1. MVP 是否新增 decisions 表。
2. 是否采用轻量数字签名。
3. 老板确认前是否禁止生成正式任务。
4. Manager 是否只默认拥有本部门权限。
5. MVP 是否优先接飞书。
6. Dream 是否只做最小决策生成能力。
```

这 6 个问题确认后，技术实现路径会明显收敛。

19.14 本章小结

本章列出的风险和待确认问题，本质上都围绕一个核心：MVP 阶段要先证明 AutoMage-2 的三级 Agent 决策闭环成立，而不是一次性完成所有理想能力。

当前最建议的产品取舍是：

```
先通用 Schema，不急着做行业模板；
先 IM 表单，不急着做屏幕记忆；
先人确认，不急着让 Agent 自动决策；
先飞书卡片，不急着做老板 App；
先轻量签名，不急着做法律电子签；
先 Decision 表，不把老板决策长期塞进 meta；
先最小 Dream，不急着做复杂战略推演。
```

只要这些边界确认清楚，MVP 就可以围绕一条主链路快速推进：

```
员工确认日报
  → Manager 汇总部门事实
  → Dream 生成老板决策项
  → 老板确认方案
  → 系统生成任务
  → Staff Agent 接收执行
  → 全链路审计追溯
```

这一条链路跑通，比一次性堆满所有能力更重要。

20 后续版本规划

20.1 MVP v0.1：Mock 闭环

v0.1 的目标不是做完整系统，而是先用最小成本证明 AutoMage-2 的核心逻辑成立。

这一阶段可以不接真实 IM、不接完整数据库、不做复杂权限系统，重点是把三级 Agent 的数据流和任务闭环跑通：

```
员工输入
  ↓
Staff Agent 生成 Staff Schema
  ↓
Manager Agent 生成 Manager Schema
  ↓
Executive Agent / Dream 生成老板决策项
  ↓
老板选择方案
  ↓
系统生成任务草案
  ↓
Staff Agent 展示任务
```

v0.1 的核心交付物包括：

模块	交付内容
Staff Agent Mock	能把员工自然语言整理成 Staff Schema
Manager Agent Mock	能读取 Mock Staff Schema 并生成部门汇总
Executive Agent Mock	能生成老板决策项和 A/B 方案
Dream Mock	能基于部门汇总生成组织摘要和任务草案
Task Mock	能根据老板确认生成任务对象
Demo 数据	准备一组完整样例数据
演示脚本	能按 12 步演示主链路

v0.1 可以先使用本地 JSON 或内存对象模拟数据库，例如：

```
mock_staff_records.json
mock_manager_summaries.json
mock_executive_decisions.json
mock_tasks.json
```

这一阶段不要求：

1. 真实用户登录。
2. 真实 IM 回调。
3. 完整数据库写入。
4. 完整权限校验。
5. 完整数字签名。

6. 完整审计日志。

但 v0.1 必须保证数据结构和后续真实系统一致。也就是说，Mock 可以是假的，但 Schema 不能随便写。

v0.1 的验收标准：

1. 能从一条员工输入生成 Staff Schema。
2. 能从 Staff Schema 生成 Manager Schema。
3. 能从 Manager Schema 生成 Executive 决策项。
4. 决策项包含 A/B 方案、推荐方案和任务草案。
5. 老板确认后能生成 Task Schema。
6. Task 能追溯到 Decision、Summary 和 Staff Record。
7. 演示者能完整讲清楚这条链路。

v0.1 的推荐周期：

```
1-3 天
```

这一版的价值是快速统一产品、后端、Agent 客制化和前端对主链路的理解，避免团队一开始就陷入数据库字段、IM 平台和权限细节。

20.2 MVP v0.2：真实数据库联调

v0.2 的目标是把 v0.1 的 Mock 数据落到真实数据库中，让 AutoMage-2 从“流程演示”进入“数据可信”的阶段。

这一阶段的重点是：

```
Schema 不再只存在于 Agent 输出中；
每一步关键结果都必须写入数据库；
每个对象都能被查询、追溯和复用。
```

v0.2 需要优先联调以下数据库链路：

Schema / 对象	目标表
Staff Schema	`work_records`、`work_record_items`
Manager Schema	`summaries`、`summary_source_links`
Executive Schema	`summaries` / `decisions`
Task Schema	`tasks`、`task_assignments`、`task_updates`
Incident Schema	`incidents`、`incident_updates`
Audit	`audit_logs`

v0.2 的核心任务包括：

1. 完成 Staff 日报提交接口。
2. 完成 Manager 汇总提交接口。
3. 完成 Summary 来源追溯。

4. 完成老板决策临时落库或正式 Decision 表。
5. 完成任务生成和任务分配写库。
6. 完成任务查询接口。
7. 完成基础审计日志写入。
8. 完成幂等键处理。
9. 完成基础 422 错误返回。

v0.2 最关键的链路是：

```
Staff Schema
  → work_records
  → work_record_items
  → Manager Schema
  → summaries
  → summary_source_links
  → Executive Decision
  → tasks
```

如果此阶段时间有限，建议优先新增两张表：

表	原因
`agent_nodes`	Agent 初始化和权限绑定需要事实源
`decisions`	老板决策不应长期塞在 `summaries.meta`

数字签名、Dream Run、Schema Contract 可以先用 `meta` 或审计日志承载，但要保留后续迁移方案。

v0.2 的验收标准：

1. Staff 日报可以真实写入数据库。
2. Manager 汇总可以真实写入数据库。
3. Summary 可以追溯来源 Staff Record。
4. 老板决策可以被记录。
5. 老板确认后可以生成真实任务。
6. Staff Agent 可以从数据库读取任务。
7. 关键动作有审计日志。
8. 同一日报或同一决策不会重复写入。

v0.2 的推荐周期：

```
3-7 天
```

这一版完成后，AutoMage-2 就不再只是 Agent Demo，而是具备了可联调的数据底座。

20.3 MVP v0.3：真实 IM 接入

v0.3 的目标是接入真实 IM 平台，让员工、Manager 和老板都能在实际沟通入口中完成操作。

MVP 阶段建议优先接飞书，原因是飞书卡片能力强，适合展示老板决策卡片、按钮确认、任务提醒和来源查看。企业微信可以在后续版本接入。

v0.3 的核心目标是：

```
用户不需要打开复杂后台；
通过 IM 就能完成日报提交、决策确认和任务接收。
```

v0.3 需要完成以下能力：

场景	IM 能力
Staff 日报	IM 对话或表单填写
日报确认	员工点击确认或回复确认提交
Manager 汇总	向 Manager 推送部门摘要
老板决策	向老板推送决策卡片
老板确认	按钮选择方案 A / B
任务下发	向员工或 Manager 推送任务
异常提醒	推送阻塞或高风险事项
账号绑定	IM 用户与系统用户绑定

v0.3 不建议把业务逻辑写死在飞书中。应抽象出统一 IM Adapter：

```
send_text
send_card
send_task_notification
send_decision_card
handle_button_callback
handle_user_message
bind_im_user
```

这样后续接企业微信、钉钉、Slack 或 Webhook 时，不需要重写业务主链路。

v0.3 的建议实现路径：

```
先接 Staff 日报提交
  ↓
再接老板决策卡片
  ↓
再接任务下发
  ↓
最后接 Manager 汇总和异常提醒
```

不要一开始就做所有消息类型，否则联调成本会快速上升。

v0.3 的验收标准：

1. 员工能在 IM 中提交日报。

2. Staff Agent 能在 IM 中展示结构化确认内容。
3. 员工确认后，日报真实写库。
4. 老板能在 IM 中收到决策卡片。
5. 老板点击方案后，后端能收到回调。
6. 老板确认后能生成正式任务。
7. 员工能在 IM 中收到任务提醒。
8. IM 回调失败时不会重复生成任务。
9. IM 用户身份能映射到系统用户。

   v0.3 的推荐周期：

```
5-10 天
```

这一版完成后，AutoMage-2 就具备了真实使用入口，不再需要通过手动脚本或调试页面演示。

20.4 MVP v0.4：老板决策卡片

v0.4 的目标是把老板侧体验打磨到可演示、可理解、可点击确认的程度。

老板侧不需要一开始做独立 App，但决策卡片必须足够清楚。老板应该能在 30 秒内看懂：

1. 今天发生了什么。
2. 风险是什么。
3. 为什么需要我决策。
4. 有哪些方案。
5. Agent 推荐哪个方案。
6. 确认后会生成什么任务。

   老板决策卡片建议包含以下结构：

```
今日组织摘要
关键风险
待决策事项
方案 A
方案 B
推荐方案
推荐理由
确认后任务
来源摘要
操作按钮
```

v0.4 的核心交付物包括：

模块	交付内容
决策卡片模板	固定卡片结构和字段
A/B 方案展示	支持老板快速比较
推荐理由展示	控制在短文本内
任务草案展示	老板知道确认后会发生什么
来源查看	可查看 Manager Summary 来源
按钮回调	支持选择 A、选择 B、需要补充信息、驳回
确认审计	确认动作写入审计日志
幂等控制	重复点击不重复生成任务

老板卡片示例结构：

```
【今日待决策】

事项：是否新增独立 Decision 表

背景：
当前 Executive 决策链路需要记录老板确认结果，但正式 DDL 尚未建立独立 Decision 表。

方案 A：新增独立 decisions 表
优点：决策对象清晰，便于审计和任务来源追踪
缺点：增加 0.5-1 天开发工作量

方案 B：暂时复用 audit_logs 和 tasks.meta
优点：短期更快
缺点：长期复盘和查询不清晰

推荐：方案 A

确认后将生成任务：
- 补充 decisions 表结构草案
- 负责人角色：后端
- 截止时间：今日 18:00
```

按钮建议：

按钮	动作
选择方案 A	确认 A 并生成任务
选择方案 B	确认 B 并生成任务
需要补充信息	回传 Manager
暂不处理	关闭或延后
查看来源	展开 Manager Summary

v0.4 的验收标准：

1. 老板卡片不依赖长篇说明也能看懂。
2. 决策项至少包含 A/B 方案。
3. 推荐方案有明确理由。
4. 确认后任务草案可见。

5. 老板点击确认后，决策状态更新。
6. 任务正式生成并下发。
7. 确认动作有签名和审计。
8. 重复点击不会生成重复任务。
9. “需要补充信息”能回传给 Manager 或相关节点。

   v0.4 的推荐周期：

```
3-7 天
```

这一版完成后，AutoMage-2 的老板侧价值会更清楚，也更适合对外演示。

20.5 MVP v1.0：完整三级数据闭环

v1.0 的目标是完成 AutoMage-2 MVP 的第一阶段完整闭环。

这一版不要求实现所有未来设想，但必须让以下链路稳定可用：

```
Staff Agent
  → Manager Agent
  → Executive Agent / Dream
  → 老板确认
  → 任务生成
  → Staff Agent 执行反馈
  → Manager 汇总回流
  → 审计追溯
```

v1.0 应具备以下能力：

20.5.1 数据闭环

1. 员工日报可以通过 IM 或 Web 提交。
2. Staff Schema 可以被后端校验和写库。
3. Manager Agent 可以读取本部门 Staff 数据。
4. Manager Schema 可以写入 Summary。
5. Executive Agent 可以读取部门 Summary。
6. Dream 可以生成老板决策项。
7. 老板确认后可以生成任务。
8. 任务可以下发给执行人。
9. 员工可以更新任务状态。
10. 任务结果可以进入下一轮 Staff / Manager 汇总。

20.5.2 权限闭环

1. Staff 只能读写本人数据。
2. Manager 只能读取本部门数据。
3. Executive 默认读取汇总，不默认展开所有个人明细。
4. 跨部门协作通过任务或异常处理。
5. 老板决策必须由授权用户确认。
6. 越权请求被拒绝并写入审计。

20.5.3 签名闭环

1. 员工日报有员工确认。
2. Manager 汇总有确认状态。
3. 老板决策有确认记录。
4. 关键确认动作有 `payload_hash`。
5. 签名后修改需要重新确认。
6. 签名和确认动作写入审计日志。

20.5.4 任务闭环

1. 任务有来源。
2. 任务有负责人。
3. 任务有优先级。
4. 任务有截止时间。
5. 任务有状态流转。
6. 任务有执行结果。
7. 任务能追溯到决策、Summary 和 Staff Record。

20.5.5 异常闭环

1. Staff 层可以上报阻塞。
2. Manager 层可以合并异常。
3. Executive 层可以看到组织级风险。
4. 异常可以生成任务。
5. 任务完成后异常可以关闭。
6. 异常处理过程有审计。

   v1.0 的最终验收标准可以写成一句话：

```
系统能从员工确认的工作记录出发，经部门汇总、Dream 分析和老板确认，生成正式任务并下发给执行人；
↪ 同时全链路具备来源追溯、权限控制、签名确认、异常处理和审计记录。
```

v1.0 的推荐周期：

```
2-4 周
```

这一版完成后，AutoMage-2 就具备了作为企业 AI 中枢 MVP 的基本形态。

20.6 后续行业化 Schema 模板

v1.0 跑通后，下一阶段重点应从“通用闭环”转向“行业适配”。

通用 Staff Schema 可以表达大多数工作记录，但不同行业和岗位的高价值字段并不相同。如果长期只使用通用字段，系统会逐渐变成“高级日报工具”，很难深入企业业务流程。

后续建议引入行业化 Schema 模板。

20.6.1 岗位模板

先从岗位维度做模板，比直接做行业模板更容易落地。

岗位	模板重点
研发	任务、代码、接口、Bug、联调、技术风险
产品	需求、原型、评审、用户反馈、版本风险
销售	客户、商机、报价、合同、成交概率
客服	工单、投诉、满意度、升级处理
运营	活动、数据、投放、转化、异常
项目经理	里程碑、资源、延期、依赖、风险
HR	招聘、面试、入职、绩效、组织风险

20.6.2 行业模板

在岗位模板稳定后，再做行业模板。

行业	模板重点
软件研发	需求、迭代、代码、测试、发布
制造业	工单、设备、产线、质量、交付
教育	学员、课程、作业、反馈、续费
医疗服务	患者流程、服务质量、合规事项
电商	商品、订单、客服、投放、库存
咨询服务	客户项目、交付物、会议、风险

行业模板不应完全替代通用 Schema。推荐结构是：

```
通用 Schema
  +
岗位扩展字段
  +
行业扩展字段
```

例如研发 Staff Schema 可以在通用字段之外增加：

```
{
  "dev_ext": {
    "related_repo": "automage-backend",
    "commit_ids": [],
    "api_changes": [],
    "bug_ids": [],
    "ddl_changes": [],
    "blocker_type": "schema_contract"
  }
}
```

20.6.3 模板管理方式

后续建议通过 `schema_contracts` 管理模板版本。

每个模板应包含：

1. Schema ID。
2. Schema 版本。
3. 适用行业。
4. 适用岗位。
5. 字段定义。
6. 必填规则。
7. 枚举规则。
8. Agent Prompt 约束。
9. 前端表单配置。
10. 后端校验规则。

行业化模板的目标不是增加填报负担，而是提高 Agent 理解业务的准确性。

本阶段建议顺序：

```
研发模板
  ↓
销售模板
  ↓
客服模板
  ↓
项目管理模板
  ↓
行业综合模板
```

20.7 后续屏幕记忆自动采集

屏幕记忆和自动采集是 AutoMage-2 后续的重要增强方向，但应在三级数据闭环稳定后再进入。

它的目标是减少员工主动填报负担，让系统能从真实工作流中自动提取部分事实。

可采集的数据包括：

数据来源	可提取内容
文档系统	文档创建、修改、评审、链接
代码仓库	commit、PR、review、issue
项目管理工具	任务状态、里程碑、阻塞
客服系统	工单、投诉、处理结果
CRM	客户跟进、商机状态、回款
日历会议	会议主题、参与人、行动项
IM 群聊	明确任务、风险、决策片段
屏幕记忆	当前工作上下文、应用使用痕迹

但屏幕记忆必须谨慎设计。它涉及隐私、合规、员工信任和数据安全，不能简单做成“自动监控”。

后续建议采用分阶段路线：

20.7.1 v1.1：系统连接器

优先接入已有系统，而不是直接采集屏幕。

可先接：

1. GitHub / GitLab。
2. Jira / Linear。
3. 飞书文档。
4. Notion。
5. 企业微信 / 飞书群消息。
6. CRM 或工单系统。

这些系统本身就是工作事实源，采集阻力低，价值明确。

20.7.2 v1.2：产出物自动回填

系统可以根据任务和日报自动关联产出物。

例如：

1. 员工完成代码提交后，Staff Agent 提醒是否关联到今日任务。
2. 文档修改后，自动作为任务产出物候选。
3. PR 合并后，自动更新任务进度。
4. 会议纪要生成后，自动生成待办草案。

这一阶段仍以“用户确认”为主，不自动替用户提交正式日报。

20.7.3 v1.3：授权工作记忆

在用户明确授权后，Agent 可以记录部分工作上下文。

原则：

1. 用户知道采集什么。
2. 用户可以暂停采集。
3. 用户可以查看和删除采集记录。
4. 敏感窗口和敏感字段默认过滤。
5. 采集内容先作为草稿，不直接进入正式汇总。
6. 提交前仍需用户确认。

20.7.4 v1.4：屏幕记忆

屏幕记忆应作为高级能力谨慎推出。

必须具备：

1. 本地敏感信息过滤。
2. 应用白名单。
3. 采集时间段控制。
4. 用户可见采集状态。
5. 原始截图最小化保存。
6. 自动摘要优先，不长期保存原始内容。
7. 企业管理员策略配置。
8. 完整审计和权限隔离。

屏幕记忆的最终目标不是监控员工，而是减少重复填报，让 Agent 更准确理解工作上下文。

推荐产品表述：

工作记忆助手

不建议使用：

屏幕监控

20.8 后续绩效分析与组织诊断

当 Staff、Manager、Executive、Task、Incident、Audit 的数据稳定沉淀后，AutoMage-2 可以进一步发展为组织诊断系统。

但绩效分析要非常谨慎。如果过早把系统定位成“考核员工”，会影响员工真实填报，也会降低团队接受度。

建议后续分三层推进：

```
先做组织运行诊断
再做团队效能分析
最后才做个人绩效辅助
```

20.8.1 组织运行诊断

这是最适合优先做的方向。

组织运行诊断关注的是系统性问题，而不是直接评价某个员工。

可分析内容包括：

1. 哪些任务经常阻塞。
2. 哪些部门经常缺失日报。
3. 哪类异常反复出现。
4. 哪些决策长期未确认。
5. 哪些老板任务经常逾期。
6. 哪些跨部门协作成本高。
7. 哪些 Schema 字段经常缺失。
8. 哪些流程节点最容易失败。

输出可以是：

```
本周组织运行报告
部门风险趋势
异常类型分布
任务逾期原因分析
决策响应时间分析
跨部门依赖图
```

这类分析对老板和 Manager 有价值，也不容易引发员工抵触。

20.8.2 团队效能分析

团队效能分析关注部门和小组层面。

可分析指标包括：

指标	说明
日报提交完整率	部门数据覆盖情况
任务完成率	部门任务推进情况
阻塞解决时长	部门处理问题速度
高风险事项数量	部门风险压力
决策上推质量	Manager 上推事项是否清晰
任务返工率	完成后重新打开或修改比例
产出物提交率	任务是否有明确产出
跨部门协作响应时间	部门协作效率

这些指标应主要用于发现流程问题，而不是直接作为个人绩效结论。

例如，系统可以提示：

```
本周架构组有 3 个任务阻塞都与数据库字段确认有关，建议提前建立 Schema Review 流程。
```

这种诊断比直接说“某某效率低”更有价值。

20.8.3 个人绩效辅助

个人绩效辅助应放在更后期，并且必须明确边界。

可以做的：

1. 汇总员工本人完成任务。
2. 汇总员工本人产出物。
3. 汇总员工参与的关键项目。
4. 帮员工生成周报、月报。
5. 帮 Manager 准备一对一沟通材料。
6. 记录员工遇到的阻塞和支持需求。
7. 帮员工展示贡献证据。

不建议 MVP 或早期版本做的：

1. 自动给员工打绩效分。
2. 自动排名。
3. 自动判断员工是否努力。
4. 自动生成负面评价。
5. 将屏幕采集直接用于绩效判断。
6. 将 Agent 推断作为正式考核依据。

个人绩效辅助应以“证据整理”为主，而不是“自动裁判”。

推荐定位：

```
绩效材料辅助生成
```

不建议定位：

```
AI 自动绩效评估
```

20.8.4 组织诊断看板

后续可以形成老板侧组织诊断看板。

看板模块包括：

1. 今日组织健康度。

2. 本周关键风险。
3. 决策待处理列表。
4. 高优任务执行状态。
5. 部门健康趋势。
6. 异常分布。
7. 跨部门依赖。
8. Dream 建议记录。
9. 决策执行效果复盘。
10. 审计与责任追溯。

这部分可以作为老板 App 或 Web 工作台的核心价值。

20.8.5 从数据闭环到管理闭环

绩效分析和组织诊断的前提，是前面的数据链路足够可信。

如果 Staff 日报不真实，Manager 汇总无来源，老板决策无签名，任务无状态，异常无关闭，审计无记录，那么任何绩效分析都会失真。

因此后续版本规划必须遵守顺序：

```
先数据可信
再任务闭环
再异常闭环
再组织诊断
最后绩效辅助
```

不要跳过前面的可信链路直接做分析，否则结果容易漂亮但不可用。

20.9 本章小结

AutoMage-2 的后续版本规划应围绕一条主线推进：

```
先跑通闭环
再接入真实系统
再优化老板体验
再扩展行业模板
再做自动采集
最后做组织诊断
```

推荐版本节奏如下：

版本	核心目标
v0.1	Mock 闭环，验证三级 Agent 主链路
v0.2	真实数据库联调，建立事实源
v0.3	真实 IM 接入，进入实际交互入口
v0.4	老板决策卡片，强化确认体验
v1.0	完整三级数据闭环，形成 MVP 交付
v1.1+	行业模板、系统连接器、自动采集、组织诊断

当前阶段最重要的不是一次性做完所有愿景，而是守住 MVP 的核心闭环：

```
员工事实
  → 部门判断
  → 老板决策
  → 任务执行
  → 结果回流
  → 审计追溯
```

这条链路稳定以后，AutoMage-2 才能继续向行业化、自动化和组织智能方向扩展。

A Schema JSON 完整示例

A.1 Staff Schema 示例

A.2 Manager Schema 示例

A.3 Executive Schema 示例

A.4 Task Schema 示例

A.5 Incident Schema 示例

A.6 Decision Schema 示例

B 字段字典

B.1 通用字段

B.2 用户字段

B.3 部门字段

B.4 Staff 字段

B.5 Manager 字段

B.6 Executive 字段

B.7 任务字段

B.8 决策字段

B.9 异常字段

B.10 签名字段

C 错误码表

C.1 通用错误码

C.2 Schema 校验错误

C.3 权限错误

C.4 签名错误

C.5 Agent 执行错误

C.6 Dream 执行错误

C.7 数据库写入错误

C.8 IM 推送错误

D 待产品确认问题清单

D.1 老板侧交互问题

D.2 员工侧填报问题

D.3 Manager 权限问题

D.4 决策等级问题

D.5 数字签名问题

D.6 行业场景问题

D.7 MVP 演示问题